STUDENT GUIDE

FACULTY OF INFORMATION TECHNOLOGY

DATABASE SYSTEMS 622

RICHFIELD

Registered with the Department of Higher Education as a Private Higher Education Institution under the Higher Education Act, 1997.
Registration Certificate No. 2000/HE07/008

FACULTY OF INFORMATION TECHNOLOGY

LEARNER GUIDE

MODULE: DATABASE SYSTEMS 622 (2ND SEMESTER)

(ADVANCED DATABASE CONCEPTS)

PREPARED ON BEHALF OF

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (PTY) LTD

GRADUATE INDTITUTE OF TECHNOLOGY (PTY) LTD
Registration Number: 2000/000757/07
All rights reserved; no part of this publication may be reproduced in any form or by any means, including photocopying machines, without the written permission of the Institution.

Table of Contents

Prologue viii

Preface x

Module Outcomes and Prerequisites x

Resource Links 12

Chapter 1: Advanced SQL 13

Data Definition Commands 14

Starting Database Model 14

Creating the Database 15

The Database Schema 16

Data Types 16

Creating Table Structures 19

CREATE TABLE command 19

SQL Constraints 21

Creating a Table with a SELECT Statement 24

SQL Indexes 25

Altering Table Structures 26

Changing a Column's Data Type 27

Changing a Column's Data Characteristics 27

Adding a Column 28

Adding Primary Key, Foreign Key, and Check Constraints 28

Dropping a Column 29

Deleting a Table from the Database 29

Data Manipulation Commands 30

Adding Table Rows 30

Inserting Table Rows with a SELECT Subquery 31

Saving Table Changes 33

Updating Table Rows 33

Deleting Table Rows 36

Restoring Table Contents 36

Virtual Tables: Creating a View 37

Updatable Views 39

Auto Increment, Identity, and Sequences 40

Procedural SQL 46

Stored Procedures 48

Working with Variables 50
Conditional Execution 51
Iteration or Looping 53
SELECT Processing with Cursors 57
Stored Procedures with Parameters 60
Triggers 62
User Defined Functions 73
Embedded SQL 73
Summary 79
Review Questions 80
Chapter 2: Database Design 81
The Information System 82
The Systems Development Life Cycle 83
Planning 83
Analysis 85
Detailed Systems Design 85
Implementation 86
Maintenance 86
The Database Life Cycle 87
The Database Initial Study 87
Database Design 90
Implementation and Loading 92
Testing and Evaluation 94
Operation 96
Maintenance and Evolution 97
Conceptual Design 97
Data Analysis and Requirements 99
Entity Relationship Modeling and Normalization 101
Data Model Verification 104
Distributed Database Design 107
DBMS Software Selection 107
Logical Design 108
Map the Conceptual Model to the Logical Model Components 108
Validate the Logical Model Using Normalization 110
Validate the Logical Model Integrity Constraints 110
Validate the Logical Model against User Requirements 111

Physical Design 111
Define Data Storage Organization 112
Define Integrity and Security Measures 113
Determine Performance Measurements 113
Database Design Strategies 113
Centralized versus Decentralized Design 114
Summary 117
Review Questions 118
Chapter 3: Transaction Management and Concurrency Control 119
What Is a Transaction? 120
Evaluating Transaction Results 121
Transaction Properties 124
Transaction Management with SQL 125
The Transaction Log 126
Concurrency Control 127
Lost Updates 127
Uncommitted Data 128
Inconsistent Retrievals 129
The Scheduler 130
Concurrency Control with Locking Methods 132
Lock Granularity 132
Lock Types 135
Two-Phase Locking to Ensure Serializability 136
Deadlocks 137
Concurrency Control with Time Stamping Methods 138
Wait/Die and Wound/Wait Schemes 139
Concurrency Control with Optimistic Methods 140
ANSI Levels of Transaction Isolation 140
Database Recovery Management 142
Transaction Recovery 142
Summary 146
Review Questions 148
Chapter 4: Database Performance Tuning and Query Optimization 149
Database Performance-Tuning Concepts 150
Performance Tuning: Client and Server 151
DBMS Architecture 152

Database Query Optimization Modes 154
Database Statistics 155
Query Processing 156
SQL Parsing Phase 157
SQL Execution Phase 158
SQL Fetching Phase 158
Query Processing Bottlenecks 158
Indexes and Query Optimization 160
SQL Performance Tuning 162
Index Selectivity 163
Conditional Expressions 164
Query Formulation 166
DBMS Performance Tuning 167
Summary 170
Review Questions 171
Chapter 5: Business Intelligence and Data Warehouse 172
The Need for Data Analysis 173
Business Intelligence 173
Business Intelligence Architecture 175
Business Intelligence Benefits 178
Business Intelligence Evolution 178
Business Intelligence Technology Trends 180
Decision Support Data 181
Operational Data versus Decision Support Data 181
Decision Support Database Requirements 184
The Data Warehouse 186
Data Marts 188
Twelve Rules That Define a Data Warehouse 189
Star Schemas 189
Facts 190
Dimensions 190
Attributes 191
Attribute Hierarchies 193
Star Schema Representation 195
Online Analytical Processing 197
Multidimensional Data Analysis Techniques 197

Advanced Database Support 198
Easy-to-Use End-User Interfaces 199
OLAP Architecture 200
Relational OLAP 201
Multidimensional OLAP 203
Relational versus Multidimensional OLAP 204
Data Analytics 204
Data Mining 205
Predictive Analytics 208
Data Lake 209
Summary 210
Review Questions 211
Chapter 6: Big Data and NoSQL 212
Big Data 213
Volume 214
Velocity 215
Variety 217
Other Characteristics 217
Hadoop 218
HDFS 219
Hadoop Ecosystem 221
Hadoop Pushback 223
NoSQL 223
Key-Value Databases 224
Document Databases 225
Column-Oriented Databases 226
Graph Databases 229
Aggregate Awareness 231
Summary 232
Review Questions 234
Chapter 7: Database Connectivity and Web Technologies 235
Database Connectivity 236
Native SQL Connectivity 237
ODBC, DAO, and RDO 238
ADO.NET 243
Java Database Connectivity (JDBC) 245

Database Internet Connectivity 247
Web-to-Database Middleware: Server-Side Extensions 248
Web Server Interfaces 250
The Web Browser 251
Client-Side Extensions 252
Web Application Servers 253
Cloud Computing Services 253
Cloud Implementation Types 256
Characteristics of Cloud Services 257
Types of Cloud Services 257
Cloud Services: Advantages and Disadvantages 259
Summary 261
Review Questions 262
Chapter 8: Database Administration 263
Data as a Corporate Asset 264
The Need for a Database and Its Role in an Organization 266
Introduction of a Database: Special Considerations 267
The Evolution of Database Administration 268
The Database Environment's Human Component 271
The DBA's Managerial Role 274
The DBA's Technical Role 278
Security 279
Security Policies 280
Security Vulnerabilities 280
Database Security 281
Database Administration Tools 283
The Data Dictionary 283
Case Tools 286
The DBA's Role in the Cloud 288
Summary 290
Review Questions 292
References 294

Prologue

Diploma in Information Technology (DIT)

The Diploma in Information Technology (DIT) at Richfield College is a dynamic and future-focused program designed to equip students with advanced technical, analytical, and problem-solving skills. At the core of the qualification is a commitment to academic excellence, industry alignment, and innovation, fostering graduates who are proficient in addressing modern technological challenges. This qualification strategically integrates theoretical knowledge with practical applications, preparing students for key roles in the IT sector.

The DIT program is structured to address the growing complexity of the evolving technological landscape. Through carefully curated modules, students gain comprehensive knowledge in essential information systems areas, including:

- Database Design and Management
- Business Information Systems
- Systems Analysis and Design
- Enterprise Resource Planning
- Data Analytics and Business Intelligence
- Information Security Management
- Cloud Computing Technologies
- IT Project Management

The curriculum bridges the gap between academic learning and real-world applications, fostering innovation and an entrepreneurial mindset. Students engage with modern information systems concepts through practical projects that emphasize how organizations use technology to achieve business objectives. The program includes hands-on experience with industry-standard business applications, database management systems, and enterprise software platforms. Additionally, students develop skills in requirements analysis, system documentation, and IT service management.

Their final project demonstrates their ability to analyze business requirements, design appropriate information system solutions, and implement these solutions effectively. This comprehensive approach ensures graduates are well-prepared to support and enhance organizational information systems in various business contexts.

Database Systems 622

The Database Systems module is a cornerstone component of the Diploma in Information Technology (DIT) program, providing students with a comprehensive understanding of database management systems and their crucial role in modern organizations. The module begins with fundamental concepts of data modeling, normalization, and relational database design, progressively advancing to more complex topics such as distributed databases, data warehousing, and NoSQL solutions.

Students explore the complete database lifecycle, from conceptual design and schema development to implementation, optimization, and maintenance. This includes hands-on experience with industry-standard database management systems and tools, such as Oracle, MySQL, and MongoDB. The module emphasizes the critical importance of data integrity, consistency, and security, teaching students how to design and implement robust database solutions that meet organizational requirements and performance objectives.

The curriculum incorporates practical laboratory sessions and real-world projects that challenge students to apply theoretical knowledge to solve complex data management problems. Students learn to develop and optimize database queries, implement stored procedures and triggers, and manage database transactions effectively. Special attention is given to database performance tuning and optimization techniques, preparing students to work with large-scale database systems and handle high-performance computing requirements.

Data security and access control are integrated throughout the module, ensuring students understand the importance of protecting sensitive data while maintaining database availability and reliability. The module also covers database administration tasks, backup and recovery procedures, and disaster recovery planning, preparing students to manage enterprise-level database systems that comply with industry standards and best practices.

The Database Systems module directly supports the DIT program's overall objectives by developing professionals who can design, implement, and manage sophisticated database solutions. It equips graduates with the skills to handle complex data environments, implement data-driven solutions, and support organizational decision-making through effective data management. Through this comprehensive approach, students become proficient database professionals capable of managing enterprise data assets and driving data-centric innovations across various industries.

Preface

Welcome to Database Systems 622. This module is a continuation of Database Systems 621. The purpose of this module is to familiarize students with the advanced principles of databases and SQL Server programming. Throughout this module, you will learn database design principles as well creating, querying and maintaining a database. This module serves to enlighten students on how databases are used on the backend of various software products and how data is retrieved and inserted. You will learn the basics of SQL Queries as well more advanced concepts like stored procedures, triggers and functions.

Module Outcomes and Prerequisites

After successfully completing the module, students should be able to demonstrate the following:

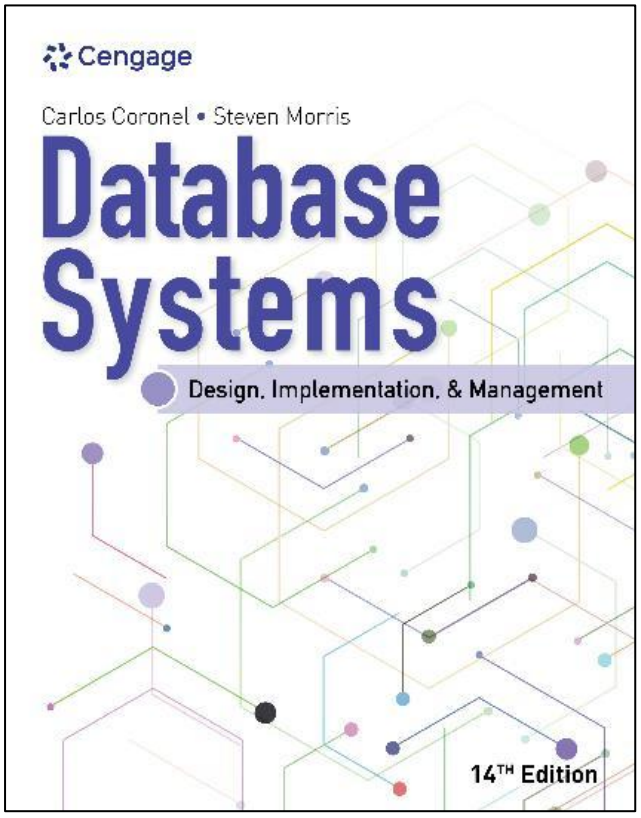
- Write complex SQL code.
- Write defensive SQL scripts.
- Write SQL Server scripts, stored procedures, and triggers.
- Perform basic SQL Server administration.
- Perform backups and restores of a database.
- The ability to manage concurrent transactions.
- An understanding of database concurrency.
- Understanding and managing SQL performance.
- Understanding distributed databases.
- An understanding of NoSQL Databases

Prerequisites

DBS621 for DIT Year 2 students

Students need a foundation of Introduction to Databases (DBS621B) in order to fully understand the concepts discussed in this module.

Prescribed Reading and Required Tools



Coronel, C and Morris, S. (2022). Database Systems: Design, Implementation & Management. 14th Ed. United Kingdom: Cengage Learning. ISBN:9780357673034

Microsoft SQL Server and Microsoft SQL Server Management Studio **(Software)**

Take note of the following symbols that will be used through the guide:



Indicates learning outcomes

Resource Links

SLQ Server Developer Edition

https://go.microsoft.com/fwlink/p/?linkid=2215158&clcid=0x1c09&culture=en-za&country=za

Please note that SQL Server is resource intensive, if your laptop cannot support the developer edition, then download SQL Server Express

SQL Server Express (for Celeron Processors)

https://go.microsoft.com/fwlink/p/?linkid=2216019&clcid=0x1c09&culture=en-za&country=za

SQL Server Management Studio

https://aka.ms/ssmsfullsetup

Chapter 1: Advanced SQL

Learning Outcomes

After this chapter you will be able to:

1. Use SQL to create a table manually
2. Use SQL to create a copy of a table using a subquery
3. Manipulate the structure of existing tables to add, modify, and remove columns and constraints
4. Use SQL to do data manipulation (insert, update, and delete rows of data)
5. Use SQL to create database views, including updatable views
6. Use procedural SQL to create triggers, stored procedures, and procedural SQL functions
7. Create embedded SQL

Data Definition Commands

Before you examine the SQL syntax for creating and defining tables and other elements, let's first examine a simple database model and the database tables that form the basis for the many SQL examples you will explore in this chapter.

Starting Database Model

The same simple database composed of CUSTOMER, INVOICE, LINE, PRODUCT, and VENDOR is used as a starting point for illustrating the SQL commands in this chapter. This database model is shown in Figure 1.1. As you can see in Figure 1.1, the database model contains many tables. However, to illustrate the initial set of data definition commands, the focus of attention will be the PRODUCT and VENDOR tables. You will have the opportunity to use the remaining tables later in this chapter and in the Problems section.

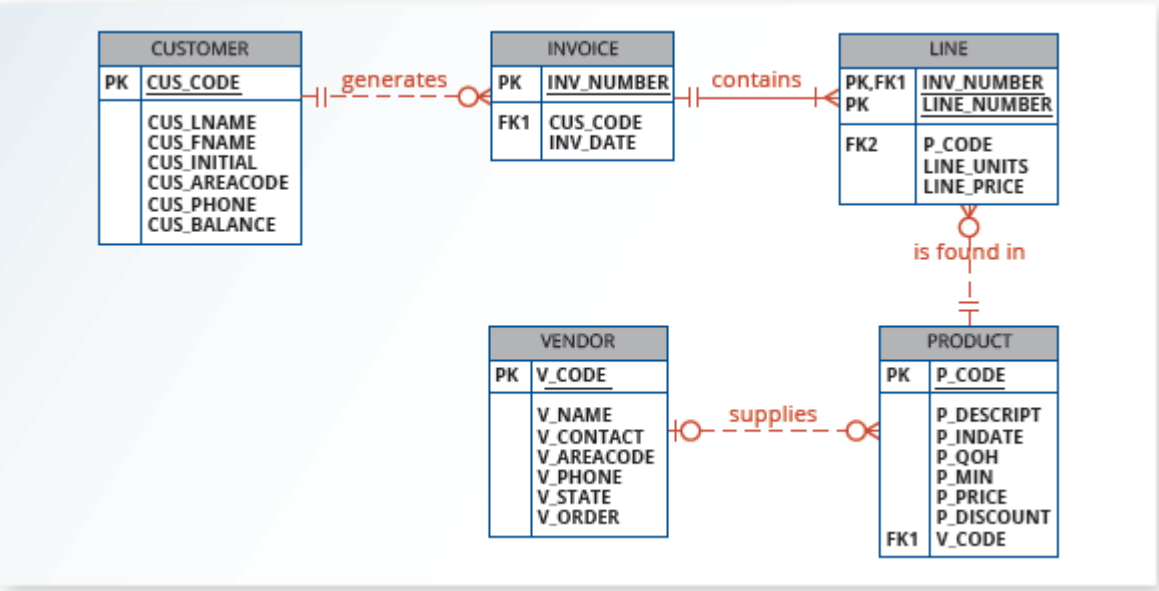


Figure 1-0-1: A database model

To give you a point of reference for understanding the effect of the SQL queries, the contents of the PRODUCT and VENDOR tables are listed in Figure 1.2. In the tables, note the following features, which correspond to the business rules reflected in the ERD shown in Figure 1.1:

- The VENDOR table contains vendors who are not referenced in the PRODUCT table. Database designers note that possibility by saying that PRODUCT is optional to VENDOR; a vendor may exist without a reference to a product.
- Existing V_CODE values in the PRODUCT table must (and do) have a match in the VENDOR table to ensure referential integrity.

A few products are supplied factory-direct, a few are made in-house, and a few may have been bought in a warehouse sale. In other words, a product is not necessarily supplied by a vendor. Therefore, VENDOR is optional to PRODUCT. A few of the conditions just described were made for the sake of illustrating specific SQL features.

Table name: VENDOR

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
21225	Bryson, Inc.	Smithson	615	223-3234	TN	Y
21226	SuperLoo, Inc.	Flushing	904	215-8995	FL	N
21231	D&E Supply	Singh	615	228-3245	TN	Y
21344	Gomez Bros.	Ortega	615	889-2546	KY	N
22567	Dome Supply	Smith	901	678-1419	GA	N
23119	Randsets Ltd.	Anderson	901	678-3998	GA	Y
24004	Brackman Bros.	Browning	615	228-1410	TN	N
24288	ORDVA, Inc.	Hakford	615	898-1234	TN	Y
25443	B&K, Inc.	Smith	904	227-0093	FL	N
25501	Damal Supplies	Smythe	615	890-3529	TN	N
25595	Rubicon Systems	Orton	904	456-0092	FL	Y

Table name: PRODUCT

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595	25	0
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595	12	0
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225	25	0
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.4	0.05		12	0
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344	25	0
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288	10	0
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00		50	0
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5	119.95	0.10	25595	10	0

Figure 1-2: Vendor and Product tables

Creating the Database

Before you can use a new RDBMS, you must complete two tasks: create the database structure and create the tables that will hold the end-user data. To complete the first task, the RDBMS creates the physical files that will hold the database. When you create a new database, the RDBMS automatically creates the data dictionary tables in which to store the metadata and creates a default database administrator. Creating the physical files that will hold the database means interacting with the operating system and the file systems supported by the operating system. Therefore, creating the database structure is one feature that tends to differ substantially from one RDBMS to another. However, it is relatively easy to create a database structure, regardless of which RDBMS you use.

If you use Microsoft Access, creating the database is simple: start Access and open a new blank database. Specify the folder in which you want to store the database, and then name the database. However, if you work in a database environment typically used by larger organizations, you will probably use an enterprise RDBMS such as Oracle, MS SQL Server, MySQL, or DB2. Given their security requirements and greater complexity, creating a database with these products is a more elaborate process.

With the exception of creating the database, most RDBMS vendors use SQL that deviates little from the ANSI standard SQL. For example, most RDBMSs require each SQL command to end with a semicolon. However, some SQL implementations do not use a semicolon. Important syntax differences among implementations are highlighted in the Note boxes in this chapter.
If you are using an enterprise RDBMS, you must be authenticated by the RDBMS before you can start creating tables. **Authentication** is the process the DBMS uses to verify that only registered users access the database. To be authenticated, you must log on to the RDBMS using a user ID

and a password created by the database administrator. In an enterprise RDBMS, every user ID is associated with a database schema.

The Database Schema

In the SQL environment, a **schema** is a logical group of database objects—such as tables and indexes—that are related to each other. Usually, the schema belongs to a single user or application. A single database can hold multiple schemas that belong to different users or applications. Schemas are useful in that they group tables by owner (or function) and enforce a first level of security by allowing each user to see only the tables that belong to that user. ANSI SQL standards define a command to create a database schema:

CREATE SCHEMA AUTHORIZATION {creator};

Therefore, if the creator is JONES, the following command is used:
CREATE SCHEMA AUTHORIZATION JONES;

Most enterprise RDBMSs support that command. However, the command is seldom used directly—that is, from the command line. (When a user is created, the DBMS automatically assigns a schema to that user.) When the DBMS *is* used, the CREATE SCHEMA AUTHORIZATION command must be issued by the user who owns the schema. That is, if you log on as JONES, you can only use CREATE SCHEMA AUTHORIZATION JONES. For most RDBMSs, the CREATE SCHEMA AUTHORIZATION command is optional, which is why this chapter focuses on the ANSI SQL commands required to create and manipulate tables.

Data Types

As discussed in DBS621B, there is a wide array of data types supported by SQL. Only a few of the most common are covered in this book. Generally speaking, the data types are character, numeric, and date (see Table 1.1). If your RDBMS is fully compliant with ANSI SQL, it supports many more data types than those shown in Table 1.1. Also, many RDBMSs support data types beyond the ones specified in ANSI SQL. In addition to the data types shown in Table 8.1, SQL supports several other data types, including TIME, TIMESTAMP, REAL, DOUBLE, and FLOAT, and intervals, such as INTERVAL DAY TO HOUR. Many RDBMSs have also expanded the list to include other types of data, such as LOGICAL, CURRENCY, and AutoNumber (Access). However, in this chapter the discussion is limited to the data types summarized in Table 1.1.

Character data types include both fixed-length and variable-length character data. A variable-length character data type, like VARCHAR, is typically specified with a maximum length. For example, VARCHAR(10) means variable-length character data up to a maximum of 10 characters. Fixed-length character data is also typically specified with a length and requires all values to be the specified length. For example, CHAR(10) means fixed length character data with all values being 10 characters long. If the value ‘Smith’ is stored in a VARCHAR(10) column, then only five characters of data will be stored. If ‘Smith’ is stored in a CHAR(10) column, then five blank spaces will be added to the end of the value to force it to be the specified 10 characters long so the actual value stored will be ‘Smith’. Numeric data types often vary on the size of the number that can be stored and whether the number can contain a decimal value. Numeric data types that allow decimal places often provide the option to provide values for the overall number of digits possible (L) and another value for the number of those digits that will be to the right of the decimal place (D). Depending on the DBMS used, the overall number of digits possible may

be called the *length* or *precision*, and the number of digits to the right of the decimal may be called the *decimals* or *scale*.

Table 1.1: Some common SQL Data Types

Data Type	Format	Comments
Numeric	NUMERIC(L,D) or NUMBER(L,D)	The declaration NUMERIC(7,2) or NUMBER (7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or –134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMERIC specification, but the storage length is a *minimum* specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable. In most popular DBMSs, this difference is not implemented so NUMERIC and DECIMAL operate the same.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as *Smith* and *Katzenjammer* are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR(25) or VARCHAR2(25) will let you store up to 25 characters. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

If you use DATE fields, you can also determine a future date using a simple command. For example, you can determine the date that is 60 days from a given P_INDATE by using P_INDATE + 60 in most DBMSs. MySQL requires a function for adding dates. For example, the MySQL AddDate() function used in "AddDate(P_INDATE, 60)" determines the date that is 60 days from the P_INDATE. Or, you can use the RDBMS's system date—SYSDATE in Oracle, SYSDATE() or NOW() in MySQL, GETDATE() in MS SQL Server, and Date() in Access—to answer questions such as "What will be the date 60 days from today?" For example, you might use SYSDATE + 60 in Oracle, AddDate (SYSDATE(), 60) in MySQL, GETDATE()+ 60 in MS SQL Server, or Date() + 60 in Access.

Note

Oracle uses DATE data types to store complete dates, that is, a date and time. Access uses Date/Time as the data type to store these types of values. MySQL and MS SQL Server use the DATE data type to store only dates without a time component. Storing a complete date with time component in MySQL or MS SQL Server requires the DATETIME data type.

Date arithmetic capability is particularly useful in billing. Perhaps you want your system to start charging interest on a customer balance 60 days after the invoice is generated. Such simple date arithmetic would be impossible if you used a character data type. Many DBMSs also include support for computed columns that do not take a normal data type definition. Instead, these columns allow the developer to specify a formula or query instead of a data type. The actual implementation varies based on DBMS vendor. Some vendors allow computed columns as virtual columns that are not physically stored in the database, so the values cannot be altered directly by the users, but the virtual column is available for use in SELECT statements. Other vendors allow the computed column to be physically stored so that the values can be computed and evaluated at the time the row is added or modified in the table.

In the data dictionary in Table 1.2, note the data types. Keep in mind that data-type selection is usually dictated by the nature and intended use of the data. For example:

- P_PRICE clearly requires some kind of numeric data type; defining it as a character field is not acceptable.
- Just as clearly, a vendor name is an obvious candidate for a character data type. For example, VARCHAR(35) fits well because vendor names are variable-length character strings, and in this case, such strings may be up to 35 characters long.
- At first glance, it might seem logical to select a numeric data type for V_AREACODE because it contains only digits. However, adding and subtracting area codes does not yield meaningful results. Therefore, selecting a character data type is more appropriate. This is true for many common attributes found in business data models. For example, even though zip codes contain all digits, they must be defined as character data because some zip codes begin with the digit zero (0), and a numeric data type would cause the leading zero to be dropped.
- U.S. state abbreviations are always two characters, so CHAR(2) is a logical choice.
- Selecting P_INDATE to be a (Julian) DATE field rather than a character field is desirable because Julian dates allow you to make simple date comparisons and perform date arithmetic.

For instance, if you have used DATE fields, you can determine the number of days between dates.

Table 2.2: Data Dictionary for the SaleCo Database

Table Name	Attribute Name	Contents	Type	Format	Range	Required	PK OR FK	FK Referenced Table
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPT	Product description	VARCHAR(35)	Xxxxxxxxxxxx	NA	Y		
	P_INDATE	Stocking date	DATE	YYYY-MM-DD	NA	Y		
	P_QOH	Units available	SMALLINT	####	0–9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0–9999	Y		
	P_PRICE	Product price	DECIMAL(8,2)	####.##	0.00–9999.00	Y		
	P_DISCOUNT	Discount rate	DECIMAL(5,2)	0.##	0.00–0.20	Y		
	V_CODE	Vendor code	INTEGER	###	100–999		FK	VENDOR
VENDOR	V_CODE	Vendor code	INTEGER	#####	1000–9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	Xxxxxxxxxxxxxx	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	Xxxxxxxxxxxxxx	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999–9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		

FK = Foreign key
PK = Primary key
CHAR = Fixed-length character data, 1 to 255 characters
VARCHAR = Variable-length character data, 1 to 2,000 characters. VARCHAR is automatically converted to VARCHAR2 in Oracle.
DECIMAL = Numeric data. DECIMAL(9,2) is used to specify numbers that have two decimal places and are up to nine digits long, including the decimal places. Some RDBMSs permit the use of a MONEY or a CURRENCY data type.

NUMERIC = Numeric data. DBMSs that do not support the NUMBER data type typically use NUMERIC instead.
INT = Integer values only. INT is automatically converted to NUMBER in Oracle.
SMALLINT = Small integer values only. SMALLINT is automatically converted to NUMBER in Oracle.
DATE formats vary. Commonly accepted formats are DD-MON-YYYY, DD-MON-YY, MM/DD/YYYY, and MM/DD/YY.

Data-type selection sometimes requires professional judgment. For example, you must make a decision about the V_CODE's data type as follows:

- If you want the computer to generate new vendor codes by adding 1 to the largest recorded vendor code, you must classify V_CODE as a numeric attribute. (You cannot perform mathematical procedures on character data.) The designation INTEGER will ensure that only the counting numbers (integers) can be used. Most SQL implementations also permit the use of SMALLINT for integer values up to six digits.
- If you do not want to perform mathematical procedures based on V_CODE, you should classify it as a character attribute, even though it is composed entirely of numbers. When there is no need to perform mathematical procedures on the attribute, store it as a character attribute.

The first option is used to demonstrate the SQL procedures in this chapter. When you define the attribute's data type, you must pay close attention to the expected use of the attributes for sorting and data-retrieval purposes. For example, in a real estate application, an attribute that represents the number of bathrooms in a home (H_BATH_NUM) could be assigned the CHAR(3) data type because the application will probably not do any addition, multiplication, or division with the number of bathrooms. Based on the CHAR(3) data-type definition, valid H_BATH_NUM values would be '2','1','2.5','10'. However, this data-type decision creates potential problems. For example, if an application sorts the homes by number of bathrooms, a query would "see" the value '10' as less than '2', which is clearly incorrect. So, you must consider the expected use of the data to properly define the attribute data type.

Creating Table Structures

After the database has been created and the appropriate data types for each attribute have been determined, it is time to create the actual database tables. Recall that when implementing the database design, every entity becomes a table, and the attributes of each entity become the columns in that table.

CREATE TABLE command

Now you are ready to implement the PRODUCT and VENDOR table structures with the help of SQL, using the **CREATE TABLE** syntax shown next.

CREATE TABLE *tablename* (
column1 data type [*constraint*] [,
column2 data type [*constraint*]] [,
PRIMARY KEY (*column1* [, *column2*])] [,
FOREIGN KEY (*column1* [, *column2*]) REFERENCES *tablename*] [,
CONSTRAINT *constraint*]);

To make the SQL code more readable, most SQL programmers use one line per column (attribute) definition. In addition, spaces are used to line up the attribute characteristics and constraints. Finally, both table and attribute names are fully capitalized.

```
CREATE TABLE VENDOR (
V_CODE INTEGER NOT NULL UNIQUE,
V_NAME VARCHAR(35) NOT NULL,
V_CONTACT VARCHAR(25) NOT NULL,
V_AREACODE CHAR(3) NOT NULL,
V_PHONE CHAR(8) NOT NULL,
V_STATE CHAR(2) NOT NULL,
V_ORDER CHAR(1) NOT NULL,
PRIMARY KEY (V_CODE));
```

Note

- Because the PRODUCT table contains a foreign key that references the VENDOR table, create the VENDOR table first. (In fact, the "M" side of a relationship always references the "1" side. Therefore, in a 1:M relationship, you must *always* create the table for the "1" side first.)
- If your RDBMS does not support the VARCHAR2 and FCHAR format, use CHAR.
- Oracle accepts the VARCHAR data type and automatically converts it to VARCHAR2.
- If your RDBMS does not support SINT or SMALLINT, use INTEGER or INT. If INTEGER is not supported, use NUMBER (Oracle or Access) or NUMERIC (MS SQL Server or MySQL).
- If you use Access, you can use the NUMBER data type, but you cannot use the number delimiters at the SQL level. For example, using NUMBER(8,2) to indicate numbers with up to eight digits with two digits to the right of the decimal place is fine in Oracle, but you cannot use it in Access—you must use NUMBER without the delimiters.
- If your RDBMS does not support primary and foreign key designations or the UNIQUE specification, delete them from the SQL code shown here.
- If you use the PRIMARY KEY designation in Oracle, you do not need the NOT NULL and UNIQUE specifications.
- The ON UPDATE CASCADE clause is part of the ANSI standard, but it may not be supported by your RDBMS. In that case, delete the ON UPDATE CASCADE clause.

```
CREATE TABLE PRODUCT (
P_CODE VARCHAR(10) NOT NULL UNIQUE,
P_DESCRIPT VARCHAR(35) NOT NULL,
P_INDATE DATE NOT NULL,
P_QOH SMALLINT NOT NULL,
P_MIN SMALLINT NOT NULL,
P_PRICE NUMBER(8,2) NOT NULL,
P_DISCOUNT NUMBER(5,2) NOT NULL,
V_CODE INTEGER,
PRIMARY KEY (P_CODE),
FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE) ON UPDATE CASCADE);
```

As you examine the preceding SQL table-creating command sequences, note the following features:

- The NOT NULL specifications for the attributes ensure that a data entry will be made. When it is crucial to have the data available, the NOT NULL specification will not allow the end user to leave the attribute empty (with no data entry at all). Because this specification is made at the table level and stored in the data dictionary, application programs can use this information to create the data dictionary validation automatically.
- The UNIQUE specification creates a unique index on the respective attribute. Use it to avoid having duplicated values in a column.
- The primary key attributes contain both a NOT NULL and UNIQUE specification, which enforce the entity integrity requirements. If the NOT NULL and UNIQUE specifications are not supported, use PRIMARY KEY without the specifications. (For example, if you designate the PK in MS Access, the NOT NULL and UNIQUE specifications are automatically assumed and are not spelled out.)
- The entire table definition is enclosed in parentheses. A comma is used to separate each table element definition (attributes, primary key, and foreign key).
- The ON UPDATE CASCADE specification ensures that if you make a change in any VENDOR's V_CODE that change is automatically applied to all foreign key references throughout the system to ensure that referential integrity is maintained. (Although the ON UPDATE CASCADE clause is part of the ANSI standard, some RDBMSs, such as Oracle, do not support it. If your RDBMS does not support the clause, delete it from the code shown here.)
- An RDBMS automatically enforces referential integrity for foreign keys. That is, you cannot have an invalid entry in the foreign key column; at the same time, you cannot delete a vendor row as long as a product row references that vendor.
- The command sequence ends with a semicolon. (Remember that your RDBMS may require you to omit the semicolon.)

Note

Note About Column Names

Do *not* use mathematical symbols such as +, −, and / in your column names; instead, use an underscore to separate words, if necessary. For example, PER-NUM might generate an error message, but PER_NUM is acceptable. Also, do *not* use reserved words. **Reserved words** are words used by SQL to perform specific functions. For example, in some RDBMSs, the column name INITIAL will generate the message "invalid column name."

SQL Constraints

In Chapter 3: The Relational Database Model (DSB621B), you learned that adherence to rules for entity integrity and referential integrity is crucial in a relational database environment. Fortunately, most SQL implementations support both integrity rules. Entity integrity is enforced automatically when the primary key is specified in the CREATE TABLE command sequence. For example, you can create the VENDOR table structure and set the stage for the enforcement of entity integrity rules by using the following:

PRIMARY KEY (V_CODE)
In the PRODUCT table's CREATE TABLE sequence, note that referential integrity has

been enforced by specifying the following in the PRODUCT table:
FOREIGN KEY (V_CODE) REFERENCES VENDOR(V_CODE) ON UPDATE CASCADE
The foreign key constraint definition ensures that:

- You cannot delete a vendor from the VENDOR table if at least one product row references that vendor. This is the default behavior for the treatment of foreign keys.
- On the other hand, if a change is made in an existing VENDOR table's V_CODE, that change must be reflected automatically in any PRODUCT table V_CODE reference (ON UPDATE CASCADE). That restriction makes it impossible for a V_CODE value to exist in the PRODUCT table if it points to a nonexistent VENDOR table V_CODE value. In other words, the ON UPDATE CASCADE specification ensures the preservation of referential integrity. (Oracle is a strict adherent to the philosophy that the value of a primary key should never change, so it does not support ON UPDATE CASCADE.)

In general, ANSI SQL permits the use of ON DELETE and ON UPDATE clauses to cover CASCADE, SET NULL, or SET DEFAULT. Besides the PRIMARY KEY and FOREIGN KEY constraints, the ANSI SQL standard also defines the following constraints:

- The NOT NULL constraint ensures that a column does not accept nulls.
- The UNIQUE constraint ensures that all values in a column are unique.
- The DEFAULT constraint assigns a value to an attribute when a new row is added to a table. The end user may, of course, enter a value other than the default value.
- The CHECK constraint is used to validate data when an attribute value is entered. The CHECK constraint does precisely what its name suggests: it checks to see that a specified condition exists. Examples of such constraints include the following:
    - The minimum order value must be at least 10.
    - The date must be after April 15, 2018.
    - If the CHECK constraint is met for the specified attribute (i.e., the condition is true),the data is accepted for that attribute. If the condition is found to be false, an error message is generated and the data is not accepted.
    -

Note that the CREATE TABLE command lets you define constraints in two different places:

- When you create the column definition (known as a *column constraint*)
- When you use the CONSTRAINT keyword (known as a *table constraint*)

A column constraint applies to just one column; a table constraint may apply to many columns. Those constraints are supported at varying levels of compliance by enterprise RDBMSs. In this chapter, Oracle is used to illustrate SQL constraints. For example, note that the following SQL command sequence uses the DEFAULT and CHECK constraints to define the table named CUSTOMER.

```
CREATE TABLE CUSTOMER (
CUS_CODE INTEGER PRIMARY KEY,
CUS_LNAME VARCHAR(15) NOT NULL,
CUS_FNAME VARCHAR(15) NOT NULL,
CUS_INITIAL CHAR(1),
CUS_AREACODE CHAR(3) DEFAULT '615' NOT NULL
CHECK(CUS_AREACODE IN ('615','713','931')),
CUS_PHONE CHAR(8) NOT NULL,
CUS_BALANCE DECIMAL(9,2) DEFAULT 0.00,
CONSTRAINT CUS_UI1 UNIQUE (CUS_LNAME, CUS_FNAME));
```

In this case, the CUS_AREACODE attribute is assigned a default value of '615'. Therefore, if a new CUSTOMER table row is added and the end user makes no entry for the area code, the '615' value will be recorded. Also, the CHECK condition restricts the values for the customer's area code to 615, 713, and 931; any other values will be rejected. The DEFAULT value applies only when new rows are added to a table and then only when no value is entered for the customer's area code. (The default value is not used when an existing row in the table is modified.) In contrast, the CHECK condition is validated whether a customer row is added or *modified*. However, while the CHECK condition may include any valid expression, it applies only to the attributes in the table being checked. If you want to check for conditions that include attributes in other tables, you must use triggers, as discussed later in this chapter.

Finally, the last line of the CREATE TABLE command sequence creates a unique index constraint (named CUS_UI1) on the customer's last name and first name. The index will prevent the entry of two customers with the same last name and first name. (This index merely illustrates the process. Clearly, it should be possible to have more than one person named John Smith in the CUSTOMER table.)

In the following SQL command to create the INVOICE table, the DEFAULT constraint assigns a default date to a new invoice, and the CHECK constraint validates that the invoice date is greater than January 1, 2022.

```
CREATE TABLE INVOICE (
INV_NUMBER INTEGER PRIMARY KEY,
CUS_CODE INTEGER NOT NULL REFERENCES CUSTOMER(CUS_CODE),
INV_DATE DATETIME DEFAULT NOW() NOT NULL,
CONSTRAINT INV_CK1 CHECK (INV_DATE > '2022-01-01'));
```

In this case, notice the following:

- The CUS_CODE attribute definition contains REFERENCES CUSTOMER(CUS_CODE) to indicate that the CUS_CODE is a foreign key. This is another way to define a foreign key.
- The DEFAULT constraint uses the NOW() function. This function always returns today's date.
- The invoice date (INV_DATE) attribute is automatically given today's date (returned by NOW()) when a new row is added if no value is given for the attribute.
- A CHECK constraint is used to validate that the invoice date is greater than "January 1, 2022."

The final SQL command sequence creates the LINE table. The LINE table has a composite primary key (INV_NUMBER, LINE_NUMBER) and uses a UNIQUE constraint in INV_NUMBER and P_CODE to ensure that the same product is not ordered twice in the same invoice.

```
CREATE TABLE LINE (
INV_NUMBER INTEGER NOT NULL,
LINE_NUMBER DECIMAL(2,0) NOT NULL,
P_CODE VARCHAR(10) NOT NULL,
LINE_UNITS DECIMAL(9,2) DEFAULT 0.00 NOT NULL,
LINE_PRICE DECIMAL(9,2) DEFAULT 0.00 NOT NULL,
PRIMARY KEY (INV_NUMBER, LINE_NUMBER),
FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE (INV_NUMBER) ON
DELETE CASCADE,
FOREIGN KEY (P_CODE) REFERENCES PRODUCT(P_CODE),
CONSTRAINT LINE_UI1 UNIQUE(INV_NUMBER, P_CODE));
```

In the creation of the LINE table, note that a UNIQUE constraint is added to prevent the duplication of an invoice line. A UNIQUE constraint is enforced through the creation of a unique index. Also note that the ON DELETE CASCADE foreign key enforces referential integrity. The use of ON DELETE CASCADE is recommended for weak entities to ensure that the deletion of a row in the strong entity automatically triggers the deletion of the corresponding rows in the dependent weak entity. In that case, the deletion of an INVOICE row will automatically delete all of the LINE rows related to the invoice. In the following section, you will learn more about indexes and how to use SQL commands to create them.

Creating a Table with a SELECT Statement

SQL provides a way to rapidly create a new table based on selected columns and rows of an existing table using a subquery. Recall that a subquery, also known as a nested query or an inner query, is a query that is embedded (or nested) inside another query. The inner query is always executed first by the RDBMS. In this case, the new table copies the attribute names, data characteristics, and rows of the original table as retrieved by the subquery. The MySQL version of the command is:

```
CREATE TABLE [IF NOT EXISTS] PART AS
SELECT P_CODE AS PART_CODE, P_DESCRIPT AS PART_DESCRIPT, P_PRICE
AS PART_PRICE, V_CODE
FROM PRODUCT;
```

If the PART table already exists, MySQL will not let you overwrite the existing table. If you include the optional IF NOT EXISTS clause to the command, the table is not overwritten, and no error message is generated. If you omit the optional IF NOT EXISTS clause, the table is not overwritten, and an error message is generated. In either case, to run this command if the table already exists, you must first delete the existing PART table. The SQL command just shown creates a new PART table with PART_CODE, PART_DESCRIPT, PART_PRICE, and V_CODE columns. In the previous SQL statement, the CREATE TABLE portion represents the outer query, and the SELECT portion represents the subquery. You can nest queries (place queries inside queries) many levels deep. In every case, the output of the inner query is used as the input for the outer (higher-level) query. The MS Access and SQL Server version of this command is:

```
SELECT P_CODE AS PART_CODE, P_DESCRIPT AS PART_DESCRIPT,
P_PRICE AS PART_PRICE, V_CODE INTO PART
FROM PRODUCT;
```

If the PART table already exists, MS Access asks if you want to delete the existing table and continue with the creation of the new PART table. SQL Server will not offer to delete the existing table for you automatically.

In addition to creating the table structure, all of the data rows returned by the SELECT statement are copied automatically. *However, note that no entity integrity (primary key) or referential integrity (foreign key) rules are automatically applied to the new table*. You will learn how to define the PK to enforce entity integrity and the FK to enforce referential integrity in a later section when you examine how to change the characteristics of a table.

SQL Indexes

Indexes can be used to improve the efficiency of searches and to avoid duplicate column values. In the previous section, you saw how to declare unique indexes on selected attributes when the table is created. In fact, when you declare a primary key, the DBMS automatically creates a unique index. Even with this feature, you often need additional indexes. The ability to create indexes quickly and efficiently is important. Using the **CREATE INDEX** command, SQL indexes can be created on the basis of any selected attribute.
The syntax is:

CREATE [UNIQUE]INDEX *indexname* ON *tablename*(*column1* [, *column2*])

For example, based on the attribute P_INDATE stored in the PRODUCT table, the following command creates an index named P_INDATEX:

CREATE INDEX P_INDATEX ON PRODUCT(P_INDATE);

SQL does not let you write over an existing index without warning you first, thus preserving the index structure within the data dictionary. Using the UNIQUE index qualifier, you can even create an index that prevents you from using a value that has been used before. Such a feature is especially useful when the index attribute is a candidate key whose values must not be duplicated:

CREATE UNIQUE INDEX P_CODEX ON PRODUCT(P_CODE);

If you now try to enter a duplicate P_CODE value, SQL produces the error message “duplicate value in index.” Many RDBMSs, including Access, automatically create a unique index on the PK attribute(s) when you declare the PK. A common practice is to create an index on any field that is used as a search key, in comparison operations in a conditional expression, or when you want to list rows in a specific order. For example, if you want to create a report of all products by vendor, it would be useful to create an index on the V_CODE attribute in the PRODUCT table. Remember that a vendor can supply many products. Therefore, you should not create a UNIQUE index in this case. Better yet, to make the search as efficient as possible, using a composite index is recommended. Unique composite indexes are often used to prevent data duplication. For example, consider the case illustrated in Table 1.3, in which required employee test scores are stored. (An employee can take a test only once on a given date.) Given the structure of Table 1.3, the PK is EMP_NUM + TEST_NUM. The third test entry for employee 111 meets entity integrity requirements—the combination 111,3 is unique—yet the WEA test entry is clearly duplicated.

Table 3.3: A duplicated test record

Emp_num	Test_num	Test_code	Test_date	Test_score
110	1	WEA	2022-01-15	93
110	2	WEA	2022-01-12	87
111	1	HAZ	2021-12-14	91
111	2	WEA	2022-02-18	95
111	3	WEA	2022-02-18	95
112	1	CHEM	2021-08-17	91

Such duplication could have been avoided through the use of a unique composite index, using the attributes EMP_NUM, TEST_CODE, and TEST_DATE:

CREATE UNIQUE INDEX EMP_TESTDEX ON TEST(EMP_NUM, TEST_CODE, TEST_DATE);

By default, all indexes produce results that are listed in ascending order, but you can create an index that yields output in descending order. For example, if you routinely print a report that lists all products ordered by price from highest to lowest, you could create an index named PROD_PRICEX by typing:

CREATE INDEX PROD_PRICEX ON PRODUCT(P_PRICE DESC);

To delete an index, use the **DROP INDEX** command:

DROP INDEX *indexname*

While the ANSI standard for SQL only requires the index name, some DBMSs such as MySQL require specifying the table name as well. For example, if you want to eliminate the PROD_PRICEX index in MySQL, type:

DROP INDEX PROD_PRICEX ON PRODUCT;

After creating the tables and some indexes, you are ready to start entering data. The following sections use two tables (VENDOR and PRODUCT) to demonstrate most of the data manipulation commands.

Altering Table Structures

In this section, you will learn how to change table structures by changing attribute characteristics and by adding columns. Finally, you will learn how to copy tables or parts of tables and how to delete tables. All changes in the table structure are made by using the **ALTER TABLE** command followed by a keyword that produces the specific change you want to make. Three options are available: ADD, MODIFY, and DROP. You use ADD to add a column, MODIFY to change column characteristics, and DROP to delete a column from a table. Most RDBMSs do not allow you to delete a column unless the column does not contain any values; otherwise, such an action might delete crucial data used by other tables. The basic syntax to add or modify columns is:

ALTER TABLE *tablename*
{ADD | MODIFY | ALTER} [COLUMN] (*columnname datatype* [{ADD | MODIFY |

ALTER} *columnname datatype*]);

The ALTER TABLE command can also be used to add table constraints. In those cases, the syntax would be:
ALTER TABLE *tablename*
ADD *constraint* [ADD *constraint*];

You could also use the ALTER TABLE command to remove a column or table constraint. The syntax would be as follows:

ALTER TABLE *tablename*
DROP {PRIMARY KEY | COLUMN *columnname* | CONSTRAINT *constraintname* };

Notice that when removing a constraint, you need to specify it by name, which is one reason you should always name constraints in your CREATE TABLE or ALTER TABLE statement.

Changing a Column's Data Type

Using the ALTER syntax, the integer V_CODE in the PRODUCT table can be changed to a character V_CODE by using the following command:

ALTER TABLE PRODUCT
MODIFY V_CODE CHAR(5);

Some RDBMSs, such as Oracle, do not let you change data types unless the column to be changed is empty. For example, if you want to change the V_CODE field from the current number definition to a character definition, the preceding command will yield an error message because the V_CODE column already contains data. The error message is easily explained. Remember that the V_CODE in PRODUCT references the V_CODE in VENDOR. If you change the V_CODE data type, the data types do not match, and there is a referential integrity violation, which triggers the error message. If the V_CODE column does not contain data, the preceding command sequence will alter the table structure as expected (if the foreign key reference was not specified during the creation of the PRODUCT table).

Changing a Column's Data Characteristics

If the column to be changed already contains data, you can make changes in the column's characteristics if those changes do not alter the data type. For example, if you want to increase the width of the P_PRICE column to nine digits, use the following command:
ALTER TABLE PRODUCT
MODIFY P_PRICE DECIMAL(9,2);

If you now list the table contents, you can see that the column width of P_PRICE has increased by one digit.

Note

Some DBMSs impose limitations on when it is possible to change attribute characteristics. For example, Oracle lets you increase (but not decrease) the size of a column because an attribute modification will affect the integrity of the data in the database. In fact, some attribute changes can be made only when there is no data in any rows for the affected attribute.

Adding a Column

You can alter an existing table by adding one or more columns. In the following example, you add the column named P_SALECODE to the PRODUCT table. (This column will be used later to determine whether goods that have been in inventory for a certain length of time should be placed on special sale.) Suppose that you expect the P_SALECODE entries to be 1, 2, or 3. Because no arithmetic will be performed with the P_SALECODE, the P_SALECODE will be classified as a single-character attribute. Note the inclusion of all required information in the following ALTER command:

```
ALTER TABLE PRODUCT
ADD (P_SALECODE CHAR(1));
```

When adding a column, be careful not to include the NOT NULL clause for the new column. Doing so will cause an error message; if you add a new column to a table that already has rows, the existing rows will default to a value of null for the new column. Therefore, it is not possible to add the NOT NULL clause for this new column. (Of course, you can add the NOT NULL clause to the table structure after all the data for the new column has been entered and the column no longer contains nulls.)

Adding Primary Key, Foreign Key, and Check Constraints

When you create a new table based on another table, the new table does not include integrity rules from the old table. In particular, there is no primary key. To define the primary key for the new PART table, use the following command:

```
ALTER TABLE PART
ADD PRIMARY KEY (PART_CODE);
```

Several other scenarios could leave you without entity and referential integrity. For example, you might have forgotten to define the primary and foreign keys when you created the original tables. Or, if you imported tables from a different database, you might have discovered that the importing procedure did not transfer the integrity rules. In any case, you can re-establish the integrity rules by using the ALTER command. For example, if the PART table's foreign key has not yet been designated, it can be designated by:

```
ALTER TABLE PART
ADD FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE);
```

Just as primary key and foreign key constraints can be added after the table structure is created, check constraints can also be applied to the table. For example, if there should be a constraint that the part price cannot be a negative value (i.e., it should be greater than or equal to 0), then a check constraint can be designated by:

```
ALTER TABLE PART
ADD CHECK (PART_PRICE >5 0);
```

Alternatively, if the PART table's primary key, foreign key, or check constraint has not been designated, you can incorporate all three changes at once:

```
ALTER TABLE PART
ADD PRIMARY KEY (PART_CODE),
ADD FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE),
ADD CHECK (PART_PRICE >5 0);
```

Even composite primary keys and multiple foreign keys can be designated in a single SQL command. For example, if you want to enforce the integrity rules for the LINE table shown in Figure 8.1, you can use:

```
ALTER TABLE LINE
ADD PRIMARY KEY (INV_NUMBER, LINE_NUMBER)
ADD FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE
(INV_NUMBER)
ADD FOREIGN KEY (P_CODE) REFERENCES PRODUCT (P_CODE);
```

Dropping a Column

Occasionally, you might want to modify a table by deleting a column. Suppose that you want to delete the V_ORDER attribute from the VENDOR table. You would use the following command:

```
ALTER TABLE VENDOR
DROP COLUMN V_ORDER;
```

Again, some RDBMSs impose restrictions on attribute deletion. For example, you may not drop attributes that are involved in foreign key relationships, nor may you delete an attribute if it is the only one in a table.

Deleting a Table from the Database

A table can be deleted from the database using the **DROP TABLE** command. For example, you can delete the PART table you just created with the following command:

DROP TABLE PART;

You can drop a table only if it is not the “one” side of any relationship. If you try to drop a table, otherwise, the RDBMS will generate an error message indicating that a foreign key integrity violation has occurred. Therefore, the order in which multiple tables must be dropped is influenced by the foreign key constraints. You should drop tables from the “many” side first, and then drop the table on the “one” side.

Data Manipulation Commands

In this section, you will learn how to use the basic SQL data manipulation commands INSERT, UPDATE, and DELETE, and the transaction management commands COMMIT and ROLLBACK.

Adding Table Rows

SQL requires the use of the **INSERT** command to enter data into a table. The INSERT command’s basic syntax looks like this:
INSERT INTO *tablename* VALUES (*value1*, *value2*, ..., *valuen*)

Because the PRODUCT table uses its V_CODE to reference the VENDOR table’s V_CODE, an integrity violation will occur if the VENDOR table V_CODE values do not yet exist. Therefore, you need to enter the VENDOR rows before the PRODUCT rows. Given the VENDOR table structure defined earlier and the sample VENDOR data shown in Figure 8.2, you would enter the first two data rows as follows:

```
INSERT INTO VENDOR
VALUES (21225, 'Bryson, Inc.', 'Smithson', '615', '223-3234', 'TN', 'Y');
INSERT INTO VENDOR
VALUES (21226, 'Superloo, Inc.', 'Flushing', '904', '215-8995', 'FL', 'N');
```

and so on, until all of the VENDOR table records have been entered. (To see the contents of the VENDOR table, use the SELECT * FROM VENDOR; command.) The PRODUCT table rows would be entered in the same fashion, using the PRODUCT data shown in Figure 8.2. For example, the first two data rows would be entered as follows, pressing Enter at the end of each line:

```
INSERT INTO PRODUCT
VALUES ('11QER/31', 'Power painter, 15 psi., 3-nozzle', '2021-11-03', 8, 5, 109.99,
0.00, 25595);
INSERT INTO PRODUCT
VALUES ('13-Q2/P2', '7.25-in. pwr. saw blade', '2021-12-13', 32,15, 14.99, 0.05, 21344);
```

(To see the contents of the PRODUCT table, use the SELECT * FROM PRODUCT; command.) The previous commands inserted rows into the VENDOR table and then into the PRODUCT table. It is significant that the rows are entered in this order. Just as when creating tables or dropping tables, the order of the operations matters due to foreign key constraints. You must enter data in the table on the “one” side of a one-to-many relationship before entering the related data in the table on the “many” side.

In the preceding data-entry lines, observe that:

- The row contents are entered between parentheses. Note that the first character after VALUES is a parenthesis and that the last character in the command sequence is also a parenthesis.
- Character (string) and date values must be entered between apostrophes (').
- Numerical entries are *not* enclosed in apostrophes.
- Attribute entries are separated by commas.
- A value is required for each column in the table.

This version of the INSERT command adds one table row at a time.

Inserting Rows with Null Attributes Thus far, you have entered rows in which all of the attribute values are specified. But what do you do if a product does not have a vendor or if you do not yet know the vendor code? In those cases, you would want to leave the vendor code null. To enter a null, use the following syntax:

```
INSERT INTO PRODUCT
VALUES ('BRT-345', 'Titanium drill bit', '18-Oct-21', 75, 10, 4.50, 0.06, NULL);
```

Incidentally, note that the NULL entry is accepted only because the V_CODE attribute is optional—the NOT NULL declaration was not used in the CREATE TABLE statement for this attribute.

Inserting Rows with Optional Attributes Sometimes, more than one attribute is optional. Rather than declaring each attribute as NULL in the INSERT command, you can indicate just the attributes that have required values. You do that by listing the attribute names inside parentheses after the table name. For the purpose of this example, assume that the only required attributes for the PRODUCT table are P_CODE and P_DESCRIPT:

```
INSERT INTO PRODUCT(P_CODE, P_DESCRIPT) VALUES ('BRT-345', 'Titanium drill bit');
```

Inserting Table Rows with a SELECT Subquery

As you will discover in later chapters on database design, sometimes it is necessary to break up a table structure into several component parts (or smaller tables). Fortunately, SQL allows you to copy the contents of selected table columns so that the data need not be re-entered manually into the newly created table(s). Previously, you saw that a subquery can be used with the CREATE TABLE command to copy parts of a table, including both the structure and the data. If the table structure that you want to copy the data into already exists, then the CREATE TABLE command is not appropriate. It is possible to use a similar technique to copy only the data from one table into another already existing table. For example, assume you already have the PART table created with the structure produced by the following command:

```
CREATE TABLE PART(
PART_CODE CHAR(8),
PART_DESCRIPT CHAR(35),
```

PART_PRICE DECIMAL(8,2),
V_CODE INTEGER,
PRIMARY KEY (PART_CODE));

Now you want to copy data from the P_CODE, P_DESCRIPT, P_PRICE, and V_CODE columns of the PRODUCT table into the PART table. Note that the PART column names need not be identical to those of the original table and that the new table need not have the same number of columns as the original table. In this case, the first column in the PART table is PART_CODE, rather than the original P_CODE in the PRODUCT table. Also, the PART table contains only four columns rather than the eight columns in the PRODUCT table. However, column characteristics must match; you cannot copy a character-based attribute into a numeric structure, and vice versa. You need to add the rows to the PART table, using the PRODUCT table rows. So far, you have added rows one at a time. Using a subquery with the INSERT command, it is possible to add multiple rows to a table, using another table as the source of the data, at the same time. The syntax is:

INSERT INTO *target_tablename*[(*target_columnlist*)]
SELECT *source_columnlist*
FROM *source_tablename*;
Note that the target column list is required if the source column list does not match all of the attribute names and characteristics of the target table (including the order of the columns). Otherwise, you do not need to specify the target column list. In this example, you must specify the target column list in the following INSERT command because the column names of the target table are different:

INSERT INTO PART (PART_CODE, PART_DESCRIPT, PART_PRICE, V_CODE)
SELECT P_CODE, P_DESCRIPT, P_PRICE, V_CODE
FROM PRODUCT;

The results are shown in Figure 1.3:

PART_CODE	PART_DESCRIPT	PART_PRICE	V_CODE
11QER/31	Power painter, 15 psi., 3-nozzle	109.99	25595
13-Q2/P2	7.25-in. pwr. saw blade	14.99	21344
14-Q1/L3	9.00-in. pwr. saw blade	17.49	21344
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	23119
1558-QVI	Hrd. cloth, 1/2-in., 3x50	43.99	23119
2232/QTY	B&D jigsaw, 12-in. blade	109.92	24288
2232/QWE	B&D jigsaw, 8-in. blade	99.87	24288
2238/QPD	B&D cordless drill, 1/2-in.	38.95	25595
23109-HB	Claw hammer	9.95	21225
23114-AA	Sledge hammer, 12 lb.	14.4	
54778-2T	Rat-tail file, 1/8-in. fine	4.99	21344
89-JWRE-Q	Hicut chain saw, 16 in.	256.99	24288
PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	
SM-18277	1.25-in. metal screw, 25	6.99	21225
SW-23116	2.5-in. wd. screw, 50	8.45	21231
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	119.95	25595

Figure 1-3: Part Table Data Copied from the Product Table

Given the previous SQL statement, the INSERT portion represents the outer query, and the SELECT portion represents the subquery. Just as when a subquery was used with the CREATE TABLE command, the output of the inner query is used as the input for the outer query. The values

returned by the SELECT subquery should match the attributes and data types of the table in the INSERT statement. If the table into which you are inserting rows has one date attribute, one number attribute, and one character attribute, the SELECT subquery should return one or more rows in which the first column has date values, the second column has number values, and the third column has character values.

Saving Table Changes

Any changes made to the table contents are not saved on disk until you close the database, close the program you are using, or use the **COMMIT** command. If the database is open and a power outage or some other interruption occurs before you issue the COMMIT command, your changes will be lost and only the original table contents will be retained. The syntax for the COMMIT command is:

COMMIT [WORK | TRANSACTION]

The COMMIT command permanently saves all changes—such as rows added, attributes modified, and rows deleted—made to any table in the database. Therefore, if you intend to make your changes to the PRODUCT table permanent, it is a good idea to save those changes by using the following command:

COMMIT;

Note

Note to MS Access and MySQL Users

MS Access does not support the COMMIT command because it automatically saves changes after the execution of each SQL command. By default, MySQL also automatically commits changes with each command. However, if START TRANSACTION or BEGIN is placed at the beginning of a series of commands, MySQL will delay committing the commands until the COMMIT or ROLLBACK command is issued.

However, the COMMIT command's purpose is not just to save changes. In fact, the ultimate purpose of the COMMIT and ROLLBACK commands is to ensure database update integrity in transaction management.

Updating Table Rows

Use the **UPDATE** command to modify data in a table. The syntax for this command is as follows:

UPDATE *tablename*
SET *columnname* 5 *expression* [, *columnname* 5 *expression*]
[WHERE *conditionlist*];

For example, if you want to change P_INDATE from December 13, 2021, to January 18, 2022, in the second row of the PRODUCT table (see Figure 8.2), use the primary key (13-Q2/ P2) to locate the correct row. Therefore, type:

```
UPDATE PRODUCT
SET P_INDATE 5 '2022-01-18'
WHERE P_CODE 5 '13-Q2/P2';
```

If more than one attribute is to be updated in the row, separate the corrections with commas:

```
UPDATE PRODUCT
SET P_INDATE 5 '2022-01-18', P_PRICE 5 17.99, P_MIN 5 10
WHERE P_CODE 5 '13-Q2/P2';
```

What would have happened if the previous UPDATE command had not included the WHERE condition? The P_INDATE, P_PRICE, and P_MIN values would have been changed in *all* rows of the PRODUCT table. Remember, the UPDATE command is a set-oriented operator. Therefore, if you do not specify a WHERE condition, the UPDATE command will apply the changes to *all* rows in the specified table. Confirm the correction(s) by using the following SELECT command to check the PRODUCT table's listing:

```
SELECT * FROM PRODUCT;
```

To make changes to data in the columns of existing rows, use the UPDATE command. Do not confuse the INSERT and UPDATE commands: INSERT creates new rows in the table, while UPDATE changes rows that already exist. For example, to enter the P_SALECODE value '2' in the fourth row, use the UPDATE command together with the primary key P_CODE of 1546-QQ2. Enter the value by using the following command sequence:

```
UPDATE PRODUCT
SET P_SALECODE 5 '2'
WHERE P_CODE 5 '1546-QQ2';
```

Subsequent data can be entered the same way, defining each entry location by its primary key (P_CODE) and its column location (P_SALECODE). For example, if you want to enter the P_SALECODE value '1' for the P_CODE values '2232/QWE' and '2232/QTY', you use:

```
UPDATE PRODUCT
SET P_SALECODE 5 '1'
WHERE P_CODE IN ('2232/QWE', '2232/QTY');
```

If your RDBMS does not support IN, use the following command:

```
UPDATE PRODUCT
SET P_SALECODE 5 '1'
WHERE P_CODE 5 '2232/QWE' OR P_CODE 5 '2232/QTY';
```

You can check the results of your efforts by using the following commands:

```
SELECT P_CODE, P_DESCRIPT, P_INDATE, P_PRICE, P_SALECODE
FROM PRODUCT;
```

Although the UPDATE sequences just shown allow you to enter values into specified table cells, the process is very cumbersome. Fortunately, if a relationship can be established between the entries and the existing columns, the relationship can be used to assign values to their appropriate slots. For example, suppose that you want to place sales codes into the table based on the P_INDATE using the following schedule:

P_INDATE	P_SALECODE
before December 25, 2021	2
between January 16, 2022 and February 10, 2022	1

Using the PRODUCT table, the following two command sequences make the appropriate assignments:

```
UPDATE PRODUCT
SET P_SALECODE = '2'
WHERE P_INDATE < '2021-12-25';
UPDATE PRODUCT
SET P_SALECODE = '1'
WHERE P_INDATE >= '2022-01-16' AND P_INDATE <='2022-02-10';
```

To check the results of those two command sequences, use:

```
SELECT P_CODE, P_DESCRIPT, P_INDATE, P_PRICE, P_SALECODE
FROM PRODUCT;
```

If you have made *all* of the updates shown in this section using Oracle, your PRODUCT table should look like Figure 1.4. *Make sure that you issue a COMMIT statement to save these changes*.

P_CODE	P_DESCRIPT	P_INDATE	P_PRICE	P_SALECODE
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	109.99	2
13-Q2/P2	7.25-in. pwr. saw blade	18-Jan-22	17.99	1
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	17.49	2
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	39.95	2
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	43.99	
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	109.92	1
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	99.87	2
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	38.95	1
23109-HB	Claw hammer	20-Jan-22	9.95	1
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	14.4	
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	4.99	2
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	256.99	1
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	5.87	
SM-18277	1.25-in. metal screw, 25	01-Mar-22	6.99	
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	8.45	
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	119.95	1

Figure 1-4: Cumulative Effect of the Multiple Updates

The arithmetic operators are particularly useful in data updates. For example, if the quantity on hand in your PRODUCT table has dropped below the minimum desirable value, you will order more of the product. Suppose, for example, that you have ordered 20 units of product 2232/QWE. When the 20 units arrive, you will want to add them to inventory using the following commands:

```
UPDATE PRODUCT
SET P_QOH = P_QOH + 20
WHERE P_CODE = '2232/QWE';
```

If you want to add 10 percent to the price for all products that have current prices below $50, you can use:

```
UPDATE PRODUCT
SET P_PRICE = P_PRICE * 1.10
WHERE P_PRICE < 50.00;
```

Deleting Table Rows

It is easy to delete a table row using the **DELETE** statement. The syntax is:

DELETE FROM *tablename*
[WHERE *conditionlist*];

For example, if you want to delete the product you added earlier whose code (P_CODE) is BRT-345, use the following command:

```
DELETE FROM PRODUCT
WHERE P_CODE 5 'BRT-345';
```

In this example, the primary key value lets SQL find the exact record to be deleted from the PRODUCT table. However, deletions are not limited to a primary key match; any attribute may be used. For example, in your PRODUCT table, you will see several products for which the P_MIN attribute is equal to 5. Use the following command to delete all rows from the PRODUCT table for which the P_MIN is equal to 5:

```
DELETE FROM PRODUCT
WHERE P_MIN = 5;
```

Check the PRODUCT table's contents again to verify that all products with P_MIN equal to 5 have been deleted. Finally, remember that DELETE is a set-oriented command and that the WHERE condition is optional. Therefore, if you do not specify a WHERE condition, *all* rows from the specified table will be deleted!

Restoring Table Contents

If you have not yet used the COMMIT command to store the changes permanently in the database, you can restore the database to its previous condition with the **ROLLBACK** command. ROLLBACK undoes any changes since the last COMMIT command and brings all of the data back

to the values that existed before the changes were made. To restore the data to its "pre-change" condition, type:

ROLLBACK;

and then press Enter. Use the SELECT statement again to verify that the ROLLBACK restored the data to its original values.

Note

MS Access does not support the ROLLBACK command.

COMMIT and ROLLBACK work only with data manipulation commands that add, modify, or delete table rows. For example, assume that you perform these actions:

1. CREATE a table called SALES.
2. INSERT 10 rows in the SALES table.
3. UPDATE two rows in the SALES table.
4. Execute the ROLLBACK command.

Will the SALES table be removed by the ROLLBACK command? No, the ROLLBACK command will undo *only* the results of the INSERT and UPDATE commands. All data definition commands (CREATE TABLE) are automatically committed to the data dictionary and cannot be rolled back. Some RDBMSs, such as Oracle, automatically COMMIT data changes when issuing data definition commands. For example, if you had used the CREATE INDEX command after updating the two rows in the previous example, all previous changes would have been committed automatically; doing a ROLLBACK afterward would not have undone anything. *Check your RDBMS manual to understand these subtle differences*.

Virtual Tables: Creating a View

As you learned earlier, the output of a relational statement such as SELECT is another relation (or table). Suppose that at the end of each day, you would like to have a list of all products to reorder—that is, products with a quantity on hand that is less than or equal to the minimum quantity. Instead of typing the same query at the end of each day, wouldn't it be better to permanently save that query in the database? That is the function of a relational view. A **view** is a virtual table based on a SELECT query. The query can contain columns, computed columns, aliases, and aggregate functions from one or more tables. The tables on which the view is based are called **base tables**.

You can create a view by using the **CREATE VIEW** command:

CREATE VIEW *viewname* [*(column list)*] AS SELECT *query*

The CREATE VIEW statement is a data definition command that stores the subquery specification— the SELECT statement used to generate the virtual table—in the data dictionary.

The first SQL command set in Figure 1.5 shows the syntax used to create a view named PRICEGT50. This view contains only the designated three attributes (P_DESCRIPT, P_QOH, and P_PRICE) and only rows in which the price is over $50. The second SQL command sequence in Figure 1.5 shows the rows that make up the view.

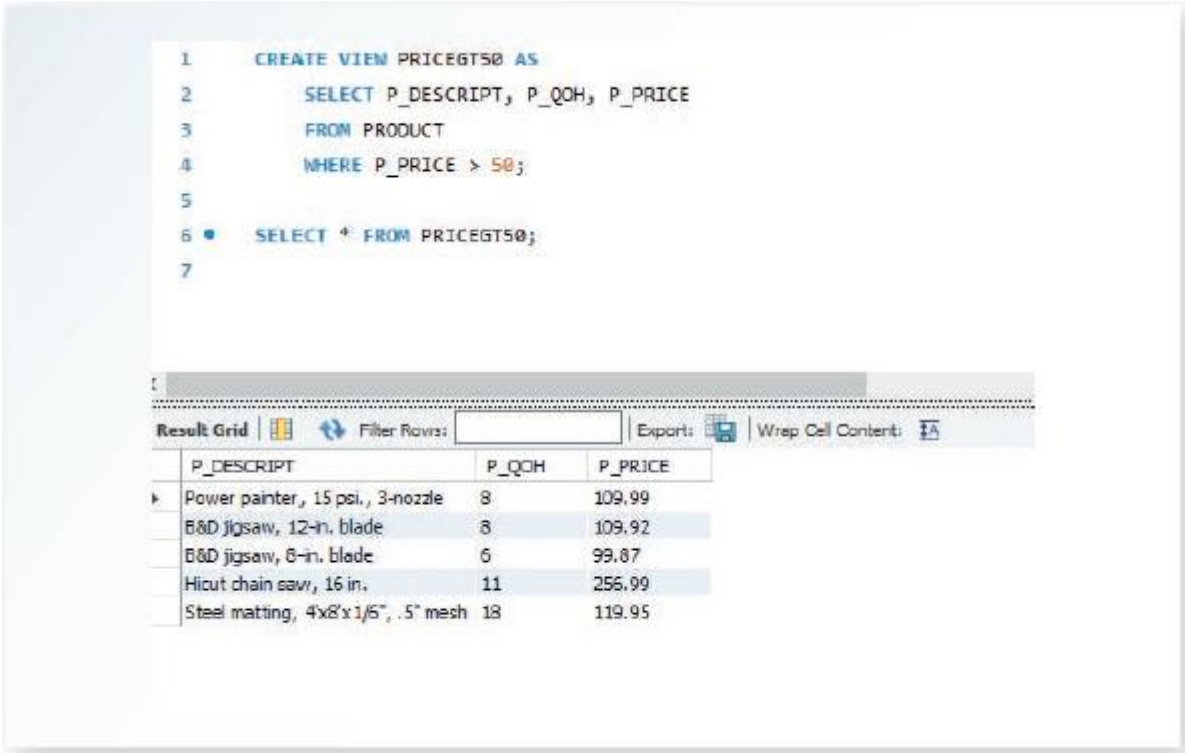


Figure 1-5: Creating a virtual table with the Create View command

A relational view has several special characteristics:

- You can use the name of a view anywhere a table name is expected in a SQL statement.
- Views are dynamically updated. That is, the view is re-created on demand each time it is invoked. Therefore, if new products are added or deleted to meet the criterion P_PRICE > 50.00, those new products will automatically appear or disappear in the PRICEGT50 view the next time the view is invoked.
- Views provide a level of security in the database because they can restrict users to seeing only specified columns and rows in a table. For example, if you have a company with hundreds of employees in several departments, you could give each department administrative assistant a view of certain attributes only for the employees who belong to that assistant's department.
- Views may also be used as the basis for reports. For example, if you need a report that shows a summary of total product cost and quantity-on-hand statistics grouped by vendor, you could create a PROD_STATS view as:

  CREATE VIEW PROD_STATS AS
  SELECT V_CODE, SUM(P_QOH*P_PRICE) AS TOTCOST, MAX(P_QOH)
  AS MAXQTY, MIN(P_QOH) AS MINQTY, AVG(P_QOH) AS AVGQTY
  FROM PRODUCT
  GROUP BY V_CODE;

Updatable Views

One of the most common operations in production database environments is to use batch update routines to update a master table attribute (field) with transaction data. As the name implies, a **batch update routine** pools multiple transactions into a single batch to update a master table field in a *single operation*. For example, a batch update routine is commonly used to update a product's quantity on hand based on summary sales transactions. Such routines are typically run as overnight batch jobs to update the quantity on hand of products in inventory. For example, the sales transactions performed by traveling salespeople can be entered during periods when the system is offline. To perform a batch update routine, begin by defining the master product table (PRODMASTER) and the product monthly sales totals table (PRODSALES) shown in Figure 1.6. Note the 1:1 relationship between the two tables.

Table name: PRODMASTER

PROD_ID	PROD_DESC	PROD_QOH
A123	SCREWS	67
BX34	NUTS	37
C583	BOLTS	50

Table name: PRODSALES

PROD_ID	PS_QTY
A123	7
BX34	3

Figure 1-6: The PRODMASTER and PRODSALES Tables

Using the tables in Figure 1.6, update the PRODMASTER table by subtracting the PRODSALES table's product monthly sales quantity (PS_QTY) from the PRODMASTER table's PROD_QOH. To produce the required update, the update query would be written like this:

```
UPDATE PRODMASTER, PRODSALES
SET PRODMASTER.PROD_QOH 5 PROD_QOH – PS_QTY
WHERE PRODMASTER.PROD_ID 5 PRODSALES.PROD_ID;
```

Note that the update statement reflects the following sequence of events:

- Join the PRODMASTER and PRODSALES tables.
- Update the PROD_QOH attribute (using the PS_QTY value in the PRODSALES table) for each row of the PRODMASTER table with matching PROD_ID values in the PRODSALES table.

To be used in a batch update, the PRODSALES data must be stored in a base table rather than in a view. The query works in MySQL and Access, but Oracle returns an error message that the keyword SET is missing.

Note

Updating using multiple tables in MS SQL Server requires the UPDATE FROM syntax. The above code would be written in MS SQL Server as the following:

```
UPDATE PRODMASTER
SET PROD_QOH = PROD_QOH – PS_QTY
FROM PRODMASTER JOIN PRODSALES ON PRODMASTER.PROD_ID = PRODSALES.
PROD_ID;
```

Oracle produced the error message because it expected to find a single table name in the UPDATE statement. In fact, you cannot join tables in the UPDATE statement in Oracle. To solve that problem, one solution is to create an *updatable* view. As its name suggests, an **updatable view** can be used to update attributes in any base table(s) used in the view. You must realize that *not all views are updatable*. Several restrictions govern updatable views, and some of them are vendor-specific.

The most common updatable view restrictions are as follows:

- GROUP BY expressions or aggregate functions cannot be used.
- You cannot use set operators such as UNION, INTERSECT, and MINUS.
- Most restrictions are based on the use of JOINs or group operators in views. More specifically, the base table to be updated must be key-preserved, meaning that the values of the primary key of the base table must still be unique by definition in the view.

An updatable view named PSVUPD has been created, as shown:

```
CREATE VIEW PSVUPD AS
(SELECT PRODMASTER.PROD_ID, PROD_QOH, PS_QTY
FROM PRODMASTER JOIN PRODSALES
ON PRODMASTER.PROD_ID = PRODSALES.PROD_ID);
```

One easy way to determine whether a view can be used to update a base table is to examine the view's output. If the primary key columns of the base table you want to update still have unique values in the view, the base table is updatable. For example, if the PROD_ID column of the view returns the A123 or BX34 values more than once, the PRODMASTER table cannot be updated through the view. After creating the updatable view, you can use the UPDATE command to update the view, thereby updating the PRODMASTER table, with the command:

```
UPDATE PSVUPD
SET PROD_QOH = PROD_QOH - PS_QTY;
```

Although the batch update procedure just illustrated meets the goal of updating a master table with data from a transaction table, the preferred real-world solution to the update problem is to use procedural SQL, which you will learn about later in this chapter.

Auto Increment, Identity, and Sequences

If you use MS Access, you might be familiar with the AutoNumber data type, which you can use to define a column in your table that is automatically populated with unique numeric values. In fact, if you create a table in MS Access and forget to define a primary key, MS Access offers to create a primary key column; if you accept, MS Access creates a column named "ID" with an AutoNumber data type. After you define a column as an AutoNumber type, every time you insert a row in the table, MS Access automatically adds a value to that column, starting with 1 and increasing the value by 1 in every new row you add. Also, you cannot include that column in your INSERT statements—Access will not let you edit that value at all. MS SQL Server traditionally has

used the Identity column property to serve a similar purpose. In MS SQL Server, a table can have at most one column defined as an Identity column. This column behaves similarly to an MS Access column with the AutoNumber data type. The value for the Identity column is generated automatically during the execution of an INSERT operation. If you need to determine the value generated by an Identity column, the options are Scope_Identity() and Ident_Current(). The options differ in two major aspects, table specification and session-awareness. Scope_Identity() does not allow you to specify the identity on a particular table. It returns the last identity value generated on any table. Ident_Current() allows specifying a table, such that

SELECT Ident_Current('PRODUCT');

would return the last identity value generated for the PRODUCT table. Ident_Current(), however, is not session-aware. This means that the function will return the last identity value generated for that table by any session—that is, by any user in the database server. Scope_Identity() is session-aware so the value it returns is the last value that was generated for an Identity column during your current session. That means the value returned is not influenced by the work done by other users of the system. For example, imagine both the CUSTOMER and INVOICE tables use Identity columns for their primary keys. Tom and Jane are both connected to the database server, and they perform the following actions on the CUSTOMER and INVOICE tables.

- Tom creates a customer named "Mohammad Patel".
- Jane creates a customer named "Hannah Mwanse".
- Jane creates an invoice for Hannah.

If Tom runs a command using Scope_Identity(), it will return the value generated for Mohammad's customer row because that is the last Identity value generated in Tom's session. If Jane runs a command using Scope_Identity(), it will return the value generated for Hannah's invoice because that is the last value generated in Jane's session. If either Tom or Jane run a command with Ident_Current('CUSTOMER'), it will return Hannah's customer identity value.

Note

MS Access supports the use of a system variable @@IDENTITY that operates essentially the same as the Scope_Identity() function in SQL Server.

MySQL uses the AUTO_INCREMENT property during table creation to indicate that values for an attribute should be generated in the same fashion. AUTO_INCREMENT can be adjusted to start with a value other than 1. Similar to IDENTITY columns in MS SQL Server, only one column in a table can have AUTO_INCREMENT specified, and that column must also be defined as the primary key of the table.
Auto_increment is a property of an integer attribute. A table can contain only one attribute that has auto_increment enabled. The data type of the attribute must be a form of integer (INT, INTEGER, SMALLINT, BIGINT). MySQL has depreciated the ability to specify a maximum digit length for integer data types (e.g., INTEGER (5)), but this type of limitation is possible using a NUMERIC or DECIMAL data type (e.g., NUMERIC (5,0)). Even though a data type specification of NUMERIC (5,0) will allow only integer values, it is not possible to enable the auto_increment property on that data type. Therefore, the requirements to specify the maximum number of digits in an integer attribute and to enable auto_increment for that attribute are conflicting in MySQL, and both requirements cannot be satisfied. The database programmer will have to prioritize one requirement over the other and use other means, such as a trigger, to programmatically enforce

the other. To be clear, the maximum values of the integer-based data types is not completely unrestricted—that is the purpose of having SMALLINT, INT, and BIGINT to restrict the range of values possible in a broad sense. These data type options should be used to their best advantage in meeting the requirements of the system.

MySQL provides a function named Last_Insert_ID() which will return the last value generated by an auto_increment property. Last_Insert_ID() is session specific, but not table or attribute specific. Last_Insert_ID() will return the last value generated by any auto_increment in the session. There is no way to specify the auto_increment of a particular attribute. Last_Insert_ID() operates like Scope_Identity() in a SQL Server environment. If you need to use the Last_Insert_ID() function, be certain to use it immediately after you perform the insert that required auto_increment to generate a new value and to ensure that you are getting back the value intended.

Oracle does not support the AutoNumber data type or Auto_Increment column properties. Traditionally, Oracle uses a **sequence** to assign values to a column in a table. However, in recent years, Oracle added support for Identity columns, and MS SQL Server added support for sequences. These DBMSs share many similarities in the use of sequences, so a database programmer who is comfortable with one should be able to easily transition to the other. However, a sequence is different from the Access AutoNumber data type or the MySQL auto_increment property and deserves closer scrutiny:

- Sequences are an independent object in the database. (Sequences are not a data type.)
- Sequences have a name.
- Sequences can be used anywhere a value is expected.
- Sequences are not tied to a table or a column.
- Sequences generate a numeric value that can be assigned to any column in any table.
- The table attribute to which you assigned a value based on a sequence can be edited and modified.

The basic syntax to create a sequence is as follows:

CREATE SEQUENCE *name* [START WITH *n*] [INCREMENT BY *n*] [CACHE | NOCACHE]
where

- *name* is the name of the sequence.
- *n* is an integer value that can be positive or negative.
- *START WITH* specifies the initial sequence value. (The default value is 1.)
- *INCREMENT BY* determines the value by which the sequence is incremented. (The default increment value is 1. The sequence increment can be positive or negative to enable you to create ascending or descending sequences.)
- The *CACHE* or *NOCACHE/NO CACHE* clause indicates whether the DBMS will pre-allocate sequence numbers in memory.

Oracle uses *NOCACHE* as one word and pre-allocates 20 values by default. SQL Server uses *NO CACHE* as two words. If a cache size is not specified in SQL Server, then the DBMS will determine a default cache size that is not guaranteed to be consistent across different databases.

For example, you could create a sequence to automatically assign values to the customer code each time a new customer is added and create another sequence to automatically assign values

to the invoice number each time a new invoice is added. The SQL code to accomplish those tasks is:

CREATE SEQUENCE CUS_CODE_SEQ START WITH 20010 NOCACHE;
CREATE SEQUENCE INV_NUMBER_SEQ START WITH 4010 NOCACHE;

Note

Remember, SQL Server uses NO CACHE as two words so the corresponding commands in SQL Server would be:

CREATE SEQUENCE CUS_CODE_SEQ START WITH 20010 NO CACHE;
CREATE SEQUENCE INV_NUMBER_SEQ START WITH 4010 NO CACHE;

You can check all of the sequences you have created by using the following SQL command, as illustrated in Figure 1.7.

SELECT * FROM USER_SEQUENCES;

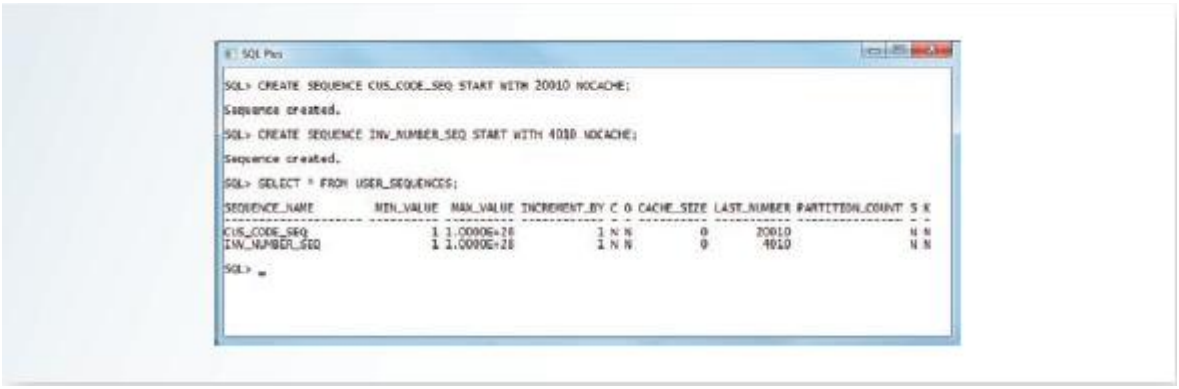


Figure 1-7: Oracle Sequence

Note

SQL Server also stores sequences as schema-level objects, so they can be viewed with the command

SELECT * FROM SYS.SEQUENCES;

Oracle and SQL Server differ slightly in the syntax for retrieving a value from the sequence. In SQL Server, the phrase NEXT VALUE FOR <sequence_name> causes the sequence to generate and return the next value. In Oracle, you must use two special pseudo-columns:

NEXTVAL and CURRVAL. NEXTVAL retrieves the next available value from a sequence, and CURRVAL retrieves the current value of a sequence. For example, you can use the following code to enter a new customer in SQL Server:

INSERT INTO CUSTOMER
VALUES (NEXT VALUE FOR CUS_CODE_SEQ, 'Walker', 'James', NULL, '615',
'898-2007', 0.00);

In Oracle, you would use:
INSERT INTO CUSTOMER
VALUES (CUS_CODE_SEQ.NEXTVAL, 'Walker', 'James', NULL, '615', '898-2007', 0.00);

The preceding SQL statement adds a new customer to the CUSTOMER table and assigns the value 20010 to the CUS_CODE attribute. Examine some important sequence characteristics:

- CUS_CODE_SEQ.NEXTVAL retrieves the next available value from the sequence.
- Each time you use NEXTVAL, the sequence is incremented.
- Once a sequence value is used (through NEXTVAL), it cannot be used again. If your SQL statement rolls back for some reason, *the sequence value does not roll back*. If you issue another SQL statement (with another NEXTVAL), the next available sequence value will be returned to the user—it will look like the sequence skips a number.
- You can issue an INSERT statement without using the sequence.

In Oracle, CURRVAL retrieves the current value of a sequence—that is, the last sequence number used, which was generated with a NEXTVAL. You cannot use CURRVAL unless a NEXTVAL was issued previously in the same session. The main use for CURRVAL is to enter rows in dependent tables. For example, the INVOICE and LINE tables are related in a one-to-many relationship through the INV_NUMBER attribute. You can use the INV_NUMBER_SEQ sequence to automatically generate invoice numbers. Then, using CURRVAL, you can get the latest INV_NUMBER used and assign it to the related INV_NUMBER foreign key attribute in the LINE table. For example:

INSERT INTO INVOICE VALUES (INV_NUMBER_SEQ.NEXTVAL, 20010, SYSDATE);
INSERT INTO LINE VALUES (INV_NUMBER_SEQ.CURRVAL, 1, '13-Q2/P2',1, 14.99);
INSERT INTO LINE VALUES (INV_NUMBER_SEQ.CURRVAL, 2, '23109-HB',1, 9.95);

COMMIT;

The results are shown in Figure 1.8.

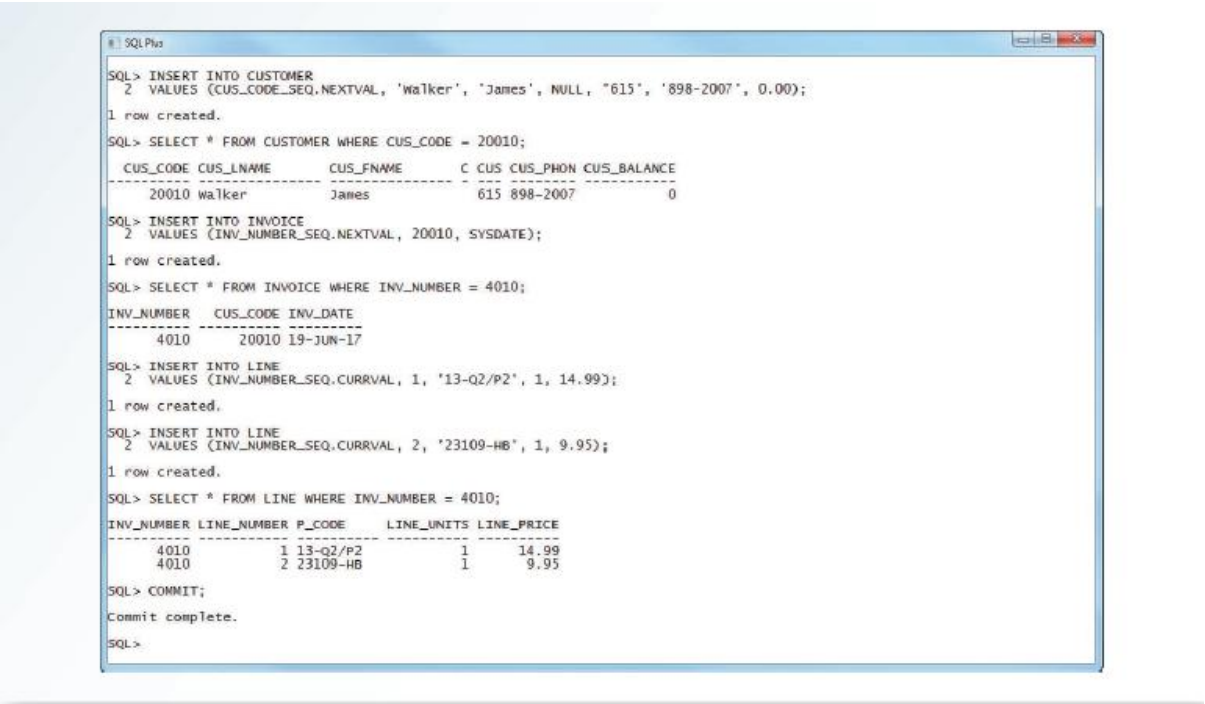


Figure 1-8: Oracle Sequence Examples

In the example shown in Figure 8.8, INV_NUMBER_SEQ.NEXTVAL retrieves the next available sequence number (4010) and assigns it to the INV_NUMBER column in the INVOICE table. Also note the use of the SYSDATE attribute to automatically insert the current date in the INV_DATE attribute. Next, the following two INSERT statements add the products being sold to the LINE table. In this case, INV_NUMBER_SEQ.CURRVAL refers to the last-used INV_NUMBER_SEQ sequence number (4010). In this way, the relationship between INVOICE and LINE is established automatically. The COMMIT statement at the end of the command sequence makes the changes permanent. Of course, you can also issue a ROLLBACK statement, in which case the rows you inserted in the INVOICE and LINE tables would be rolled back (but remember that the sequence number would not). Once you use a sequence number with NEXTVAL, there is no way to reuse it! This "no-reuse" characteristic is designed to guarantee that the sequence will always generate unique values.

Note

At this writing, SQL Server does not provide a direct equivalent to Oracle's CURRVAL. If you want to find the last number generated by a sequence in SQL Server, you can retrieve it by querying the metadata, but this only returns the last number generated by the sequence for any user. Remember, MySQL and Access do not support sequences.

Remember these points when you think about sequences:

- The use of sequences is optional. You can enter the values manually.
- A sequence is not associated with a table.

As in the examples in Figure 1.8, two distinct sequences were created (one for customer code values and one for invoice number values), but you could have created just one sequence and used it to generate unique values for both tables.

Finally, you can drop a sequence from a database with a DROP SEQUENCE command.

For example, to drop the sequences created earlier, you would type:

```
DROP SEQUENCE CUS_CODE_SEQ;
DROP SEQUENCE INV_NUMBER_SEQ;
```

Dropping a sequence does not delete the values you assigned to table attributes (CUS_CODE and INV_NUMBER); it deletes only the sequence object from the database. The *values* you assigned to the table columns (CUS_CODE and INV_NUMBER) remain in the database. Because the CUSTOMER and INVOICE tables are used in the following examples, you will want to keep the original data set. Therefore, you can delete the customer, invoice, and line rows you just added by using the following commands:

```
DELETE FROM INVOICE WHERE INV_NUMBER = 4010;
DELETE FROM CUSTOMER WHERE CUS_CODE = 20010;
COMMIT;
```

Those commands delete the recently added invoice, all of the invoice line rows associated with the invoice (the LINE table's INV_NUMBER foreign key was defined with the ON DELETE CASCADE option), and the recently added customer. The COMMIT statement saves all changes to permanent storage.

Procedural SQL

Thus far, you have learned to use SQL to read, write, and delete data in the database. For example, you learned to update values in a record, to add records, and to delete records. Unfortunately, SQL does not support the *conditional* execution of procedures that are typically supported by a programming language using the general format:

```
IF <condition>
THEN <perform procedure>
ELSE <perform alternate procedure>
END IF
```

SQL also fails to support looping operations in programming languages that permit the execution of repetitive actions typically encountered in a programming environment. The typical format is:

```
DO WHILE
<perform procedure>
END DO
```

Traditionally, if you wanted to perform a conditional or looping type of operation (i.e., a procedural type of programming using an IF-THEN-ELSE or DO-WHILE statement), you would use a programming language such as Visual Basic .NET, C#, or Java. Although that approach is still common, it usually involves the duplication of application code in many programs. Therefore, when procedural changes are required, modifications must be made in many different programs. An environment characterized by such redundancies often creates data-management problems.

A better approach is to isolate critical code and then have all application programs call the shared code. The advantage of this modular approach is that the application code is isolated in a single program, thus yielding better maintenance and logic control. In any case, the rise of distributed databases and object-oriented databases required that more application code be stored and executed within the database. To meet that requirement, most RDBMS vendors created numerous programming language extensions. Those extensions include:

- Flow-control procedural programming structures (IF-THEN-ELSE, DO-WHILE) for logic representation
- Variable declaration and designation within the procedures
- Error management

To remedy the lack of procedural functionality in SQL and to provide some standardization within the many vendor offerings, the SQL-99 standard defined the use of persistent stored modules. A **persistent storage module (PSM)** is a block of code containing standard SQL statements and procedural extensions that is stored and executed at the DBMS server. The PSM represents business logic that can be encapsulated, stored, and shared among multiple database users. A PSM lets an administrator assign specific access rights to a stored module to ensure that only authorized users can use it. Support for PSMs is left to each vendor to implement.

In fact, for many years, some RDBMSs (such as Oracle, SQL Server, and DB2) supported stored procedure modules within the database before the official standard was promulgated. MS SQL Server implements PSMs via Transact-SQL (TSQL) and other language extensions, the most notable of which is the .NET family of programming languages. Oracle implements PSMs through its procedural SQL language named PL/SQL. MySQL uses a procedural SQL that is similar in many respects to the Oracle procedural language. **Procedural SQL** is an extension of the SQL language that makes it possible to use and store procedural code and SQL statements within the database and to merge SQL and traditional programming constructs, such as variables, conditional processing (IF-THEN-ELSE), basic loops (FOR and WHILE loops), and error trapping. The procedural code is executed as a unit by the DBMS when it is invoked (directly or indirectly) by the end user. End users can use procedural SQL to create:

- Stored procedures
- Triggers
- Procedural SQL functions

Do not confuse procedural SQL functions with SQL's built-in aggregate functions such as MIN and MAX. SQL built-in functions can be used only within SQL statements, while procedural SQL functions are mainly invoked within programs such as triggers and stored procedures. Functions can also be called within SQL statements, provided that they conform to very specific rules that are dependent on your DBMS environment.

Note

Although SQL is a universal language for all RDBMS products, it is a declarative language, not procedural. Most DBMS products provide some procedural language support, but there is no universal procedural language for all RDBMS. MS SQL Server uses Transact-SQL (TSQL). Oracle uses PL/SQL. MySQL has its own language that is similar to PL/SQL in many ways but varies in other significant aspects.

The following procedural SQL, stored procedures, and triggers are illustrated within the context of a MySQL DBMS. All examples in the following sections assume the use of MySQL.

You can write a procedural SQL code block by enclosing commands inside BEGIN and END clauses. For example, the following block inserts a new row in the VENDOR table.

```
BEGIN
INSERT INTO VENDOR
VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
END;
```

Some DBMSs will allow anonymous code blocks such as this. MySQL, however, requires that all code blocks be named and defined in such a way that they are stored inside the database. When a code block is named and saved in the database, it is generally called a stored procedure.

Stored Procedures

A **stored procedure** is a named collection of procedural and SQL statements. Stored procedures are stored in the database. One of the major advantages of stored procedures is that they can be used to encapsulate and represent business transactions. For example, you can create a stored procedure to represent a product sale, a credit update, or the addition of a new customer. By doing that, you can encapsulate SQL statements within a single stored procedure and execute them as a single transaction. Using stored procedures offers two clear advantages:

- Stored procedures substantially reduce network traffic and increase performance. Because the procedure is stored at the server, there is no transmission of individual SQL statements over the network. The use of stored procedures improves system performance because all transactions are executed locally on the RDBMS, so each SQL statement does not have to travel over the network.
- Stored procedures help reduce code duplication by means of code isolation and code sharing (creating unique modules that are called by application programs), thereby minimizing the chance of errors and the cost of application development and maintenance.

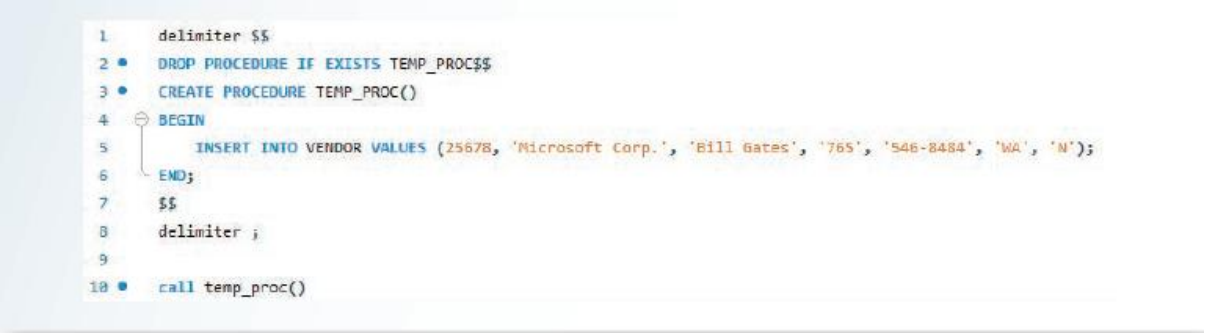
To create a stored procedure, you use the following syntax:

```
CREATE PROCEDURE procedure_name ([IN | OUT | INOUT] argument data-type, ...)
BEGIN
[DECLARE variable_name data type [DEFAULT initial_value] ]
[BEGIN]
Procedural SQL and SQL statements;
...
[END;]
END;
```

Note the following important points about stored procedures and their syntax:

- *argument* specifies the parameters that are passed to the stored procedure. A stored procedure could have zero or more arguments or parameters.
- *IN* | *OUT* | *INOUT* indicates whether the parameter is for input, output, or both.
- *data-type* is one of the procedural SQL data types used in the RDBMS. The data types normally match those used in the RDBMS table creation statement.
- The body of the stored procedure must be inside a BEGIN...END; block. Within that

- block can be one or more DECLARE statements and other blocks of executable code.
- Variables can be declared in a DECLARE statement. You must specify the variable name, its data type, and (optionally) an initial value.



```
1    delimiter $$
2    DROP PROCEDURE IF EXISTS TEMP_PROC$$
3    CREATE PROCEDURE TEMP_PROC()
4    BEGIN
5        INSERT INTO VENDOR VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
6    END;
7    $$
8    delimiter ;
9
10   call temp_proc()
```

Figure 1-9: Block Program with Insert

All MySQL programs must be stored. Therefore, even simple programs, such as the one shown in Figure 1.9 must be named and saved as a stored procedure. The syntax for this program includes several interesting elements. MySQL expects a program to be a statement. In SQL, statements are delimited by a semicolon, which means the end of a statement is marked using a semicolon. Because this program will be a compound statement made of many other statements, a different delimiter must be used to mark the end of the collection of statements that constitute the program. The first line changes the statement delimiter for the interactive session to be $$. It doesn't matter what delimiter you use, as long as it is something that you do not also use when writing your code. Using $$ is common, as is using // and several other options. Line 8 returns to the normal semicolon delimiter once the program is finished.

Line 2 of the program drops any program that currently has the name temp_proc (the $$ delimiter used for this line as it was defined in line 1). Line 3 specifies the name of the program to be created. The program is a compound statement that can include other statements. Notice that MySQL requires parentheses after the procedure name. These parentheses are used to define formal parameters, which are discussed later in this chapter. The parentheses are required even when no formal parameters are being specified, as in this program. Within the program, an executable block is used to contain the INSERT command on line 5. The block is ended on line 6. The compound statement that comprises the program is ended on line 7 using the $$ delimiter. As stated earlier, line 8 changes the delimiter back to the semicolon.

The CREATE PROCEDURE command causes the program to be created, compiled, and stored in the database, but it does not execute the program. Line 10 of Figure 1.9 illustrates using the CALL command to execute the stored procedure, that is, it runs the program. Suppose that you want to display a more specific message after a procedure is completed, such as "New Vendor Added." Because stored procedures are meant to run inside the database, they are not designed to interact directly with the users. They are meant to be called by other programs and to interact with them in a programmatic fashion. As a result, there is little support for displaying information messages within a stored procedure. You can mimic displaying an information message through the use of a SELECT query. You will learn more advanced uses of SELECT within a stored procedure later in the chapter. However, a simple SELECT query within the stored procedure will display its results to the screen. Therefore, you can add a SELECT query to the previous program to display a message. The program shown in Figure 1.10 inserts a row in the VENDOR table and displays the message "New Vendor Added!"

Notice that a SELECT query was used to send the results to the screen. Because this query returns only a literal, the SELECT query does not include a FROM clause. Remember that this is possible in some DBMSs, but not all of them. MySQL clearly allows SELECT without FROM.

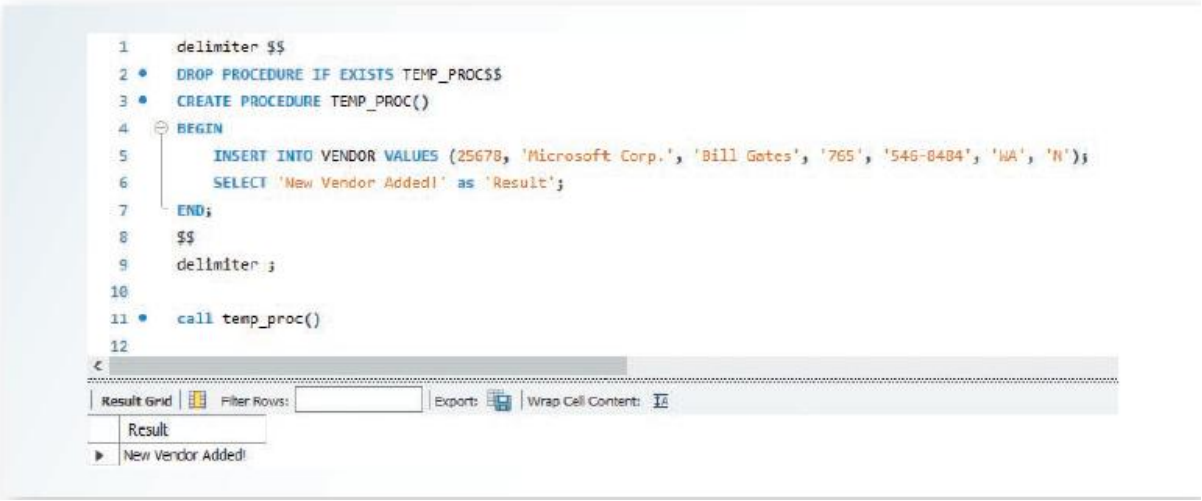


Figure 1-0-20: Output to the screen

Working with Variables

MySQL variables are either local variables or user-defined variables. A user-defined variable is indicated by placing "@" at the beginning of the variable name. User-defined variables persist throughout a session, allowing them to persist values across programs. Local variables are used within a program and are lost when the program execution ends. MySQL local variables are declared inside the executable BEGIN...END block and should be the first part of the executable block. Examine the code in Figure 1.11 and notice the declaration of the variables. Unlike Oracle, the declarations are not a separate block section; they are statements. Therefore, each variable is declared separately. A variable can be initialized to a starting value using the DEFAULT keyword. If a starting value is not specified, the variable begins with a NULL value. Beyond the initial value specified in the declaration of the variable, values are placed in variables using the SET command. Figure 1.11 illustrates the declaration of variables as well as how to manipulate them.

```
1  • DROP PROCEDURE IF EXISTS temp_proc;
2    delimiter $$
3  • CREATE PROCEDURE TEMP_PROC()
4    BEGIN
5        DECLARE MYNUM1 NUMERIC(1) DEFAULT 5;
6        DECLARE MYNUM2 NUMERIC(1);
7        DECLARE TOTAL INT;
8        DECLARE MESSAGE VARCHAR(15) DEFAULT 'The total is ';
9
10       SET MYNUM2 = 6;
11       SET TOTAL = MYNUM1 * MYNUM2;
12       SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
13   END;
14   $$
15   delimiter ;
16
17 • call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
The total is 30

Figure 1-0-31: Variable Manipulation

Conditional Execution

Conditional execution of code can be performed using IF statements in MySQL. An IF structure is composed of one or more sections of conditional code. The structure is defined by the IF and END IF keywords. Figure 1.12 illustrates a basic IF structure. If the condition specified evaluates to true, then the code in the first section, between THEN and ELSE in this example, is executed. If the condition is not true, then the code in the second section, between ELSE and END IF in Figure 1.12, is executed. The condition specified in the IF structure can be very complex, using the same comparison operators, special operators, and logical connectors that are used in the WHERE clause of a SELECT query.

```
1  DROP PROCEDURE IF EXISTS temp_proc;
2  delimiter $$
3  CREATE PROCEDURE TEMP_PROC()
4  BEGIN
5      DECLARE MYNUM1 INT DEFAULT 5;
6      DECLARE MYNUM2 INT DEFAULT 60;
7      DECLARE TOTAL INT;
8      DECLARE MESSAGE VARCHAR(50);
9
10     SET TOTAL = MYNUM1 * MYNUM2;
11     IF TOTAL < 100 THEN
12         SET MESSAGE = 'The total is small: ';
13     ELSE
14         SET MESSAGE = 'The total is large: ';
15     END IF;
16     SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
17 END;
18 $$
19 delimiter ;
20
21 call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
The total is large: 300

Figure 1-12: Conditional Execution (Coronel C., Morris S. 2022)

It is possible to have more than two sections in a single IF structure. The keyword ELSEIF (notice that it is a single word) can be used to create IF structures with as many logical, conditional sections as needed. For example, Figure 1.13 illustrates an IF structure with four conditional sections. Using ELSE IF as two separate keywords creates a nested IF structure, which has a complete IF structure inside a logical section of another, outer IF structure. A single IF structure, as shown in Figure 1.13, is useful when the logical conditions are mutually exclusive. With such a structure, the DBMS will execute the first logical section that evaluates to true, or it executes the ELSE section if none of the other sections evaluate to true. This means you must ensure the logical sections do not overlap. For example, in Figure 1.13, both the second and third sections, beginning on lines 12 and 14 respectively, will be true when the total value is exactly 500. In this case, as the figure shows, only the first section found to be true is executed.

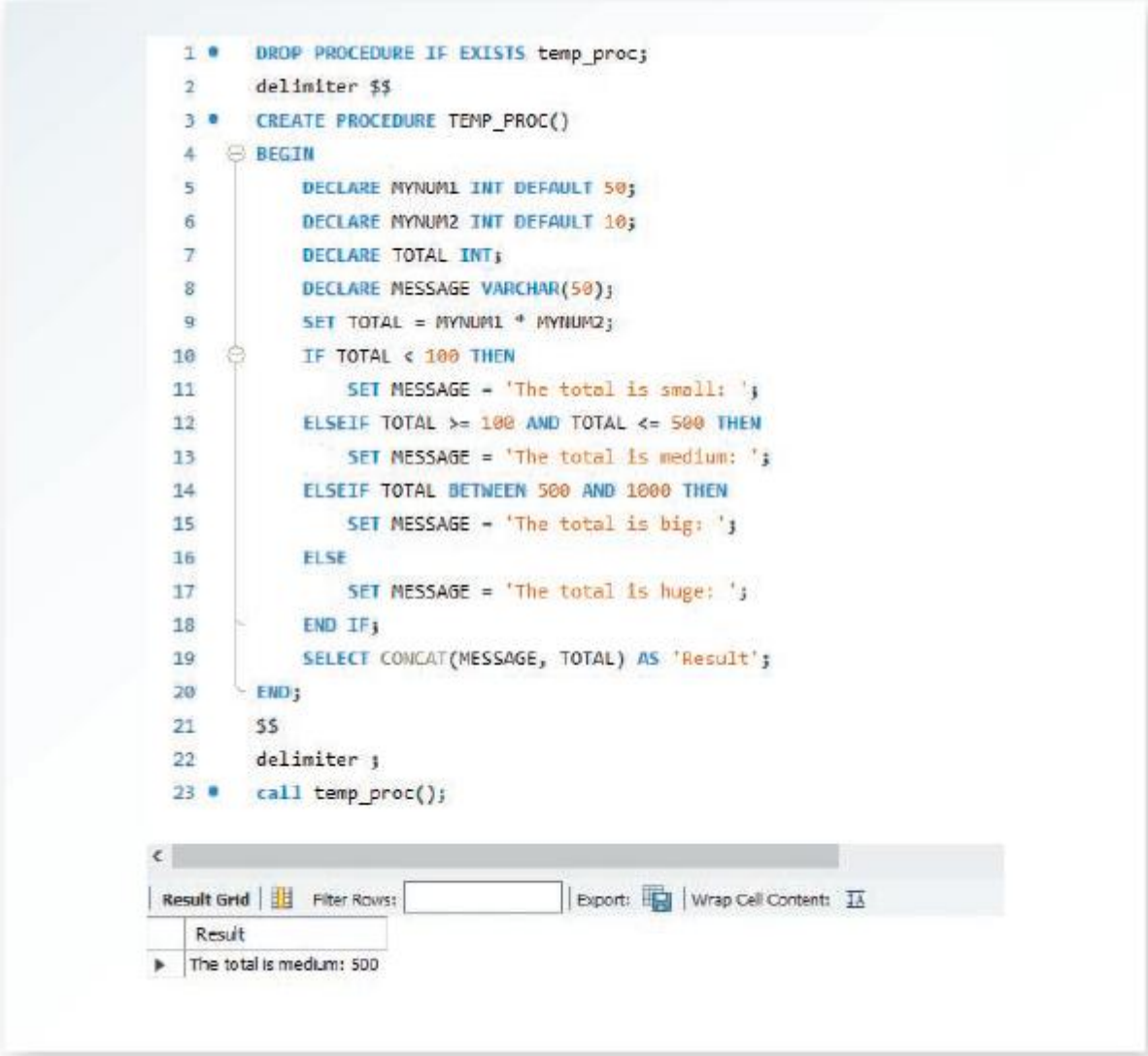


Figure 1-0-43:ElseIF for complex IF Structure(Coronel C., Morris S. 2022)

Iteration or Looping

Iteration, or looping, is a fundamental program flow control operation, and it can easily be incorporated in a MySQL program. Various types of loops can be created, but the primary difference between them is in how the loop is ended. Loops need an indicator as to when the loop should end and program control should continue through the remainder of the program. This section considers two types of loops and how to exit them. The first type of loop uses the LOOP keyword to mark the beginning of the loop. Loops in MySQL often have a label to name the loop. When using the LOOP keyword, a loop label is required. The label ends with a colon and precedes the LOOP keyword, as shown in line 7 of Figure 1.14, where the counter loop is defined. The END LOOP statement marks the end of the loop. When logical program control reaches the END LOOP statement, control automatically passes back to the beginning of the loop. The LEAVE statement is used to exit the loop. Exiting the loop means that logical program control passes immediately to the statement that follows the END LOOP statement. The LEAVE statement is typically

included as a conditional expression; that is, it is typically inside an IF structure. The LEAVE keyword is followed by the loop label to indicate which loop is to be exited.

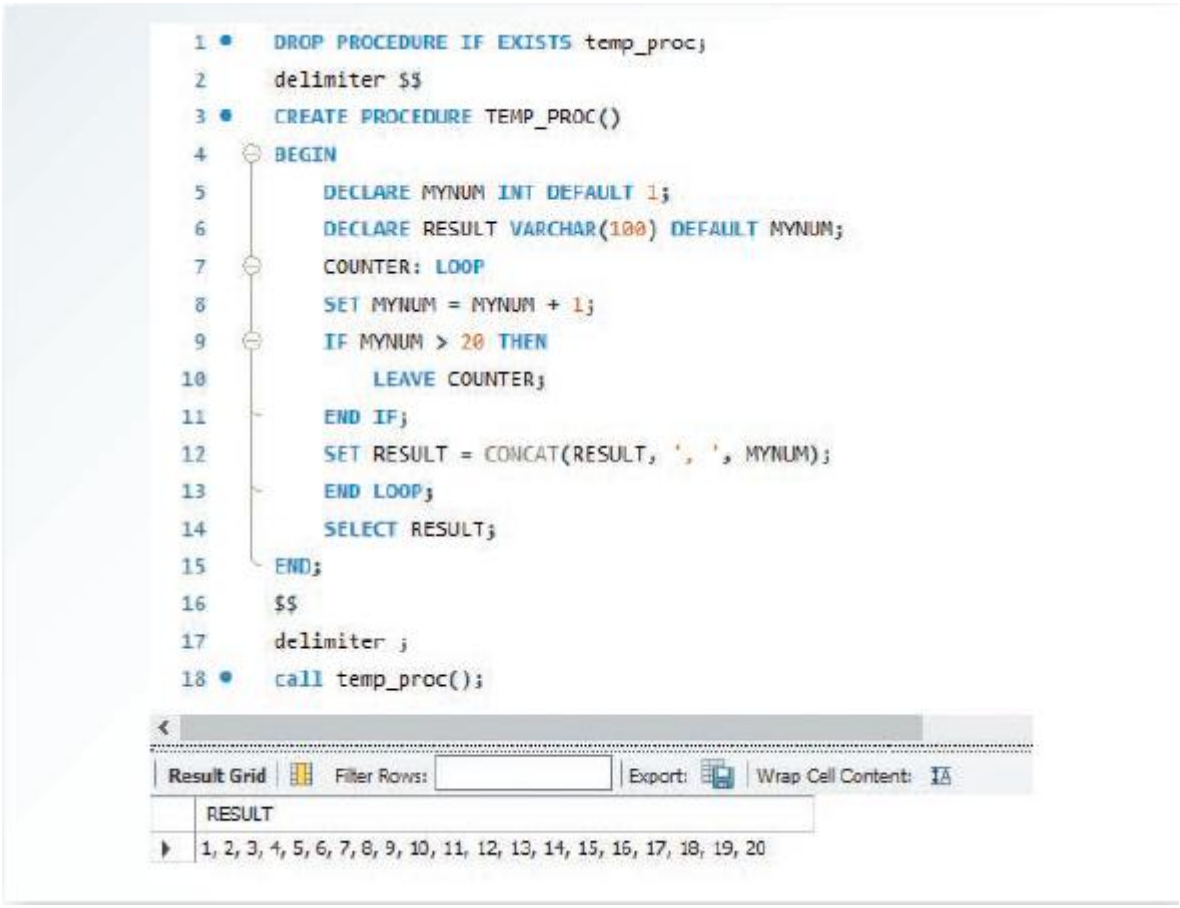


Figure 1-14: A Simple loop (Coronel C., Morris S. 2022)

The second type of loop to consider is called a while loop because it uses the keyword WHILE to define the exiting condition for the loop. With a WHILE loop, the loop label is optional. WHILE loops place the exiting condition immediately after the WHILE keyword, thus eliminating the need for a LEAVE statement. Figure 1.15 illustrates the same program as the previous figure but implemented with a WHILE loop instead.

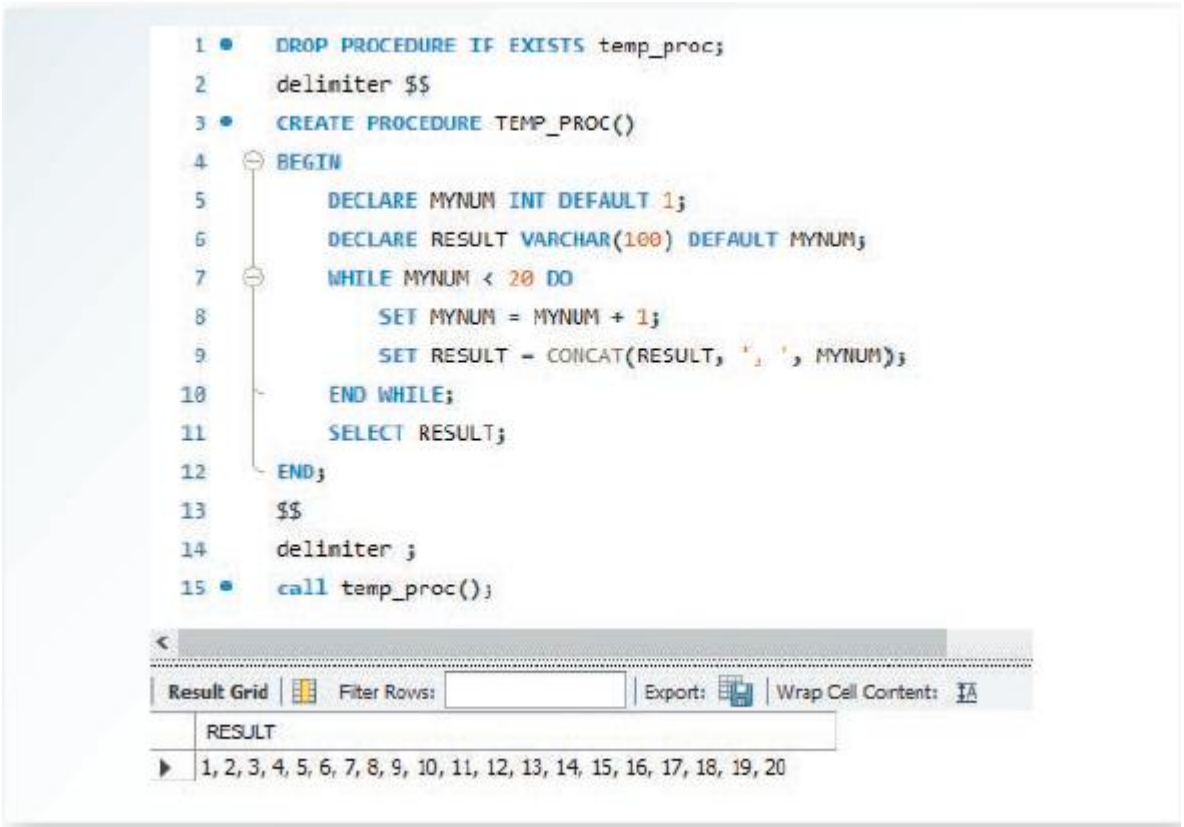


Figure 1-0-5: Implementation of a while loop (Coronel C., Morris S. 2022)

One capability that is often associated with looping that is not easily supported in MySQL is dealing with multiple lines of output to the console. In the programs illustrated thus far, only a single line of output has been sent to the console by each program. This output is generated using a SELECT query. MySQL Workbench displays a result window for each SELECT query in a program. The result window is generated by the SELECT query. Therefore, if multiple SELECT queries are generating output to the screen, the result window for the first query is separate from the result window for the second query, which is separate from the result window for the third query, and so on for all queries in the program. The consequence of this behavior is that what should have been a single, coherent output is spread across numerous, independent output windows. This can be illustrated by changing the loop program from Figure 1.15 to include a line of output with each loop iteration, as shown in Figure 1.16. Notice that only the result of the last SELECT query is shown.



Figure 1-0-66: Multiple Select Outputs (Coronel C., Morris S. 2022)

The SELECT query that produces the output can return multiple lines, but there can essentially be only one SELECT query producing output during any given execution of the program. The solution to getting the query that produces the output to produce multiple lines of output is typically solved by creating a temporary table, writing the intended lines of output to the temporary table, then querying that table to display the output. While this is not an especially elegant solution, it is generally considered the best solution currently available in MySQL for most programs. Temporary tables are limited in scope so that they are only visible to the user that creates them. Temporary tables are automatically dropped when the database session ends, but they can be dropped sooner if the program is finished with them. Figure 1.17 illustrates a basic example of using a temporary table to allow multiple lines of output from a program.

```
1  DROP PROCEDURE IF EXISTS temp_proc;
2  delimiter $$
3  CREATE PROCEDURE TEMP_PROC()
4  BEGIN
5      DECLARE MYNUM INT DEFAULT 1;
6      CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
7      WHILE MYNUM <= 5 DO
8          INSERT INTO MYTAB VALUES (CONCAT('The current value is ', MYNUM));
9          SET MYNUM = MYNUM + 1;
10     END WHILE;
11     SELECT * FROM MYTAB;
12     DROP TEMPORARY TABLE MYTAB;
13 END;
14 $$
15 delimiter ;
16 call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RESULT
The current value is 1
The current value is 2
The current value is 3
The current value is 4
The current value is 5

Figure 1-0-77: Temporary Table for Outputs (Coronel C., Morris S. 2022)

SELECT Processing with Cursors

Until now, all of the SELECT statements you have used inside a stored procedure have returned data for output. However, you often need to retrieve data from the database to manipulate within the program. In this case, the results of the SELECT query must go into one or more variables so they can be manipulated. Figure 1.18 illustrates a simple SELECT query to retrieve data into a variable in a stored procedure. If the SELECT statement returns more than one value, you will generate an error. If you want to use a SELECT statement that returns more than one value inside your program code, you need to use a cursor. A **cursor** is a special construct used in procedural SQL to hold the data rows returned by a SELECT query. You can think of a cursor as a reserved area of memory in which the output of the query is stored, like an array holding columns and rows. Cursors are held in a reserved memory area in the DBMS server, not in the client computer.

There are two types of cursors: implicit and explicit. An **implicit cursor** is automatically created in procedural SQL when the SELECT statement returns only one value. The program in Figure 1.18 created an implicit cursor. An **explicit cursor** is created to hold the output of a SQL statement that may return two or more rows (but could return zero rows or only one). To create an explicit cursor, you use the following syntax in a DECLARE statement:

DECLARE *cursor_name* CURSOR FOR *select-query*;

After you have declared a cursor, you can use specific cursor processing commands (OPEN, FETCH, and CLOSE) anywhere between the BEGIN and END keywords of the executable block. Table 1.4 summarizes the main use of each command.

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE W_P1 INT DEFAULT 0;
    DECLARE W_P2 INT DEFAULT 10;
    DECLARE W_NUM INT DEFAULT 0;
    CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
    WHILE W_P2 < 300 DO
        SELECT COUNT(P_CODE) INTO W_NUM FROM PRODUCT
        WHERE P_PRICE BETWEEN W_P1 AND W_P2;
        INSERT INTO MYTAB VALUES (CONCAT('There are ', W_NUM, ' products with a price between ', W_P1, ' and ', W_P2));
        SET W_P1 = W_P2 + 1;
        SET W_P2 = W_P2 + 50;
    END WHILE;
    SELECT * FROM MYTAB;
    DROP TEMPORARY TABLE MYTAB;
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RESULT
There are 5 products with a price between 0 and 10
There are 6 products with a price between 11 and 60
There are 3 products with a price between 61 and 110
There are 1 products with a price between 111 and 160
There are 0 products with a price between 161 and 210
There are 1 products with a price between 211 and 260

Figure 1-0-88: Select into a Variable (Coronel C., Morris S. 2022)

Table 4.4: Cursor Processing Commands

Cursor Command	Explanation
OPEN	Opening the cursor executes the SQL command and populates the cursor with data, opening the cursor for processing. The cursor declaration command only reserves a named memory area for the cursor; it does not populate the cursor with the data. Before you can use a cursor, you need to open it. For example: OPEN *cursor_name*
FETCH	After the cursor is opened, you can use the FETCH command to retrieve data from the cursor and copy it to the variables for processing. The syntax is: FETCH cursor_name INTO variable1 [, variable2, ...] The variables used to hold the data must be declared in DECLARE statements and must have data types compatible with the columns retrieved by the SQL command. If the cursor's SQL statement returns five columns, there must be five variables to receive the data from the cursor. This type of processing resembles the one-record-at-a-time processing used in previous database models. The first time you fetch a row from the cursor, the first row of data from the cursor is copied to the variables; the second time you fetch a row from the cursor, the second row of data is placed in the variables; and so on.
CLOSE	The CLOSE command closes the cursor for processing.

Cursor-style processing involves retrieving data from the cursor one row at a time. After you open a cursor; it becomes an active data set. That data set contains a "current" row pointer. Therefore, after opening a cursor, the current row is the first row of the cursor. When you fetch a row from the cursor, the data from the "current" row in the cursor is copied to the PL/SQL variables. After the fetch, the "current" row pointer moves to the next row in the set and continues until it reaches the end of the cursor. How do you know when you have reached the end of the cursor data set?

In MySQL, you must create a special object known as a *handler* that will signal the end of the cursor has been reached. The basic format for a handler is

DECLARE [CONTINUE | EXIT] HANDLER FOR NOT FOUND <*statement*>;

Whenever the program attempts to FETCH a row from the cursor after there are no rows left to process, the program will check with the handler to determine what action to take. The handler action can either be to CONTINUE or EXIT. The CONTINUE action means that the handler will let the program continue to execute, while the EXIT action means that the program will immediately end. If the program is allowed to CONTINUE, then the <statement> will be executed and the program will continue processing at the point at which it realized the cursor was finished.

To illustrate the use of cursors, use a simple stored procedure example to list the products that have a greater quantity on hand than the average quantity on hand for all products. The code is shown in Figure 1.19. As you examine the stored procedure code shown in Figure 1.19, note the following important characteristics:

- Line 8 declares a variable to be manipulated by the handler so the program will know when the cursor is finished.
- Lines 9, 10, and 11 declare the PROD_CURSOR cursor.
- Line 12 declares the handler for the cursor. The handler is defined to set the variable *NOTFOUND* to the value 1 then allow the program to CONTINUE when a new row is NOT FOUND in the cursor during a fetch operation.
- Line 16 opens the PROD_CURSOR cursor and populates it.
- Line 17 uses the LOOP statement to mark the beginning of the loop that will process the data in the cursor, one row at a time.
- Line 18 uses the FETCH command to retrieve a row from the cursor and place it in the respective variables. If the FETCH command finds no row to process (when the cursor is finished), it will check with the handler to decide what to do.
- Line 19 checks to see if the handler has set the variable *NOTFOUND* to be 1 yet. The handler will only do this when the FETCH command indicates that it could not find another row in the cursor.
- Line 20 uses the LEAVE command to exit the loop if Line 19 determined that the *NOTFOUND* variable had been changed to the value 1.
- Line 25 issues the CLOSE PROD_CURSOR command to close the cursor.

The use of cursors, combined with standard SQL, makes working with relational databases very desirable because programmers can work in the best of both worlds: set-oriented processing and record-oriented processing. Any experienced programmer knows to use the tool that best fits the job. Sometimes you will be better off manipulating data in a set-oriented environment; at other times, it might be better to use a record-oriented environment. Procedural SQL lets you have your proverbial cake and eat it too. Procedural SQL provides functionality that enhances the capabilities of the DBMS while maintaining a high degree of manageability.



Figure 1-0-99: An explicit cursor example(Coronel C., Morris S. 2022)

Stored Procedures with Parameters

One of the most valuable features of working with stored procedures is their ability to use parameters. A parameter is a value that is provided to the program at the time of execution. To illustrate, assume that you want to create a procedure (PRC_PROD_DISCOUNT) to assign an additional 5 percent discount for all products when the quantity on hand is more than or equal to twice the minimum quantity. Figure 1.20 shows how the stored procedure is created. Notice that line 13 of Figure 1.20 begins a transaction so the changes made during the execution of the program can be rolled back (line 16). You must test the execution of programs and using a transaction so the test can be rolled back is a good practice.

```
1  DROP PROCEDURE IF EXISTS prc_prod_discount;
2  deliniter $$
3  CREATE PROCEDURE PRC_PROD_DISCOUNT()
4  BEGIN
5      UPDATE PRODUCT
6      SET P_DISCOUNT = P_DISCOUNT + .05
7      WHERE P_QOH >= P_MIN * 2;
8      SELECT '** Update finished.';
9  END;
10 $$
11 deliniter ;
12
13 START TRANSACTION;
14 call PRC_PROD_DISCOUNT();
15 SELECT P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_DISCOUNT FROM PRODUCT;
16 ROLLBACK;
17
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

P_CODE	P_DESCRIPT	P_QOH	P_MIN	P_DISCOUNT
11QER/31	Power painter, 15 psi., 3-nozzle	8	5	0.00
13-Q2/P2	7.25-in. pwr. saw blade	32	15	0.10
14-Q1/L3	9.00-in. pwr. saw blade	18	12	0.00
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15	8	0.00
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23	5	0.05
2232/QTY	B&D jigsaw, 12-in. blade	8	5	0.05
2232/QWE	B&D jigsaw, 8-in. blade	6	5	0.05
2238/QPD	B&D cordless drill, 1/2-in.	12	5	0.10
23109-HB	Claw hammer	23	10	0.15
23114-AA	Sledge hammer, 12 lb.	8	5	0.05
54778-2T	Rat-tail file, 1/8-in. fine	43	20	0.05
89-WRE-Q	Hicut chain saw, 16 in.	11	5	0.10
PVC23DRT	PVC pipe, 3.5-in., 8-ft	188	75	0.05
SM-18277	1.25-in. metal screw, 25	172	75	0.05
SW-23116	2.5-in. wd. screw, 50	237	100	0.05
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	18	5	0.15

Figure 1-20: Creating the PRC_PROD_DISCOUNT Stored Procedure (Coronel C., Morris S. 2022)

One of the main advantages of procedures is that you can pass values to them. For example, the previous PRC_PROD_DISCOUNT procedure worked well, but what if you want to make the percentage increase an input variable? In that case, you can pass an argument, or parameter, to represent the rate of increase to the procedure. Figure 1.21 shows the code for that procedure. Figure 1.21 shows the execution of the second version of the PRC_PROD_DISCOUNT stored procedure. Note that if the procedure requires arguments, they must be enclosed in parentheses and separated by commas. Line 3 includes the format parameter, or the definition of the parameter. In this case, the parameter WPI was specified as being an input for the program (IN) and as having the data type DECIMAL (2,2). Line 18 that calls the procedure provides the actual parameter, or the value to be used in the variable specified by the formal parameter. In this case, during the current execution of the stored procedure, WPI will have the value .33. If a different value is specified in the actual parameter during a later execution of the program, the program will execute with WPI having that other value.



```
DROP PROCEDURE IF EXISTS prc_prod_discount;
delimiter $$
CREATE PROCEDURE PRC_PROD_DISCOUNT(IN WPI DECIMAL(2,2))
BEGIN
    IF WPI <=0 OR WPI >= 1 THEN  -- validate WPI parameter
        SELECT 'Error: Value must be greater than 0 and less than 1' AS RESULT;
    ELSE
        UPDATE PRODUCT
        SET P_DISCOUNT = P_DISCOUNT + WPI
        WHERE P_QOH >= P_MIN * 2;
        SELECT '** Update finished.';
    END IF;
END;
$$
delimiter ;

START TRANSACTION;
call PRC_PROD_DISCOUNT(.33);
SELECT P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_DISCOUNT FROM PRODUCT;
ROLLBACK;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

P_CODE	P_DESCRIPT	P_QOH	P_MIN	P_DISCOUNT
11QER/31	Power painter, 15 psi., 3-nozzle	8	5	0.00
13-Q2/P2	7.25-in. pwr. saw blade	32	15	0.38
14-Q1/L3	9.00-in. pwr. saw blade	18	12	0.00
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15	8	0.00
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23	5	0.33
2232/QTY	B&D jigsaw, 12-in. blade	8	5	0.05
2232/QWE	B&D jigsaw, 8-in. blade	6	5	0.05
2238/QPD	B&D cordless drill, 1/2-in.	12	5	0.38
23109-HB	Claw hammer	23	10	0.43
23114-AA	Sledge hammer, 12 lb.	8	5	0.05
54778-2T	Rat-tail file, 1/8-in. fine	43	20	0.33
89-WRE-Q	Hicut chain saw, 16 in.	11	5	0.38
PVC23DRT	PVC pipe, 3.5-in., 8-ft	188	75	0.33
SM-18277	1.25-in. metal screw, 25	172	75	0.33
SW-23116	2.5-in. wd. screw, 50	237	100	0.33
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	18	5	0.43

Figure 1-20-10: Iteration of the PRC_PROD_DISCOUNT Stored Procedure (Coronel C., Morris S. 2022)

Stored procedures are also useful to encapsulate shared code to represent business transactions. For example, you can create a simple stored procedure to add a new customer. By using a stored procedure, all programs can call it by name each time a new customer is added. Naturally, if new customer attributes are added later, you will need to modify the stored procedure. However, the programs that use the stored procedure will not need to know the name of the newly added attribute; they will need to add only a new parameter to the procedure call.

Triggers

Automating business procedures and automatically maintaining data integrity and consistency are critical in a modern business environment. While stored procedures can help to encapsulate the business logic, they must be manually executed. Triggers advance stored procedures by creating a type of program that the DBMS can run automatically under specified conditions.

For example, one of the most critical business procedures is proper inventory management. You want to make sure that current product sales can be supported with sufficient product availability. Therefore, you must ensure that a product order is written to a vendor when that product's inventory drops below its minimum allowable quantity on hand. Better yet, how about ensuring that the task is completed automatically?

To automate product ordering, you first must make sure the product's quantity on hand reflects an up-to-date and consistent value. After the appropriate product availability requirements have been set, two key issues must be addressed:

1. Business logic requires an update of the product quantity on hand each time there is a sale of that product.
2. If the product's quantity on hand falls below its minimum allowable inventory level, the product must be reordered.

To accomplish these two tasks, you could write multiple SQL statements: one to update the product quantity on hand and another to update the product reorder flag. Next, you would have to run each statement in the correct order each time there was a new sale. Such a multistage process would be inefficient because a series of SQL statements must be written and executed each time a product is sold. Even worse, this SQL environment requires that someone must remember to perform the SQL tasks.

A **trigger** is procedural SQL code that is *automatically* invoked by the RDBMS upon the occurrence of a given data manipulation event. It is useful to remember that:

- A trigger is invoked before or after a data row is inserted, updated, or deleted.
- A trigger is associated with a database table.
- Each database table may have one or more triggers.
- A trigger is executed as part of the transaction that triggered it.

Triggers are critical to proper database operation and management. For example:

- Triggers can be used to enforce constraints that cannot be enforced at the DBMS design and implementation levels.
- Triggers add functionality by automating critical actions and providing appropriate warnings and suggestions for remedial action. In fact, one of the most common uses for triggers is to facilitate the enforcement of referential integrity.
- Triggers can be used to update table values, insert records in tables, and call other stored
- procedures.
- Triggers cannot output data to the screen. Because triggers are designed to run unattended, the DBMS will not assume that there is any user interface capable of accepting display messages or results of SELECT queries on the user interface.
- Triggers play a critical role in making the database truly useful; they also add processing power to the RDBMS and to the database system as a whole.

To see how a trigger is created and used, examine a simple inventory management problem. For example, if a product's quantity on hand is updated when the product is sold, the system should automatically check whether the quantity on hand falls below its minimum allowable quantity. To demonstrate that process, use the PRODUCT table in Figure 1.22. Note the use of the minimum quantity (P_MIN) and product reorder flag (P_REORDER) columns. The P_MIN indicates the minimum quantity on hand allowable before restocking an item. The P_REORDER column is a numeric field that indicates whether the product needs to be reordered (1 = Yes, 0 = No). The initial P_REORDER values are set to 0 (No) to serve as the basis for the initial trigger development.

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595	25	0
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595	12	0
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225	25	0
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.4	0.05		12	0
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344	25	0
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288	10	0
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00		50	0
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5	119.95	0.10	25595	10	0

Figure 1-22: The Product Table (Coronel C., Morris S. 2022)

Given the PRODUCT table listing shown in Figure 1.22, create a trigger to evaluate the product's quantity on hand, P_QOH. If the quantity on hand is below the minimum quantity shown in P_MIN, the trigger will set the P_REORDER column to 1, which represents "Yes." The syntax to create a trigger in MySQL is as follows:

CREATE TRIGGER *trigger_name*
[BEFORE / AFTER] [DELETE / INSERT / UPDATE] ON *table_name*
FOR EACH ROW
[DECLARE]
[*variable_namedata type*[5*initial_value*]]
BEGIN
Procedural SQL instructions;
...
END;

As you can see, a trigger definition contains the following parts:

- *The triggering timing*: BEFORE or AFTER. This timing indicates when the trigger's code executes—in this case, before or after the triggering statement is completed.
- *The triggering statement or triggering event*: The statement that causes the trigger to execute (INSERT, UPDATE, or DELETE).
- *The triggering level*: The two types of triggers are statement-level triggers and row-level triggers. Although MySQL does not currently support statement-level triggers, other DBMSs do, and such functionality may be added to MySQL in the future, so it is good to be familiar with the concept.
- A **statement-level trigger** is executed once, before or after the triggering statement is completed. The presence of the FOR EACH ROW clause makes a trigger a row-level trigger and is required in MySQL.
- A **row-level trigger** requires use of the FOR EACH ROW keywords. This type of trigger is executed once for each row affected by the triggering statement. (In other words, if you update 10 rows, the trigger executes 10 times.)
- *The triggering action:* The SQL and procedural SQL code enclosed between the BEGIN and END keywords. Each statement inside the triggering action must end with a semicolon (;).

In the PRODUCT table, the P_REORDER attribute is a derived attribute (calculated from P_MIN and P_QOH), and like all derived attributes is redundant. One of the most common uses for triggers is to control redundant data. Remember, it is not so much that redundancy is problematic for the integrity of the data, it is *uncontrolled* redundancy that compromises the integrity of the data. When you want to use redundant data in a database, triggers can be created to control that redundancy so that it can never become inconsistent, thus avoiding the primary problems with the redundancy. In this case, we will need to introduce several new concepts into our procedural SQL coding to create an appropriate trigger. To avoid introducing too many new concepts at once, we will work through different versions of a trigger to control the P_REORDER redundancy.

One complication with triggers is that you will be creating a situation in which multiple SQL commands are trying to operate on the same rows of data at the same time. The trigger timing, BEFORE or AFTER, addresses this to some extent but does not deal with it completely. Consider the unsuccessful trigger shown in Figure 1.23. The trigger will generate an error.

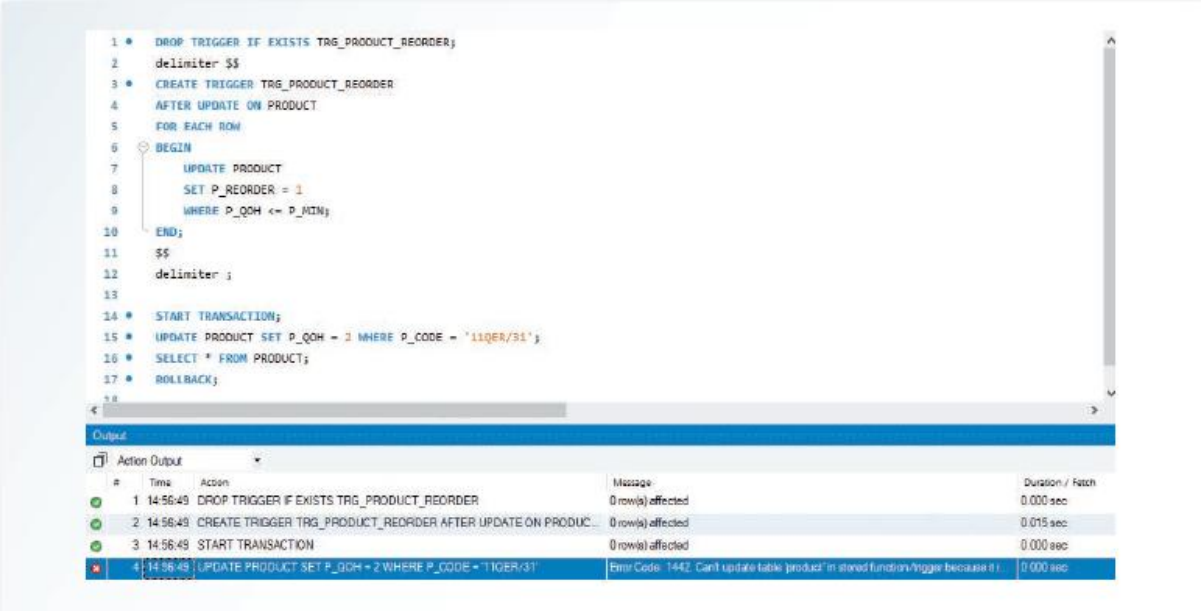


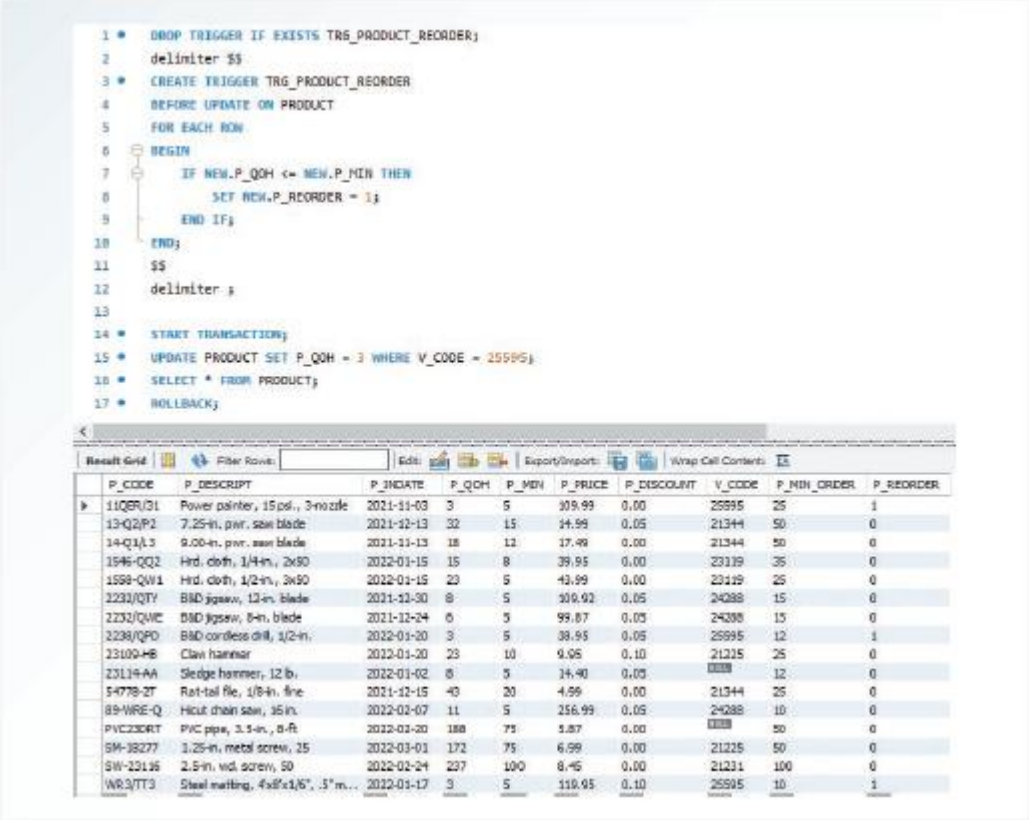
Figure 1-23: Unsuccessful Product Reorder Trigger (Coronel C., Morris S. 2022)

The triggering statement is UPDATE ON PRODUCT. In other words, if the user attempts to update a row in the PRODUCT table, the DBMS will automatically execute the code in the triggering action inside this trigger. The timing of the trigger is AFTER the triggering statement. In other words, the DBMS will perform the update from the triggering statement before it attempts the update in the triggering action. The trigger will compile successfully. However, if you try to issue a command that would cause the trigger to execute, such as the following command, an error will be generated.

```
UPDATE PRODUCT
SET P_QOH = 2
WHERE P_CODE = '11QER/31';
```

Remember, this is a row-level trigger, as all triggers are in MySQL. This means the trigger runs once for every row being manipulated by the triggering statement. Essentially, the DBMS begins execution of the triggering statement to update the P_QOH attribute. When the DBMS finds a row that matches the criterion in the WHERE clause, it updates that row, then "pauses" to run the trigger. When the trigger finishes, the DBMS "unpauses" the triggering statement to look for any other rows that satisfy the WHERE clause. If it finds a second row, it will update it and again pause the triggering statement to run the trigger again, before continuing to search for additional rows. The error message tells you that the problem is that the triggering action to update the P_REORDER attribute is being blocked by the triggering statement query that is currently paused. Only one update statement can be operating on row 11QER/31 at a time, but the code has created a situation where both the triggering statement and the triggering action are trying to manipulate that row.

This is made worse because the update in the triggering action is attempting to evaluate, and potentially update, every row in the PRODUCT table. This is a common issue with triggers. The better solution is to have the triggering action only attempt to manipulate the row in the PRODUCT table that the triggering statement "paused" on. You need a way to reference just that paused row, and you need to reference it in such a way that the DBMS does not see the statements as conflicting with each other. The version of the trigger shown in Figure 1.24 uses such a reference.



```
DROP TRIGGER IF EXISTS TRG_PRODUCT_REORDER;
delimiter $$
CREATE TRIGGER TRG_PRODUCT_REORDER
BEFORE UPDATE ON PRODUCT
FOR EACH ROW
BEGIN
    IF NEW.P_QOH <= NEW.P_MIN THEN
        SET NEW.P_REORDER = 1;
    END IF;
END;
$$
delimiter ;

START TRANSACTION;
UPDATE PRODUCT SET P_QOH = 3 WHERE V_CODE = 25595;
SELECT * FROM PRODUCT;
ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	2021-11-03	3	5	109.99	0.00	25595	25	1
13-Q2/P2	7.25-in. pwr. saw blade	2021-12-13	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	2021-11-13	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	2022-01-15	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	2022-01-15	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	2021-12-30	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	2021-12-24	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	2022-01-20	3	5	38.95	0.05	25595	12	1
23109-HB	Claw hammer	2022-01-20	23	10	9.95	0.10	21225	25	0
Z3114-AA	Sledge hammer, 12 lb.	2022-01-02	8	5	14.40	0.05	NULL	12	0
54778-2T	Rat-tail file, 1/8-in. fine	2021-12-15	40	20	4.99	0.00	21344	25	0
89-WRE-Q	Hicut chain saw, 16 in.	2022-02-07	11	5	256.99	0.05	24288	10	0
PVC23DRT	PVC pipe, 3.5-in., 8-ft	2022-02-20	188	75	5.87	0.00	NULL	50	0
SM-18277	1.25-in. metal screw, 25	2022-03-01	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	2022-02-24	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	2022-01-17	3	5	119.95	0.10	25595	10	1

Figure 1-24: Successful Product Reorder Trigger (Coronel C., Morris S. 2022)

Notice that this version of the trigger now executes BEFORE the triggering statement. It also uses a NEW reference. To understand the use of the NEW reference, you must first consider a basic computing tenet: *all changes are done first in primary memory and then transferred to permanent memory*. In other words, the computer cannot change anything directly in permanent storage (on disk). It must first read the data from permanent storage to primary memory, then make the

change in primary memory, and finally write the changed data back to permanent memory (on disk). This means that the definition of BEFORE and AFTER trigger timing must be considered more precisely. The triggering statement is always executed *in memory* before the triggering action is performed. A BEFORE trigger timing means that the triggering action is performed before the results of the triggering statement are made *permanent*.

An AFTER timing means that the triggering action is performed after the results of the triggering statement are made permanent. The DBMS operates in the same way, with one addition. Because ensuring data integrity is critical, the DBMS makes two copies of every row being changed by a DML (INSERT, UPDATE, or DELETE) statement. The first copy contains the original ("old") values of the attributes in the row before the changes are made. The second copy contains the changed ("new") values of the attributes that will be permanently saved to the database after any changes made by the INSERT, UPDATE, or DELETE. You can use OLD to refer to the original values; you can use NEW to refer to the changed values (the values that will be stored in the table). You can use NEW and OLD attribute references only within the SQL code of a database trigger action. For example:

- IF NEW.P_QOH < = NEW.P_MIN compares the quantity on hand with the minimum quantity of a product. Remember that this is a row-level trigger. Therefore, this comparison is made for each row that is updated by the triggering statement.
- Although the trigger is a BEFORE trigger, this does not mean that the triggering statement has not executed yet. To the contrary, the triggering statement has already taken place; otherwise, the trigger would not have fired, and the NEW values would not exist. Remember, BEFORE means *before* the changes are permanently saved to disk, but *after* the changes are made in memory.
- The trigger uses the NEW reference to assign a value to the P_REORDER column before the UPDATE results are permanently stored in the table. The assignment is always made to the NEW value (never to the OLD value). The OLD values are *read-only* values; you cannot change them. Note that SET NEW.P_REORDER = 1; assigns the value 1 to the P_REORDER column and SET NEW.P_REORDER = 0; assigns the value 0 to the P_REORDER column.
- This new trigger version does not use any DML statements!

The trigger manipulates the values in the row that the triggering statement paused on while those values are still in memory for the triggering statement. Whatever values are in those attributes in memory when the triggering statement is finished are written permanently to storage. A value is written regardless of whether it is in that row in memory because it was unchanged from the original version of the row, was changed by the triggering statement, or was changed by the triggering action. That is why changes to NEW must be made in a BEFORE trigger. If the trigger is BEFORE, then the row is still being manipulated in memory. If the trigger is AFTER, then the results from memory have already been written to permanent storage. To make a change, your trigger must make the change before the values are permanently written.

After compiling the trigger shown in Figure 1.24, test the trigger with the following statements. First start a transaction so you can roll back the changes after testing.

```
START TRANSACTION;
UPDATE PRODUCT
SET P_QOH = 3
WHERE V_CODE = 25595;
```

If you look at the data in the PRODUCT table, you will now see that the P_REORDER attribute for several rows has been set to 1 based on the P_QOH and P_MIN values. Continue the test with the following code:

```
UPDATE PRODUCT
SET P_MIN = 1
WHERE V_CODE = 25595;
```

If you look at the data in the PRODUCT table again, you will see that none of the 1 values changed back to 0. Our data is inconsistent! Use the ROLLBACK command to return your data to the original state. The problem is that the trigger only knows when to set the P_REORDER equal to 1, but it does not know about setting P_REORDER equal to 0. In practice, you must consider all of the possibilities for your data. Including redundant data in the database is risky. Creating triggers to control that redundancy reduces the risks but failing to anticipate all of the potential threats to consistency can have devastating effects on the quality of the data. The trigger can be modified as shown in Figure 1.25 to enable the trigger to return P_REORDER to 0 when appropriate. (Be certain you remembered to ROLLBACK the previous testing. Compiling a procedural program will commit any outstanding DML actions.)

```
1  DROP TRIGGER IF EXISTS TRG_PRODUCT_REORDER;
2  delimiter $$
3  CREATE TRIGGER TRG_PRODUCT_REORDER
4  BEFORE UPDATE ON PRODUCT
5  FOR EACH ROW
6  BEGIN
7      IF NEW.P_QOH <= NEW.P_MIN THEN
8          SET NEW.P_REORDER = 1;
9      ELSE
10         SET NEW.P_REORDER = 0;
11     END IF;
12 END;
13 $$
14 delimiter ;
15
16 START TRANSACTION;
17 UPDATE PRODUCT SET P_QOH = 3 WHERE V_CODE = 25595;
18 SELECT * FROM PRODUCT;
19 UPDATE PRODUCT SET P_MIN = 1 WHERE V_CODE = 25595;
20 SELECT * FROM PRODUCT;
21 ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

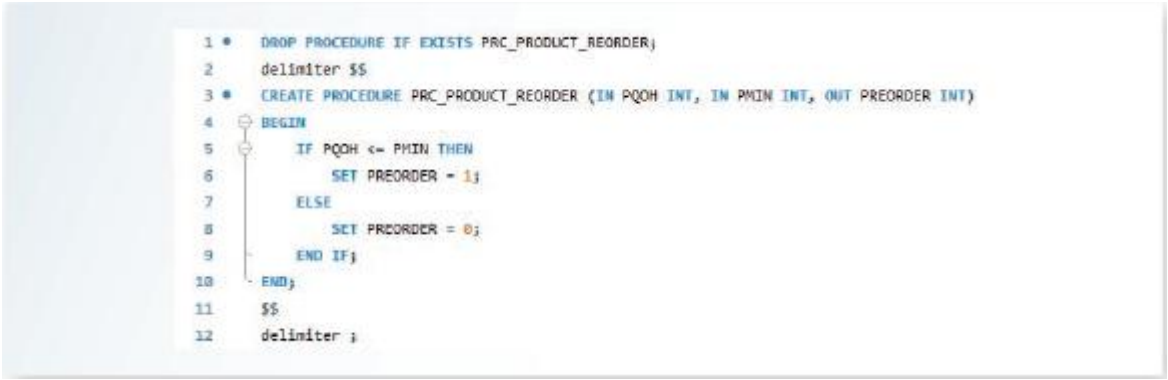
P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	2021-11-03	3	1	109.99	0.00	25595	25	0
13-Q2/P2	7.25-in. pwr. saw blade	2021-12-13	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	2021-11-13	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	2022-01-15	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	2022-01-15	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	2021-12-30	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	2021-12-24	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	2022-01-20	3	1	38.95	0.05	25595	12	0
23109-HB	Claw hammer	2022-01-20	23	10	9.95	0.10	21225	25	0
23114-AA	Sledge hammer, 12 lb.	2022-01-02	8	5	14.40	0.05	NULL	12	0
54778-2T	Rat-tail file, 1/8-in. fine	2021-12-15	43	20	4.99	0.00	21344	25	0
89-WRE-Q	Hicut chain saw, 16 in.	2022-02-07	11	5	256.99	0.05	24288	10	0
PVC23DRT	PVC pipe, 3.5-in., 8-ft	2022-02-20	188	75	5.87	0.00	NULL	50	0
SM-18277	1.25-in. metal screw, 25	2022-03-01	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	2022-02-24	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	2022-01-17	3	1	119.95	0.10	25595	10	0

Figure 1-25: Second Successful Product Reorder Trigger (Coronel C., Morris S. 2022)

After you have created the trigger shown in Figure 1.25, you can test it using the same code that you used to test the previous trigger. You will find that this newest version of the trigger

successfully changes values to 1 or to 0 as appropriate. However, there is still one remaining issue. As discussed earlier, you must consider all of the conditions that could lead to inconsistency in the data, and there is one scenario that has not been covered. It is possible that a user could insert a new row into the PRODUCT table, and to insert it with P_REORDER being inconsistent with the values entered for P_MIN and P_QOH. In fact, because P_REORDER is redundant data, it is likely that users will not be allowed to insert values for it directly. In which case, the DBMS will have to determine, based on the P_QOH and P_MIN being inserted, which value should be used in P_REORDER.

Many DBMS products will allow a trigger to have multiple triggering statements in the trigger definition (e.g., INSERT OR UPDATE ON PRODUCT). MySQL, however, only allows a trigger to contain a single DML operation as a triggering statement. One solution to this limitation is to create multiple triggers with different triggering events that all perform the same processing. Because each trigger would have a different triggering event, the trigger processing will still occur only once for any given triggering DML command. While this works, it is not ideal because it creates redundant program code in multiple triggers. The standard workaround for this limitation is to create a stored procedure that contains the processing that should be done by the different triggers, then simply have the triggers call the stored procedure. Figure 1.26 shows the code for such a shared stored procedure.



```
1  DROP PROCEDURE IF EXISTS PRC_PRODUCT_REORDER;
2  delimiter $$
3  CREATE PROCEDURE PRC_PRODUCT_REORDER (IN PQOH INT, IN PMIN INT, OUT PREORDER INT)
4  BEGIN
5      IF PQOH <= PMIN THEN
6          SET PREORDER = 1;
7      ELSE
8          SET PREORDER = 0;
9      END IF;
10 END;
11 $$
12 delimiter ;
```

Figure 1-26: Stored Procedure for Common Trigger Processing (Coronel C., Morris S. 2022)

Notice that the stored procedure does not work with the NEW references like a trigger does. It only works with variables. Triggers can use both variables and NEW and OLD references, but procedures can only work with variables. The triggers to work with this procedure are shown in Figure 1.27 and Figure 1.28. In this case, the trigger in Figure 1.27 is executed when the triggering INSERT occurs. The trigger will take the value in the NEW.P_QOH reference and pass that value to the procedure as a value. The procedure receives the value from the trigger and places it in a parameter named PQOH. The value in the parameter now corresponds to the value in NEW.P_QOH. Even though the stored procedure cannot work directly with the NEW reference, the procedure now has the value from the NEW reference in a parameter that it can work with. The trigger passes the value from NEW.P_MIN to the procedure as well, and it is placed in another parameter named PMIN. Again, the procedure cannot directly address the NEW reference, but the trigger has passed the values that are in the NEW reference to the procedure in a manner that the procedure can work with.

```
DROP TRIGGER IF EXISTS TRG_INSERT_PRODUCT_REORDER;
delimiter $$
CREATE TRIGGER TRG_INSERT_PRODUCT_REORDER
BEFORE INSERT ON PRODUCT
FOR EACH ROW
BEGIN
    CALL PRC_PRODUCT_REORDER(NEW.P_QOH, NEW.P_MIN, NEW.P_REORDER);
END;
$$
delimiter ;
```

Figure 1-27: Trigger for Insert Commands (Coronel C., Morris S. 2022)

```
DROP TRIGGER IF EXISTS TRG_UPDATE_PRODUCT_REORDER;
delimiter $$
CREATE TRIGGER TRG_UPDATE_PRODUCT_REORDER
BEFORE UPDATE ON PRODUCT
FOR EACH ROW
BEGIN
    CALL PRC_PRODUCT_REORDER(NEW.P_QOH, NEW.P_MIN, NEW.P_REORDER);
END;
$$
delimiter ;
```

Figure 1-28: Trigger of Update Commands (Coronel C., Morris S. 2022)

The stored procedure also makes use of an OUT parameter. Remember that parameters can be IN, OUT, or INOUT. A parameter is essentially a variable whose values can be passed in and/or out of the procedure. An IN parameter is used to pass a value into the stored procedure from the statement that called the procedure, as was done to pass the values of NEW.P_QOH and NEW.P_MIN from the trigger to the procedure. An OUT parameter is used to pass a value from the stored procedure out to a variable in the statement that called the procedure; in this case, the trigger in either Figure 1.27 or Figure 1.28. The procedure is only working with variables. Based on the values in the two IN parameters, the procedure is determining the appropriate value to put in the PREORDER variable. Once the correct value has been placed in PREORDER, the procedure ends, and the value in PREORDER (an OUT parameter) is passed out of the procedure to the trigger that called the procedure. The trigger then places that value in the NEW.P_REORDER reference.

Because the programs in Figure 1.27 and Figure 1.28 are triggers, they can use the NEW reference to manipulate the values in the row being manipulated by the triggering statement. The trigger in Figure 1.27 will execute when a new row is inserted into PRODUCT, while the trigger in Figure 1.28 will execute when an existing row is updated in PRODUCT. Regardless of which triggering statement is used, only one of the triggers will execute, and either will call the same procedure so the execution is identical. The triggers provide values from NEW.P_QOH and NEW.P_MIN to the IN parameters of the stored procedure, and both triggers will accept the value from the OUT parameter of the stored procedure into the NEW.P_REORDER attribute.

In all of the previous trigger examples, you created a trigger to manipulate the same row that the triggering statement was manipulating. This is not the only time that triggers are useful. You can

also use a trigger to update an attribute in a table other than the one being modified. For example, suppose that you would like to create a trigger that automatically reduces the quantity on hand of a product with every sale. To accomplish that task, you must create a trigger for the LINE table that updates a row in the PRODUCT table. The sample code for that trigger is shown in Figure 1.29.

```
DROP TRIGGER IF EXISTS TRG_LINE_PROD;
delimiter $$
CREATE TRIGGER TRG_LINE_PROD
AFTER INSERT ON LINE
FOR EACH ROW
BEGIN
    UPDATE PRODUCT
    SET P_QOH = P_QOH - NEW.LINE_UNITS
    WHERE PRODUCT.P_CODE = NEW.P_CODE;
END;
$$
delimiter ;

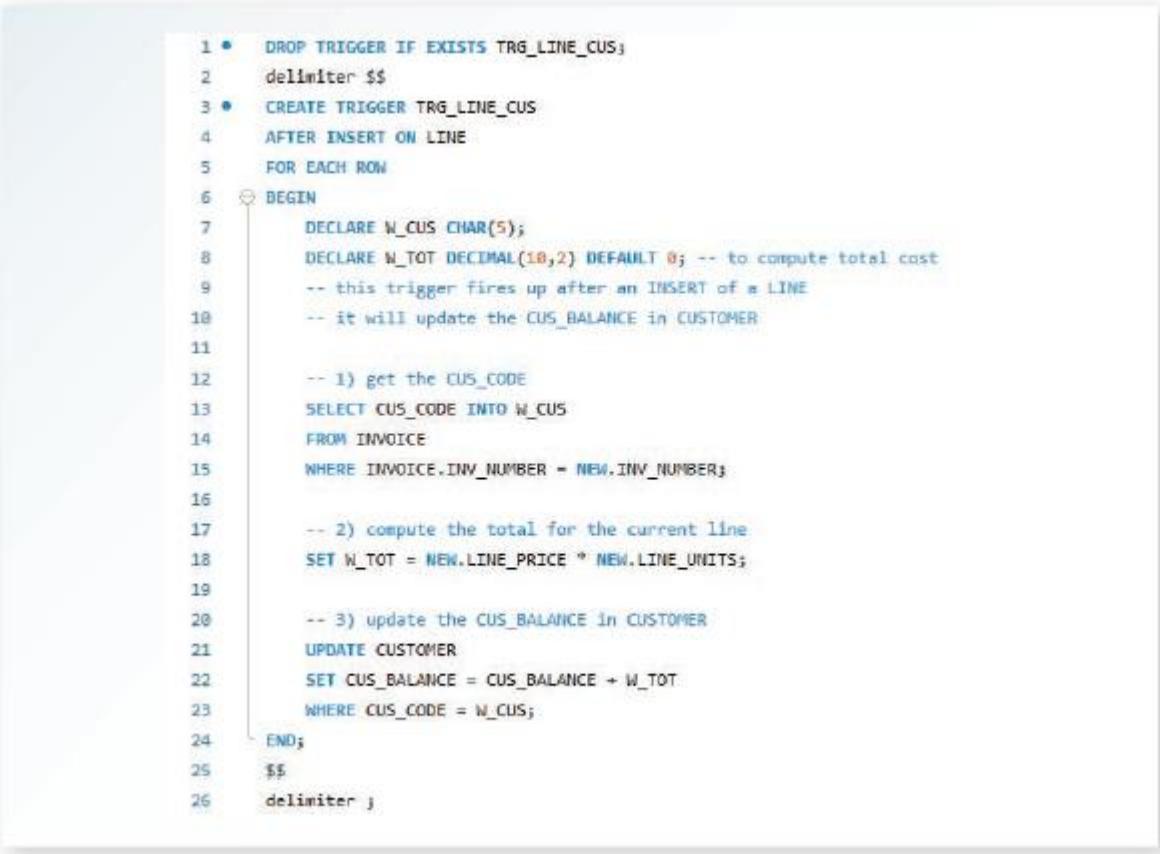
START TRANSACTION;
INSERT INTO LINE VALUES (1001, 3, '11QER/31', 5, 10);
SELECT * FROM LINE;
SELECT * FROM PRODUCT;
ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_MIN_ORDER	P_REORDER
11QER/31	Power painter, 15 psi., 3-nozzle	2021-11-03	3	5	109.99	0.00	25595	25	1
13-Q2/P2	7.25-in. pwr. saw blade	2021-12-13	32	15	14.99	0.05	21344	50	0
14-Q1/L3	9.00-in. pwr. saw blade	2021-11-13	18	12	17.49	0.00	21344	50	0
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	2022-01-15	15	8	39.95	0.00	23119	35	0
1558-QW1	Hrd. cloth, 1/2-in., 3x50	2022-01-15	23	5	43.99	0.00	23119	25	0
2232/QTY	B&D jigsaw, 12-in. blade	2021-12-30	8	5	109.92	0.05	24288	15	0
2232/QWE	B&D jigsaw, 8-in. blade	2021-12-24	6	5	99.87	0.05	24288	15	0
2238/QPD	B&D cordless drill, 1/2-in.	2022-01-20	12	5	38.95	0.05	25595	12	0
23109-HB	Claw hammer	2022-01-20	23	10	9.95	0.10	21225	25	0
23114-AA	Sledge hammer, 12 lb.	2022-01-02	8	5	14.40	0.05	NULL	12	0
54778-2T	Rat-tail file, 1/8-in. fine	2021-12-15	43	20	4.99	0.00	21344	25	0
89-WRE-Q	Hicut chain saw, 16 in.	2022-02-07	11	5	256.99	0.05	24288	10	0
PVC23DRT	PVC pipe, 3.5-in., 8-ft	2022-02-20	188	75	5.87	0.00	NULL	50	0
SM-18277	1.25-in. metal screw, 25	2022-03-01	172	75	6.99	0.00	21225	50	0
SW-23116	2.5-in. wd. screw, 50	2022-02-24	237	100	8.45	0.00	21231	100	0
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	2022-01-17	18	5	119.95	0.10	25595	10	0

Figure 1-29: TRG_LINE_PROD Trigger to update PQOH(Coronel C., Morris S. 2022)

Note that the TRG_LINE_PROD trigger executes after inserting a new invoice's LINE and reduces the quantity on hand of the recently sold product by the number of units sold. This trigger updates a row in a different table (PRODUCT), using the :NEW values of the recently added LINE row. In this case, an AFTER trigger is appropriate because the trigger does not need to change the values in the NEW reference, only read them, so it is okay for those changes to have already been permanently written to the database. Another trigger example shows the use of variables within a trigger. In this case, you want to update the customer balance (CUS_BALANCE) in the CUSTOMER table after inserting a new LINE row. This trigger code is shown in Figure 1.30.



```
DROP TRIGGER IF EXISTS TRG_LINE_CUS;
delimiter $$
CREATE TRIGGER TRG_LINE_CUS
AFTER INSERT ON LINE
FOR EACH ROW
BEGIN
    DECLARE W_CUS CHAR(5);
    DECLARE W_TOT DECIMAL(10,2) DEFAULT 0; -- to compute total cost
    -- this trigger fires up after an INSERT of a LINE
    -- it will update the CUS_BALANCE in CUSTOMER

    -- 1) get the CUS_CODE
    SELECT CUS_CODE INTO W_CUS
    FROM INVOICE
    WHERE INVOICE.INV_NUMBER = NEW.INV_NUMBER;

    -- 2) compute the total for the current line
    SET W_TOT = NEW.LINE_PRICE * NEW.LINE_UNITS;

    -- 3) update the CUS_BALANCE in CUSTOMER
    UPDATE CUSTOMER
    SET CUS_BALANCE = CUS_BALANCE + W_TOT
    WHERE CUS_CODE = W_CUS;
END;
$$
deliniter ;
```

Figure 1-30: : TRG_LINE_CUS Trigger to update the customer balance (Coronel C., Morris S. 2022)

Carefully examine the trigger in Figure 1.30.

- The trigger executes after each new LINE row is inserted.
- The DECLARE statements in the trigger are used to declare variables used inside the trigger code.
- You can declare a variable by assigning a name, a data type, and (optionally) an initial value, as in the case of the W_TOT variable.
- The first step in the trigger code is to get the customer code (CUS_CODE) from the related INVOICE table. Note that the SELECT statement returns only one attribute (CUS_CODE) from the INVOICE table. Also note that the attribute returns only one value as specified by the use of the WHERE clause, *to restrict the query output to a single value*.
- Note the use of the INTO clause within the SELECT statement. You use the INTO clause to assign a value from a SELECT statement to a variable (W_CUS) used within a trigger.
- The second step in the trigger code computes the total of the line by multiplying NEW. LINE_UNITS by NEW.LINE_PRICE and assigning the result to the W_TOT variable.
- The final step updates the customer balance by using an UPDATE statement and the W_TOT and W_CUS trigger variables.
- Double dashes (--) are used to indicate comments within the trigger.

The use of triggers facilitates the automation of multiple data management tasks. Although triggers are independent objects, they are associated with database tables. When you delete a table, all its trigger objects are deleted with it. However, if you needed to delete a trigger without deleting the table, you could use the following command:

DROP TRIGGER *trigger_name;*

User Defined Functions

Using programmable or procedural SQL, you can also create your own stored functions. Stored procedures and functions are very similar. A **user defined function (UDF)** is basically a named group of procedural and SQL statements that returns a value, as indicated by a RETURN statement in its program code. To create a function, you use the following syntax:

CREATE FUNCTION *function_name* (IN *argument data-type*, ...) RETURNS *data-type*
BEGIN
Procedure SQL statements;
...
RETURN (*value or expression*);
END;

User defined functions can be invoked from within stored procedures, triggers, and usually from SQL statements such as SELECT, INSERT, UPDATE, and DELETE. Remember not to confuse built-in SQL functions (such as MIN, MAX, and AVG) with stored functions.

Embedded SQL

There is little doubt that SQL's popularity as a data manipulation language is due in part to its ease of use and its powerful data-retrieval capabilities. In the real world, however, database systems are related to other systems and programs, and you still need a conventional programming language such as C#, Java, or COBOL to integrate database systems with other programs and systems. If you are developing web applications, you are most likely familiar with Java, ASP, or ColdFusion. Yet, almost regardless of the programming tools you use, if your web application or Windows-based GUI system requires access to a database such as MS Access, MySQL, SQL Server, Oracle, or DB2, you will likely need to use SQL to manipulate the data in the database.

Embedded SQL is a term used to refer to SQL statements contained within an application programming language such as Python, C#, COBOL, or Java. The program being developed might be a standard binary executable in Windows or Linux, or it might be a web application designed to run over the Internet. No matter what language you use, if it contains embedded SQL statements, it is called the **host language**. Embedded SQL is still the most common approach to maintaining procedural capabilities in DBMS-based applications. However, mixing SQL with procedural languages requires that you understand some key differences between the two.

- *Run-time mismatch*. Remember that SQL is a nonprocedural, interpreted language; that is, each instruction is parsed, its syntax is checked, and it is executed one instruction at a time. (The authors are particularly grateful for the thoughtful comments provided by Emil T. Cipolla.) All of the processing takes place at the server side. Meanwhile, the host language is generally a binary-executable program (also known as a *compiled program*). The host program typically runs at the client side in its own memory space, which is different from the DBMS environment.

- *Processing mismatch*. Conventional programming languages process one data element at a time. Although you can use arrays to hold data, you still process the array elements one row at a time. This is especially true for file manipulation, where the host language

typically manipulates data one record at a time. However, newer programming environments such as C# have adopted object-oriented approaches that help the programmer manipulate data sets in a cohesive manner.

- *Data type mismatch*. SQL provides several data types, but some of them might not match data types used in different host languages (e.g., the DATETIME and VARCHAR data types).

Another way to interface host languages and SQL is through the use of a call-level interface (CLI), in which the programmer writes to an application programming interface (API). A common CLI in Windows is provided by the Open Database Connectivity (ODBC) interface. Many new programming languages, like Python, allow the establishment of database connections through the DBMS to issue SQL statements to the database. Python has a large collection of resources that can be included in projects. Importing a MySQL connector library into a Python program provides the necessary facilities to connect a database server (import mysql.connector). The MySQL connector includes the definition of a "connection" object for a database connection. Using this object is as simple as providing the host name for the database server, the username to connect to the server as, and the password for that user on the database. connection = mysql.connector.connect (*host_name*, *user_name*, *password*)

The connector also includes a defined "cursor" object that is used to interact with the database through the connection. For example, you could use the following code in a Python program to retrieve data from the PRODUCT table. (Python uses triple quotes when a string value spans multiple lines.)

```
cursor = connection.cursor()
query = """
SELECT P_CODE, P_DESCRIPT, P_PRICE
FROM PRODUCT
WHERE V_CODE IN (25595, 23118, 21225);
"""
cursor.execute(query)
results = cursor.fetchall()
for result in results:
print(result)
```

Other enterprise-level programs use more complex methods to embed SQL. The SQL standard defines an embedded SQL framework to integrate SQL within several programming languages like COBOL and C that are still popular in enterprise software. This framework allows program pre-compilers to convert the embedded SQL into routines within the host language. The embedded SQL framework defines the following:

- A standard syntax to identify embedded SQL code within the host language (EXEC SQL/ END-EXEC).
- A standard syntax to identify host variables, which are variables in the host language that receive data from the database (through the embedded SQL code) and process the data in the host language. All host variables are preceded by a colon (:).
- A communication area used to exchange status and error information between SQL and the host language. This communication area contains two variables—SQLCODE and SQLSTATE.

Before continuing, you should explore the process required to create and run an executable program with embedded SQL statements. If you have ever programmed in Java or C++, you are familiar with the multiple steps required to generate the final executable program. Although the specific details vary among language and DBMS vendors, the following general steps are standard:

1. The programmer writes embedded SQL code within the host language instructions. The code follows the standard syntax required for the host language and embedded SQL.
2. A preprocessor is used to transform the embedded SQL into specialized procedure calls that are DBMS- and language-specific. The preprocessor is provided by the DBMS vendor and is specific to the host language.
3. The program is compiled using the host language compiler. The compiler creates an object code module for the program containing the DBMS procedure calls.
4. The object code is linked to the respective library modules and generates the executable program. This process binds the DBMS procedure calls to the DBMS run-time libraries. Additionally, the binding process typically creates an "access plan" module that contains instructions to run the embedded code at run time.
5. The executable is run, and the embedded SQL statement retrieves data from the database.

Note that you can embed individual SQL statements or even an entire procedural SQL block. Up to this point in the book, you have used a DBMS-provided application (MySQL Workbench) to write SQL statements and procedural SQL blocks in an interpretive mode to address one-time or ad hoc data requests. However, it is extremely difficult and awkward to use ad hoc queries to process transactions inside a host language. Programmers typically embed SQL statements within a host language that is compiled once and executed as often as needed. To embed SQL into a host language, follow this syntax:

```
EXEC SQL
SQL statement;
END-EXEC.
```

The preceding syntax will work for SELECT, INSERT, UPDATE, and DELETE statements. For example, the following embedded SQL code will delete employee 109, George Smith, from the EMPLOYEE table:

```
EXEC SQL
DELETE FROM EMPLOYEE WHERE EMP_NUM = 109;
END-EXEC.
```

Remember, the preceding embedded SQL statement is compiled to generate an executable statement. Therefore, the statement is fixed permanently and cannot change (unless, of course, the programmer changes it). Each time the program runs, it deletes the same row. In short, the preceding code is good only for the first run; all subsequent runs will likely generate an error.

Clearly, this code would be more useful if you could specify a variable to indicate the employee number to be deleted. In embedded SQL, all host variables are preceded by a colon (:). The host variables may be used to send data from the host language to the embedded SQL, or they may be used to receive the data from the embedded SQL. To use a host variable, you must first declare

it in the host language. Common practice is to use similar host variable names as the SQL source attributes.
For example, if you are using COBOL, you would define the host variables in the Working Storage section. Then you would refer to them in the embedded SQL section by preceding them with a colon. For example, to delete an employee whose employee number is represented by the host variable W_EMP_NUM, you would write the following code:

```
EXEC SQL
DELETE FROM EMPLOYEE WHERE EMP_NUM = :W_EMP_NUM;
END-EXEC.
```

At run time, the host variable value will be used to execute the embedded SQL statement. What happens if the employee you are trying to delete does not exist in the database? How do you know that the statement has been completed without errors? As mentioned previously, the embedded SQL standard defines a SQL communication area to hold status and error information. In COBOL, such an area is known as the SQLCA area and is defined in the Data Division as follows:

```
EXEC SQL
INCLUDE SQLCA
END-EXEC.
```

The SQLCA area contains two variables for status and error reporting. Table 1.5 shows some of the main values returned by the variables and their meaning.

Table 5.5: SQL Status and Error Reporting Variables

Variable Name	Value	Explanation
SQLCODE		Old-style error reporting supported for backward compatibility only; returns an integer value (positive or negative)
	0	Successful completion of command
	100	No data; the SQL statement did not return any rows and did not select, update, or delete any rows
	–999	Any negative value indicates that an error occurred
SQLSTATE		Added by SQL-92 standard to provide predefined error codes; defined as a character string (5 characters long)
	00000	Successful completion of command
		Multiple values in the format XXYYY where: XX-> represents the class code YYY-> represents the subclass code

The following embedded SQL code illustrates the use of the SQLCODE within a COBOL program.

```
EXEC SQL
SELECT EMP_FNAME, EMP_LNAME INTO :W_EMP_ FNAME, :W_EMP_
LNAME WHERE EMP_NUM = :W_EMP_NUM;
END-EXEC.
IF SQLCODE = 0 THEN
PERFORM DATA_ROUTINE
ELSE
PERFORM ERROR_ROUTINE
END-IF.
```

In this example, the SQLCODE host variable is checked to determine whether the query completed successfully. If it did, the DATA_ROUTINE is performed; otherwise, the ERROR_ROUTINE is performed. Just as with PL/SQL, embedded SQL requires the use of cursors to hold data from a query that returns more than one value. If COBOL is used, the cursor can be declared either in the Working Storage section or in the Procedure Division. The cursor must be declared and processed. To declare a cursor, you use the syntax shown in the following example:

```
EXEC SQL
DECLARE PROD_CURSOR FOR
SELECT P_CODE, P_DESCRIPT FROM PRODUCT
WHERE P_QOH > (SELECT AVG(P_QOH) FROM PRODUCT);
END-EXEC.
```

Next, you must open the cursor to make it ready for processing:

```
EXEC SQL
OPEN PROD_CURSOR;
END-EXEC.
```

To process the data rows in the cursor, you use the FETCH command to retrieve one row of data at a time and place the values in the host variables. The SQLCODE must be checked to ensure that the FETCH command completed successfully. This section of code typically constitutes part of a routine in the COBOL program. Such a routine is executed with the PERFORM command. For example:

```
EXEC SQL
FETCH PROD_CURSOR INTO :W_P_CODE, :W_P_DESCRIPT;
END-EXEC.

IF SQLCODE = 0 THEN
PERFORM DATA_ROUTINE
ELSE
PERFORM ERROR_ROUTINE
END-IF.
```

When all rows have been processed, you close the cursor as follows:

```
EXEC SQL
CLOSE PROD_CURSOR;
END-EXEC.
```

Thus far, you have seen examples of embedded SQL in which the programmer used predefined SQL statements and parameters. Therefore, the end users of the programs are limited to the actions that were specified in the application programs. That style of embedded SQL is known as **static SQL**, meaning that the SQL statements will not change while the application is running. For example, the SQL statement might read like this:

```
SELECT P_CODE, P_DESCRIPT, P_QOH, P_PRICE FROM PRODUCT
WHERE P_PRICE > 100;
```

Note that the attributes, tables, and conditions are known in the preceding SQL statement. Unfortunately, end users seldom work in a static environment. They are more likely to require the flexibility of defining their data access requirements on the fly. Therefore, the end user requires that SQL be as dynamic as the data access requirements. **Dynamic SQL** is a term used to describe an environment in which the SQL statement is not known in advance; instead, the SQL statement is generated at run time. At run time in a dynamic SQL environment, a program can generate the SQL statements that are required to respond to ad hoc queries. In such an environment, neither the programmer nor the end user is likely to know precisely what kind of queries will be generated or how they will be structured. For example, a dynamic SQL equivalent of the preceding example could be:

```
SELECT :W_ATTRIBUTE_LIST
FROM :W_TABLE
WHERE :W_CONDITION;
```

Note that the attribute list and the condition are not known until the end user specifies them. W_TABLE, W_ATTRIBUTE_LIST, and W_CONDITION are text variables that contain the end-user input values used in the query generation. Because the program uses the end-user input to build the text variables, the end user can run the same program multiple times to generate varying outputs. For example, in one instance, the end user might want to know which products cost less than $100; in another case, the end user might want to know how many units of a given product are available for sale at any given moment. Although dynamic SQL is clearly flexible, such flexibility carries a price. Dynamic SQL tends to be much slower than static SQL. Dynamic SQL also requires more computer resources (overhead). Finally, you are more likely to find inconsistent levels of support and incompatibilities among DBMS vendors.

Summary

- The ANSI standard data types are supported by all RDBMS vendors in different ways. The basic data types are numeric (e.g., NUMBER, NUMERIC, INTEGER), character (e.g., CHAR, VARCHAR), and DATE.
- The basic data definition commands allow you to create tables and indexes. Tables may also be created using SELECT statements. Many SQL constraints can be used with columns. The commands are CREATE TABLE, CREATE INDEX, ALTER TABLE, DROP TABLE, and DROP INDEX.
- Data manipulation commands allow you to add, modify, and delete rows from tables. The basic DML commands are SELECT (covered in Chapter 7), INSERT, UPDATE, and DELETE.
- The INSERT command is used to add new rows to tables. The UPDATE command is used to modify data values in existing rows of a table. The DELETE command is used to delete rows from tables. The COMMIT and ROLLBACK commands are used to permanently save or roll back changes made to the rows. After you COMMIT the changes, you cannot undo them with a ROLLBACK command.
- Views can be created to expose subsets of data to end users primarily for security and privacy reasons. Normally, views only store the SELECT statement to produce the view. Materialized views store a separate copy of the data and must be refreshed regularly.
- In Oracle and SQL Server, sequences may be used to generate values to be assigned to a record. For example, a sequence may be used to number invoices automatically. MS Access uses an AutoNumber data type to generate numeric sequences, and MySQL uses the AUTO_INCREMENT property during table creation. Oracle and SQL Server can use the Identity column property to designate the column that will have sequential numeric values automatically assigned to it. There can only be one Identity column per table.
- Procedural SQL can be used to create triggers, stored procedures, and user defined functions. A trigger is procedural SQL code that is automatically invoked by the DBMS upon the occurrence of a specified data manipulation event (UPDATE, INSERT, or DELETE). Triggers are critical to proper database operation and management. They help automate various transaction and data management processes, and they can be used to enforce constraints that are not enforced at the DBMS design and implementation levels.
- A stored procedure is a named collection of SQL statements. Just like database triggers, stored procedures are stored in the database. One of the major advantages of stored procedures is that they can be used to encapsulate and represent complete business transactions. Use of stored procedures substantially reduces network traffic and increases system performance. Stored procedures also help reduce code duplication by creating unique modules that are called by the application programs, thereby minimizing the chance of errors and the cost of application development and maintenance.
- When SQL statements are designed to return more than one value inside the procedural SQL code, a cursor is needed. You can think of a cursor as a reserved area of memory in which the output of the query is stored, like an array holding columns and rows. Cursors are held in a reserved memory area in the DBMS server, rather than in the client computer. There are two types of cursors: implicit and explicit.
- Embedded SQL refers to the use of SQL statements within an application programming language such as Python, C#, COBOL, or Java. The language in which the SQL statements are embedded is called the host language. Embedded SQL is still the most common approach to maintaining procedural capabilities in DBMS-based applications.

Review Questions

1. What type of integrity is enforced when a primary key is declared?
2. Explain why it might be more appropriate to declare an attribute that contains only digits as a character data type instead of a numeric data type.
3. What is the difference between a column constraint and a table constraint?
4. What are “referential constraint actions”?
5. What is the purpose of a CHECK constraint?
6. Explain when an ALTER TABLE command might be needed.
7. What is the difference between an INSERT command and an UPDATE command?
8. What is the difference between using a subquery with a CREATE TABLE command and using a subquery with an INSERT command?
9. What is a sequence? Write its syntax.
10. What is a trigger, and what is its purpose? Give an example.
11. What is a stored procedure, and why is it particularly useful? Give an example.

Chapter 2: Database Design

Learning Outcomes

After this chapter you will be able to:

1. Describe the role of database design as the foundation of a successful information system
2. Describe the five phases in the Systems Development Life Cycle (SDLC)
3. Design databases using the six phases in the Database Life Cycle (DBLC) framework
4. Conduct evaluation and revision within the SDLC and DBLC frameworks
5. Distinguish between top-down and bottom-up approaches in database design
6. Distinguish between centralized and decentralized conceptual database design

The Information System

Basically, a database is a carefully designed and constructed repository of facts. The database is part of a larger whole known as an **information system (IS)**, which provides for data collection, storage, transformation, and retrieval. The information system also helps transform data into information, and it allows for the management of both data and information. Thus, a complete information system is composed of people, hardware, software, the database(s), application programs, and procedures. **Systems analysis** is the process that establishes the need for an information system and its extent. The process of creating an information system is known as **systems development**.
One key characteristic of current information systems is the strategic value of information in the age of global business. Therefore, information systems should always be aligned with the strategic business mission and goals; the view of isolated and independent information systems is no longer valid.

Within the framework of systems development, applications transform data into the information that forms the basis for decision making. Applications usually generate formal reports, tabulations, and graphic displays designed to produce insight from the information. Figure 2.1 illustrates that every application is composed of two parts, the data and the code (program instructions) by which the data is transformed into information. The data and code work together to represent real-world business functions and activities. At any given moment, physically stored data represents a snapshot of the business, but the picture is not complete without an understanding of the business activities represented by the code.

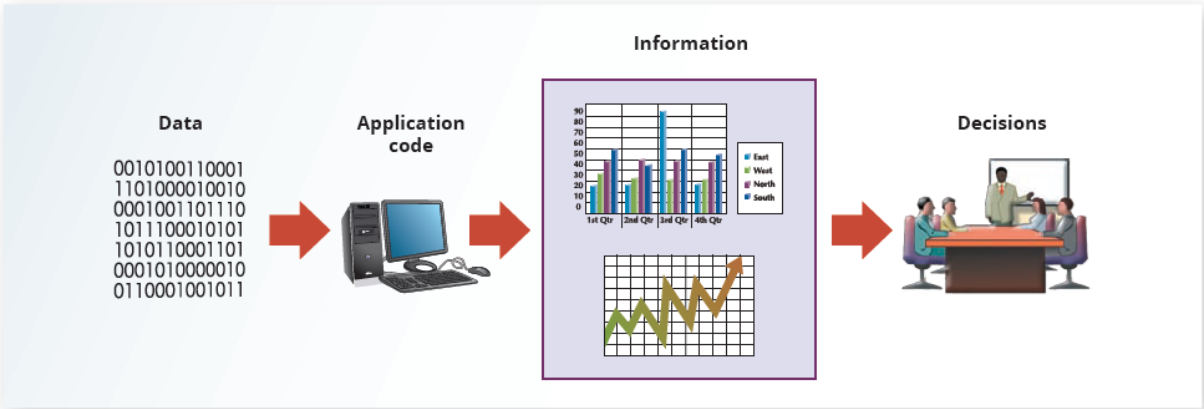


Figure 2-1: Generating Information for Decision Making (Coronel C., Morris S. 2022)

The performance of an information system depends on three factors:

- Database design and implementation
- Application design and implementation
- Administrative procedures

Creating a sound information system is hard work, systems analysis and development require extensive planning to ensure that all of the activities will interface with each other, that they will complement each other, and that they will be completed on time. In a broad sense, the term **database development** describes the process of database design and implementation. The primary objective in database design is to create complete, normalized, nonredundant (to the greatest extent possible), and fully integrated conceptual, logical, and physical database models. The implementation phase includes creating the database storage structure, loading data into the database, and providing for data management. Consideration should be taken to

design and implement a database that is flexible and scalable over time. Although most designs typically focus on solving current problems, it is important to create a design that is flexible enough to adapt to future changes (such as performance, size, or reporting requirements). To make the procedures discussed in this chapter broadly applicable, the chapter focuses on the elements that are common to all information systems. Most of the processes and procedures described in this chapter do not depend on the size, type, or complexity of the database being implemented. However, the procedures that would be used to design a small database, such as one for a neighborhood shoe store, do not precisely scale up to the procedures that would be needed to design a database for a large corporation or even a segment of such a corporation. To use an analogy, building a small house requires a blueprint, just as building the Golden Gate Bridge did, but the bridge required far more complex planning, analysis, and design.

The next sections will trace the overall Systems Development Life Cycle and the related Database Life Cycle. Once you are familiar with those processes and procedures, you will learn about various approaches to database design, such as top-down versus bottom-up and centralized versus decentralized design.

The Systems Development Life Cycle

The **Systems Development Life Cycle (SDLC)** traces the history of an information system. Perhaps more important to the system designer, the SDLC provides the big picture within which the database design and application development can be mapped out and evaluated. As illustrated in Figure 2.2, the traditional SDLC is divided into five phases; planning, analysis, detailed systems design, implementation, and maintenance. The SDLC is an iterative process rather than a sequential process. For example, the details of the feasibility study might help refine the initial assessment, and the details discovered during the user requirements portion of the SDLC might help refine the feasibility study. Because the Database Life Cycle fits into and resembles the SDLC, a brief description of the SDLC is in order.

Planning

The SDLC planning phase yields a general overview of the company and its objectives. An initial assessment of the information flow-and-extent requirements must be made during this discovery portion of the SDLC. Such an assessment should answer some important questions:

- *Should the existing system be continued?* If the information generator does its job well, there is no point in modifying or replacing it. To quote an old saying, “If it ain’t broke, don’t fix it.”
- *Should the existing system be modified?* If the initial assessment indicates deficiencies in the extent and flow of the information, minor (or even major) modifications might be needed. When considering modifications, the participants in the initial assessment must remember the distinction between wants and needs.
- *Should the existing system be replaced?* The initial assessment might indicate that the current system’s flaws are beyond fixing. Given the effort required to create a new system, a careful distinction between wants and needs is perhaps even more important in this case than it is when modifying the system.

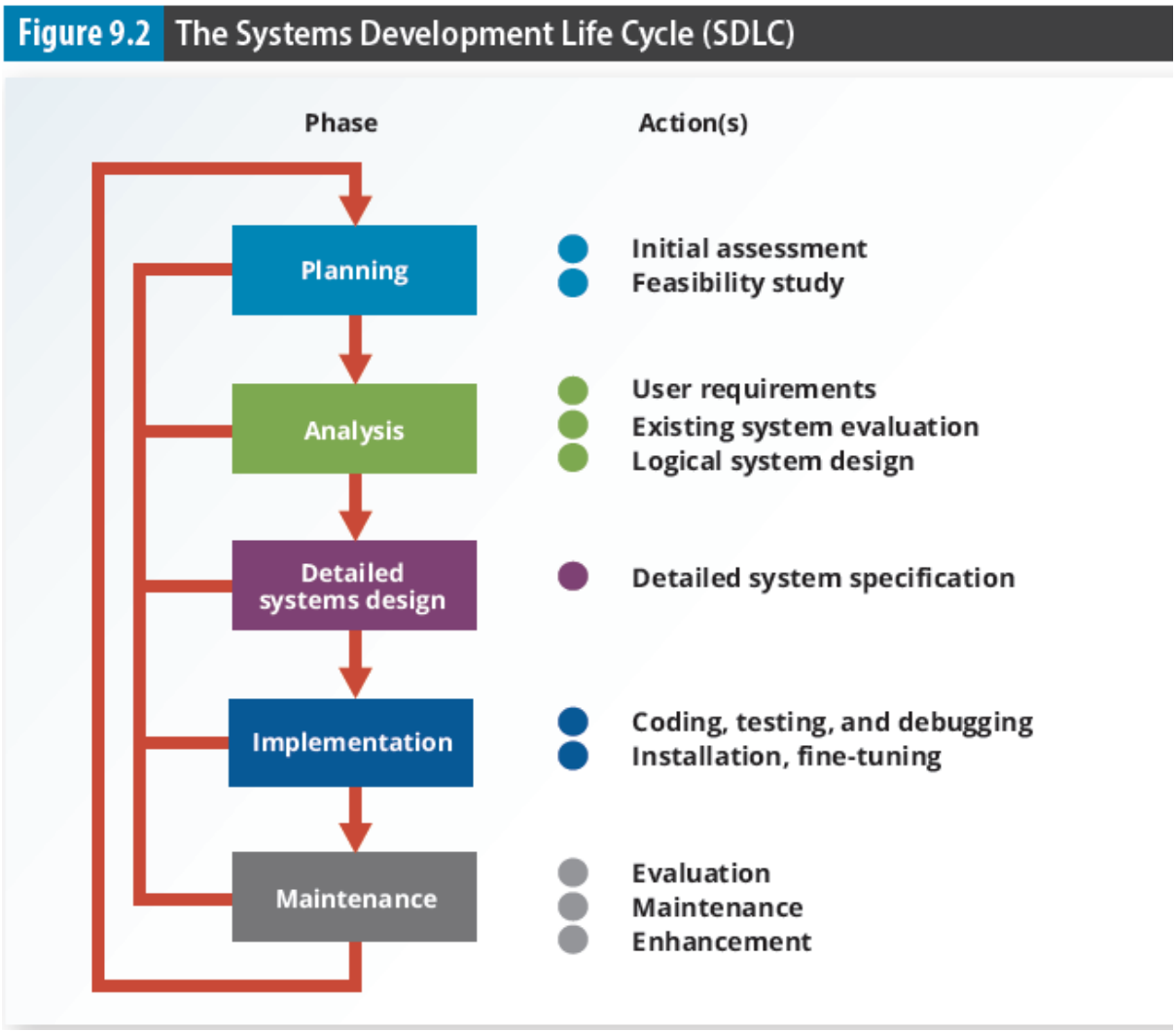


Figure 2-2: The Systems Development Life Cycle (SDLC) (Coronel C., Morris S. 2022)

Participants in the SDLC's initial assessment must begin to study and evaluate alternative solutions. If a new system is necessary, the next question is whether it is feasible. The feasibility study must address the following:

- *The technical aspects of hardware and software requirements*. The decisions might not yet be vendor-specific, but they must address the nature of the hardware requirements (desktop computer, mainframe, supercomputer, or mobile device) and the software requirements (single-user or multiuser operating systems, database type and software, programming languages to be used by the applications, and so on).
- *The system cost*. The admittedly mundane question "Can we afford it?" is crucial. The answer might force a careful review of the initial assessment. A million-dollar solution to a thousand-dollar problem is not defensible. At some point, the decision may be between building a system "in-house" or buying (and customizing) a third-party vendor system. In the long run, you need to find a cost-effective solution that best serves the needs (present and future) of the organization.
- *The operational cost*. Does the company possess the human, technical, and financial resources to keep the system operational? Should the feasibility study include the cost of management and end-user support needed to implement operational procedures to ensure the success of this system? What would be the impact of this new system in the company's culture? People's resistance to change should never be underestimated.

Even if you choose to “buy” rather than to “build,” the system implementation must be carefully planned for it to be successful. Whatever the chosen option (build or buy), an analysis must be done to deploy the solution across the organization in ways that minimize cost and culture changes, while maximizing value. The SDLC provides a framework for sound planning and implementation.

Analysis

Problems defined during the planning phase are examined in greater detail during the analysis phase. A macro analysis must be made both of individual needs and organizational needs, addressing questions such as:

- What are the requirements of the current system’s end users?
- Do those requirements fit into the overall information requirements?

The analysis phase of the SDLC is, in effect, a thorough *audit* of user requirements. The existing hardware and software systems are also studied during the analysis phase. The result of the analysis should be a better understanding of the system’s functional areas, actual and potential problems, and opportunities. End users and the system designer(s) must work together to identify processes and uncover potential problem areas. Such cooperation is vital to defining the appropriate performance objectives by which the new system can be judged.

Along with a study of user requirements and the existing systems, the analysis phase also includes the creation of a logical systems design. The logical design must specify the appropriate conceptual data model, inputs, processes, and expected output requirements. When creating a logical design, the designer might use tools such as system flowcharts, data flow diagrams (DFDs), process diagrams such as hierarchical input process output (HIPO) diagrams, context diagrams, use cases, entity relationship (ER) diagrams, and even some application prototypes. The database design’s data-modeling activities take place at this point to discover and describe all entities and their attributes, and the relationships among the entities within the database.

Defining the logical system also yields functional descriptions of the system’s components (modules) for each process within the database environment. All data transformations (processes) are described and documented, using systems analysis tools such as DFDs. The conceptual data model is validated against those processes.

Detailed Systems Design

In the detailed systems design phase, the designer completes the design of the system’s processes. The design includes all the necessary technical specifications for the screens, menus, reports, and other devices that might help make the system a more efficient information generator. The steps are laid out for conversion from the old system to the new system. Training principles and methodologies are also planned and must be submitted for management’s approval.

Note

When trying to develop solutions, the database designer must look for the source of the problems. Many database systems have failed to satisfy the end users because they were designed to treat the *symptoms* of the problems rather than their source.

Implementation

During the implementation phase, the hardware, DBMS software, and application programs are installed, and the database design is implemented. During the initial stages of the implementation phase, the system enters into a cycle of coding, testing, and debugging until it is ready to be delivered. The actual database is created, and the system is customized by the creation of tables and views, user authorizations, and so on. The database contents might be loaded interactively or in batch mode, using a variety of methods and devices:

- Customized user programs
- Database interface programs
- Conversion programs that import the data from a different file structure, using batch programs, a database utility, or both

The system is subjected to exhaustive testing until it is ready for use. Traditionally, the implementation and testing of a new system took 50 to 60 percent of the total development time. However, the advent of sophisticated application generators and debugging tools has substantially decreased coding and testing time. After testing is concluded, the final documentation is reviewed and printed and end users are trained. The system is in full operation at the end of this phase, but it will be continuously evaluated and fine-tuned.

Maintenance

Almost as soon as the system is operational, end users begin to request changes in it. Those changes generate system maintenance activities, which can be grouped into three types:

- *Corrective maintenance* in response to systems errors
- *Adaptive maintenance* due to changes in the business environment
- *Perfective maintenance* to enhance the system

Because every request for structural change requires retracing the SDLC steps, the system is, in a sense, always at some stage of the SDLC. Each system has a predetermined operational life span, but its actual life span depends on its perceived utility. There are several reasons for reducing the operational life of certain systems. Rapid technological change is one reason, especially for systems based on processing speed and expandability. Another common reason is the cost of maintaining a system. If the system's maintenance cost is high, its value becomes suspect. **Computer-aided software engineering (CASE)** tools, such as System Architect or Relational Rose, help produce better systems within a reasonable amount of time and at a reasonable cost. In addition, CASE-produced applications are more structured, better documented, and especially *standardized*, which tends to prolong the operational life of systems by making them easier and cheaper to update and maintain.

The Database Life Cycle

Within the larger information system, the database is subject to a life cycle as well. The **Database Life Cycle (DBLC)** contains six phases, as shown in Figure 2.3; database initial study, database design, implementation and loading, testing and evaluation, operation, and maintenance and evolution.

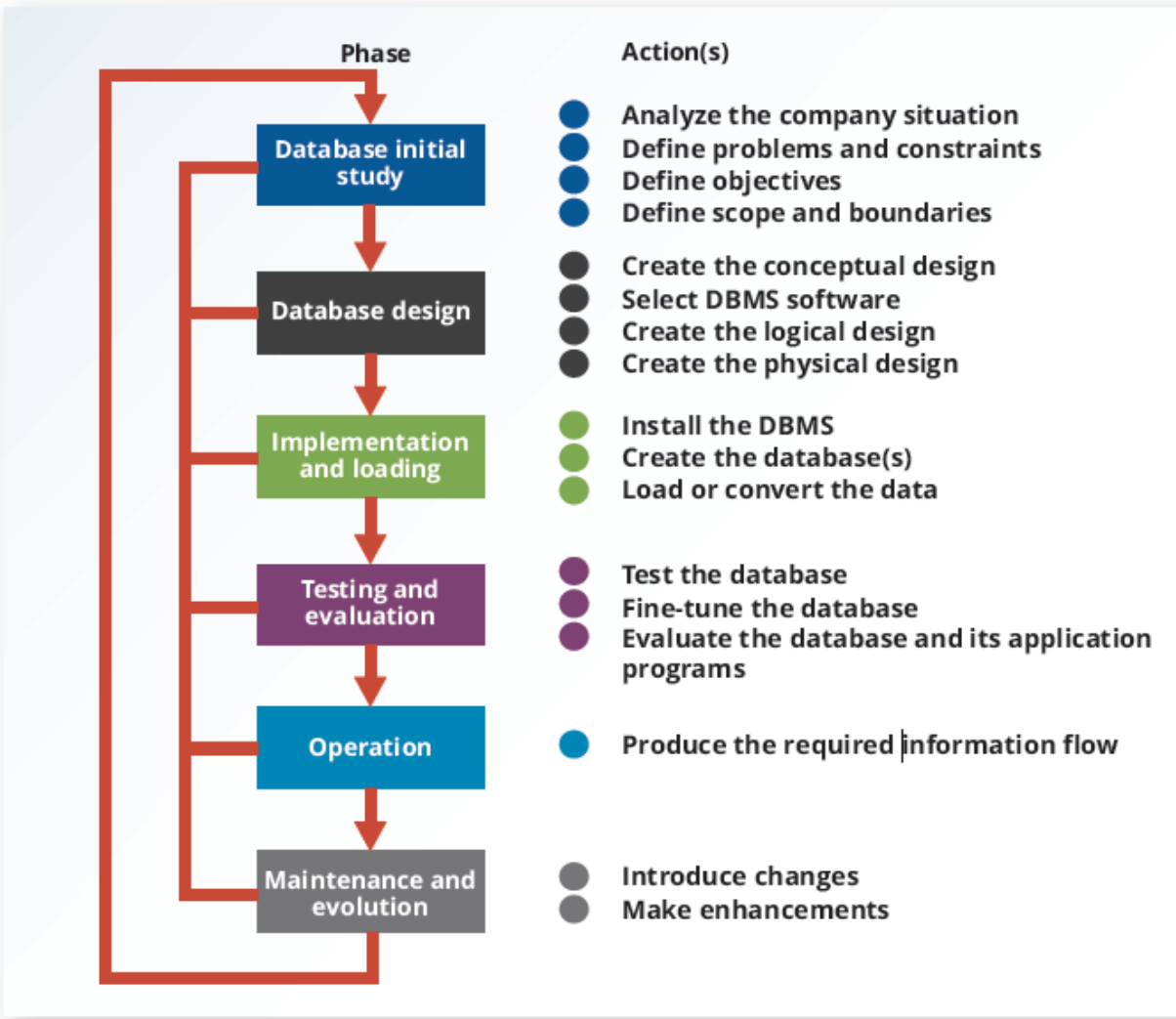


Figure 2-3: The Database Life Cycle (BDLC) (Coronel C., Morris S. 2022)

The Database Initial Study

If a designer has been called in, chances are that the current system has failed to perform functions deemed vital by the company. (You don't call the plumber unless the pipes leak.) Therefore, in addition to examining the current system's operation within the company, the designer must determine how and why the current system fails. That means spending a lot of time talking and listening to end users. Although database design is a technical business, it is also people-oriented. Database designers must be excellent communicators and must have finely tuned interpersonal skills. Depending on the complexity and scope of the database environment, the database designer might be a lone operator or part of a systems development team composed of a project leader, one or more senior systems analysts, and one or more junior

systems analysts. The word *designer* is used generically here to cover a wide range of design team compositions. The overall purpose of the database initial study is to:

- Analyze the company situation
- Define problems and constraints
- Define objectives
- Define scope and boundaries

Figure 2.4 depicts the interactive and iterative processes required to complete the first phase of the DBLC successfully. Note that the database initial study phase leads to the development of database system objectives. Using Figure 2.4 as a discussion template, examine each of its components in greater detail.

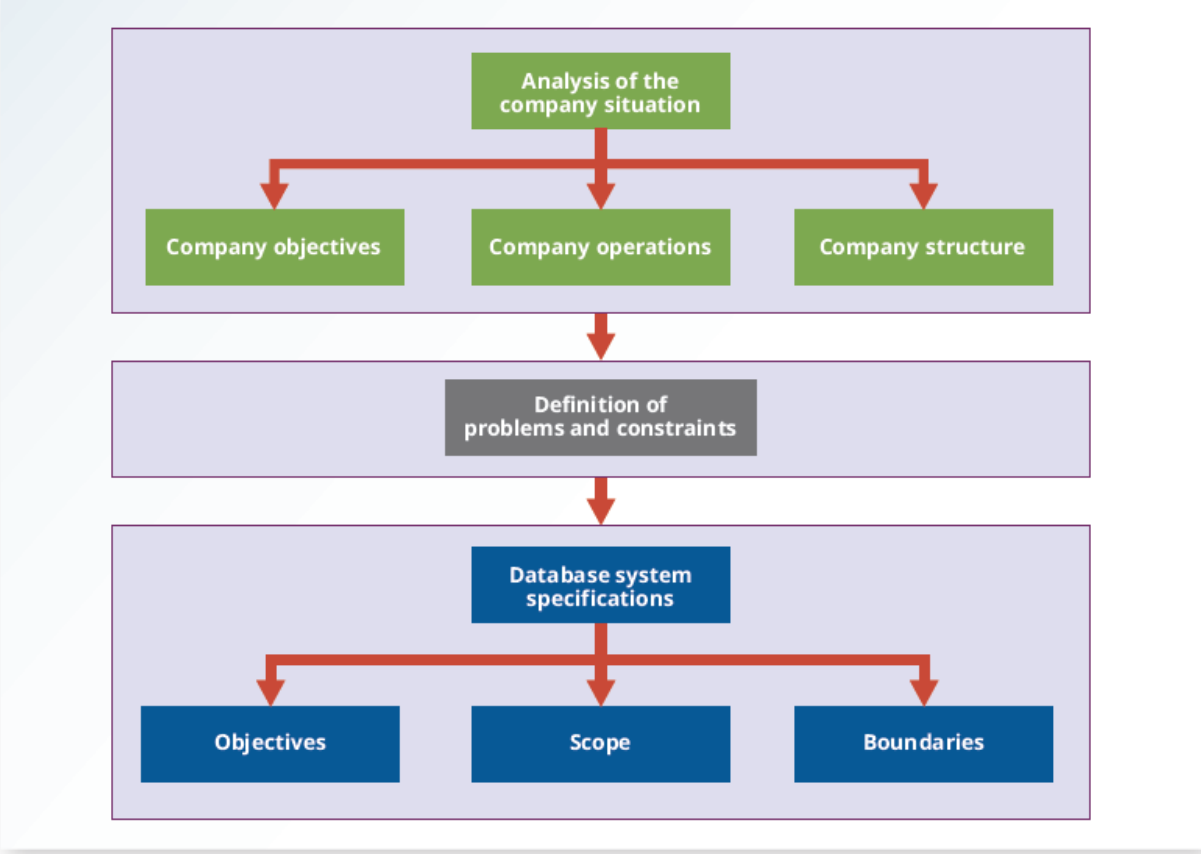


Figure 2-4: A summary of activities in the database initial study (Coronel C., Morris S. 2022)

Analyze the Company Situation

The *company situation* describes the general conditions in which a company operates, its organizational structure, and its mission. To analyze the company situation, the database designer must learn the company's operational components, how they function, and how they interact. The following issues must be resolved:

- *What is the organization's general operating environment, and what is its mission within that environment?* The design must satisfy the operational demands created by the organization's mission. For example, a mail-order business probably has operational requirements for its database that are quite different from those of a manufacturing business.
- *What is the organization's structure?* Knowing who controls what and who reports to whom is quite useful when you need to define required information flows, specific report and query formats, and so on.

Define Problems and Constraints

The designer has both formal and informal sources of information. If the company has existed for any length of time, it already has a system in place (either manual or computer-based). How does the existing system function? What input does the system require? What documents does the system generate? By whom and how is the system output used? Studying the paper trail can be very informative. In addition to the official version of the system's operation, there is also the more informal, perhaps more real version; the designer must be shrewd enough to see how these differ. The process of defining problems might initially appear to be unstructured. Company end users often cannot precisely describe the larger scope of company operations or identify the real problems encountered during company operations. Often the managerial view of a company's operation and its problems is different from that of the end users who perform the actual routine work.

After the initial declarations, the database designer must continue to probe carefully to generate additional information that will help define the problems within the larger framework of company operations. How does the problem of the marketing manager's customer fit within the broader set of marketing department activities? How does the solution to the customer's problem help meet the objectives of the marketing department and the rest of the company? How do the marketing department's activities relate to those of the other departments? That last question is especially important. Note that there are common threads in the problems described by the marketing and production department managers. If the inventory query process can be improved, both departments are likely to find simple solutions to at least some of their problems.

Finding precise answers is important, especially concerning the operational relationships among business units. If a proposed system will solve the marketing department's problems but exacerbate those of the production department, not much progress will have been made. Using an analogy, suppose that your home water bill is too high. You have determined the problem, the faucets leak. The solution? You step outside and cut off the water supply to the house. However, is that an adequate solution, or would the replacement of faucet washers do a better job of solving the problem? You might find this scenario simplistic, yet almost any experienced database designer can find similar instances of database problem solving, although they are admittedly more complicated. Even the most complete and accurate problem definition does not always lead to the perfect solution. The real world usually intrudes to limit the design of even the most elegant database by imposing constraints such as time, budget, and personnel. If you must have a solution within a month and within a $12,000 budget, you cannot take two years to develop a database at a cost of $100,000. *The designer must learn to distinguish between what's perfect and what's possible*.

Define Objectives

A proposed database system must be designed to help solve at least the major problems identified during the problem discovery process. As the list of problems unfolds, several common sources are likely to be discovered. In the previous example, both the marketing manager and the production manager seem to be plagued by inventory inefficiencies. If the designer can create a database that sets the stage for more efficient parts management, both departments gain. The initial objective, therefore, might be to create an efficient inventory query and management system. Note that the initial study phase also yields proposed problem solutions. The designer's job is to make sure that his or her database system objectives correspond to those envisioned by the end user(s). In any case, the database designer must begin to address the following questions:

- What is the proposed system's initial objective?
- Will the system interface with other existing or future systems in the company?
- Will the system share the data with other systems or users?

Define Scope and Boundaries

The designer must recognize two sets of limits, scope and boundaries. The system's **scope** defines the extent of the design according to operational requirements. Will the database design encompass the entire organization, one or more departments within the organization, or one or more functions of a single department? The designer must know the "size of the ballpark." Knowing the scope helps define the required data structures, the type and number of entities, the physical size of the database, and so on. The proposed system is also subject to limits known as **boundaries**, which are external to the system. Has any designer ever been told, "We have all the time in the world" or "Use an unlimited budget and as many people as needed to make the design come together"? Boundaries are also imposed by existing hardware and software. Ideally, the designer can choose the hardware and software that will best accomplish the system goals. In fact, software selection is an important aspect of the Systems Development Life Cycle. Unfortunately, in the real world, a system must often be designed around existing hardware. Thus, the scope and boundaries become the factors that force the design into a specific mold, and the designer's job is to design the best system possible within those constraints. (Note that problem definitions and the objectives must sometimes be reshaped to meet the system scope and boundaries.)

Database Design

The second phase of the DBLC focuses on the design of the database model that will support company operations and objectives. This is arguably the most critical DBLC phase, making sure that the final product meets user and system requirements. In the process of database design, you must concentrate on the data characteristics required to build the database model. At this point, there are two views of the data within the system, the business view of data as a source of information and the designer's view of the data structure, its access, and the activities required to transform the data into information. Figure 2.5 contrasts those views. Note that you can summarize the different views by looking at the terms *what* and *how*. Defining data is an integral part of the DBLC's second phase.

Figure 9.5 Two Views of Data: Business Manager and Designer

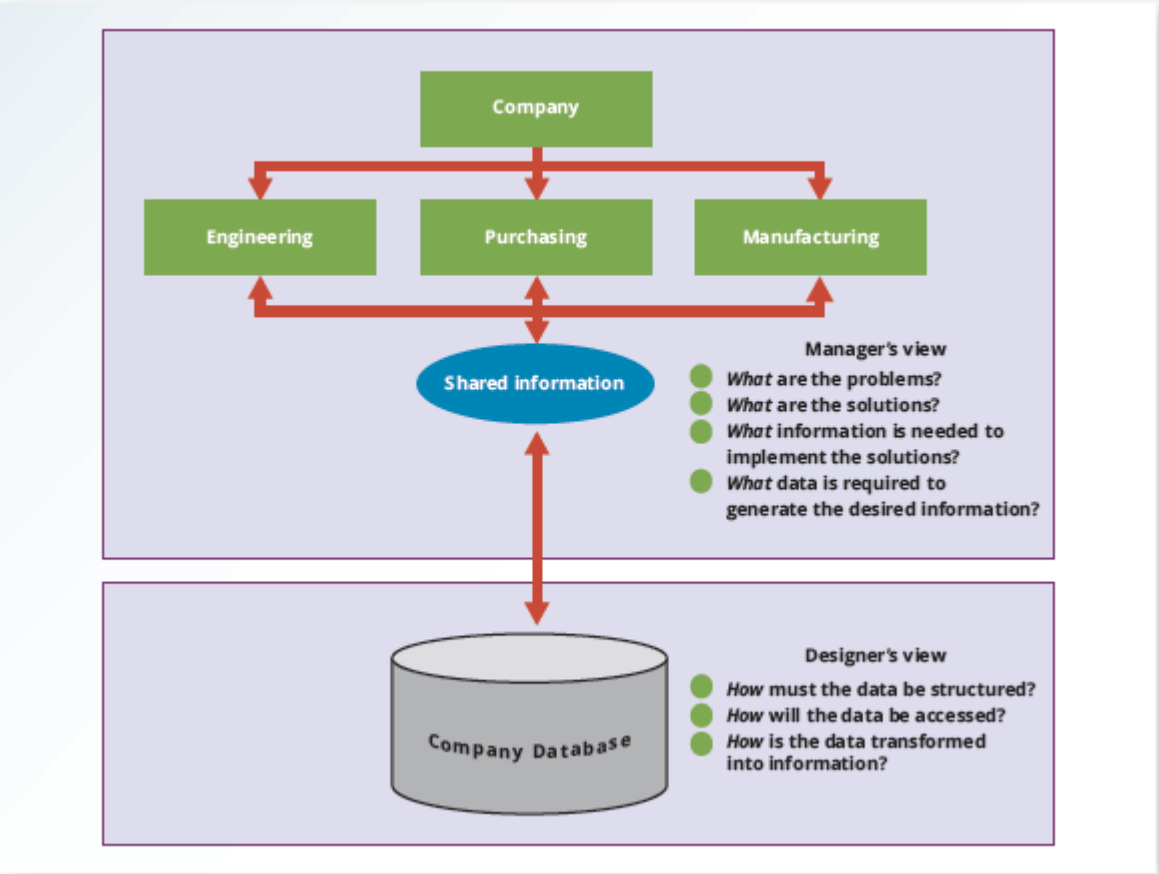


Figure 2-5: Two views of data: Business Manager and Designer (SDLC) (Coronel C., Morris S. 2022)

As you examine the procedures required to complete the design phase in the DBLC, remember these points:

- The process of database design is loosely related to the analysis and design of a larger system. The data component is only one element of a larger information system.
- The systems analysts or systems programmers are in charge of designing the other system components. Their activities create the procedures that will help transform the data within the database into useful information.
- The database design does not constitute a sequential process. Rather, it is an iterative process that provides continuous feedback designed to trace previous steps. The database design process is depicted in Figure 9.6.

The figure shows that there are three essential stages, conceptual, logical, and physical design, plus the DBMS selection decision, which is critical to determine the type of logical and physical designs to be created. The design process starts with conceptual design and moves to the logical and physical design stages. At each stage, more details about the data model design are determined and documented. You could think of the conceptual design as the overall data as seen by the end user, the logical design as the data as seen by the DBMS, and the physical design as the data as seen by the operating system's storage management devices. The overwhelming majority of database designs and implementations are based on the relational model, and therefore use the relational model constructs and techniques. When you finish the design activities, you will have a complete database design ready to be implemented.

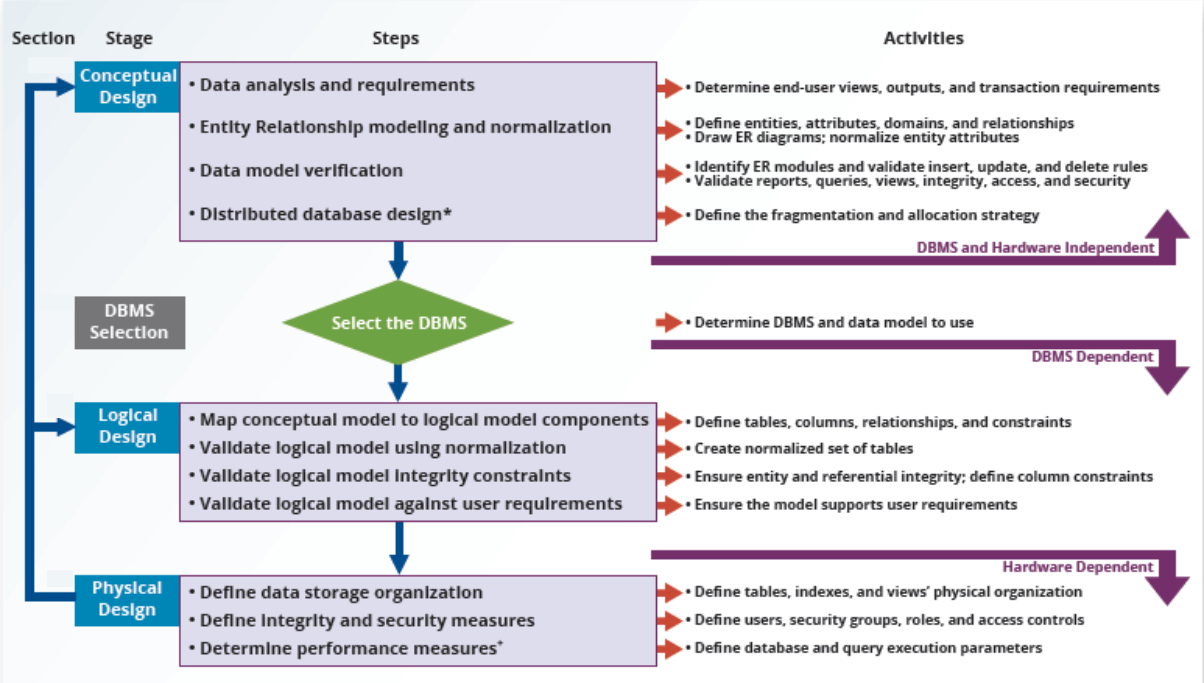


Figure 2-6: Database Design Process (Coronel C., Morris S. 2022)

Implementation and Loading

The output of the database design phase is a series of instructions detailing the creation of tables, attributes, domains, views, indexes, security constraints, and storage and performance guidelines. In this phase, you actually implement all these design specifications.

Install the DBMS

This step is required only when a new dedicated instance of the DBMS is necessary for the system. In many cases, the organization will have made a particular DBMS the standard to leverage investments in the technology and the skills that employees have already developed. The DBMS may be installed on a new server or on existing servers. One current trend is called virtualization. **Virtualization** is a technique that creates logical representations of computing resources that are independent of the underlying physical computing resources. This technique is used in many areas of computing, such as the creation of virtual servers, virtual storage, and virtual private networks. In a database environment, database virtualization refers to the installation of a new instance of the DBMS on a virtual server running on shared hardware. This is normally a task that involves system and network administrators to create appropriate user groups and services in the server configuration and network routing. Another common trend is the use of cloud database services such Microsoft Azure SQL Database Service or Amazon Relational Database Services (RDS). This new generation of services allows users to create databases that could be easily managed, tested, and scaled up as needed.

Create the Database(s)

In most modern relational DBMSs, a new database implementation requires the creation of special storage-related constructs to house the end-user tables. The constructs usually include the storage group (or file groups), the table spaces, and the tables. Figure 9.7 shows that a storage group can contain more than one table space and that a table space can contain more than one table. The implementation of the logical design would require the database administrator to create the storage groups, table spaces, tables, and finally assign access rights to the tables to specific users or groups. For example, using the following command, access rights to a table named PROFESSOR may be granted to the user Madison Hendley, whose identification code is mhendley:

GRANT SELECT ON PROFESSOR TO MHENDLEY;

Access rights may be limited to views rather than to whole tables. The creation of views is not required for database access in the relational environment, but views are desirable from a security standpoint.

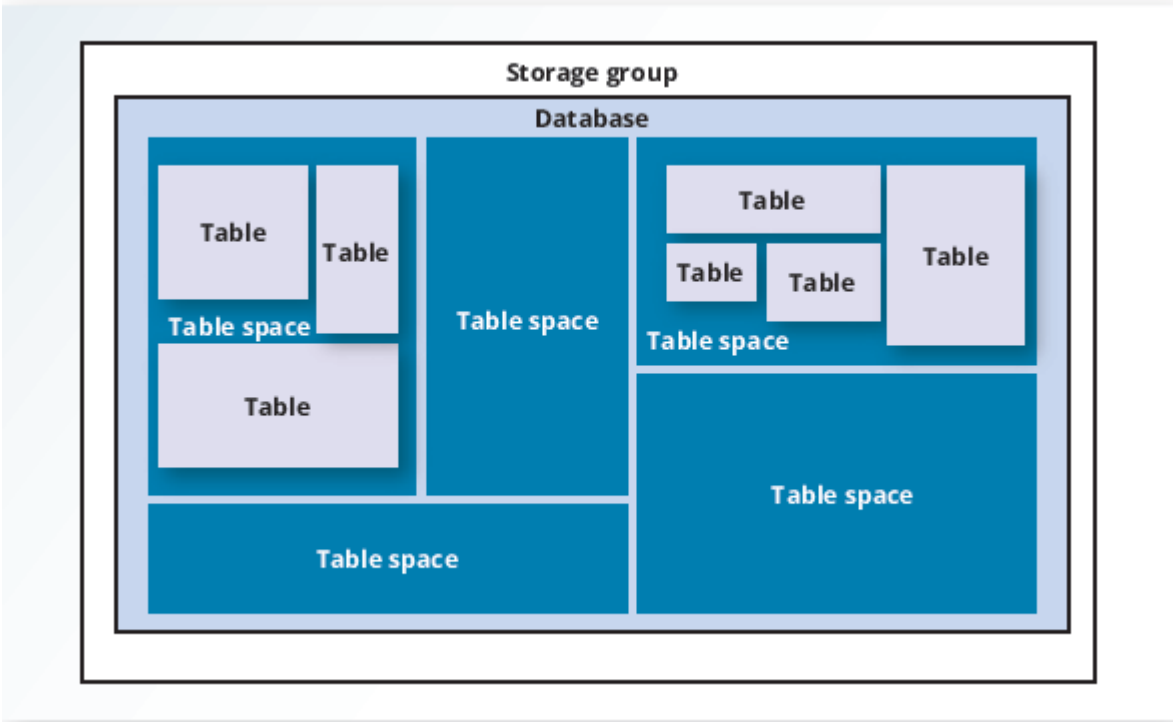


Figure 2-7: Physical Organization of a Database Environment (Coronel C., Morris S. 2022)

Load or Convert the Data

After the database has been created, the data must be loaded into the database tables. Typically, the data will have to be migrated from the prior version of the system. Often, data to be included in the system must be aggregated from multiple sources. In a best-case scenario, all of the data will be in a relational database so that it can be readily transferred to the new database. However, in some cases data may have to be imported from other relational databases, nonrelational databases, flat files, legacy systems, or even manual paper-and-pencil systems. If the data format does not support direct importing into the new database, conversion programs may have to be created to reformat the data for importing. In a worst-case scenario, much of the data may have to be manually entered into the database. Once the data has been loaded, the DBA works with the application developers to test and evaluate the database. Loading existing data into a

cloud-based database service sometimes can be expensive. The reason for this is that most cloud services are priced based not only on the volume of data to be stored but also on the amount of data that travels over the network. In such cases, loading a 1 TB database could be a very expensive proposition. Therefore, system administrators must be very careful in reading and negotiating the terms of cloud service contracts to ensure that there will be no "hidden" costs.

Testing and Evaluation

In the design phase, decisions were made to ensure integrity, security, performance, and recoverability of the database. During implementation and loading, these plans were put into place. In testing and evaluation, the DBA tests and fine-tunes the database to ensure that it performs as expected. This phase occurs in conjunction with application programming. Programmers use database tools to *prototype* the applications during coding of the programs. Tools such as report generators, screen painters, and menu generators are especially useful to application programmers.

Test the Database

During this step, the DBA tests the database to ensure that it maintains the integrity and security of the data. Data integrity is enforced by the DBMS through the proper use of primary and foreign key rules. Many DBMSs also support the creation of domain constraints and database triggers. Testing will ensure that these constraints were properly designed and implemented. Data integrity is also the result of properly implemented data management policies, which are part of a comprehensive data administration framework. For a more detailed study of this topic, see "The DBA's Managerial Role" section in Chapter 16, Database Administration and Security. Previously, users and roles were created to grant users access to the data. In this stage, not only must those privileges be tested, but the broader view of data privacy and security must be addressed. Data stored in the company database must be protected from access by unauthorized users. (It does not take much imagination to predict the likely results if students have access to a student database or if employees have access to payroll data!) Consequently, you must test for at least the following:

- *Physical security* allows only authorized personnel physical access to specific areas. Depending on the type of database implementation, however, establishing physical security might not always be practical. For example, a university student research database is not a likely candidate for physical security.
- *Password security* allows the assignment of access rights to specific authorized users. Password security is usually enforced at login time at the operating system level. *Access rights* can be established through the use of database software. The assignment of access rights may restrict operations (CREATE, UPDATE, DELETE, and so on) on predetermined objects such as databases, tables, views, queries, and reports.
- *Audit trails* are usually provided by the DBMS to check for access violations. Although the audit trail is an after-the-fact device, its mere existence can discourage unauthorized use.
- *Application code and database security*. Test the application for code vulnerabilities, such as SQL injections, and unnecessary privileges. The database group should work closely with the network and security groups to build the necessary safeguards to protect the application.
- *Data encryption* can render data useless to unauthorized users who might have violated some of the database security layers.

- *Diskless workstations* allow end users to access the database without being able to download the information from their workstations.

Fine-Tune the Database

Database performance can be difficult to evaluate, but it is typically one of the most important factors in database implementation. Different systems will place different performance requirements on the database. Typical performance indicators are query throughput (the number of concurrent queries supported), query response time (the average amount of time a query takes to complete), and query wait time (the average amount of time a query waits in queue). Customer-facing systems typically require superior performance during high volumes of inserts, updates, and deletes. Other systems, like decision support systems, may require superior performance for complex data retrieval tasks. Many factors can affect the database's performance on various tasks, including the hardware and software environment in which the database exists. Naturally, the characteristics and volume of the data also affect database performance—a search of 10 tuples is faster than a search of 100,000 tuples. Other important factors in database performance include system and database configuration parameters such as data placement, access path definition, the use of indexes, and buffer size.

Evaluate the Database and Its Application Programs

As the database and application programs are created and tested, the system must also be evaluated using a more holistic approach. Testing and evaluation of the individual components should culminate in a variety of broader system tests to ensure that all of the components interact properly to meet the needs of the users. At this stage, integration issues and deployment plans are refined, user training is conducted, and system documentation is finalized. Once the system receives final approval, it must be a sustainable resource for the organization. To ensure that the data contained in the database is protected against loss, backup and recovery plans are tested. Timely data availability is crucial for almost every database. Unfortunately, the database can lose data through unintended deletions, power outages, and other causes. Data backup and recovery procedures create a safety valve, ensuring the availability of consistent data. Typically, database vendors encourage the use of fault-tolerant components such as uninterruptible power supply (UPS) units, RAID storage devices, clustered servers, and data replication technologies to ensure the continuous operation of the database in case of a hardware failure. Even with these components, backup and restore functions constitute a very important part of daily database operations. Some DBMSs provide functions that allow the database administrator to schedule automatic database backups to permanent storage devices such as disks, DVDs, tapes, and online storage. Database backups can be performed at different levels:

- A **full backup**, or *dump*, of the entire database. In this case, all database objects are backed up in their entirety.
- A **differential backup** of the database, in which only the objects that have been updated or modified since the last full backup are backed up.
- A **transaction log backup**, which backs up only the transaction log operations that are not reflected in a previous backup copy of the database. In this case, no other database objects are backed up.

The database backup is stored in a secure place, usually in a different building from the database itself, and is protected against dangers such as fire, theft, flood, and other potential calamities. The main purpose of the backup is to guarantee database restoration following a hardware or software failure. Failures that plague databases and systems are generally induced by software,

hardware, programming exemptions, transactions, or external factors. Table 2.1 summarizes the most common sources of database failure.

Table 2.1: Common Sources of Database Failure

Source	Description	Example
Software	Software-induced failures may be traceable to the operating system, the DBMS software, application programs, or viruses and other malware.	A vulnerability found in the Oracle E-Business Suite allowed an unauthenticated attacker to create, modify, or delete critical data.[6]
Hardware	Hardware-induced failures may include memory chip errors, disk crashes, bad disk sectors, and disk-full errors.	A bad memory module or a multiple hard disk failure in a database system can bring it to an abrupt stop.
Programming exemptions	Application programs or end users may roll back transactions when certain conditions are defined. Programming exemptions can also be caused by malicious or improperly tested code that can be exploited by hackers.	Recently, a group of unidentified hackers fraudulently instructed the New York Federal Reserve Bank to transfer $81 million from the central bank of Bangladesh to accounts in the Philippines. The hackers used fraudulent messages injected by malware disguised as a PDF reader.[7]
Transactions	The system detects deadlocks and aborts one of the transactions. (See Chapter 10.)	Deadlock occurs when executing multiple simultaneous transactions.
External factors	Power Backup generators are especially important when a system suffers from external factors such as an earthquake, flood, or other natural disaster.	In the past few years, Louisiana has experienced several hurricanes that caused general power interruptions. The outages affected IT service providers as well as general businesses.

Depending on the type and extent of the failure, the recovery process ranges from a minor short-term inconvenience to a major long-term rebuild. Regardless of the extent of the required recovery process, recovery is not possible without a usable backup. Database recovery generally follows a predictable scenario. First, the type and extent of the required recovery are determined. If the entire database needs to be recovered to a consistent state, the recovery uses the most recent backup copy of the database in a known consistent state. The backup copy is then rolled forward to restore all subsequent transactions by using the transaction log information. If the database needs to be recovered but the committed portion of the database is still usable, the recovery process uses the transaction log to "undo" all of the transactions that were not committed. At the end of this phase, the database completes an iterative process of testing, evaluation, and modification that continues until the system is certified as ready to enter the Operational phase.

Operation

Once the database has passed the evaluation and testing stage, it is considered operational. At that point, the database, its management, its users, and its application programs constitute a complete information system. The beginning of the operational phase invariably starts the process of system evolution. As soon as all of the targeted end users have entered the operations phase, problems that could not have been foreseen during the testing phase begin to surface. Some of the problems are serious enough to warrant emergency "patchwork," while others are merely minor annoyances. For example, if the database design is implemented to interface with the web, the sheer volume of transactions might cause even a well-designed system to bog down. In that case, the designers have to identify the source of the bottleneck and produce alternative solutions. Those solutions may include using load-balancing software to distribute the transactions among multiple computers, increasing the available cache for the DBMS, and so on. Another critical aspect of an operational database system is compliance. The database and its data must comply with ever changing state and federal regulations regarding data confidentiality, privacy, intellectual property, etc. The demand for change is the designer's constant concern, which leads to phase 6, maintenance and evolution.

Maintenance and Evolution

The database administrator must be prepared to perform routine maintenance activities within the database. Some of the required periodic maintenance activities include:

- Preventive maintenance (backup)
- Corrective maintenance (recovery)
- Adaptive maintenance (enhancing performance, adding entities and attributes, and so on)
- Assignment of access permissions and their maintenance for new and old users
- Generation of database access statistics to improve the efficiency and usefulness of system audits and to monitor system performance
- Periodic security audits based on the system-generated statistics
- Monthly, quarterly, or yearly system usage summaries for internal billing or budgeting purposes

The likelihood of new information requirements and the demand for additional reports and new query formats require application changes and possible minor changes in the database components and contents. These changes can be easily implemented only when the database design is flexible and when all documentation is updated and online. Eventually, even the best-designed database environment will no longer be capable of incorporating such evolutionary changes, and then the whole DBLC process begins anew. As you can see, many of the activities described in the DBLC are similar to those in the SDLC. This should not be surprising because the SDLC is the framework within which the DBLC activities take place. A summary of the parallel activities that occur within the SDLC and DBLC is shown in Figure 2.8.

Conceptual Design

Recall that the second phase of the DBLC is database design, which comprises three stages, conceptual design, logical design, and physical design, plus the critical decision of DBMS selection. **Conceptual design** is the first stage in the database design process. The goal at this stage is to design a database that is independent of database software and physical details. The output of this process is a conceptual data model that describes the main data entities, attributes, relationships, and constraints of a given problem domain. This design is descriptive and narrative in form. In other words, it is generally composed of a graphical representation as well as textual descriptions of the main data elements, relationships, and constraints. In this stage, data modeling is used to create an abstract database structure that represents real-world objects in the most realistic way possible. The conceptual model must embody a clear understanding of the business and its functional areas. At this level of abstraction, the type of hardware and database model to be used might not have been identified yet. Therefore, the design must be software- and hardware-independent so that the system can be set up within any platform chosen later.

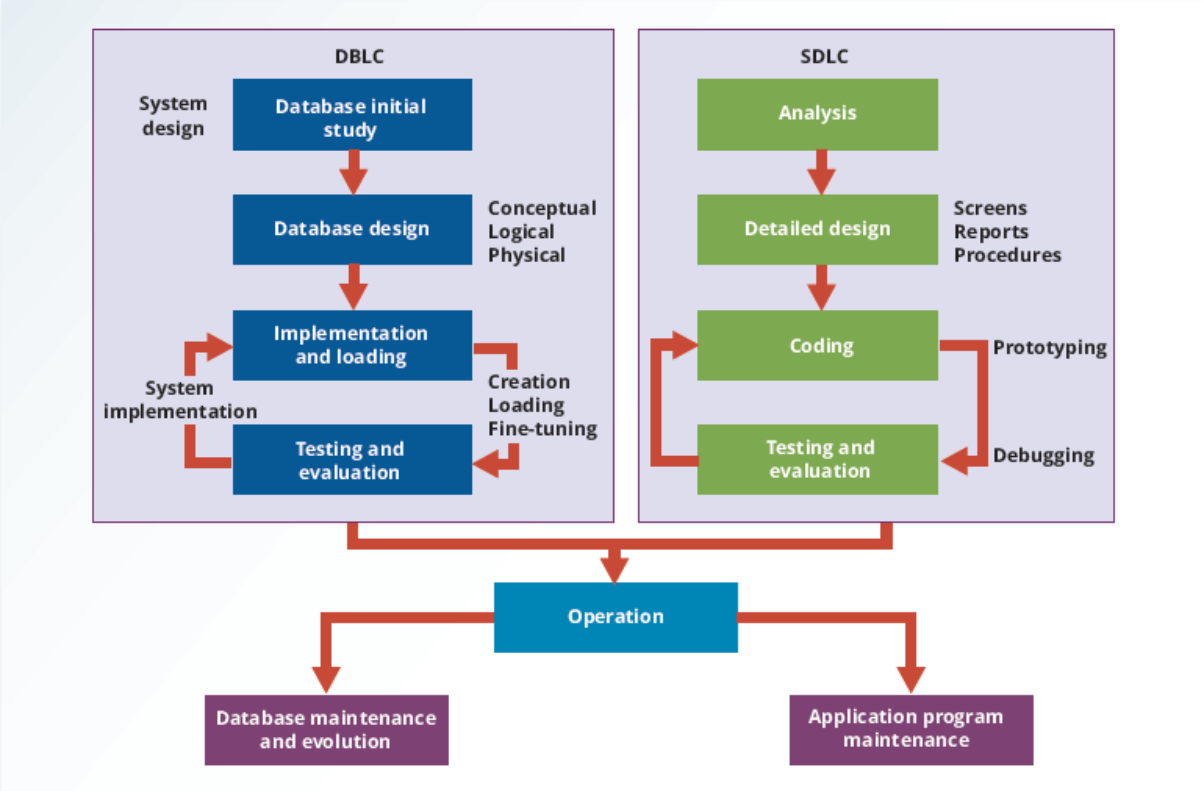


Figure 2-8: Parallel Activities in the DBLC and the SDLC (Coronel C., Morris S. 2022)

Keep in mind the following **minimal data rule**:
All that is needed is there, and all that is there is needed. In other words, make sure that all data needed is in the model and that all data in the model is needed. All data elements required by the database transactions must be defined in the model, and all data elements defined in the model must be used by at least one database transaction.

However, as you apply the minimal data rule, avoid excessive short-term bias. Focus not only on the immediate data needs of the business but on future data needs. Thus, the database design must leave room for future modifications and additions, ensuring that the business's investment in information resources will endure. The conceptual design has four steps, which are listed in Table 2.2.

Table 2.2: Conceptual Design Steps

STEP	ACTIVITY
1	Data analysis and requirements
2	Entity relationship modeling and normalization
3	Data model verification
4	Distributed database design

The following sections cover these steps in more detail.

Data Analysis and Requirements

The first step in conceptual design is to discover the characteristics of the data elements. An effective database is an information factory that produces key ingredients for successful decision making. Appropriate data element characteristics are those that can be transformed into appropriate information. Therefore, the designer's efforts are focused on:
Information needs. What kind of information is needed? That is, what output (reports and queries) must be generated by the system, what information does the current system generate, and to what extent is that information adequate?

- *Information users*. Who will use the information? How is the information to be used? What are the various end-user data views?
- *Information sources*. Where is the information to be found? How is the information to be extracted once it is found?
- *Information constitution*. What data elements are needed to produce the information? What are the data attributes? What relationships exist in the data? What is the data volume? How frequently is the data used? What data transformations will be used to generate the required information?

The designer obtains the answers to those questions from a variety of sources to compile the necessary information:

- *Developing and gathering end-user data views*. The database designer and the end user(s) jointly develop a precise description of end-user data views, which in turn are used to help identify the database's main data elements.
- *Directly observing the current system—existing and desired output*. The end user usually has an existing system in place, whether it is manual or computer-based. The designer reviews the existing system to identify the data and its characteristics. The designer examines the input forms and files (tables) to discover the data type and volume. If the end user already has an automated system in place, the designer carefully examines the current and desired reports to describe the data required to support the reports.
- *Interfacing with the systems design group*. As noted earlier in this chapter, the database design process is part of the SDLC. In some cases, the systems analyst in charge of designing the new system will also develop the conceptual database model. (This is usually true in a decentralized environment.) In other cases, the database design is considered part of the DBA's job. The presence of a DBA usually implies the existence of a formal data-processing department. The DBA designs the database according to the specifications created by the systems analyst.

To develop an accurate data model, the designer must have a thorough understanding of the company's data types and their extent and uses. But data does not, by itself, yield the required understanding of the total business. From a database point of view, the collection of data becomes meaningful only when business rules are defined.

Data Models, that a *business rule* is a brief and precise description of a policy, procedure, or principle within a specific organization's environment. Business rules, derived from a detailed description of an organization's operations, help to create and enforce actions within that organization's environment. When business rules are written properly, they define entities, attributes, relationships, connectivities, cardinalities, and constraints. To be effective, business rules must be easy to understand, and they must be widely disseminated to ensure that every person in the organization shares a common interpretation of the rules. Using simple language, business rules describe the main and distinguishing characteristics of the data as *viewed by the company*. Examples of business rules are as follows:

- A customer may make many payments on an account.
- Each payment on an account is credited to only one customer.
- A customer may generate many invoices.
- Each invoice is generated by only one customer.

Given their critical role in database design, business rules must not be established casually. Poorly defined or inaccurate business rules lead to database designs and implementations that fail to meet the needs of the organization's end users. Ideally, business rules are derived from a formal **description of operations**, which is a document that provides a precise, up-to-date, and thoroughly reviewed description of the activities that define an organization's operating environment. (To the database designer, the operating environment is both the data sources and the data users.) Naturally, an organization's operating environment is dependent on the organization's mission. For example, the operating environment of a university would be quite different from that of a steel manufacturer, an airline, or a nursing home. Yet, no matter how different the organizations may be, the *data analysis and requirements* component of the database design is enhanced when the data environment and data use are described accurately and precisely within a description of operations.

In a business environment, the main sources of information for the description of operations—and therefore of business rules—are company managers, policymakers, department managers, and written documentation such as company procedures, standards, and operations manuals. A faster and more direct source of business rules is direct interviews with end users. Unfortunately, because perceptions differ, the end user can be a less reliable source when it comes to specifying business rules. For example, a maintenance department mechanic might believe that any mechanic can initiate a maintenance procedure, when actually only mechanics with inspection authorization should perform such a task. This distinction might seem trivial, but it has major legal consequences. Although end users are crucial contributors to the development of business rules, it pays to verify end-user perceptions. Often, interviews with several people who perform the same job yield very different perceptions of their job components. While such a discovery might point to "management problems," that general diagnosis does not help the database designer. Given the discovery of such problems, the database designer's job is to reconcile the differences and verify the results of the reconciliation to ensure that the business rules are appropriate and accurate.

Knowing the business rules enables the designer to fully understand how the business works and what role the data plays within company operations. Consequently, the designer must identify the company's business rules and analyze their impact on the nature, role, and scope of data. Business rules yield several important benefits in the design of new systems:

- They help standardize the company's view of data.
- They constitute a communications tool between users and designers.
- They allow the designer to understand the nature, role, and scope of the data.
- They allow the designer to understand business processes.
- They allow the designer to develop appropriate relationship participation rules and foreign key constraints.

The last point is especially noteworthy; whether a given relationship is mandatory or optional is usually a function of the applicable business rule.

Entity Relationship Modeling and Normalization

Before creating the ER model, the designer must communicate and enforce appropriate standards to be used in the documentation of the design. The standards include the use of diagrams and symbols, documentation writing style, layout, and any other conventions to be followed during documentation. Designers often overlook this very important requirement, especially when they are working as members of a design team. Failure to standardize documentation often means a failure to communicate later, and communications failures often lead to poor design work. In contrast, well-defined and enforced standards make design work easier and promise (but do not guarantee) a smooth integration of all system components. Because the business rules usually define the nature of the relationship(s) among the entities, the designer must incorporate them into the conceptual model. The process of defining business rules and developing the conceptual model using ER diagrams can be described using the steps shown in Table 2.3.

Table 2.3 : Developing the Conceptual Model Using ER Diagrams

STEP	ACTIVITY
1	Identify, analyze, and refine the business rules.
2	Identify the main entities, using the results of Step 1.
3	Define the relationships among the entities, using the results of Steps 1 and 2.
4	Define the attributes, primary keys, and foreign keys for each of the entities.
5	Normalize the entities. (Remember that entities are implemented as tables in an RDBMS.)
6	Complete the initial ER diagram.
7	Validate the ER model against the end users' information and processing requirements.
8	Modify the ER model, using the results of Step 7.

Some of the steps listed in Table 2.3 take place concurrently, and some, such as the normalization process, can generate a demand for additional entities and/or attributes, thereby causing the designer to revise the ER model. For example, while identifying two main entities, the designer might also identify the composite bridge entity that represents the many-to-many relationship between the two main entities. To review, suppose that you are creating a conceptual model for the JollyGood Movie Rental Corporation, whose end users want to track customers' DVD movie kiosk rentals. The simple ER diagram presented in Figure 2.9 shows a composite entity that helps track customers and their video rentals. Business rules define the optional nature of the relationships between the entities VIDEO and CUSTOMER. For example, customers are not required to check out a video. A video need not be checked out in order to exist in the kiosk. A customer may rent many videos, and a video may be rented by many customers. In particular, note the composite RENTAL entity that connects the two main entities.

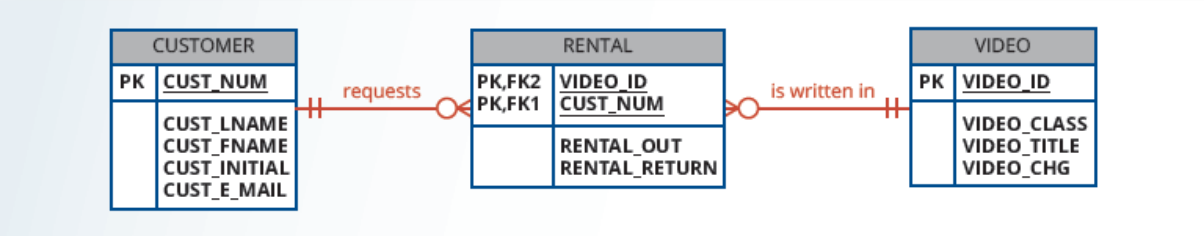


Figure 2-9: Jollygood Movie Rental ERD (Coronel C., Morris S. 2022)

As you will likely discover, the initial ER model may be subjected to several revisions before it meets the system's requirements. Such a revision process is quite natural. Remember that the ER model is a communications tool as well as a design blueprint. Therefore, when you meet with the proposed system users, the initial ER model should give rise to questions such as "Is this really what you meant?" For example, the ERD shown in Figure 2.9 is far from complete. Clearly, many more attributes must be defined, and the dependencies must be checked before the design can be implemented. In addition, the design cannot yet support typical video rental transactions. For example, each video is likely to have many copies available for rental purposes. However, if the VIDEO entity shown in Figure 2.9 is used to store the titles as well as the copies, the design triggers the data redundancies shown in Table 2.4.

Table 2.4: Data Redundancies in the Video Table

VIDEO_ID	VIDEO_TITLE	VIDEO_COPY	VIDEO_CHG	VIDEO_DAYS
SF-12345FT-1	Adventures on Planet III	1	$1.09	1
SF-12345FT-2	Adventures on Planet III	2	$1.09	1
SF-12345FT-3	Adventures on Planet III	3	$1.09	1
WE-5432GR-1	TipToe Canoe and Tyler 2: A Journey	1	$1.09	2
WE-5432GR-2	TipToe Canoe and Tyler 2: A Journey	2	$1.09	2

The initial ERD shown in Figure 2.9 must be modified to reflect the answer to the question "Is more than one copy available for each title?" Also, payment transactions must be supported. From the preceding discussion, you might get the impression that ER modeling activities such as entity and attribute definition, normalization, and verification take place in a precise sequence. In fact, once you have completed the initial ER model, chances are that you will move back and forth among the activities until you are satisfied that the ER model accurately represents a database design that can meet the required system demands. The activities often take place in parallel, and the process is iterative. Figure 2.10 summarizes the ER modelling interactions. Figure 2.11 summarizes the array of design tools and information sources that the designer can use to produce the conceptual model.

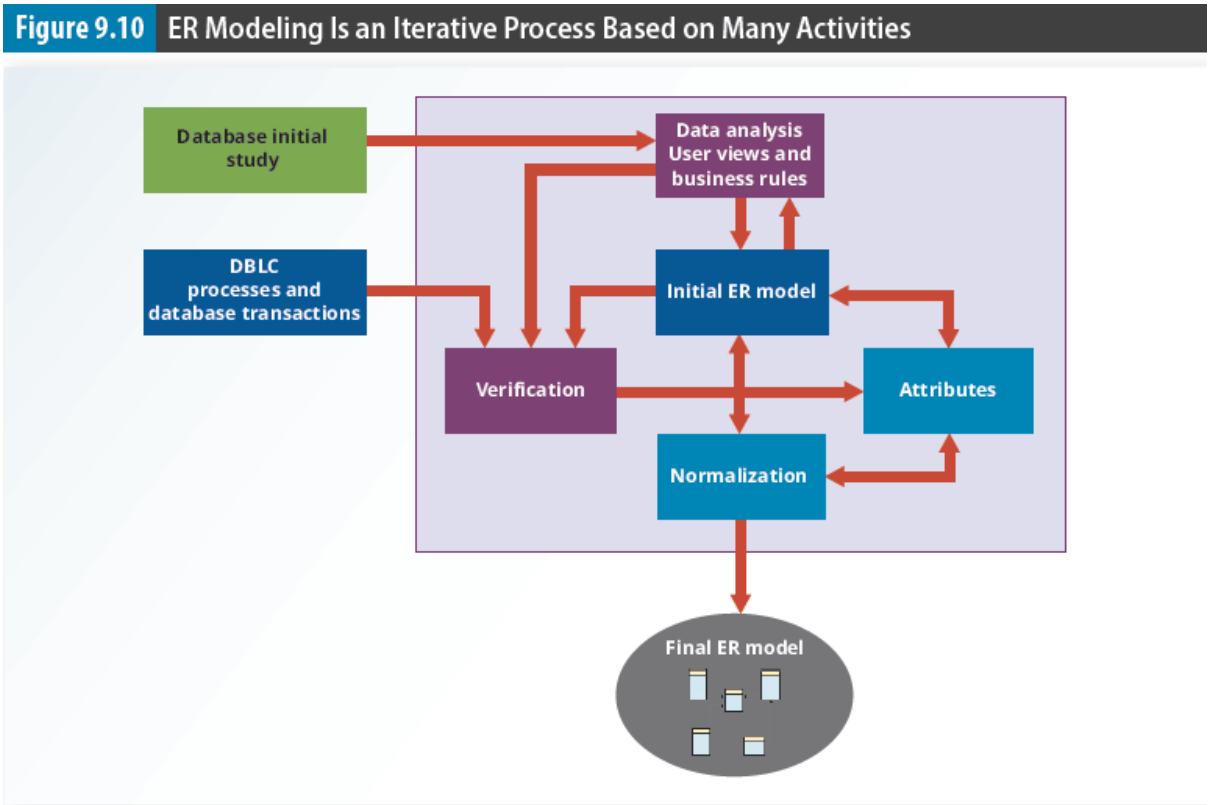


Figure 2-10: ER Modelling (An iterative process based on many activities) (Coronel C., Morris S. 2022)

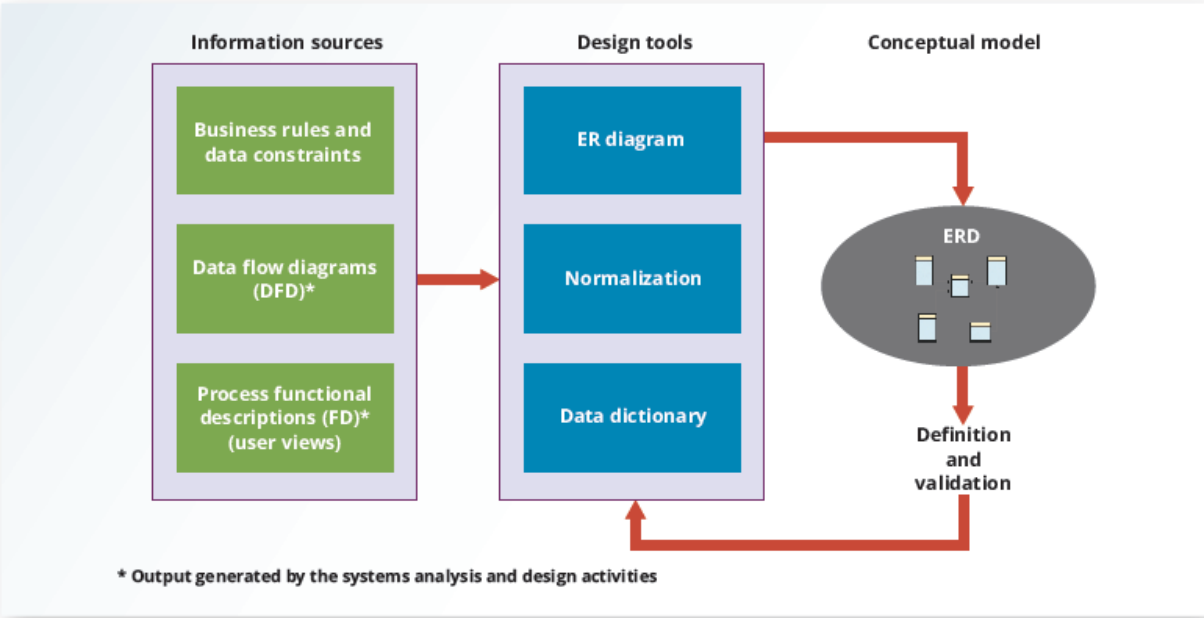


Figure 2-11: Conceptual Design Tools and Information Sources (Coronel C., Morris S. 2022)

All objects (entities, attributes, relations, views, and so on) are defined in a data dictionary, which is used in tandem with the normalization process to help eliminate data anomalies and redundancy problems. During this ER modeling process, the designer must:

- Define entities, attributes, primary keys, and foreign keys. (The foreign keys serve as the basis for the relationships among the entities.)
- Make decisions about adding new primary key attributes to satisfy end-user and processing requirements.
- Make decisions about the treatment of composite and multivalued attributes.
- Make decisions about adding derived attributes to satisfy processing requirements.
- Make decisions about the placement of foreign keys in 1:1 relationships.
- Avoid unnecessary ternary or higher degree relationships.
- Draw the corresponding ER diagram.
- Normalize the entities.
- Include all data element definitions in the data dictionary.
- Make decisions about standard naming conventions.

The naming conventions requirement is important, yet it is frequently ignored at the designer's risk. Real database design is generally accomplished by teams. Therefore, it is important to ensure that team members work in an environment in which naming standards are defined and enforced. Proper documentation is crucial to the successful completion of the design, and adherence to the naming conventions serves database designers well.

Data Model Verification

Data model verification is one of the last steps in the conceptual design stage, and it is one of the most critical. In this step, the ER model must be verified against the proposed system processes to corroborate that they can be supported by the database model. Verification requires that the model be run through a series of tests against:

- End-user data views
- All required transactions: SELECT, INSERT, UPDATE, and DELETE operations
- Access rights and security
- Business-imposed data requirements and constraints

Because real-world database design is generally done by teams, the database design is probably divided into major components known as modules. A **module** is an information system component that handles a specific business function, such as inventory, orders, or payroll.
Under these conditions, each module is supported by an ER segment that is a subset or fragment of an enterprise ER model. Working with modules accomplishes several important ends:

- The modules (and even the segments within them) can be delegated to design groups within teams, greatly speeding up the development work.
- The modules simplify the design work. The large number of entities within a complex design can be daunting. Each module contains a more manageable number of entities.
- The modules can be prototyped quickly. Implementation and application programming trouble spots can be identified more readily. Quick prototyping is also a great confidence builder.
- Even if the entire system cannot be brought online quickly, the implementation of one or more modules will demonstrate that progress is being made and that at least part of the system is ready to begin serving the end users.

As useful as modules are, they represent a loose collection of ER model fragments that could wreak havoc in the database if left unchecked. For example, the ER model fragments:

- Might present overlapping, duplicated, or conflicting views of the same data
- Might not be able to support all processes in the system's modules

To avoid these problems, it is better if the modules' ER fragments are merged into a single enterprise ER model. This process starts by selecting a central ER model segment and iteratively adding more ER model segments one at a time. At each stage, for each new entity added to the model, you need to validate that the new entity does not overlap or conflict with a previously identified entity in the enterprise ER model. Merging the ER model segments into an enterprise ER model triggers a careful re-evaluation of the entities, followed by a detailed examination of the attributes that describe those entities. This process serves several important purposes:

- The emergence of the attribute details might lead to a revision of the entities themselves. Perhaps some of the components first believed to be entities will instead turn out to be attributes within other entities. Or, a component that was originally considered an attribute might turn out to contain a sufficient number of subcomponents to warrant the introduction of one or more new entities.
- The focus on attribute details can provide clues about the nature of relationships as they are defined by the primary and foreign keys. Improperly defined relationships lead to implementation problems first and to application development problems later.
- To satisfy processing and end-user requirements, it might be useful to create a new primary key to replace an existing primary key. For example, in the example illustrated in Figure 2.9, a surrogate primary key (RENTAL_ID) could be introduced to replace the original primary key composed of VIDEO_ID and CUST_NUM.
- Unless the entity details (the attributes and their characteristics) are precisely defined, it is difficult to evaluate the extent of the design's normalization. Knowledge of the normalization levels helps guard against undesirable redundancies.
- A careful review of the rough database design blueprint is likely to lead to revisions. Those revisions will help ensure that the design is capable of meeting end-user requirements. After finishing the merging process, the resulting enterprise ER model is verified against each of the module's processes. The ER model verification process is detailed in Table 2.5.

Table 2.5: The ER Model Verification Process

STEP	ACTIVITY
1	Identify the ER model's central entity.
2	Identify each module and its components.
3	Identify each module's transaction requirements: Internal: updates/inserts/deletes/queries/reports External: module interfaces
4	Verify all processes against the module's processing and reporting requirements.
5	Make all necessary changes suggested in Step 4.
6	Repeat Steps 2–5 for all modules.

Keep in mind that this process requires the continuous verification of business transactions as well as system and user requirements. The verification sequence must be repeated for each of the system's modules. Figure 2.12 illustrates the iterative nature of the process.

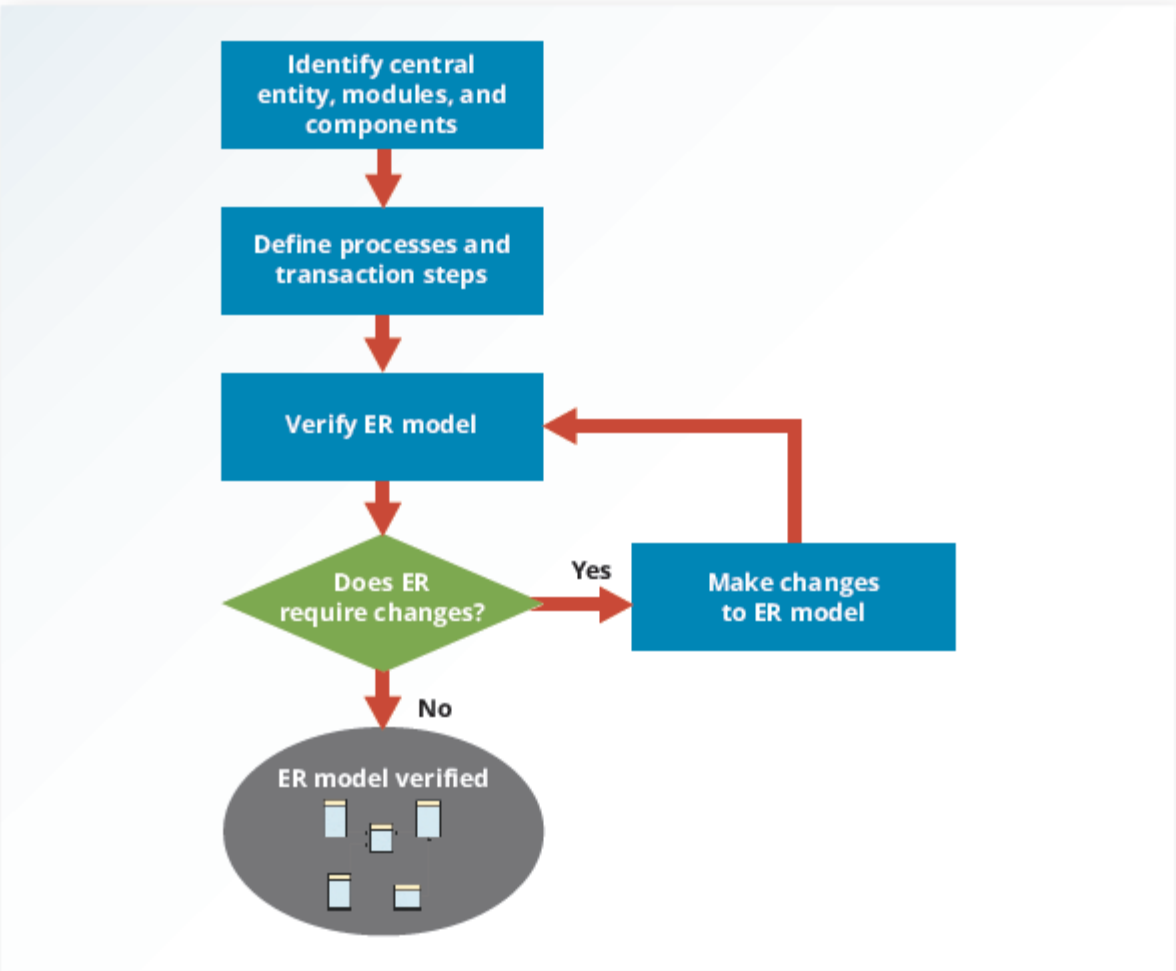


Figure 2-12: Iterative ER Model Verification Process (Coronel C., Morris S. 2022)

The verification process starts with selecting the central (most important) entity, which is the focus for most of the system's operations. To identify the central entity, the designer selects the entity involved in the greatest number of the model's relationships. In the ER diagram, it is the entity with more lines connected to it than any other. The next step is to identify the module or subsystem to which the central entity belongs and to define that module's boundaries and scope. The entity belongs to the module that uses it most frequently. Once each module is identified, the central entity is placed within the module's framework to let you focus on the module's details. Within the central entity/module framework, you must

- *Ensure the module's cohesivity*. The term **cohesivity** describes the strength of the relationships found among the module's entities. A module must display *high cohesivity*—that is, the entities must be strongly related, and the module must be complete and self-sufficient.
- *Analyze each module's relationships with other modules to address module coupling*.

Module coupling describes the extent to which modules are independent of one another. Modules must display *low coupling*, indicating that they are independent of other modules. Low coupling decreases unnecessary intermodule dependencies, thereby allowing the creation of a truly modular system and eliminating unnecessary relationships among entities.

Processes may be classified according to their:

- Frequency (daily, weekly, monthly, yearly, or exceptions)
- Operational type (INSERT or ADD, UPDATE or CHANGE, DELETE, queries and reports, batches, maintenance, and backups)

All identified processes must be verified against the ER model. If necessary, appropriate changes are implemented. The process verification is repeated for all of the model's modules. You can expect that additional entities and attributes will be incorporated into the conceptual model during its validation. At this point, a conceptual model has been defined as hardware- and software-independent. Such independence ensures the system's portability across platforms. Portability can extend the database's life by making it possible to migrate to another DBMS and hardware platform.

Distributed Database Design

Although not a requirement for most databases, some may need to be distributed among multiple geographical locations. Processes that access the database may also vary from one location to another. For example, a retail process and a warehouse storage process are likely to be found in different physical locations. If the database data and processes will be distributed across the system, portions of a database, known as database fragments, may reside in several physical locations. A **database fragment** is a subset of a database that is stored at a given location. The database fragment may be a subset of rows or columns from one or multiple tables. Distributed database design defines the optimum allocation strategy for database fragments to ensure database integrity, security, and performance. The allocation strategy determines how to partition the database and where to store each fragment.

DBMS Software Selection

The selection of DBMS software is critical to the information system's smooth operation. Consequently, the advantages and disadvantages of the proposed DBMS software should be carefully studied. To avoid false expectations, the end user must be made aware of the limitations of both the DBMS and the database. Although the factors that affect the purchasing decision vary from company to company, some of the most common are:

- *Cost*. This includes the original purchase price, along with maintenance, operational, license, installation, training, and conversion costs.
- *DBMS features and tools*. Some database software includes a variety of tools that facilitate application development. For example, the availability of query by example (QBE), screen painters, report generators, application generators, and data dictionaries helps to create a more pleasant work environment for both the end user and the application programmer. Database administrator facilities, query facilities, ease of use, performance, security, concurrency control, transaction processing, and third-party support also influence DBMS software selection.
- *Underlying model*. The database can be based on a hierarchical, network, relational, object/ relational, NoSQL, document, or even a graph data model. Although most of the methodologies covered in this chapter focus on the relational data model, some of the basic design processes could also apply to other data models.

- *Portability*. A DBMS can be portable across platforms, systems, and languages. *DBMS hardware requirements*. Items to consider include processor(s), RAM, disk space, and so on.

Logical Design

Logical design is the second stage in the database design process. The logical design goal is to design an enterprise-wide database that is based on a specific data model but independent of physical-level details. Logical design requires that all objects in the conceptual model be mapped to the specific constructs used by the selected database model. For example, the logical design for a relational DBMS includes the specifications for the relations (tables), relationships, and constraints (in other words, domain definitions, data validations, and security views). The logical design is generally performed in four steps, which are listed in Table 2.6.

Table 2.6: Logical Design Steps

Table 9.6 Logical Design Steps

STEP	ACTIVITY
1	Map the conceptual model to logical model components.
2	Validate the logical model using normalization.
3	Validate the logical model integrity constraints.
4	Validate the logical model against user requirements.

Map the Conceptual Model to the Logical Model Components

The first step in creating the logical design is to map the conceptual model to the chosen database constructs. Because this book deals primarily with relational databases, and because most current database design projects are based on the relational database model, this section focuses on logical design using relational constructs. In the real world, logical design generally involves translating the ER model into a set of relations (tables), columns, and constraint definitions. The process of translating the conceptual model into a set of relations is depicted in Table 2.7.

Table 2.7: Mapping the Conceptual Model to the Relational Model

STEP	ACTIVITY
1	Map strong entities.
2	Map supertype/subtype relationships.
3	Map weak entities.
4	Map binary relationships.
5	Map higher-degree relationships.

The steps indicated in Table 9.7 are not sequential but iterative. The example of the Simple College ER model shown in Figure 2.13 illustrates this process.

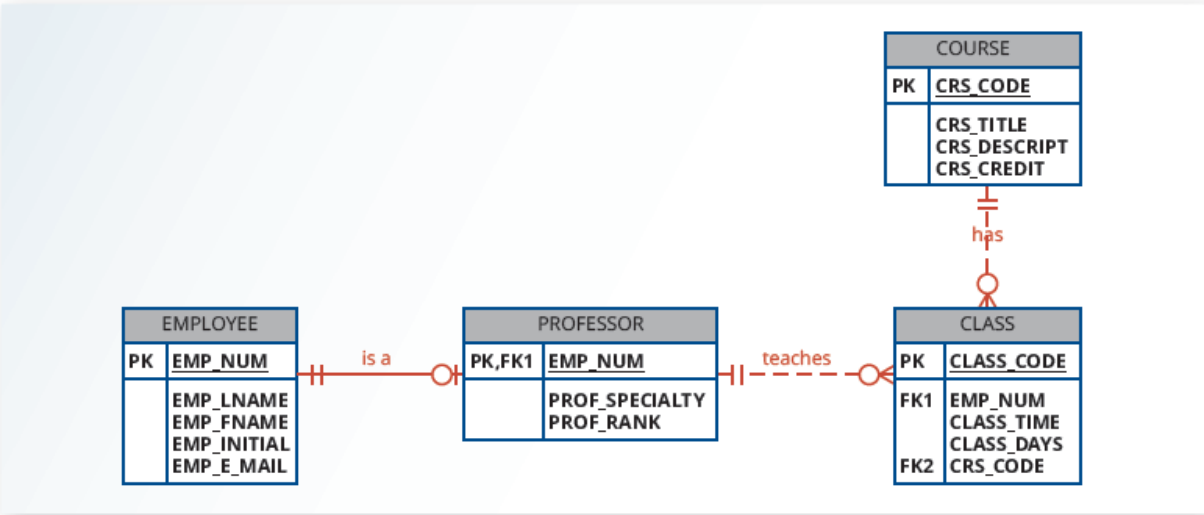


Figure 2-13: Simple College Conceptual Model (Coronel C., Morris S. 2022)

As indicated in Table 2.7, the first step in the logical design stage is to map strong entities to tables. A strong entity is one that resides in the "1" side of all its relationships—that is, an entity that does not have a mandatory attribute that is a foreign key to another table. Therefore, the first entities to be translated into tables would be the EMPLOYEE and COURSE entities. In this case, you define the table name, its columns, and their characteristics. For example, the relation definitions for the strong entities of Simple College would be:

COURSE (CRS_CODE, CRS_TITLE, CRS_DESCRIPT, CRS_CREDIT)
PRIMARY KEY: CRS_CODE
EMPLOYEE (EMP_NUM, EMP_LNAME, EMP_FNAME, EMP_INITIAL, EMP_E_MAIL)
PRIMARY KEY: EMP_NUM

Once all strong entities are mapped, you are ready to map any entities involved in a supertype/subtype relationship or any weak entities. In the case of Simple College, the PROFESSOR entity is a subtype of the EMPLOYEE entity. PROFESSOR is also a weak entity because it inherits its primary key from EMPLOYEE and is existence-dependent on EMPLOYEE. At this point, you could also start defining the relationships between supertype and subtype entities. For example:

PROFESSOR (EMP_NUM, PROF_SPECIALTY, PROF_RANK)
PRIMARY KEY: EMP_NUM
FOREIGN KEY: EMP_NUM REFERENCES EMPLOYEE

Next, you start mapping all binary relationships. In the previous example, you defined the supertype/subtype relationship between EMPLOYEE and PROFESSOR. This is an instance that demonstrates the iterative nature of the process. Continuing with the Simple College ER model, you would define the CLASS relation and define its 1:M relationships with PROFESSOR and COURSE:

CLASS (CLASS_CODE, *EMP_NUM*, CLASS_TIME, CLASS_DAYS, *CRS_CODE*)
PRIMARY KEY: CLASS_CODE
FOREIGN KEYS: EMP_NUM REFERENCES PROFESSOR
CRS_CODE REFERENCES COURSE

Next, you will proceed with all relationships between three or more entities until all relationships in the model are clearly defined. The logical design's tables must correspond to the entities (EMPLOYEE, PROFESSOR, COURSE, and CLASS) shown in the conceptual design of Figure 2.13, and the table columns must correspond to the attributes specified in the conceptual design. The final outcome of this process is a list of relations, attributes, and relationships that will be the basis for the next step.

Validate the Logical Model Using Normalization

The logical design should contain only properly normalized tables. The process of mapping the conceptual model to the logical model may unveil some new attributes or the discovery of new multivalued or composite attributes. Therefore, it's very likely that new attributes may be added to tables, or that entire new tables may be added to the logical model. For each identified table (old and new), you must ensure that all attributes are fully dependent on the identified primary key and that the tables are in at least third normal form (3NF). As indicated throughout this section, database design is an iterative process. Activities such as normalization take place at different stages in the design process. Each time you reiterate a step, the model is further refined and better documented. New attributes may be created and assigned to the proper entities. Functional dependencies among determinant and dependent attributes are evaluated and data anomalies are prevented via normalization.

Validate the Logical Model Integrity Constraints

The translation of the conceptual model into a logical model also requires definition of the attribute domains and appropriate constraints. For example, the domain definitions for the CLASS_CODE, CLASS_DAYS, and CLASS_TIME attributes displayed in the CLASS entity in Figure 2.13 are written this way:

CLASS_CODE is a valid class code.
Type: numeric
Range: low value = 1000 high value = 9999
Display format: 9999
Length: 4

CLASS_DAYS is a valid day code.
Type: character
Display format: XXX
Valid entries: MWF, TR, M, T, W, R, F, S
Length: 3

CLASS_TIME is a valid time.
Type: character
Display format: 99:99 (24-hour clock)
Display range: 06:00 to 22:00
Length: 5

All these defined constraints must be supported by the logical data model. In this stage, you must map these constraints to the proper relational model constraints. For example, the CLASS_DAYS attribute is character data that should be restricted to a list of valid character combinations.

Here, you define this attribute to have a CHECK IN constraint to enforce that the only allowed values are “MWF,” “TR,” “M,” “T,” “W,” “R,” “F,” and “S.” During this step, you also define which attributes are mandatory and which are optional, and you ensure that all entities maintain entity and referential integrity.

The right to use the database is also specified during the logical design phase. Who will be allowed to use the tables, and what portions of the tables will be available to which users? Within a relational framework, the answers to those questions require the definition of appropriate views. For example, a given process may require the creation of the following view to get data about the class schedules:

```
CREATE VIEW vSCHEDULE AS
SELECT EMP_LNAME, EMP_FNAME, CLASS_CODE, CRS_TITLE,
CLASS_TIME, CLASS_DAYS
FROM PROFESSOR, CLASS, COURSE
WHERE PROFESSOR.EMP_NUM = CLASS.EMP_NUM AND
CLASS.CRS_CODE = COURSE.CRS_CODE
```

Special attention is needed at this stage to ensure that all views can be resolved and that security is enforced to ensure the privacy of the data. Additionally, if you are working with a distributed database design, data could be stored at multiple locations, and each location may have different security restrictions. After validating the logical model integrity constraints, you are ready to validate the model against the end-user requirements.

Validate the Logical Model against User Requirements

The logical design translates the software-independent conceptual model into a software-dependent model. The final step in the logical design process is to validate all logical model definitions against all end-user data, transaction, and security requirements. A process similar to the one depicted in Table 2.5 takes place again to ensure the correctness of the logical model. The stage is now set to define the physical requirements that allow the system to function within the selected DBMS/hardware environment.

Physical Design

Physical design is the process of determining the data storage organization and data access characteristics of the database to ensure its integrity, security, and performance. This is the last stage in the database design process. The storage characteristics are a function of the types of devices supported by the hardware, the type of data access methods supported by the system, and the DBMS. Physical design can become a very technical job that affects not only the accessibility of the data in the storage device(s) but the performance of the system. The physical design stage consists of the steps in Table 2.8.

Table 2.8: Physical design steps

STEP	ACTIVITY
1	Define data storage organization.
2	Define integrity and security measures.
3	Determine performance measurements.

The following sections cover these steps in more detail.

Define Data Storage Organization

Before you can define data storage organization, you must determine the volume of data to be managed and the data usage patterns.

- Knowing the data volume will help you determine how much storage space to reserve for the database. To do this, the designer follows a process similar to the one used during ER model verification. For each table, identify all possible transactions, their frequency, and volume. For each transaction, you determine the amount of data to be added or deleted from the database. This information will help you determine the amount of data to be stored in the related table.

- Conversely, knowing how frequently new data is inserted, updated, and retrieved will help the designer determine the data usage patterns. Usage patterns are critical, particularly in distributed database design. For example, are there any weekly batch uploads or monthly aggregation reports to be generated? How frequently is new data added to the system? This information is also useful to determine the query performance parameters for the system. Equipped with the two previous pieces of information, the designer must:

    - *Determine the location and physical storage organization for each table*. Tables are stored in table spaces, and a table space can hold data from multiple tables. In this step, the designer assigns which tables will use which table spaces and the location of the table spaces. For example, a useful technique available in most relational databases is the use of clustered tables. The **clustered tables** storage technique stores related rows from two related tables in adjacent data blocks on disk. This ensures that the data is stored in sequentially adjacent locations, thereby reducing data access time and increasing system performance.
    - *Identify indexes and the type of indexes to be used for each table*. As you saw in previous chapters, indexes are useful for ensuring the uniqueness of data values in a column and to facilitate data lookups. You also know that the DBMS automatically creates a unique index for the primary key of each table. In this step, you identify all required indexes and determine the best type of organization to use based on the data usage patterns and performance requirements.
    - *Identify the views and the type of views to be used on each table.* A view is useful to limit access to data based on user or transaction needs. Views can also be used to simplify processing and end-user data access. In this step the designer must ensure that all views can be implemented and that they provide only the required data. The designer must also become familiar with the types of views supported by the DBMS and how they could help meet system goals.

Define Integrity and Security Measures

Once the physical organization of the tables, indexes, and views are defined, the database is ready for the end users. However, before users can access the data in the database, they must be properly authenticated. In this step of physical design, two tasks must be addressed:

- *Define user and security groups and roles*. User management is more a function of database administration than database design. However, as a designer you must know the different types of users and groups of users to properly enforce database security. Most DBMS implementations support the use of database roles. A **database role** is a set of database privileges that could be assigned as a unit to a user or group. For example, you could define an Advisor role that has Read access to the vSCHEDULE view.

- *Assign security controls*. The DBMS also allows administrators to assign specific access rights for database objects to a user or group of users. For example, you could assign the SELECT and UPDATE access rights to the user mhendley on the CLASS table. An access right could also be revoked from a specific user or groups of users. This feature could come in handy during database backups, scheduled maintenance events, or even during data breach incidents.

Determine Performance Measurements

Physical design becomes more complex when data is distributed at different locations because the performance is affected by the communication media's throughput. Given such complexities, it is not surprising that designers favor database software that hides as many of the physical-level activities as possible. Despite the fact that relational models tend to hide the complexities of the computer's physical characteristics, the performance of relational databases is affected by physical storage properties. For example, performance can be affected by characteristics of the storage media, such as seek time, sector and block (page) size, buffer pool size, and the number of disk platters and read/write heads. In addition, factors such as the creation of an index can have a considerable effect on the relational database's performance—that is, data access speed and efficiency.

In summary, physical design performance measurement deals with fine-tuning the DBMS and queries to ensure that they will meet end-user performance requirements as identified in the previous stages.

Database Design Strategies

There are two classical approaches to database design:

- **Top-down design** starts by identifying the data sets and then defines the data elements for each of those sets. This process involves the identification of different entity types and the definition of each entity's attributes.
- **Bottom-up design** first identifies the data elements (items) and then groups them together in data sets. In other words, it first defines attributes, and then groups them to form entities.

The two approaches are illustrated in Figure 2.14. Selecting a primary emphasis on top-down or bottom-up procedures often depends on the scope of the problem or on personal preferences. Although the two methodologies are complementary rather than mutually exclusive, a primary emphasis on a bottom-up approach may be more productive for small databases with few entities, attributes, relations, and transactions. For situations in which the number, variety, and complexity of entities, relations, and transactions is overwhelming, a primarily top-down approach may be easier. Most companies have standards for systems development and database design already in place.

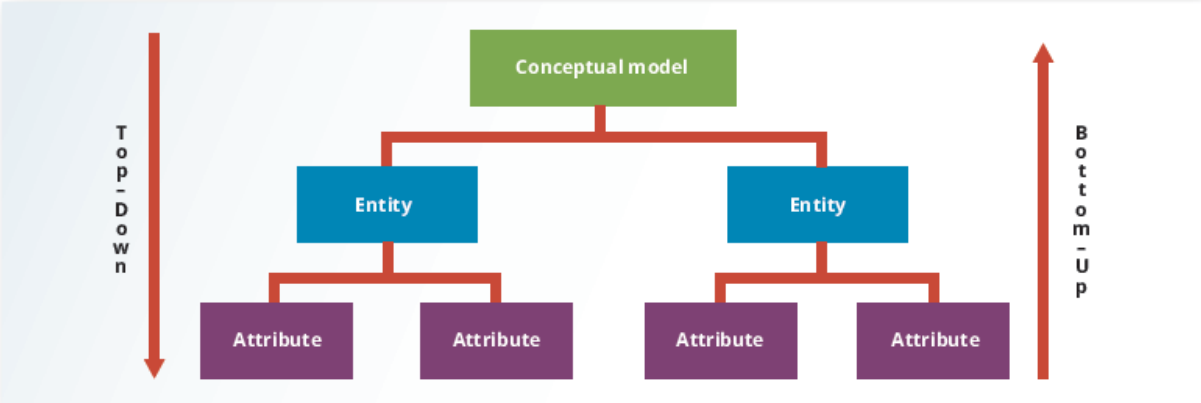


Figure 2-14: Top-Down vs Bottom-up Design Sequencing (Coronel C., Morris S. 2022)

Centralized versus Decentralized Design

The two general approaches to database design (bottom-up and top-down) can be influenced by factors such as the scope and size of the system, the company's management style, and the company's structure (centralized or decentralized). Depending on these factors, the database design may be based on two very different design philosophies: centralized and decentralized. **Centralized design** is productive when the data component has a relatively small number of objects and procedures. The design can be carried out and represented in a fairly simple database. Centralized design is typical of relatively simple, small databases and can be successfully done by a single database administrator or by a small, informal design team. The company operations and the scope of the problem are sufficiently limited to allow even a single designer to define the problem(s), create the conceptual design, verify the conceptual design with the user views, define system processes and data constraints to ensure the efficacy of the design, and ensure that the design will comply with all the requirements. (Although centralized design is typical for small companies, do not make the mistake of assuming that it is limited to them. Even large companies can operate within a relatively simple database environment.) Figure 2.15 summarizes the centralized design option. Note that a single conceptual design is completed and then validated in the centralized design approach.

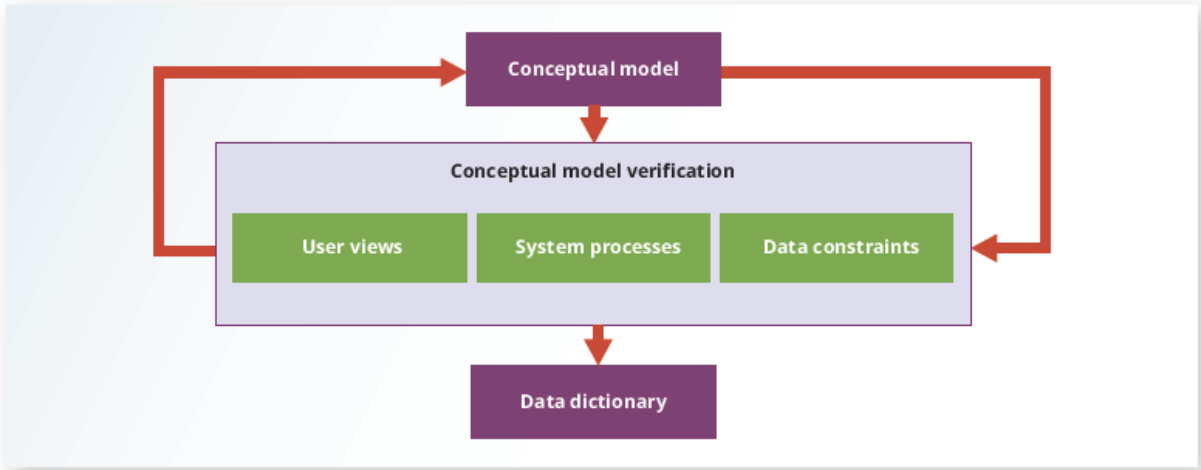


Figure 2-15: Centralized Design (Coronel C., Morris S. 2022)

Decentralized design might be used when the system's data component has a considerable number of entities and complex relations on which very complex operations are performed. Decentralized design is also often used when the problem itself is spread across several operational sites and each element is a subset of the entire data set. (See Figure 2.16.)

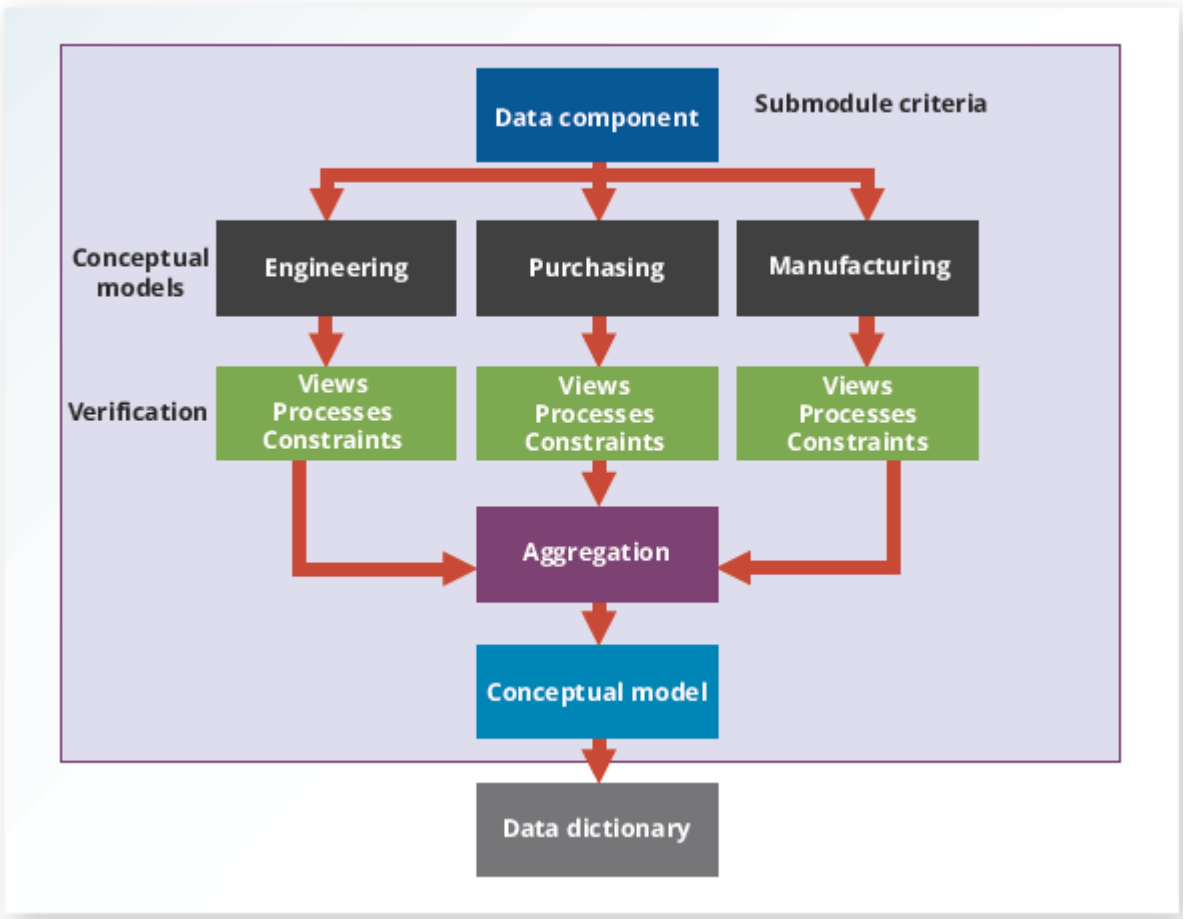


Figure 2-16: Decentralized Design (Coronel C., Morris S. 2022)

In large and complex projects, the database typically cannot be designed by only one person. Instead, a carefully selected team of database designers tackles a complex database project. Within the decentralized design framework, the database design task is divided into several

modules. Once the design criteria have been established, the lead designer assigns design subsets or modules to design groups within the team.

Because each design group focuses on modeling a subset of the system, the definition of boundaries and the interrelation among data subsets must be very precise. Each design group creates a conceptual data model corresponding to the subset being modeled. Each conceptual model is then verified individually against the user views, processes, and constraints for each of the modules. After the verification process has been completed, all modules are integrated into one conceptual model. Because the data dictionary describes the characteristics of all objects within the conceptual data model, it plays a vital role in the integration process. After the subsets have been aggregated into a larger conceptual model, the lead designer must verify that it still can support all of the required transactions.
Keep in mind that the aggregation process requires the designer to create a single model in which various aggregation problems must be addressed. (See Figure 2.17.)

- *Synonyms and homonyms*. Various departments might know the same object by different names (synonyms), or they might use the same name to address different objects (homonyms). The object can be an entity, an attribute, or a relationship.
- *Entity and entity subtypes*. An entity subtype might be viewed as a separate entity by one or more departments. The designer must integrate such subtypes into a higher-level entity.
- *Conflicting object definitions*. Attributes can be recorded as different types (character, numeric), or different domains can be defined for the same attribute. Constraint definitions can vary as well. The designer must remove such conflicts from the model.

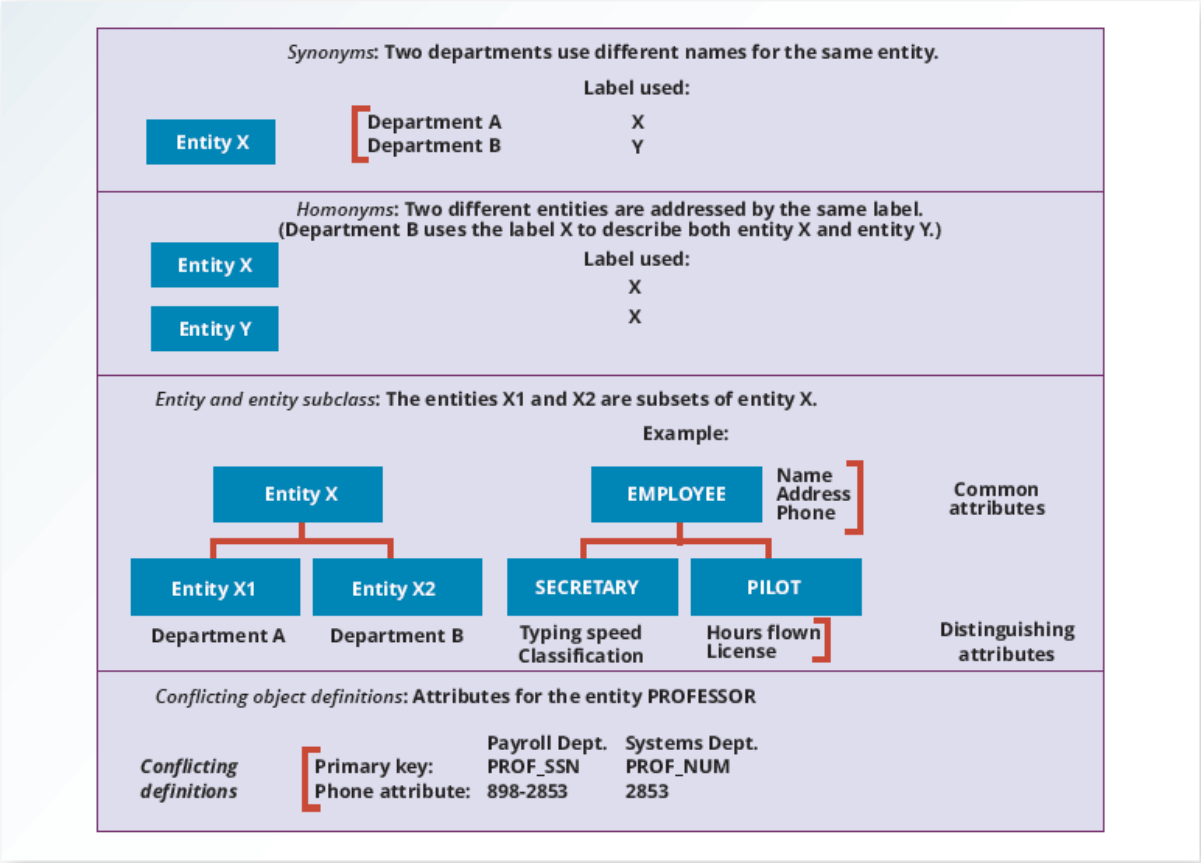


Figure 2-17: Summary of Aggregation Problems (Coronel C., Morris S. 2022)

Summary

- An information system is designed to help transform data into information and to manage both data and information. Thus, the database is a very important part of the information system. Systems analysis is the process that establishes the need for an information system and its extent. Systems development is the process of creating an information system.
- The Systems Development Life Cycle (SDLC) traces the history of an application within the information system. The SDLC can be divided into five phases: planning, analysis, detailed systems design, implementation, and maintenance. The SDLC is an iterative process rather than a sequential process.
- The Database Life Cycle (DBLC) describes the history of the database within the information system. The DBLC is composed of six phases: database initial study, database design, implementation and loading, testing and evaluation, operation, and maintenance and evolution. Like the SDLC, the DBLC is iterative rather than sequential.
- The conceptual portion of the design may be subject to several variations based on two basic design philosophies: bottom-up versus top-down and centralized versus decentralized.

Review Questions

1. What is an information system? What is its purpose?
2. How do systems analysis and systems development fit into a discussion about information systems?
3. What does the acronym SDLC mean, and what does an SDLC portray?
4. What does the acronym DBLC mean, and what does a DBLC portray?
5. Discuss the distinction between centralized and decentralized conceptual database design.
6. What is the minimal data rule in conceptual design? Why is it important?
7. Discuss the distinction between top-down and bottom-up approaches in database design.
8. What are business rules? Why are they important to a database designer?
9. What is the data dictionary's function in database design?
10. What steps are required in the development of an ER diagram?
11. List and briefly explain the activities involved in the verification of an ER model.
12. What factors are important in a DBMS software selection?
13. List and briefly explain the four steps performed during the logical design stage.
14. List and briefly explain the three steps performed during the physical design stage.
15. What three levels of backup may be used in database recovery management? Briefly describe what each backup level does.

Chapter 3: Transaction Management and Concurrency Control



Learning Outcomes

After this chapter you will be able to:

1. Describe the database transaction management process
2. Identify the four properties of a database transaction
3. Explain concurrency control and its role in maintaining database integrity
4. Describe how locking methods are used for concurrency control
5. Describe how stamping methods are used for concurrency control
6. Describe how optimistic methods are used for concurrency control
7. List and explain the ANSI levels of transaction isolation
8. Describe the role of database recovery management in maintaining database integrity

What Is a Transaction?

To illustrate what transactions are and how they work, use the SaleCo database. The relational diagram for the database is shown in Figure 3.1.

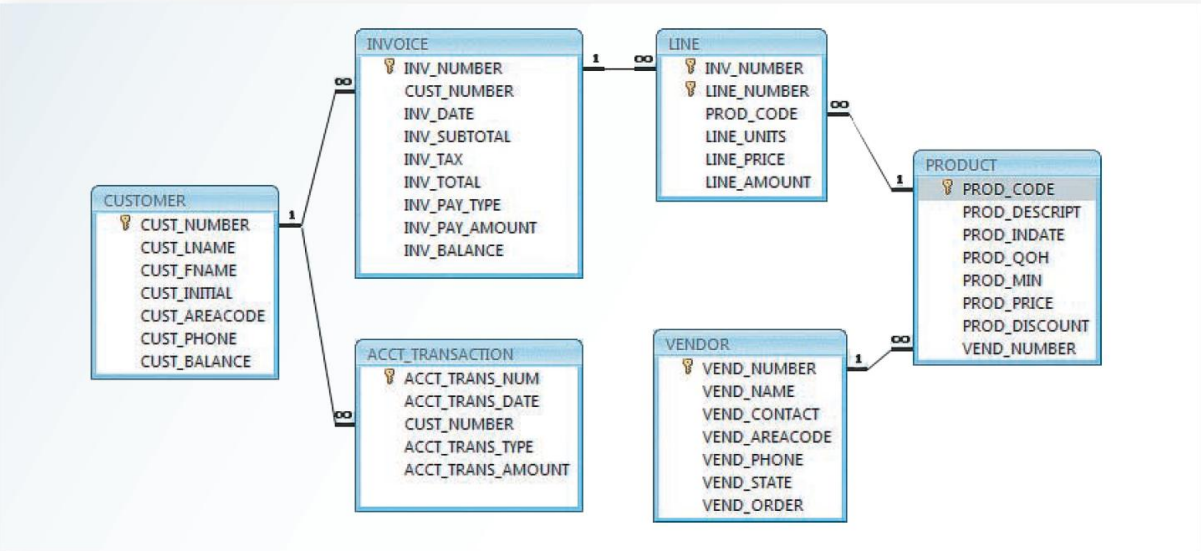


Figure 3-1: The SaleCo Database Relational Diagram (Coronel C., Morris S. 2022)

As you examine the relational diagram in Figure 3.1, note the following characteristics:

- The design stores the customer balance (CUST_BALANCE) value in the CUSTOMER table to indicate the total amount owed by the customer. The CUST_BALANCE attribute is increased when the customer makes a purchase on credit, and it is decreased when the customer makes a payment. Including the current customer account balance in the CUSTOMER table makes it easy to write a query to determine the current balance for any customer and to generate important summaries such as total, average, minimum, and maximum balances.
- The ACCT_TRANSACTION table records all customer purchases and payments to track the details of customer account activity.

You could change the design of the SaleCo database to reflect accounting practice more precisely, but the implementation provided here will enable you to track the transactions well enough to understand the chapter's discussions. To understand the concept of a transaction, suppose that you sell a product to a customer. Furthermore, suppose that the customer may charge the purchase to his or her account. Given that scenario, your sales transaction consists of at least the following parts:

- You must write a new customer invoice.
- You must reduce the quantity on hand in the product's inventory.
- You must update the account transactions.
- You must update the customer balance.

The preceding sales transaction must be reflected in the database. In database terms, a transaction is any action that reads from or writes to a database. A transaction may consist of the following:

- A simple SELECT statement to generate a list of table contents.

- A series of related UPDATE statements to change the values of attributes in various tables.
- A series of INSERT statements to add rows to one or more tables.
- A combination of SELECT, UPDATE, and INSERT statements.

The sales transaction example includes a combination of INSERT and UPDATE statements. Given the preceding discussion, you can augment the definition of a transaction. A **transaction** is a *logical* unit of work that must be entirely completed or entirely aborted; no intermediate states are acceptable. In other words, a multicomponent transaction, such as the previously mentioned sale, must not be partially completed. Updating only the inventory or only the accounts receivable is not acceptable. All of the SQL statements in the transaction must be completed successfully. If any of the SQL statements fail, the entire transaction is rolled back to the original database state that existed before the transaction started. A successful transaction changes the database from one consistent state to another. A **consistent database state** is one in which all data integrity constraints are satisfied.

To ensure consistency of the database, every transaction must begin with the database in a known consistent state. If the database is not in a consistent state, the transaction will yield an inconsistent database that violates its integrity and business rules. For that reason, subject to limitations discussed later, all transactions are controlled and executed by the DBMS to guarantee database integrity.

Most real-world database transactions are formed by two or more database requests. A **database request** is the equivalent of a single SQL statement in an application program or transaction. For example, if a transaction is composed of two UPDATE statements and one INSERT statement, the transaction uses three database requests. In turn, each database request generates several input/output (I/O) operations that read from or write to physical storage media.

Evaluating Transaction Results

Not all transactions update the database. Suppose that you want to examine the CUSTOMER table to determine the current balance for customer number 10016. Such a transaction can be completed by using the following SQL code:

```
SELECT CUST_NUMBER, CUST_BALANCE
FROM CUSTOMER
WHERE CUST_NUMBER 5=10016;
```

Although the query does not make any changes in the CUSTOMER table, the SQL code represents a transaction because it *accesses* the database. If the database existed in a consistent state before the access, the database remains in a consistent state after the access because the transaction did not alter the database. Remember that a transaction may consist of a single SQL statement or a collection of related SQL statements. Revisit the previous sales example to illustrate a more complex transaction, using the SaleCo database. Suppose that on January 18, 2022, you register the credit sale of one unit of product 89-WRE-Q to customer 10016 for $277.55. The required transaction affects the INVOICE, LINE, PRODUCT, CUSTOMER, and ACCT_TRANSACTION tables. The SQL statements that represent this transaction are as follows:

```
INSERT INTO INVOICE
```

```
VALUES (1009, 10016, '18-Jan-2022', 256.99, 20.56, 277.55, 'cred', 0.00, 277.55);
INSERT INTO LINE
VALUES (1009, 1, '89-WRE-Q', 1, 256.99, 256.99);
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 1
WHERE PROD_CODE = '89-WRE-Q';
UPDATE CUSTOMER
SET CUST_BALANCE = CUST_BALANCE + 277.55
WHERE CUST_NUMBER = 10016;
INSERT INTO ACCT_TRANSACTION
VALUES (10007, '18-Jan-22', 10016, 'charge', 277.55);
COMMIT;
```

The results of the successfully completed transaction are shown in Figure 3.2. (All records involved in the transaction are outlined in red.)

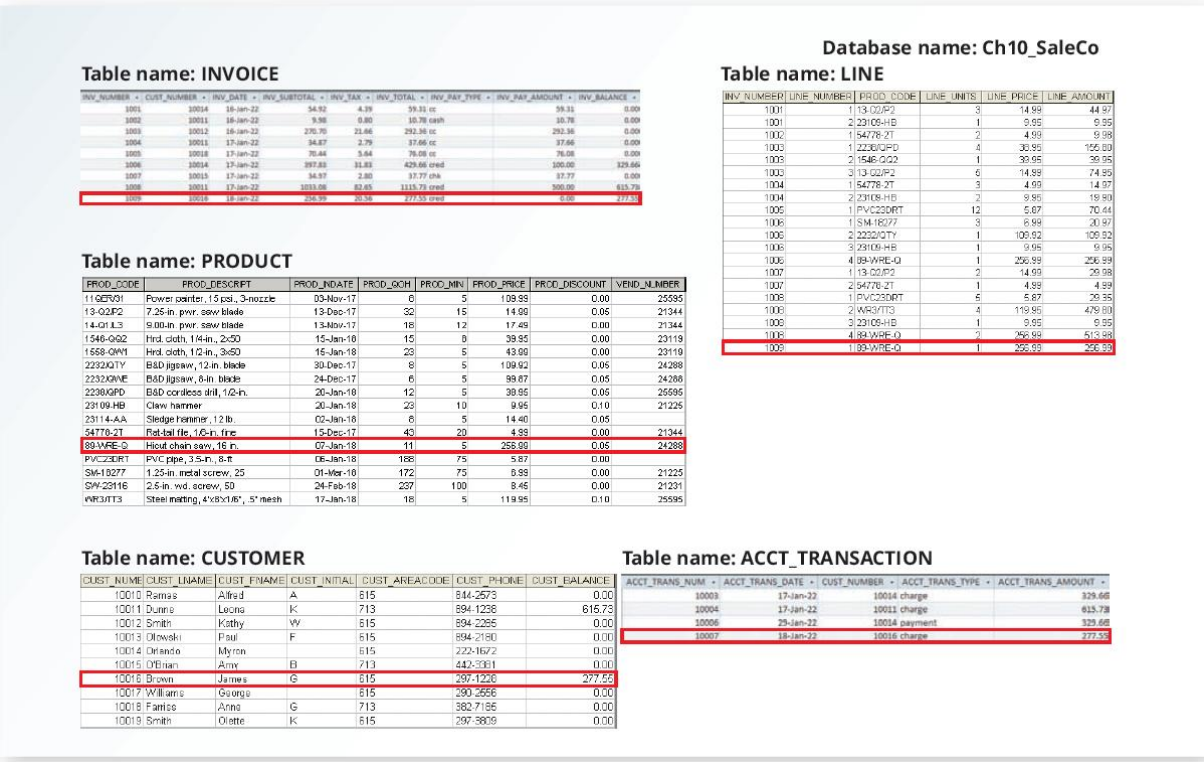


Figure 3-2: Tracing the transaction in the SaleCo Database (Coronel C., Morris S. 2022)

To better understand the transaction results, note the following:

- A new row 1009 was added to the INVOICE table. In this row, derived attribute values were stored for the invoice subtotal, the tax, the invoice total, and the invoice balance.
- The LINE row for invoice 1009 was added to reflect the purchase of one unit of product 89-WREQ with a price of $256.99. In this row, the derived attribute values for the line amount were stored.
- Product 89-WRE-Q's quantity on hand (PROD_QOH) in the PRODUCT table was reduced by one, from 12 to 11.

- The customer balance (CUST_BALANCE) for customer 10016 was updated by adding $277.55 to the existing balance (the initial value was $0.00).
- A new row was added to the ACCT_TRANSACTION table to reflect the new account transaction number 10007.
- The COMMIT statement was used to end a successful transaction.

Now suppose that the DBMS completes the first three SQL statements. Furthermore, suppose that during the execution of the fourth statement (the UPDATE of the CUSTOMER table's CUST_BALANCE value for customer 10016), the computer system loses electrical power. If the computer does not have a backup power supply, the transaction cannot be completed. Therefore, the INVOICE and LINE rows were added, and the PRODUCT table was updated to represent the sale of product 89-WRE-Q, but customer 10016 was not charged, nor was the required record written in the ACCT_TRANSACTION table. The database is now in an inconsistent state, and it is not usable for subsequent transactions. Assuming that the DBMS supports transaction management, *the DBMS will roll back the database to a previous consistent state*.

Although the DBMS is designed to recover a database to a previous consistent state when an interruption prevents the completion of a transaction, the transaction itself is defined by the end user or programmer and must be semantically correct. *The DBMS cannot guarantee that the semantic meaning of the transaction truly represents the real-world event*. For example, suppose that following the sale of 10 units of product 89-WRE-Q, the inventory UPDATE commands were written this way:

```
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH 1 10
WHERE PROD_CODE 5 '89-WRE-Q';
```

The sale should have *decreased* the PROD_QOH value for product 89-WRE-Q by 10. Instead, the UPDATE *added* 10 to product 89-WRE-Q's PROD_QOH value. Although the UPDATE command's syntax is correct, its use yields incorrect results, that is, a database inconsistent with the real-world event. Yet, the DBMS will execute the transaction anyway. The DBMS cannot evaluate whether the transaction represents the real-world event correctly; that is the end user's responsibility. End users and programmers are capable of introducing many errors in this fashion. Imagine the consequences of reducing the quantity on hand for product 1546-QQ2 instead of product 89-WRE-Q or of crediting the CUST_BALANCE value for customer 10012 rather than customer 10016.

Clearly, improper or incomplete transactions can have a devastating effect on database integrity. Some DBMSs—*especially* the relational variety—provide means by which the user can define enforceable constraints based on business rules. Other integrity rules, such as those governing referential and entity integrity, are enforced automatically by the DBMS when the table structures are properly defined, thereby letting the DBMS validate some transactions. For example, if a transaction inserts a new customer number into a customer table and the number already exists, the DBMS will end the transaction with an error code to indicate a violation of the primary key integrity rule.

Transaction Properties

Each individual transaction must display *atomicity*, *consistency*, *isolation*, and *durability*. These four properties are sometimes referred to as the ACID test. Let's look briefly at each of the properties.

- **Atomicity** requires that *all* operations (SQL requests) of a transaction be completed; if not, the transaction is aborted. If a transaction T1 has four SQL requests, all four requests must be successfully completed; otherwise, the entire transaction is aborted. In other words, a transaction is treated as a single, indivisible, logical unit of work.
- **Consistency** indicates the permanence of the database's consistent state. A transaction takes a database from one consistent state to another. When a transaction is completed, the database must be in a consistent state. If any of the transaction parts violates an integrity constraint, the entire transaction is aborted.
- **Isolation** means that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed. In other words, if transaction T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2 … T*n*) until T1 ends. This property is particularly useful in multiuser database environments because several users can access and update the database at the same time.
- **Durability** ensures that once transaction changes are done and committed, they cannot be undone or lost, even in the event of a system failure.

In addition to the individual transaction properties indicated above, another important property applies when executing multiple transactions concurrently. For example, let's assume that the DBMS has three transactions (T1, T2, and T3) executing at the same time. To properly carry out transactions, the DBMS must schedule the concurrent execution of the transaction's operations. In this case, each individual transaction must comply with the ACID properties and, at the same time, the schedule of such multiple transaction operations must exhibit the property of serializability. **Serializability** ensures that the schedule for the concurrent execution of the transactions yields consistent results. This property is important in multiuser and distributed databases in which multiple transactions are likely to be executed concurrently. Naturally, if only a single transaction is executed, serializability is not an issue.

A single-user database system automatically ensures serializability and isolation of the database because only one transaction is executed at a time. The atomicity, consistency, and durability of transactions must be guaranteed by single-user DBMSs. (Even a single-user DBMS must manage recovery from errors created by OS-induced interruptions, power interruptions, and abnormal application terminations or crashes.)

Multiuser databases are typically subject to multiple concurrent transactions. Therefore, the multiuser DBMS must implement controls to ensure serializability and isolation of transactions—in addition to atomicity and durability—to guard the database's consistency and integrity. For example, if several concurrent transactions are executed over the same data set and the second transaction updates the database before the first transaction is finished, the isolation property is violated and the database is no longer consistent. The DBMS must manage the transactions by using concurrency control techniques to avoid undesirable situations.

Transaction Management with SQL

Using the same data set and the second transaction updates the database before the first transaction is finished, the isolation property is violated and the database is no longer consistent. The DBMS must manage the transactions by using concurrency control techniques to avoid undesirable situations.

The American National Standards Institute (ANSI) has defined standards that govern SQL database transactions. Transaction support is provided by two SQL statements: COMMIT and ROLLBACK. The ANSI standards require that when a transaction sequence is initiated by a user or an application program, the sequence must continue through all succeeding SQL statements until one of the following four events occurs:

- A COMMIT statement is reached, in which case all changes are permanently recorded within the database. The COMMIT statement automatically ends the SQL transaction.
- A ROLLBACK statement is reached, in which case all changes are aborted and the database is rolled back to its previous consistent state.
- The end of a program is successfully reached, in which case all changes are permanently recorded within the database. This action is equivalent to COMMIT.
- The program is abnormally terminated, in which case the database changes are aborted and the database is rolled back to its previous consistent state. This action is equivalent to ROLLBACK.

The use of COMMIT is illustrated in the following simplified sales example, which updates a product's quantity on hand (PROD_QOH) and the customer's balance when the customer buys two units of product 1558-QW1 priced at $43.99 per unit (for a total of $87.98) and charges the purchase to the customer's account:

```
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 2
WHERE PROD_CODE = '1558-QW1';
UPDATE CUSTOMER
SET CUST_BALANCE = CUST_BALANCE + 87.98
WHERE CUST_NUMBER = '10011';
COMMIT;
```

The COMMIT statement used in the preceding example is not necessary if the UPDATE statement is the application's last action and the application terminates normally. However, good programming practice dictates that you include the COMMIT statement at the end of a transaction declaration. A transaction begins implicitly when the first SQL statement is encountered. Not all SQL implementations follow the ANSI standard; some (such as SQL Server) use transaction management statements such as the following to indicate the beginning of a new transaction:

```
BEGIN TRANSACTION;
```

Other SQL implementations allow you to assign characteristics for the transactions as parameters to the BEGIN statement. For example, the Oracle RDBMS uses the SET TRANSACTION
statement to declare the start of a new transaction and its properties.

The Transaction Log

A DBMS uses a **transaction log** to keep track of all transactions that update the database. The DBMS uses the information stored in this log for a recovery requirement triggered by a ROLLBACK statement, a program's abnormal termination, or a system failure such as a network discrepancy or a disk crash. Some RDBMSs use the transaction log to recover a database *forward* to a currently consistent state. After a server failure, for example, Oracle automatically rolls back uncommitted transactions and rolls forward transactions that were committed but not yet written to the physical database. This behavior is required for transactional correctness and is typical of any transactional DBMS. While the DBMS executes transactions that modify the database, it also automatically updates the transaction log. The transaction log stores the following:

- A record for the beginning of the transaction.
- For each transaction component (SQL statement):
    - The type of operation being performed (INSERT, UPDATE, DELETE).
    - The names of the objects affected by the transaction (the name of the table).
    - The "before" and "after" values for the fields being updated.
    - Pointers to the previous and next transaction log entries for the same transaction.
- The ending (COMMIT) of the transaction.

Although using a transaction log increases the processing overhead of a DBMS, the ability to restore a corrupted database is worth the price. Table 3.1 illustrates a simplified transaction log that reflects a basic transaction composed of two SQL UPDATE statements. If a system failure occurs, the DBMS will examine the transaction log for all uncommitted or incomplete transactions and restore (ROLLBACK) the database to its previous state on the basis of that information. When the recovery process is completed, the DBMS will write in the log all committed transactions that were not physically written to the database before the failure occurred. If a ROLLBACK is issued before the termination of a transaction, the DBMS will restore the database only for that particular transaction, rather than for all of them, to maintain the *durability* of the previous transactions. In other words, committed transactions are not rolled back. The transaction log is a critical part of the database, and it is usually implemented as one or more files that are managed separately from the actual database files. The transaction log is subject to common dangers such as disk-full conditions and disk crashes. Because the transaction log contains some of the most critical data in a DBMS, some implementations support logs on several different disks to reduce the consequences of a system failure.

Table 3-1: A Transaction Log

TRL_ID	TRX_NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_ BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				

TRL_ID = Transaction log record ID
TRX_NUM = Transaction number
PTR = Pointer to a transaction log record ID

Concurrency Control

Coordinating the simultaneous execution of transactions in a multiuser database system is known as **concurrency control**. The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment. To achieve this goal, most concurrency control techniques are oriented toward preserving the isolation property of concurrently executing transactions. Concurrency control is important because the simultaneous execution of transactions over a shared database can create several data integrity and consistency problems. The three main problems are lost updates, uncommitted data, and inconsistent retrievals.

Lost Updates

The **lost update** problem occurs when two concurrent transactions, T1 and T2, are updating the same data element and one of the updates is lost (overwritten by the other transaction). To see an illustration of lost updates, examine a simple PRODUCT table. One of the table's attributes is a product's quantity on hand (PROD_QOH). Assume that you have a product whose current PROD_QOH value is 35. Also assume that two concurrent transactions, T1 and T2, occur and update the PROD_QOH value for some item in the PRODUCT table. The transactions are shown in Table 3.2.

Table 3-2: Two Concurrent Transactions to Update PROD_QOH

Transaction	Computation
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100
T2: Sell 30 units	PROD_QOH = PROD_QOH − 30

Table 3.3 shows the serial execution of the transactions under normal circumstances, yielding the correct answer PROD_QOH =105.

Table 3-3: Serial Execution of Two Transactions

Time	Transaction	Step	Stored Value
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	PROD_QOH = 135 − 30	
6	T2	Write PROD_QOH	105

However, suppose that a transaction can read a product's PROD_QOH value from the table *before* a previous transaction has been committed, using the *same* product. The sequence depicted in Table 3.4 shows how the lost update problem can arise. Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed. Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory. In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is "lost" during the process.

Table 3-4: Lost updates

Time	Transaction	Step	Stored Value
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	PROD_QOH = 35 + 100	
4	T2	PROD_QOH = 35 – 30	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted Data

The phenomenon of **uncommitted data** occurs when two transactions, T1 and T2, are executed concurrently and the first transaction (T1) is rolled back after the second transaction (T2) has already accessed the uncommitted data—thus violating the isolation property of transactions. To illustrate that possibility, use the same transactions described during the lost updates discussion. T1 has two atomic parts, one of which is the update of the inventory; the other possible part is the update of the invoice total (not shown). T1 is forced to roll back due to an error during the updating of the invoice's total; it rolls back all the way, undoing the inventory update as well. This time the T1 transaction is rolled back to eliminate the addition of the 100 units. (See Table 3.5.) Because T2 subtracts 30 from the original 35 units, the correct answer should be 5.

Table 3-5: Transaction Creating an Uncommitted Data Problem

Transaction	Computation
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100 (Rolled back)
T2: Sell 30 units	PROD_QOH = PROD_QOH – 30

Table 3.6 shows how the serial execution of these transactions yields the correct answer under normal circumstances.

Table 3-6: Correct Execution of Two Transactions

Time	Transaction	Step	Stored Value
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	PROD_QOH = 35 – 30	
7	T2	Write PROD_QOH	5

Table 3.6

Table 3.7 shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

Table 3-7: An uncommitted data problem

Time	Transaction	Step	Stored Value
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	PROD_QOH = 135 − 30	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrievals

Inconsistent retrievals occur when a transaction accesses data before and after one or more other transactions finish working with such data. For example, an inconsistent retrieval would occur if transaction T1 calculated some summary (aggregate) function over a set of data while another transaction (T2) was updating the same data. The problem is that the transaction might read some data before it is changed and other data *after* it is changed, thereby yielding inconsistent results. To illustrate the problem, assume the following conditions:

1. T1 calculates the total quantity on hand of the products stored in the PRODUCT table.
2. At the same time, T2 updates the quantity on hand (PROD_QOH) for two of the PRODUCT table's products.

The two transactions are shown in Table 3.8.

Table 3-8 :Retrieval during update

Transaction T1	Transaction T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = 1546-QQ2
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH − 10 WHERE PROD_CODE = 1558-QW1
	COMMIT;

While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but meant to add the 10 units to product 1546-QQ2's PROD_QOH. To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH. (See the two UPDATE statements in Table 3.8.) The initial and final PROD_QOH values are reflected in Table 3.9. (Only a few PROD_CODE values are shown for the PRODUCT table. To illustrate the point, the sum for the PROD_QOH values is shown for these few products.)

Table 3-9: Transaction Results: Data Entry Correction

	Before	After
PROD_CODE	**PROD_QOH**	**PROD_QOH**
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	(15 + 10) → 25
1558-QW1	23	(23 − 10) → 13
2232-QTY	8	8
2232-QWE	6	6
Total	**92**	**92**

Although the final results shown in Table 3.9 are correct after the adjustment, Table 3.10 demonstrates that inconsistent retrievals are possible during the transaction execution, making the result of T1's execution incorrect. The "After" summation shown in Table 3.10 reflects that the value of 25 for product 1546-QQ2 was read *after* the WRITE statement was completed. Therefore, the "After" total is 40 + 25 = 65. The "Before" total reflects that the value of 23 for product 1558-QW1 was read *before* the next WRITE statement was completed to reflect the corrected update of 13. Therefore, the "Before" total is 65 + 23 = 88. The computed answer of 102 is wrong because you know from Table 3.9 that the correct answer is 92. Unless the DBMS exercises concurrency control, a multiuser database environment can create havoc within the information system.

Table 3-10: Inconsistent Retrievals

Time	Transaction	Action	Value	Total
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 − 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

The Scheduler

You now know that severe problems can arise when two or more concurrent transactions are executed. You also know that a database transaction involves a series of database I/O operations that take the database from one consistent state to another. Finally, you know that database consistency can be ensured only before and after the execution of transactions. A database always moves through an unavoidable temporary state of inconsistency during a transaction's execution if such a transaction updates multiple tables and rows. (If the transaction contains only one update, then there is no temporary inconsistency.) The temporary inconsistency exists because a computer executes the operations serially, one after another. During this serial process, the isolation property of transactions prevents them from accessing the data not yet

released by other transactions. This consideration is even more important today, with the use of multicore processors that can execute several instructions at the same time. What would happen if two transactions executed concurrently and they were accessing the same data?

In previous examples, the operations within a transaction were executed in an arbitrary order. As long as two transactions, T1 and T2, access *unrelated* data, there is no conflict and the order of execution is irrelevant to the final outcome. However, if the transactions operate on related data or the same data, conflict is possible among the transaction components and the selection of one execution order over another might have some undesirable consequences. So, how is the correct order determined, and who determines that order? Fortunately, the DBMS handles that tricky assignment by using a built-in scheduler. The **scheduler** is a special DBMS process that establishes the order in which the operations are executed within concurrent transactions. The scheduler *interleaves* the execution of database operations to ensure serializability and isolation of transactions. To determine the appropriate order, the scheduler bases its actions on concurrency control algorithms, such as locking or time stamping methods, which are explained in the next sections. However, it is important to understand that not all transactions are serializable. The DBMS determines what transactions are serializable and proceeds to interleave the execution of the transaction's operations.

Generally, transactions that are not serializable are executed on a first-come, first-served basis by the DBMS. The scheduler's main job is to create a **serializable schedule** of a transaction's operations, in which the interleaved execution of the transactions (T1, T2, T3, etc.) yields the same results as if the transactions were executed in serial order (one after another). The scheduler also makes sure that the computer's central processing unit (CPU) and storage systems are used efficiently. If there were no way to schedule the execution of transactions, all of them would be executed on a first-come, first-served basis. The problem with that approach is that processing time is wasted when the CPU waits for a READ or WRITE operation to finish, thereby losing several CPU cycles. In short, first-come, first-served scheduling tends to yield unacceptable response times within the multiuser DBMS environment. Therefore, some other scheduling method is needed to improve the efficiency of the overall system. Additionally, the scheduler facilitates data isolation to ensure that two transactions do not update the same data element at the same time. Database operations might require READ and/or WRITE actions that produce conflicts. For example, Table 3.11 shows the possible conflict scenarios when two transactions, T1 and T2, are executed concurrently over the same data. Note that in Table 3.11, two operations are in conflict when they access the same data and at least one of them is a WRITE operation.

Table 3-11: Read/Write Conflict Scenarios: Conflicting Database Operations Matrix

	Transactions		
	T1	**T2**	**Result**
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

Several methods have been proposed to schedule the execution of conflicting operations in concurrent transactions. These methods are classified as locking, time stamping, and optimistic. Locking methods, discussed next, are used most frequently.

Concurrency Control with Locking Methods

Locking methods are one of the most common techniques used in concurrency control because they facilitate the isolation of data items used in concurrently executing transactions. A **lock** guarantees exclusive use of a data item to a current transaction. In other words, transaction T2 does not have access to a data item that is currently being used by transaction T1. A transaction acquires a lock prior to data access; the lock is released (unlocked) when the transaction is completed so that another transaction can lock the data item for its exclusive use. This series of locking actions assumes that concurrent transactions might attempt to manipulate the same data item at the same time. The use of locks based on the assumption that conflict between transactions is likely is usually referred to as **pessimistic locking**.

Data consistency cannot be guaranteed *during* a transaction; the database might be in a temporary inconsistent state when several updates are executed. Therefore, locks are required to prevent another transaction from reading inconsistent data. Most multiuser DBMSs automatically initiate and enforce locking procedures. All lock information is handled by a **lock manager**, which is responsible for assigning and policing the locks used by the transactions.

Lock Granularity

Lock granularity indicates the level of lock use. Locking can take place at the following levels: database, table, page, row, or even field (attribute).

Database Level

In a **database-level lock**, the entire database is locked, thus preventing the use of any tables in the database by transaction T2 while transaction T1 is being executed. This level of locking is good for batch processes, but it is unsuitable for multiuser DBMSs. You can imagine how s-l-o-w data access would be if thousands of transactions had to wait for the previous transaction to be completed before the next one could reserve the entire database. Figure 3.3 illustrates the database-level lock; because of it, transactions T1 and T2 cannot access the same database concurrently *even when they use different tables*.

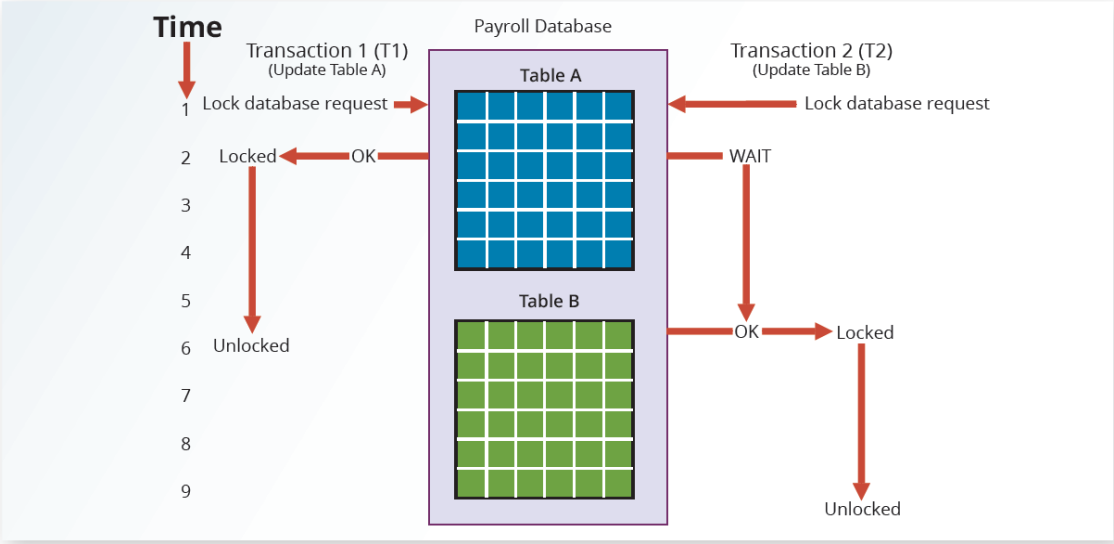


Figure 3-3: Database-Level Locking Sequence (Coronel C., Morris S. 2022)

Table Level

In a **table-level lock**, the entire table is locked, preventing access to any row by transaction T2 while transaction T1 is using the table. If a transaction requires access to several tables, each table may be locked. However, two transactions can access the same database as long as they access different tables. Table-level locks, while less restrictive than database-level locks, cause traffic jams when many transactions are waiting to access the same table. Such a condition is especially irksome if the lock forces a delay when different transactions require access to different parts of the same table—that is, when the transactions would not interfere with each other. Consequently, table-level locks are not suitable for multiuser DBMSs. Figure 3.4 illustrates the effect of a table-level lock. Note that transactions T1 and T2 cannot access the same table even when they try to use different rows; T2 must wait until T1 unlocks the table.

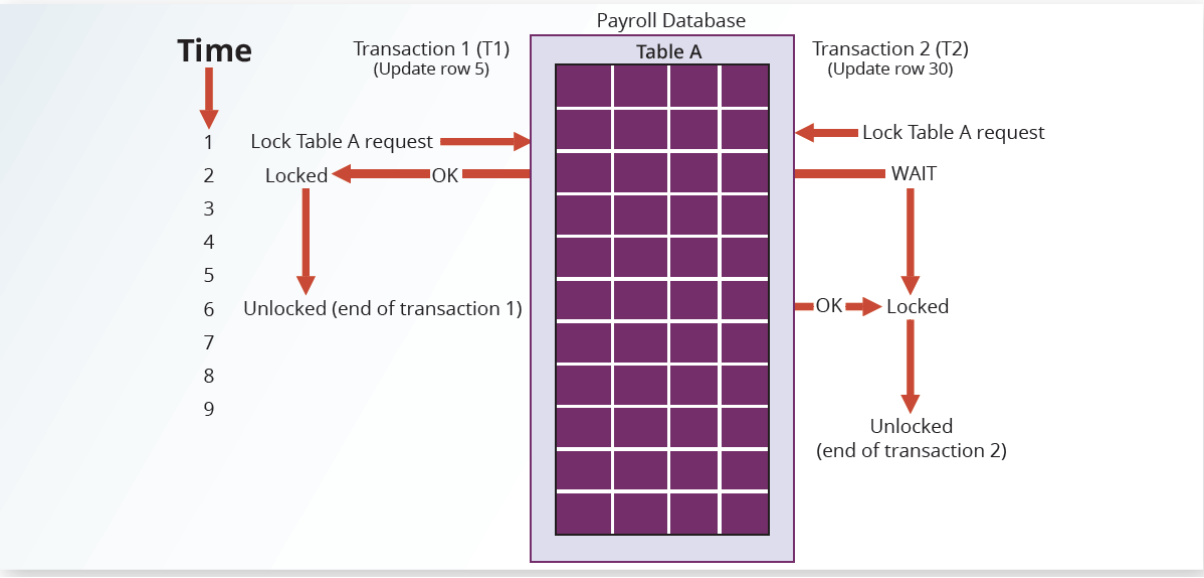


Figure 3-4: An Example of a table-level lock (Coronel C., Morris S. 2022)

Page Level

In a **page-level lock**, the DBMS locks an entire disk-page. A **diskpage**, or **page**, is the equivalent of a *disk-block*, which can be described as a directly addressable section of a disk. A page has a fixed size, such as 4K, 8K, or 16K. For example, if you want to write only 73 bytes to a 4K page, the entire 4K page must be read from disk, updated in memory, and written back to disk. A table can span several pages, and a page can contain several rows of one or more tables. Page-level locks are currently the most frequently used locking method for multiuser DBMSs. An example of a page-level lock is shown in Figure 3.5. Note that T1 and T2 access the same table while locking different diskpages. If T2 requires the use of a row located on a page that is locked by T1, T2 must wait until T1 unlocks the page.

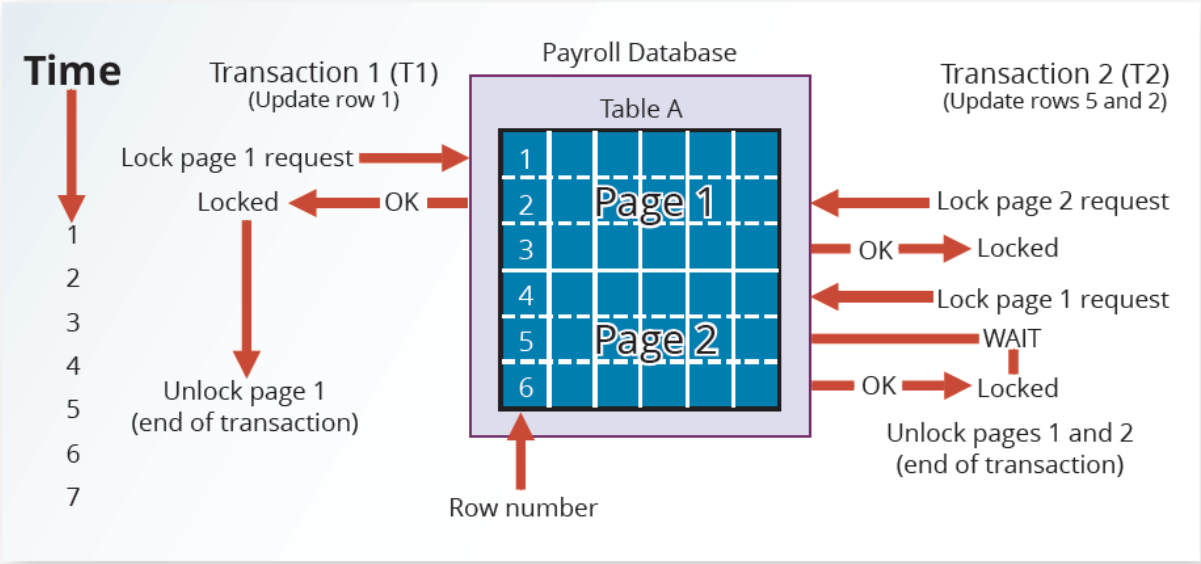


Figure 3-5: An example of a page-level lock(Coronel C., Morris S. 2022)

Row Level

A **row-level lock** is much less restrictive than the locks discussed earlier. The DBMS allows concurrent transactions to access different rows of the same table even when the rows are located on the same page. Although the row-level locking approach improves the availability of data, its management requires high overhead because a lock exists for each row in a table of the database involved in a conflicting transaction. Modern DBMSs automatically escalate a lock from a row level to a page level when the application session requests multiple locks on the same page. Figure 3.6 illustrates the use of a row-level lock.

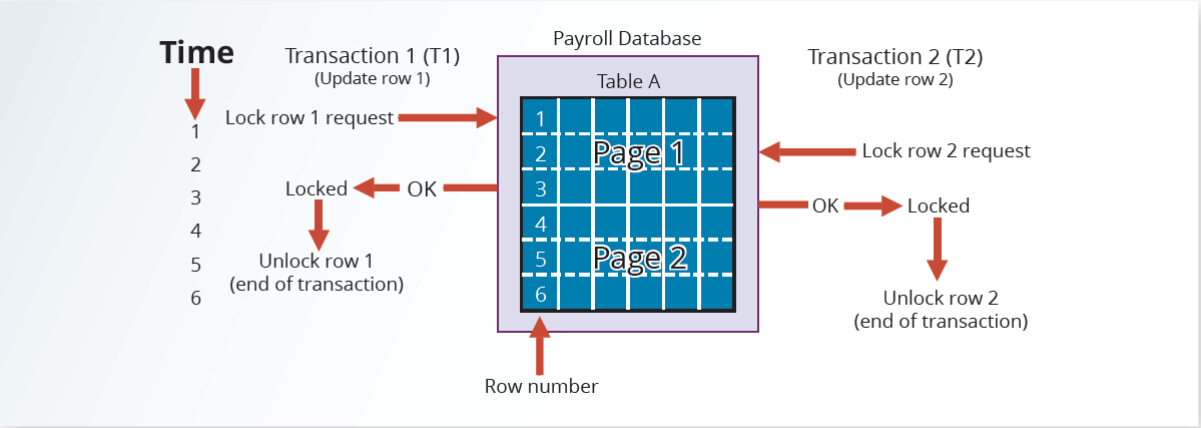


Figure 3-6: An example of a row-level lock (Coronel C., Morris S. 2022)

Note in Figure 3.6 that both transactions can execute concurrently, even when the requested rows are on the same page. T2 must wait only if it requests the same row as T1.

Field Level

The **field-level lock** allows concurrent transactions to access the same row as long as they require the use of different fields (attributes) within that row. Although field-level locking clearly yields the most flexible multiuser data access, it is rarely implemented in a DBMS because it requires an extremely high level of computer overhead and because the row-level lock is much more useful in practice.

Lock Types

Regardless of the level of granularity of the lock, the DBMS may use different lock types or modes: binary or shared/exclusive.

Binary

A **binary lock** has only two states: locked (1) or unlocked (0). If an object such as a database, table, page, or row is locked by a transaction, no other transaction can use that object. If an object is unlocked, any transaction can lock the object for its use. Every database operation requires that the affected object be locked. As a rule, a transaction must unlock the object after its termination. Therefore, every transaction requires a lock and unlock operation for each accessed data item. Such operations are automatically managed and scheduled by the DBMS; the user does not lock or unlock data items. (Every DBMS has a default-locking mechanism. If the end user wants to override the default settings, the LOCK TABLE command and other SQL commands are available for that purpose.)

The binary locking technique is illustrated in Table 3.12, using the lost update problem you encountered in Table 3.4. Note that the lock and unlock features eliminate the lost update problem because the lock is not released until the WRITE statement is completed. Therefore, a PROD_QOH value cannot be used until it has been properly updated. However, binary locks are now considered too restrictive to yield optimal concurrency conditions. For example, the DBMS will not allow two transactions to read the same database object even though neither transaction updates the database and therefore no concurrency problems can occur. Remember from Table 3.11 that concurrency conflicts occur only when two transactions execute concurrently and one of them updates the database.

Table 3-12: An example of a Binary Lock

Time	Transaction	Step	Stored Value
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	15
3	T1	PROD_QOH = 15 + 10	
4	T1	Write PROD_QOH	25
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	23
8	T2	PROD_QOH = 23 − 10	
9	T2	Write PROD_QOH	13
10	T2	Unlock PRODUCT	

Shared/Exclusive

An **exclusive lock** exists when access is reserved specifically for the transaction that locked the object. The exclusive lock must be used when the potential for conflict exists (see Table 3.11). A **shared lock** exists when concurrent transactions are granted read access on the basis of a common lock. A shared lock produces no conflict as long as all the concurrent transactions are read-only. A shared lock is issued when a transaction wants to read data from the database and no exclusive lock is held on that data item. An exclusive lock is issued when a transaction wants to update (write) a data item and no locks are currently held on that data item by any other transaction. Using the shared/exclusive locking concept, a lock can have three states: unlocked,

shared (read), and exclusive (write). As shown in Table 3.11, two transactions conflict only when at least one is a write transaction.

Because the two read transactions can be safely executed at once, shared locks allow several read transactions to read the same data item concurrently. For example, if transaction T1 has a shared lock on data item X and transaction T2 wants to read data item X, T2 may also obtain a shared lock on data item X. If transaction T2 updates data item X, an exclusive lock is required by T2 over data item X. *The exclusive lock is granted if, and only if, no other locks are held on the data item* (this condition is known as the **mutual exclusive rule**: only one transaction at a time can own an exclusive lock on an object.) Therefore, if a shared (or exclusive) lock is already held on data item X by transaction T1, an exclusive lock cannot be granted to transaction T2, and T2 must wait to begin until T1 commits. In other words, a shared lock will always block an exclusive (write) lock; hence, decreasing transaction concurrency. Although the use of shared locks renders data access more efficient, a shared/exclusive lock schema increases the lock manager's overhead for several reasons:

- The type of lock held must be known before a lock can be granted.
- Three lock operations exist: READ_LOCK to check the type of lock, WRITE_LOCK to issue the lock, and UNLOCK to release the lock.
- The schema has been enhanced to allow a lock upgrade from shared to exclusive and a lock downgrade from exclusive to shared.

Although locks prevent serious data inconsistencies, they can lead to two major problems:

- The resulting transaction schedule might not be serializable.
- The schedule might create deadlocks. A **deadlock** occurs when two transactions wait indefinitely for each other to unlock data. A database deadlock, which is similar to traffic gridlock in a big city, is caused when two or more transactions wait for each other to unlock data.

Fortunately, both problems can be managed: serializability is attained through a locking protocol known as two-phase locking, and deadlocks can be managed by using deadlock detection and prevention techniques. Those techniques are examined in the next two sections.

Two-Phase Locking to Ensure Serializability

Two-phase locking (2PL) defines how transactions acquire and relinquish locks. Two-phase locking guarantees serializability, but it does not prevent deadlocks. The two phases are:

1. A growing phase, in which a transaction acquires all required locks without unlocking any data. After all locks have been acquired, the transaction is in its locked point.
2. A shrinking phase, in which a transaction releases all locks and cannot obtain a new lock.

The two-phase locking protocol is governed by the following rules:

- Two transactions cannot have conflicting locks.
- No unlock operation can precede a lock operation in the same transaction.
- No data is affected until all locks are obtained—that is, until the transaction is in its locked point.

Figure 3.7 depicts the two-phase locking protocol. In this example, the transaction first acquires the two locks it needs. When it has the two locks, it reaches its locked point. Next, the data is modified to conform to the transaction's requirements. Finally, the transaction is completed as it releases all of the locks it acquired in the first phase. Two-phase locking increases the transaction processing cost and might cause additional undesirable effects, such as deadlocks.

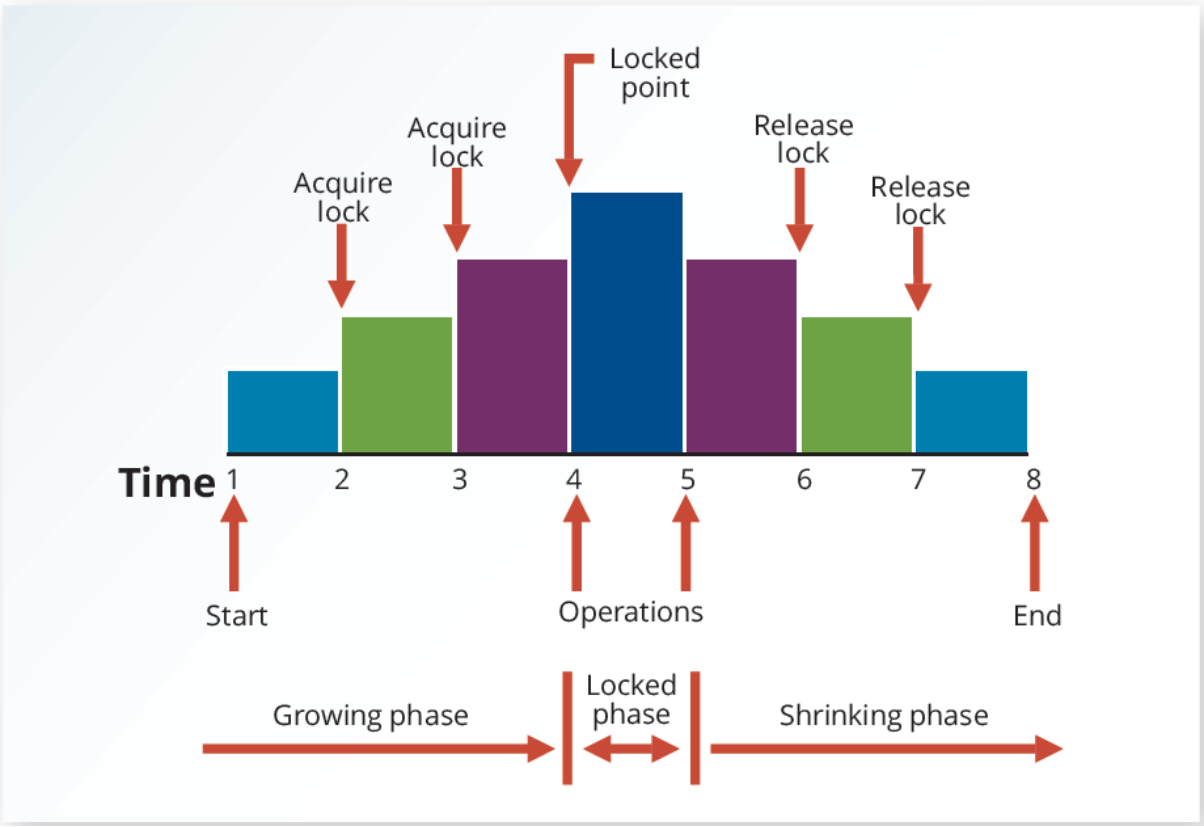


Figure 3-7: Two-phase Locking Protocol (Coronel C., Morris S. 2022)

Deadlocks

A deadlock occurs when two transactions wait indefinitely for each other to unlock data. For example, a deadlock occurs when two transactions, T1 and T2, exist in the following mode:
T1 = access data items X and Y
T2 = access data items Y and X
If T1 has not unlocked data item Y, T2 cannot begin; if T2 has not unlocked data item X, T1 cannot continue. Consequently, T1 and T2 each wait for the other to unlock the required data item. Such a deadlock is also known as a **deadly embrace**. Table 3.13 demonstrates how a deadlock condition is created.

The preceding example used only two concurrent transactions to demonstrate a deadlock condition. In a real-world DBMS, many more transactions can be executed simultaneously, thereby increasing the probability of generating deadlocks. Note that deadlocks are possible only when one of the transactions wants to obtain an exclusive lock on a data item; no deadlock condition can exist among *shared* locks.

Table 3-13: How a Deadlock Condition is Created

Time	Transaction	Reply	Lock Status	
			Data X	Data Y
0			Unlocked	Unlocked
1	T1:LOCK(X)	OK	Locked	Unlocked
2	T2:LOCK(Y)	OK	Locked	Locked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
...	.....	.....	.....	.....
...	.....	.....	.....	.....
...	.....	.....	.....	.....
...	.....	.....	.....	.....

Deadlock

The three basic techniques to control deadlocks are:

- *Deadlock prevention*. A transaction requesting a new lock is aborted when there is the possibility that a deadlock can occur. If the transaction is aborted, all changes made by this transaction are rolled back and all locks obtained by the transaction are released. The transaction is then rescheduled for execution. Deadlock prevention works because it avoids the conditions that lead to deadlocking.
- *Deadlock detection*. The DBMS periodically tests the database for deadlocks. If a deadlock is found, the "victim" transaction is aborted (rolled back and restarted) and the other transaction continues.
- *Deadlock avoidance*. The transaction must obtain all of the locks it needs before it can be executed. This technique avoids the rolling back of conflicting transactions by requiring that locks be obtained in succession. However, the serial lock assignment required in deadlock avoidance increases action response times.

The choice of which deadlock control method to use depends on the database environment. For example, if the probability of deadlocks is low, deadlock detection is recommended. However, if the probability of deadlocks is high, deadlock prevention is recommended. If response time is not high on the system's priority list, deadlock avoidance might be employed. All current DBMSs support deadlock detection in transactional databases, while some DBMSs use a blend of prevention and avoidance techniques for other types of data, such as data warehouses or XML data.

Concurrency Control with Time Stamping Methods

The **time stamping** approach to scheduling concurrent transactions assigns a global, unique time stamp to each transaction. The time stamp value produces an explicit order in which transactions are submitted to the DBMS. Time stamps must have two properties: uniqueness and monotonicity. **Uniqueness** ensures that no equal time stamp values can exist, and **monotonicity** ensures that time stamp values always increase. **Monotonicity** is a quality that ensures that time stamp values always increase. (The time stamping approach to scheduling concurrent transactions assigns a global, unique timestamp to each transaction. The time stamp value produces an explicit order in which transactions are submitted to the DBMS.)

All database operations (read and write) within the same transaction must have the same time stamp. The DBMS executes conflicting operations in time stamp order, thereby ensuring serializability of the transactions. If two transactions conflict, one is stopped, rolled back, rescheduled, and assigned a new time stamp value. The disadvantage of the time stamping approach is that each value stored in the database requires two additional time stamp fields: one for the last time the field was read and one for the last update. Time stamping thus increases memory needs and the database's processing overhead. Time stamping demands a lot of system resources because many transactions might have to be stopped, rescheduled, and restamped.

Wait/Die and Wound/Wait Schemes

Time stamping methods are used to manage concurrent transaction execution. In this section, you will learn about two schemes used to decide which transaction is rolled back and which continues executing: the wait/die scheme and the wound/wait scheme. An example illustrates the difference. Assume that you have two conflicting transactions: T1 and T2, each with a unique time stamp. Suppose that T1 has a time stamp of 11548789 and T2 has a time stamp of 19562545. You can deduce from the time stamps that T1 is the older transaction (the lower time stamp value), and T2 is the newer transaction. Given that scenario, the four possible outcomes are shown in Table 3.14.

Table 3-14: Wait/Die and Wound/Wait Concurrency Control Schemes

Transaction Requesting Lock	Transaction Owning Lock	Wait/Die Scheme	Wound/Wait Scheme
T1 (11548789)	T2 (19562545)	• T1 waits until T2 is completed and T2 releases its locks.	• T1 preempts (rolls back) T2. • T2 is rescheduled using the same time stamp.
T2 (19562545)	T1 (11548789)	• T2 dies (rolls back). • T2 is rescheduled using the same time stamp.	• T2 waits until T1 is completed and T1 releases its locks.

Using the wait/die scheme:

- If the transaction requesting the lock is the older of the two transactions, it will *wait* until the other transaction is completed and the locks are released.
- If the transaction requesting the lock is the younger of the two transactions, it will *die* (roll back) and is rescheduled using the same time stamp.

In short, in the **wait/die** scheme, the older transaction waits for the younger one to complete and release its locks.

In the wound/wait scheme:

- If the transaction requesting the lock is the older of the two transactions, it will pre-empt (*wound*) the younger transaction by rolling it back. T1 preempts T2 when T1 rolls back T2. The younger, preempted transaction is rescheduled using the same time stamp.
- If the transaction requesting the lock is the younger of the two transactions, it will wait until the other transaction is completed and the locks are released. In short, in the **wound/wait** scheme, the older transaction rolls back the younger transaction and reschedules it.

In both schemes, one of the transactions waits for the other transaction to finish and release the locks. However, in many cases, a transaction requests multiple locks. How long does a

transaction have to wait for each lock request? Obviously, that scenario can cause some transactions to wait indefinitely, causing a deadlock. To prevent a deadlock, each lock request has an associated time-out value. If the lock is not granted before the time-out expires, the transaction is rolled back.

Concurrency Control with Optimistic Methods

The **optimistic approach** is based on the assumption that the majority of database operations do not conflict. The optimistic approach requires neither locking nor time stamping techniques. Instead, a transaction is executed without restrictions until it is committed. Using an optimistic approach, each transaction moves through two or three phases, referred to as *read*, *validation*, and *write*.

- During the *read phase*, the transaction reads the database, executes the needed computations, and makes the updates to a private copy of the database values. All update operations of the transaction are recorded in a temporary update file, which is not accessed by the remaining transactions.
- During the *validation phase*, the transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database. If the validation test is positive, the transaction goes to the write phase. If the validation test is negative, the transaction is restarted and the changes are discarded.
- During the *write phase*, the changes are permanently applied to the database. The optimistic approach is acceptable for most read or query database systems that require few update transactions. In a heavily used DBMS environment, the management of deadlocks— their prevention and detection—constitutes an important DBMS function. The DBMS will use one or more of the techniques discussed here, as well as variations on those techniques. To further understand how transaction management is implemented in a database, it is important that you learn about the transaction isolation levels as defined in ANSI SQL 1992 standard.

ANSI Levels of Transaction Isolation

The ANSI SQL standard (1992) defines transaction management based on transaction isolation levels. Transaction isolation levels refer to the degree to which transaction data is "protected or isolated" from other concurrent transactions. The isolation levels are described based on what data other transactions can see (read) during execution. More precisely, the transaction isolation levels are described by the type of "reads" that a transaction allows or does not allow. The types of read operations are:

- **Dirty read**: a transaction can read data that is not yet committed.
- **Non-repeatable read**: a transaction reads a given row at time t1, and then it reads the same row at time t2, yielding different results. The original row may have been updated or deleted.
- **Phantom read**: a transaction executes a query at time t1, and then it runs the same query at time t2, yielding additional rows that satisfy the query.

Based on the above operations, ANSI defined four levels of transaction isolation: Read Uncommitted, Read Committed, Repeatable Read, and Serializable. Table 3.15 shows the four ANSI transaction isolation levels. The table also shows an additional level of isolation provided by Oracle and MS SQL Server databases.

Read Uncommitted will read uncommitted data from other transactions. At this isolation level, the database does not place any locks on the data, which increases transaction performance

but at the cost of data consistency. **Read Committed** forces transactions to read only committed data. This is the default mode of operation for most databases (including Oracle and SQL Server). At this level, the database will use exclusive locks on data, causing other transactions to wait until the original transaction commits. The **Repeatable Read** isolation level ensures that queries return consistent results. This type of isolation level uses shared locks to ensure other transactions do not update a row after the original query reads it. However, new rows are read (phantom read) as these rows did not exist when the first query ran. The **Serializable** isolation level is the most restrictive level defined by the ANSI SQL standard. However, it is important to note that even with a Serializable isolation level, deadlocks are always possible. Most databases use a deadlock detection approach to transaction management, and, therefore, they will detect "deadlocks" during the transaction validation phase and reschedule the transaction.

Table 3-15: Transaction Isolation Levels

	Isolation Level	Allowed			Comment
		Dirty Read	Nonrepeatable Read	Phantom Read	
Less restrictive ↕ More restrictive	Read Uncommitted	Y	Y	Y	Reads uncommitted data, and allows nonrepeatable reads and phantom reads.
	Read Committed	N	Y	Y	Does not allow uncommitted data reads but allows nonrepeatable reads and phantom reads.
	Repeatable Read	N	N	Y	Only allows phantom reads.
	Serializable	N	N	N	Does not allow dirty reads, nonrepeatable reads, or phantom reads.
Oracle/SQL Server Only	Read Only/Snapshot	N	N	N	Supported by Oracle and SQL Server. The transaction can only see the changes that were committed at the time the transaction started.

The reason for the different levels of isolation is to increase transaction concurrency. The isolation levels go from the least restrictive (Read Uncommitted) to the more restrictive (Serializable). The higher the isolation level the more locks (shared and exclusive) are required to improve data consistency, at the expense of transaction concurrency performance. The isolation level of a transaction is defined in the transaction statement, for example using general ANSI SQL syntax:

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED
... SQL STATEMENTS....
COMMIT TRANSACTION;
```

MySQL, Oracle, and MS SQL Server use the SET TRANSACTION ISOLATION LEVEL statement to define the level of isolation for a specific transaction. MySQL and SQL Server support all four ANSI isolation levels. Oracle supports READ COMMITED and SERIALIZABLE as defined by the ANSI standard. Oracle would suggest that it has superior implementation of the other two isolation levels. Notice that the standard defines the levels of isolation based on which problems are allowed, not on the goal that is to be achieved. Oracle uses concurrency techniques that achieve the goals of the other isolation levels without allowing the problems specified in the standard. For example, the argument can be made that the purpose of the READ UNCOMMITED level is to avoid blocking any read access by any transaction. One way to do this is to allow transactions to read uncommitted changes made by other transactions (i.e., a dirty read), which is what the ANSI standard specifies. Oracle's approach, however, is to use a multi-versioning

system that will never block the read access of any transaction, but without allowing the problem of dirty reads. As you can see from the previous discussion, transaction management is a complex subject and databases make use of various techniques to manage the concurrent execution of transactions. However, it may be necessary sometimes to employ database recovery techniques to restore the database to a consistent state.

Database Recovery Management

Database recovery restores a database from a given state (usually inconsistent) to a previously consistent state. Recovery techniques are based on the **atomic transaction property**: all portions of the transaction must be treated as a single, logical unit of work in which all operations are applied and completed to produce a consistent database. If a transaction operation cannot be completed for some reason, the transaction must be aborted and any changes to the database must be rolled back (undone). In short, transaction recovery reverses all of the changes that the transaction made to the database before the transaction was aborted. Although this chapter has emphasized the recovery of *transactions*, recovery techniques also apply to the *database* and to the *system* after some type of critical error has occurred. Critical events can cause a database to stop working and compromise the integrity of the data. Examples of critical events are:

- *Hardware/software failures*. A failure of this type could be a hard disk media failure, a bad capacitor on a motherboard, or a failing memory bank. Other causes of errors under this category include application program or operating system errors that cause data to be overwritten, deleted, or lost. Some database administrators argue that this is one of the most common sources of database problems.
- *Human-caused incidents*. This type of event can be categorized as unintentional or intentional.
    - An unintentional failure is caused by a careless end user. Such errors include deleting the wrong rows from a table, pressing the wrong key on the keyboard, or shutting down the main database server by accident.
    - Intentional events are of a more severe nature and normally indicate that the company data is at serious risk. Under this category are security threats caused by hackers trying to gain unauthorized access to data resources and virus attacks caused by disgruntled employees trying to compromise the database operation and damage the company.
- *Natural disasters*. This category includes fires, earthquakes, floods, and power failures. Whatever the cause, a critical error can render the database into an inconsistent state. The following section introduces the various techniques used to recover the database from an inconsistent state to a consistent state.

Transaction Recovery

Database transaction recovery uses data in the transaction log to recover a database from an inconsistent state to a consistent state. Before continuing, examine four important concepts that affect the recovery process:

- The **write-ahead-log protocol** ensures that transaction logs are always written *before* any database data is actually updated. This protocol ensures that, in case of a failure, the database can later be recovered to a consistent state using the data in the transaction log.
- **Redundant transaction logs** (several copies of the transaction log) ensure that a physical disk failure will not impair the DBMS's ability to recover data.

- Database **buffers** are temporary storage areas in primary memory used to speed up disk operations. To improve processing time, the DBMS software reads the data from the physical disk and stores a copy of it on a "buffer" in primary memory. When a transaction updates data, it actually updates the copy of the data in the buffer because that process is much faster than accessing the physical disk every time. Later, all buffers that contain updated data are written to a physical disk during a single operation, thereby saving significant processing time.
- Database **checkpoints** are operations in which the DBMS writes all of its updated buffers in memory (also known as *dirty buffers*) to disk. While this is happening, the DBMS does not execute any other requests. A checkpoint operation is also registered in the transaction log. As a result of this operation, the physical database and the transaction log will be in sync. This synchronization is required because update operations update the copy of the data in the buffers and not in the physical database. Checkpoints are automatically and periodically executed by the DBMS according to certain operational parameters (such as a high watermark for the transaction log size or volume of outstanding transactions) but can also be executed explicitly (as part of a database transaction statement) or implicitly (as part of a database backup operation). Of course, checkpoints that are too frequent would affect transaction performance; checkpoints that are too infrequent would affect database recovery performance. In any case, checkpoints serve a very practical function as they also play an important role in transaction recovery.

The database recovery process involves bringing the database to a consistent state after a failure. Transaction recovery procedures generally make use of deferred-write and writethrough techniques. When the recovery procedure uses a **deferred-write technique** (also called a **deferred update**), the transaction operations do not immediately update the physical database. Instead, only the transaction log is updated. The database is physically updated only with data from committed transactions, using information from the transaction log. If the transaction aborts before it reaches its commit point, no changes (no ROLLBACK or undo) need to be made to the database because it was never updated. To better visualize this example, see Figure 3.8. The recovery process for all started and committed transactions (before the failure) follows these steps:

1. Identify the last checkpoint in the transaction log. This is the last time transaction data was physically saved to disk.
2. For a transaction that started and was committed before the last checkpoint, nothing needs to be done because the data is already saved.
3. For a transaction that performed a commit operation after the last checkpoint, the DBMS uses the transaction log records to redo the transaction and update the database, using the "after" values in the transaction log. The changes are made in ascending order, from oldest to newest.
4. For any transaction that had a ROLLBACK operation after the last checkpoint or that was left active (with neither a COMMIT nor a ROLLBACK) before the failure occurred, nothing needs to be done because the database was never updated.

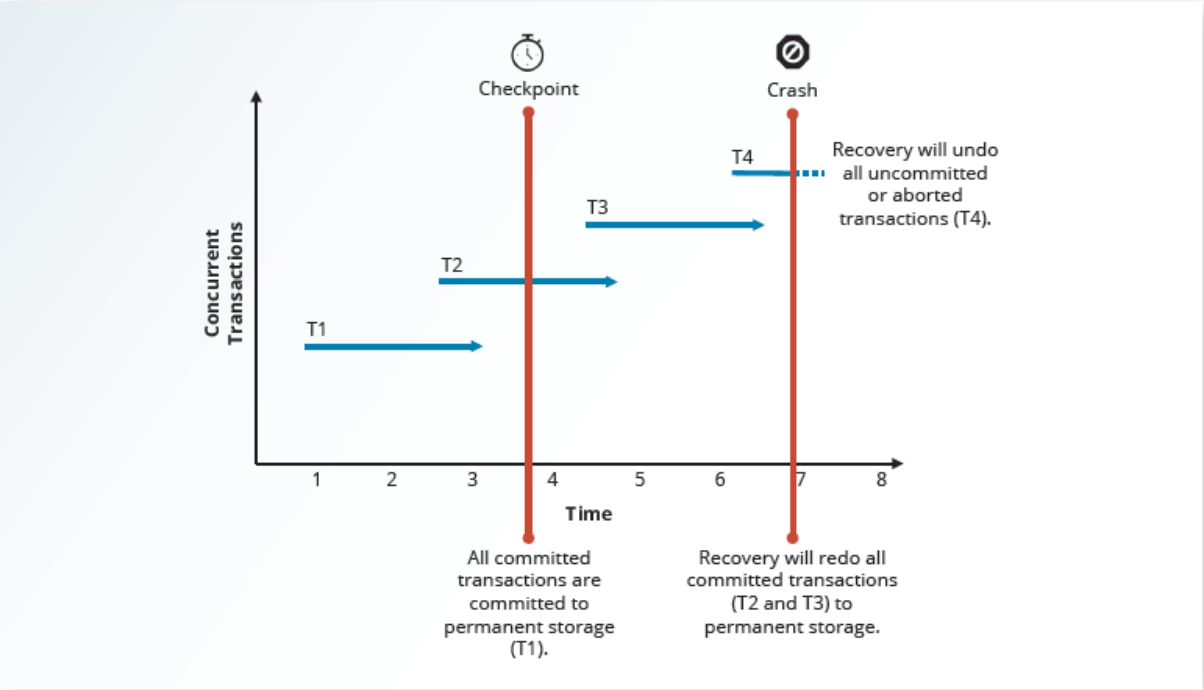


Figure 3-8: Recovery of Transactions (Coronel C., Morris S. 2022)

When the recovery procedure uses a **write-through technique** (also called an **immediate update**), the database is immediately updated by transaction operations during the transaction's execution, even before the transaction reaches its commit point. If the transaction aborts before it reaches its commit point, a ROLLBACK or undo operation needs to be done to restore the database to a consistent state. In that case, the ROLLBACK operation will use the transaction log "before" values. The recovery process follows these steps:

1. Identify the last checkpoint in the transaction log. This is the last time transaction data was physically saved to disk.
2. For a transaction that started and was committed before the last checkpoint, nothing needs to be done because the data is already saved.
3. For a transaction that was committed after the last checkpoint, the DBMS re-does the transaction, using the "after" values of the transaction log. Changes are applied in ascending order, from oldest to newest.
4. For any transaction that had a ROLLBACK operation after the last checkpoint or that was left active (with neither a COMMIT nor a ROLLBACK) before the failure occurred, the DBMS uses the transaction log records to ROLLBACK or undo the operations, using the "before" values in the transaction log. Changes are applied in reverse order, from newest to oldest.

Use the transaction log in Table 3.16 to trace a simple database recovery process. To make sure you understand the recovery process, the simple transaction log includes three transactions and one checkpoint. This transaction log includes the transaction components used earlier in the chapter, so you should already be familiar with the basic process. Given the transaction, the transaction log has the following characteristics:

- Transaction 101 consists of two UPDATE statements that reduce the quantity on hand for product 54778-2T and increase the customer balance for customer 10011 for a credit sale of two units of product 54778-2T.
- Transaction 106 is the same credit sales event seen earlier in this chapter. This transaction represents the credit sale of one unit of product 89-WRE-Q to customer

10016 for $277.55. This transaction consists of five SQL DML statements: three INSERT statements and two UPDATE statements.

- Transaction 155 represents a simple inventory update. This transaction consists of one UPDATE statement that increases the quantity on hand of product 2232/QWE from 6 units to 26 units.
- A database checkpoint writes all updated database buffers to disk. The checkpoint event writes only the changes for all previously committed transactions. In this case, the checkpoint applies all changes made by transaction 101 to the database data files.

Using Table 3.16, you can now trace the database recovery process for a DBMS using the deferred update method as follows:

1. Identify the last checkpoint—in this case, TRL ID 423. This was the last time database buffers were physically written to disk.
2. Note that transaction 101 started and finished before the last checkpoint. Therefore, all changes were already written to disk, and no additional action needs to be taken.
3. For each transaction committed after the last checkpoint (TRL ID 423), the DBMS will use the transaction log data to write the changes to disk, using the "after" values. For example, for transaction 106:
    a. Find COMMIT (TRL ID 457).
    b. Use the previous pointer values to locate the start of the transaction (TRL ID 397).
    c. Use the next pointer values to locate each DML statement, and apply the changes to disk using the "after" values. (Start with TRL ID 405, then 415, 419, 427, and 431.) Remember that TRL ID 457 was the COMMIT statement for this transaction.
    d. Repeat the process for transaction 155.
4. Any other transactions will be ignored. Therefore, for transactions that ended with ROLLBACK or that were left active (those that do not end with a COMMIT or ROLLBACK), nothing is done because no changes were written to disk.

Table 3-16: A Transaction Log for Transaction Recovery Examples

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	54778-2T	PROD_QOH	45	43
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	615.73	675.62
365	101	363	Null	COMMIT	**** End of Transaction				
397	106	Null	405	START	****Start Transaction				
405	106	397	415	INSERT	INVOICE	1009			1009,10016, ...
415	106	405	419	INSERT	LINE	1009,1			1009,1, 89-WRE-Q,1, ...
419	106	415	427	UPDATE	PRODUCT	89-WRE-Q	PROD_QOH	12	11
423				CHECKPOINT					
427	106	419	431	UPDATE	CUSTOMER	10016	CUST_BALANCE	0.00	277.55
431	106	427	457	INSERT	ACCT_TRANSACTION	10007			1007, 18-JAN-2022, ...
457	106	431	Null	COMMIT	**** End of Transaction				
521	155	Null	525	START	****Start Transaction				
525	155	521	528	UPDATE	PRODUCT	2232/QWE	PROD_QOH	6	26
528	155	525	Null	COMMIT	**** End of Transaction				
***** C*R*A*S*H****									

Summary

- A transaction is a sequence of database operations that access the database. A transaction is a logical unit of work; that is, all parts are executed or the transaction is aborted. A transaction takes a database from one consistent state to another. A consistent database state is one in which all data integrity constraints are satisfied.
- Transactions have four main properties: atomicity, consistency, isolation, and durability. Atomicity means that all parts of the transaction must be executed; otherwise, the transaction is aborted. Consistency means that the database's consistent state is maintained. Isolation means that data used by one transaction cannot be accessed by another transaction until the first one is completed. Durability means that changes made by a transaction cannot be rolled back once the transaction is committed. In addition, transaction schedules have the property of serializability—the result of the concurrent execution of transactions is the same as that of the transactions being executed in serial order.
- SQL provides support for transactions through the use of two statements: COMMIT, which saves changes to disk, and ROLLBACK, which restores the previous database state. SQL transactions are formed by several SQL statements or database requests. Each database request originates several I/O database operations. The transaction log keeps track of all transactions that modify the database. The information stored in the transaction log is used for recovery (ROLLBACK) purposes.
- Concurrency control coordinates the simultaneous execution of transactions. The concurrent execution of transactions can result in three main problems: lost updates, uncommitted data, and inconsistent retrievals. The scheduler is responsible for establishing the order in which the concurrent transaction operations are executed. The transaction execution order is critical and ensures database integrity in multiuser database systems. The scheduler uses locking, time stamping, and optimistic methods to ensure the serializability of transactions.
- A lock guarantees unique access to a data item by a transaction. The lock prevents one transaction from using the data item while another transaction is using it. There are several levels of locks: database, table, page, row, and field. Two types of locks can be used in database systems: binary locks and shared/exclusive locks. A binary lock can have only two states: locked (1) or unlocked (0). A shared lock is used when a transaction wants to read data from a database and no other transaction is updating the same data. Several shared or "read" locks can exist for a particular item. An exclusive lock is issued when a transaction wants to update (write to) the database and no other locks (shared or exclusive) are held on the data.
- Serializability of schedules is guaranteed through the use of two-phase locking. The two-phase locking schema has a growing phase, in which the transaction acquires all of the locks that it needs without unlocking any data, and a shrinking phase, in which the transaction releases all of the locks without acquiring new locks. When two or more transactions wait indefinitely for each other to release a lock, they are in a deadlock, also called a deadly embrace. There are three deadlock control techniques: prevention, detection, and avoidance.
- Concurrency control with time stamping methods assigns a unique time stamp to each transaction and schedules the execution of conflicting transactions in time stamp order. Two schemes are used to decide which transaction is rolled back and which continues executing: the wait/die scheme and the wound/wait scheme.
- Concurrency control with optimistic methods assumes that the majority of database transactions do not conflict and that transactions are executed concurrently, using private, temporary copies of the data. At commit time, the private copies are updated to

the database. The ANSI standard defines four transaction isolation levels: Read Uncommitted, Read Committed, Repeatable Read, and Serializable.

- Database recovery restores the database from a given state to a previous consistent state. Database recovery is triggered when a critical event occurs, such as a hardware error or application error.

Review Questions

1. Explain the following statement: A transaction is a logical unit of work.
2. What is a consistent database state, and how is it achieved?
3. The DBMS does not guarantee that the semantic meaning of the transaction truly represents the real-world event. What are the possible consequences of that limitation? Give an example.
4. List and discuss the four individual transaction properties.
5. What does serializability of transactions mean?
6. What is a transaction log, and what is its function?
7. What is a scheduler, what does it do, and why is its activity important to concurrency control?
8. What is a lock, and how does it work in general?
9. What are the different levels of lock granularity?
10. Why might a page-level lock be preferred over a field-level lock?
11. What is concurrency control, and what is its objective?
12. What is an exclusive lock, and under what circumstances is it granted?
13. What is a deadlock, and how can it be avoided? Discuss several strategies for dealing with deadlocks.
14. What are some disadvantages of time stamping methods for concurrency control?
15. Why might it take a long time to complete transactions when using an optimistic approach to concurrency control?
16. What are the three types of database-critical events that can trigger the database recovery process? Give some examples for each one.
17. What are the four ANSI transaction isolation levels? What type of reads does each level allow?

Chapter 4: Database Performance Tuning and Query Optimization

Learning Outcomes

After this chapter you will be able to:

1. Identify the procedures involved in database performance tuning
2. Describe how a DBMS processes SQL queries in each of its three phases
3. Define what business rules are and how they influence database design
4. Differentiate between a rule-based optimizer and a cost-based optimizer
5. Describe some common practices used to write efficient SQL code
6. Explain how to formulate queries and tune the DBMS for optimal performance

Database Performance-Tuning Concepts

One of the main functions of a database system is to provide timely answers to end users. End users interact with the DBMS through the use of queries to generate information, using the following sequence:

1. The end-user (client-end) application generates a query.
2. The query is sent to the DBMS (server end).
3. The DBMS (server end) executes the query.
4. The DBMS sends the resulting data set to the end-user (client-end) application.

End users expect their queries to return results as quickly as possible. How do you know that the performance of a database is good? Good database performance is hard to evaluate. How do you know if a 1.06-second query response time is good enough? It is easier to identify bad database performance than good database performance—all it takes is end-user complaints about slow query results. Unfortunately, the same query might perform well one day and not so well two months later. Regardless of end-user perceptions, *the goal of database performance is to execute queries as fast as possible*. Therefore, database performance must be closely monitored and regularly tuned. **Database performance tuning** refers to a set of activities and procedures designed to reduce the response time of the database system—that is, to ensure that an end-user query is processed by the DBMS in the minimum amount of time. The time required by a query to return a result set depends on many factors, which tend to be wide-ranging and to vary among environments and among vendors. In general, the performance of a typical DBMS is constrained by three main factors: CPU processing power, available primary memory (RAM), and input/output (hard disk and network) throughput. Table 4.1 lists some system components and summarizes general guidelines for achieving better query performance.

Naturally, the system will perform best when its hardware and software resources are optimized. However, in the real world, unlimited resources are not the norm; internal and external constraints always exist. Therefore, the system components should be optimized to obtain the best throughput possible with existing (and often limited) resources, which is why database performance tuning is important. Fine-tuning the performance of a system requires a holistic approach. That is, *all* factors must be checked to ensure that each one operates at its optimum level and has sufficient resources to minimize the occurrence of bottlenecks. Because database design is such an important factor in determining the database system's performance efficiency.

Table 4-1: General Guidelines for Better System Performance

	System Resources	Client	Server
Hardware	CPU	The fastest possible Quad-core CPU or higher Virtualized client desktop technologies could also be used.	The fastest possible Multiple processors (quad-core technology or higher) Cluster of networked computers Virtualized server technology could be used.
	RAM	The maximum possible to avoid OS memory to disk swapping	The maximum possible to avoid OS memory to disk swapping
	Storage	Fast SATA hard disk with sufficient free hard disk space Solid state drives (SSDs) for faster speed	Multiple high-speed, high-capacity disks Fast disk interface (SAS/SCSI/Firewire/Fibre Channel RAID configuration optimized for throughput Solid state drives (SSDs) for faster speed Separate disks for OS, DBMS, and data spaces
	Network	High-speed connection	High-speed connection
Software	Operating system (OS)	64-bit OS for larger address spaces Fine-tuned for best client application performance	64-bit OS for larger address spaces Fine-tuned for best server application performance
	Network	Fine-tuned for best throughput	Fine-tuned for best throughput
	Application	Optimize SQL in client application	Optimize DBMS server for best performance

What constitutes a good, efficient database design? From the performance-tuning point of view, the database designer must ensure that the design uses features in the DBMS that guarantee the integrity and optimal performance of the database. This chapter provides fundamental knowledge that will help you optimize database performance by selecting the appropriate database server configuration, using indexes, understanding table storage organization and data locations, and implementing the most efficient SQL query syntax.

Performance Tuning: Client and Server

In general, database performance-tuning activities can be divided into those on the client side and those on the server side.

- On the client side, the objective is to generate a SQL query that returns the correct answer in the least amount of time, using the minimum amount of resources at the server end. The activities required to achieve that goal are commonly referred to as **SQL performance tuning**.
- On the server side, the DBMS environment must be properly configured to respond to clients' requests in the fastest way possible, while making optimum use of existing resources. The activities required to achieve that goal are commonly referred to as **DBMS performance tuning**.

Keep in mind that DBMS implementations are typically more complex than just a two-tier client/server configuration. The network component plays a critical role in delivering messages between clients and servers; this is especially important in distributed databases. This chapter, however, assumes a fully optimized network, and, therefore, the focus is on the database components. Even in multi-tier client/server environments that consist of a client front end, application middleware, and database server back end, performance-tuning activities are frequently divided into subtasks to ensure the fastest possible response time between any two component points. The database administrator must work closely with the network group to ensure that database traffic flows efficiently in the network infrastructure. This is even more important when you consider that most database systems service geographically dispersed users. This chapter covers SQL performance-tuning practices on the client side and DBMS

performance-tuning practices on the server side. However, before you start learning about the tuning processes, you must first learn more about the DBMS architectural components and processes, and how those processes interact to respond to end-users' requests.

DBMS Architecture

The architecture of a DBMS is represented by the processes and structures (in memory and permanent storage) used to manage a database. Such processes collaborate with one another to perform specific functions. Figure 4.1 illustrates the basic DBMS architecture.

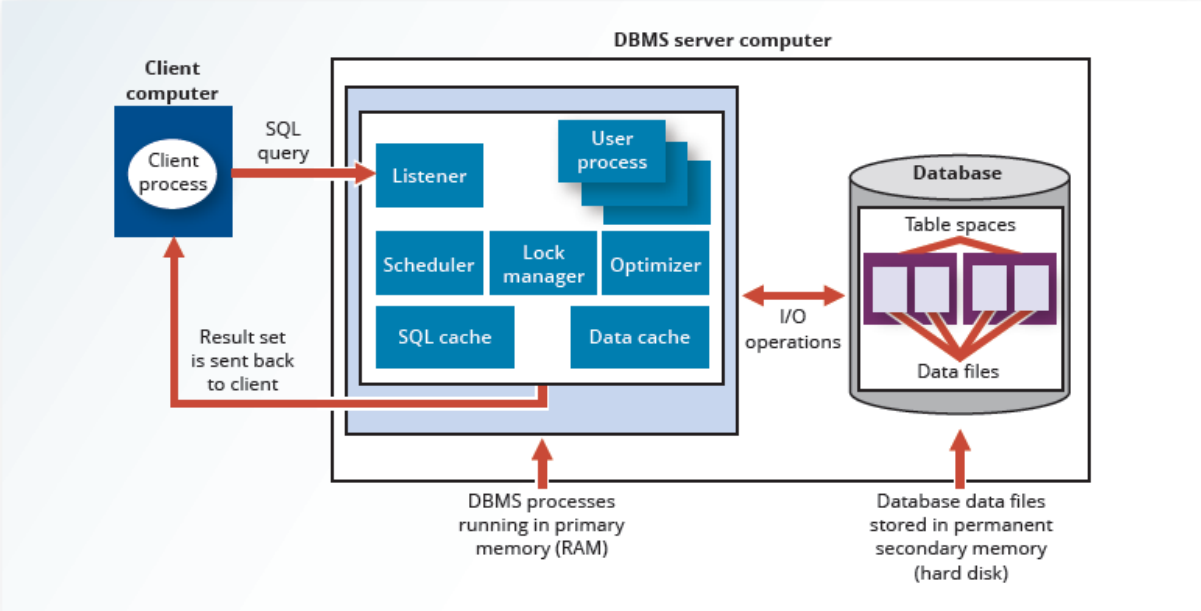


Figure 4-1: Basic DBMS Architecture (Coronel C., Morris S. 2022)

Note the following components and functions in Figure 4.1:

- All data in a database is stored in **data files**. A typical enterprise database is normally composed of several data files. A data file can contain rows from a single table, or it can contain rows from many different tables. A database administrator (DBA) determines the initial size of the data files that make up the database; however, the data files can be automatically expanded by the DBMS as required.
- **Data blocks**, sometimes called "pages," are the smallest logical units of assignable storage space in the database. The physical storage space within a data file is logically broken into a set of data blocks. For example, if a physical data file is created that is 10 MB in size, that 10 MB may be logically structured into 1,280 data blocks each 8 K in size or structured into 640 data blocks each 16 K in size. Part of the configuration of database components is to set the logical size of the data blocks that will be used. Common data block sizes tend to range from 2 K blocks to 32 K blocks, but other sizes may be available in your specific DBMS.
- **Extents** are contiguous collections of data blocks (pages) that are allocated to database objects. All data blocks in an extent must be in the same data file. When a database object is created, one or more extents are allocated to hold the data for that object. For example, when a table is created in MySQL, by default 16 K pages are used and the table is initially allocated one extent composed of 64 pages. Therefore, the table is given 1 MB of storage space to use before the first row of data is ever stored. If the table grows so that more than 1 MB of storage is needed, the DBMS will allocate another extent of 64

pages. If the table is very large or growing very quickly, the DBMS may allocate two or more extents at a time to help improve performance as the table grows. Many DBMSs will allow the DBA to configure the size of the extents that database objects are allocated as they grow.

- Data files are generally grouped in file groups or table spaces. A **table space** or **file group** is a logical grouping of several data files that store data with similar characteristics. For example, you might have a *system* table space where the data dictionary table data is stored, a *user data* table space to store the user-created tables, an *index* table space to hold all indexes, and a *temporary* table space to do temporary sorts, grouping, and so on. Each time you create a new database, the DBMS automatically creates a minimum set of table spaces.
- The **data cache**, or **buffer cache**, is a shared, reserved memory area that stores the most recently accessed data blocks in RAM. The data read from the data files is stored in the data cache after the data has been read or before the data is written to the data files. The data cache also caches system catalog data and the contents of the indexes.
- The **SQL cache**, or **procedure cache**, is a shared, reserved memory area that stores the most recently executed SQL statements or PL/SQL procedures, including triggers and functions. The SQL cache does not store the SQL written by the end user. Rather, the SQL cache stores a "processed" version of the SQL that is ready for execution by the DBMS.
- To work with the data, the DBMS must retrieve the data from permanent storage and place it in RAM. In other words, the data is retrieved from the data files and placed in the data cache.
- To move data from permanent storage (data files) to RAM (data cache), the DBMS issues I/O requests and waits for the replies. An **input/output (I/O) request** is a low-level data access operation that reads or writes data to and from computer devices, such as memory, hard disks, video, and printers. Note that an I/O disk read operation retrieves an entire physical disk block, generally containing multiple rows, from permanent storage to the data cache, even if you will use only one attribute from only one row. The physical disk block size depends on the operating system and could be 4 K, 8 K, 16 K, 32 K, 64 K, or even larger. Furthermore, depending on the circumstances, a DBMS might issue a single-block read request or a multiblock read request.
- Working with data in the data cache is many times faster than working with data in the data files because the DBMS does not have to wait for the hard disk to retrieve the data; no hard disk I/O operations are needed to work within the data cache.
- Most performance-tuning activities focus on minimizing the number of I/O operations because using I/O operations is many times slower than reading data from the data cache. For example, as of this writing, RAM access times range from 5 to 70 nanoseconds, while magnetic hard disk access times range from 5 to 15 milliseconds, and SSD access times range from 35 to 100 microseconds. This means that hard disks are several orders of magnitude slower than RAM.

Figure 4.1 also illustrates some typical DBMS processes. Although the number of processes and their names vary from vendor to vendor, the functionality is similar. The following processes are represented in Figure 4.1:

- *Listener*. The listener process listens for clients' requests and handles the processing of the SQL requests to other DBMS processes. Once a request is received, the listener passes the request to the appropriate user process.
- *User*. The DBMS creates a user process to manage each client session. Therefore, when you log on to the DBMS, you are assigned a user process. This process handles all requests you submit to the server. There are many user processes—at least one per logged-in client.

- *Scheduler*. The scheduler process organizes the concurrent execution of SQL requests.
- *Lock manager*. This process manages all locks placed on database objects, including disk pages.
- *Optimizer*. The optimizer process analyzes SQL queries and finds the most efficient way to access the data.

Database Query Optimization Modes

Most of the algorithms proposed for query optimization are based on two principles:

- The selection of the optimum execution order to achieve the fastest execution time
- The selection of sites to be accessed to minimize communication costs

Within those two principles, a query optimization algorithm can be evaluated on the basis of its *operation mode* or the *timing of its optimization*. Operation modes can be classified as manual or automatic.

- **Automatic query optimization** means that the DBMS finds the most cost-effective access path without user intervention.
- **Manual query optimization** requires that the optimization be selected and scheduled by the end user or programmer.

Automatic query optimization is clearly more desirable from the end user's point of view, but the cost of such convenience is the increased overhead that it imposes on the DBMS. Query optimization algorithms can also be classified according to when the optimization is done. Within this timing classification, query optimization algorithms can be static or dynamic.

- **Static query optimization** takes place at compilation time. In other words, the best optimization strategy is selected when the query is compiled by the DBMS. This approach is common when SQL statements are embedded in procedural programming languages such as C# or Python. When the program is submitted to the DBMS for compilation, it creates the plan necessary to access the database. When the program is executed, the DBMS uses that plan to access the database.
- **Dynamic query optimization** takes place at execution time. Database access strategy is defined when the program is executed. Therefore, access strategy is dynamically determined by the DBMS at run time, using the most up-to-date information about the database. Although dynamic query optimization is efficient, its cost is measured by run-time processing overhead. The best strategy is determined every time the query is executed; this could happen several times in the same program.

Finally, query optimization techniques can be classified according to the type of information that is used to optimize the query. For example, queries may be based on statistically based or rule-based algorithms.

- A **statistically based query optimization algorithm** uses statistical information about the database. The statistics provide information about database characteristics such as size, number of records, average access time, number of requests serviced, and number of users with access rights. These statistics are then used by the DBMS to determine the best access strategy. Within statistically based optimizers, some DBMSs allow setting a goal to specify that the optimizer should attempt to minimize the time to retrieve the first row or the last row. Minimizing the time to retrieve the first row is often used in transaction systems and interactive client environments. In these cases, the goal is to present the first several rows to the user as quickly as possible. Then, while the DBMS waits for the

user to scroll through the data, it can fetch the other rows for the query. Setting the optimizer goal to minimize retrieval of the last row is typically done in embedded SQL and inside stored procedures. In these cases, the control will not pass back to the calling application until all of the data has been retrieved; therefore, it is important to retrieve all of the data to the last row as quickly as possible so control can be returned.

The statistical information is managed by the DBMS and is generated in one of two different modes: dynamic or manual.

- In the **dynamic statistical generation mode**, the DBMS automatically evaluates and updates the statistics after each data access operation. In the **manual statistical generation mode**, the statistics must be updated periodically through a user-selected utility such as IBM's RUNSTAT command, which is used by DB2 DBMSs.

- A **rule-based query optimization algorithm** is based on a set of user-defined rules to determine the best query access strategy. The rules are entered by the end user or database administrator, and they are typically general in nature.

Database Statistics

Another DBMS process that plays an important role in query optimization is gathering database statistics. The term **database statistics** refers to a number of measurements about database objects, such as number of processors used, processor speed, and temporary space available. Such statistics provide a snapshot of database characteristics. As you will learn later in this chapter, the DBMS uses these statistics to make critical decisions about improving query processing efficiency. Database statistics can be gathered manually by the DBA or automatically by the DBMS. For example, many DBMS vendors support the ANALYZE command in SQL to gather statistics. In addition, many vendors have their own routines to gather statistics. For example, IBM's DB2 uses the RUNSTATS procedure, while Microsoft's SQL Server uses the UPDATE STATISTICS procedure and provides the Auto-Update and Auto-Create Statistics options in its initialization parameters. A sample of measurements that the DBMS may gather about various database objects is shown in Table 4.2.

Table 4-2: Sample Database Statistics Measurements

Database Object	Sample Measurements
Tables	Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes
Indexes	Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index
Environment Resources	Logical and physical disk block size, location and size of data files, and number of extends per data file

If the object statistics exist, the DBMS will use them in query processing. Most newer DBMSs (such as Oracle, MySQL, SQL Server, and DB2) automatically gather statistics; others require the DBA to gather statistics manually. To generate the database object statistics manually, each DBMS has its own commands.
In Oracle, use ANALYZE <TABLE/INDEX> object_name COMPUTE STATISTICS;
In MySQL, use ANALYZE TABLE <table_name>;
In SQL Server, use UPDATE STATISTICS <object_name>, where object name refers to a table or a view.

For example, to generate statistics for the VENDOR table, you would use:

In Oracle: ANALYZE TABLE VENDOR COMPUTE STATISTICS;
In MySQL: ANALYZE TABLE VENDOR;
In SQL Server: UPDATE STATISTICS VENDOR;

When you generate statistics for a table, all related indexes are also analyzed. However, you could generate statistics for a single index by using the following command, where VEND_ NDX is the name of the index:

ANALYZE INDEX VEND_NDX COMPUTE STATISTICS;

In SQL Server, use UPDATE STATISTICS <table_name> <index_name>. An example command would be

UPDATE STATISTICS VENDOR VEND_NDX;.

Database statistics are stored in the system catalog in specially designated tables. It is common to periodically regenerate the statistics for database objects, especially database objects that are subject to frequent change. For example, if you have a video rental DBMS, your system will likely use a RENTAL table to store the daily video rentals. That RENTAL table and its associated indexes would be subject to constant inserts and updates as you record daily rentals and returns. Therefore, the RENTAL table statistics you generated last week do not accurately depict the table as it exists today. The more current the statistics are, the better the chances that the DBMS will properly select the fastest way to execute a given query. Now that you know the basic architecture of DBMS processes and memory structures, and the importance and timing of the database statistics gathered by the DBMS, you are ready to learn how the DBMS processes a SQL query request.

Query Processing

What happens at the DBMS server end when the client's SQL statement is received? In simple terms, the DBMS processes a query in three phases:

1. *Parsing*. The DBMS parses the SQL query and chooses the most efficient access/execution plan.
2. *Execution*. The DBMS executes the SQL query using the chosen execution plan.
3. *Fetching*. The DBMS fetches the data and sends the result set back to the client.

The processing of SQL DDL statements (such as CREATE TABLE) is different from the processing required by DML statements. The difference is that a DDL statement actually updates the data dictionary tables or system catalog, while a DML statement (SELECT, INSERT, UPDATE, or DELETE) mostly manipulates end-user data. Figure 4.2 shows the general steps required for query processing.

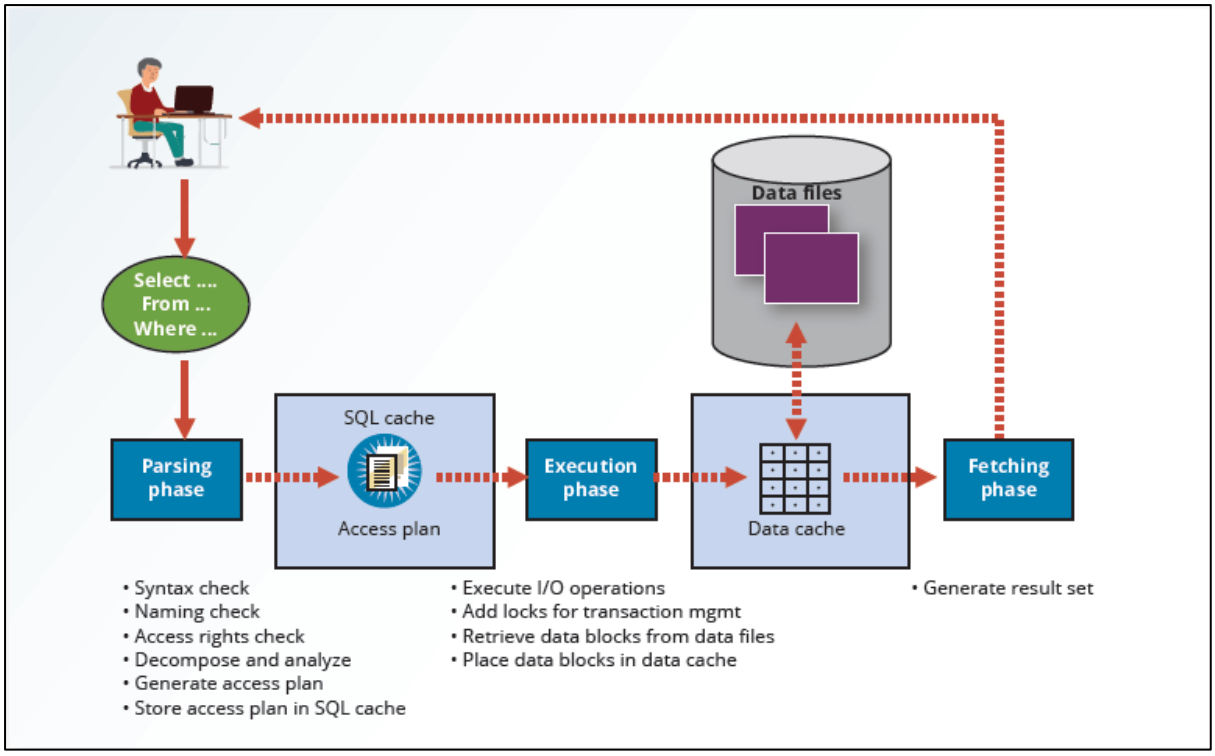


Figure 4-2: Query Processing (Coronel C., Morris S. 2022)

SQL Parsing Phase

The optimization process includes breaking down—parsing—the query into smaller units and transforming the original SQL query into a slightly different version of the original SQL code, but one that is fully equivalent and more efficient. *Fully equivalent* means that the optimized query results are always the same as the original query. *More efficient* means that the optimized query will almost always execute faster than the original query. (Note that it *almost* always executes faster because many factors affect the performance of a database, as explained earlier. Those factors include the network, the client computer's resources, and other queries running concurrently in the same database.) To determine the most efficient way to execute the query, the DBMS may use the database statistics you learned about earlier. The SQL parsing activities are performed by the **query optimizer**, which analyzes the SQL query and finds the most efficient way to access the data. This process is the most time-consuming phase in query processing. Parsing a SQL query requires several steps, in which the SQL query is:

- Validated for syntax compliance
- Validated against the data dictionary to ensure that table names and column names are correct
- Validated against the data dictionary to ensure that the user has proper access rights
- Analyzed and decomposed into more atomic components
- Optimized through transformation into a fully equivalent but more efficient SQL query
- Prepared for execution by determining the most efficient execution or access plan

Once the SQL statement is transformed, the DBMS creates what is commonly known as an access plan or execution plan. An **access plan** is the result of parsing a SQL statement; it contains the series of steps a DBMS will use to execute the query and return the result set in the most efficient way. First, the DBMS checks to see if an access plan already exists for the query in the SQL cache. If it does, the DBMS reuses the access plan to save time. If it does not, the

optimizer evaluates various plans and then decides which indexes to use and how to best perform join operations. The chosen access plan for the query is then placed in the SQL cache and made available for use and future reuse. Access plans are DBMS-specific and translate the client's SQL query into the series of complex I/O operations required to read the data from the physical data files and generate the result set. Table 4.3 illustrates some I/O operations for an Oracle RDBMS. Most DBMSs perform similar types of operations when accessing and manipulating data sets.

Table 4-3: Sample DBMS Access Plan I/O Operations

Operation	Description
Table scan (full)	Reads the entire table sequentially, from the first row to the last, one row at a time (slowest)
Table access (row ID)	Reads a table row directly, using the row ID value (fastest)
Index scan (range)	Reads the index first to obtain the row IDs and then accesses the table rows directly (faster than a full table scan)
Index access (unique)	Used when a table has a unique index in a column
Nested loop	Reads and compares a set of values to another set of values, using a nested loop style (slow)
Merge	Merges two data sets (slow)
Sort	Sorts a data set (slow)

In Table 4.3, note that a table access using a row ID is the fastest method. A row ID is a unique identification for every row saved in permanent storage; it can be used to access the row directly. Conceptually, a row ID is similar to a slip you get when you park your car in an airport parking lot. The parking slip contains the section number and lot number. Using that information, you can go directly to your car without searching every section and lot.

SQL Execution Phase

In this phase, all I/O operations indicated in the access plan are executed. When the execution plan is run, the proper locks—if needed—are acquired for the data to be accessed, and the data is retrieved from the data files and placed in the DBMS's data cache. All transaction management commands are processed during the parsing and execution phases of query processing.

SQL Fetching Phase

After the parsing and execution phases are completed, all rows that match the specified condition(s) are retrieved, sorted, grouped, and aggregated (if required). During the fetching phase, the rows of the resulting query result set are returned to the client. The DBMS might use temporary table space to store temporary data. In this stage, the database server coordinates the movement of the result set rows from the server cache to the client cache. For example, a given query result set might contain 9,000 rows; the server would send the first 100 rows to the client and then wait for the client to request the next set of rows, until the entire result set is sent to the client.

Query Processing Bottlenecks

The main objective of query processing is to execute a given query in the fastest way possible with the least amount of resources. As you have seen, the execution of a query requires the DBMS to break down the query into a series of interdependent I/O operations to be executed in a

collaborative manner. The more complex a query is, the more complex the operations are, which means that bottlenecks are more likely. A **query processing bottleneck** is a delay introduced in the processing of an I/O operation that causes the overall system to slow down. In the same way, the more components a system has, the more interfacing is required among the components, increasing the likelihood of bottlenecks. Within a DBMS, five components typically cause bottlenecks:

- *CPU*. The CPU processing power of the DBMS should match the system's expected work load. A high CPU utilization might indicate that the processor speed is too slow for the amount of work performed. However, heavy CPU utilization can be caused by other factors, such as a defective component, not enough RAM (the CPU spends too much time swapping memory blocks), a badly written device driver, or a rogue process. A CPU bottleneck will affect not only the DBMS but all processes running in the system.
- *RAM*. The DBMS allocates memory for specific usage, such as data cache and SQL cache. RAM must be shared among all running processes, including the operating system and DBMS. If not enough RAM is available, moving data among components that are competing for scarce RAM can create a bottleneck.
- *Hard disk*. Other common causes of bottlenecks are hard disk speed and data transfer rates. Current hard disk storage technology allows for greater storage capacity than in the past; however, hard disk space is used for more than just storing end-user data. Current operating systems also use the hard disk for *virtual memory*, which refers to copying areas of RAM to the hard disk as needed to make room in RAM for more urgent tasks. Therefore, more hard disk storage space and faster data transfer rates reduce the likelihood of bottlenecks.
- *Network*. In a database environment, the database server and the clients are connected via a network. All networks have a limited amount of bandwidth that is shared among all clients. When many network nodes access the network at the same time, bottlenecks are likely.
- *Application code*. Not all bottlenecks are caused by limited hardware resources. Two of the most common sources of bottlenecks are inferior application code and poorly designed databases. Inferior code can be improved with code optimization techniques, as long as the underlying database design is sound. However, no amount of coding will make a poorly designed database perform better.

Bottlenecks are the result of multiple database transactions competing for the use of database resources (CPU, RAM, hard disk, indexes, locks, buffers, etc.). As you learned earlier in this chapter, a DBMS uses many components and structures to perform its operations, such as processes, buffers, locks, table spaces, indexes, and log files. These resources are used by all transactions executing on the database, and, therefore, the transactions often compete for such resources. Because most (if not all) transactions work with data rows in tables, one of the most typical bottlenecks is caused by transactions competing for the same data rows. Another common source of contention is for shared memory resources, particularly shared buffers and locks. To speed up data update operations, the DMBS uses buffers to cache the data. At the same time, to manage access to data, the DBMS uses locks. Learning how to avoid these bottlenecks and optimize database performance is the main focus of this chapter.

Indexes and Query Optimization

Indexes are crucial in speeding up data access because they facilitate searching, sorting, and using aggregate functions and even join operations. The improvement in data access speed occurs because an index is an ordered set of values that contains the index key and pointers. The pointers are the row IDs for the actual table rows. Conceptually, a data index is similar to a book index. When you use a book index, you look up a word, which is similar to the index key. The word is accompanied by one or more page numbers where the word is used; these numbers are similar to pointers. An index scan is more efficient than a full table scan because the index data is preordered and the amount of data is usually much smaller. Therefore, when performing searches, it is almost always better for the DBMS to use the index to access a table than to scan all rows in a table sequentially. For example, Figure 4.3 shows the index representation of a CUSTOMER table with 14,786 rows and the index STATE_NDX on the CUS_STATE attribute.

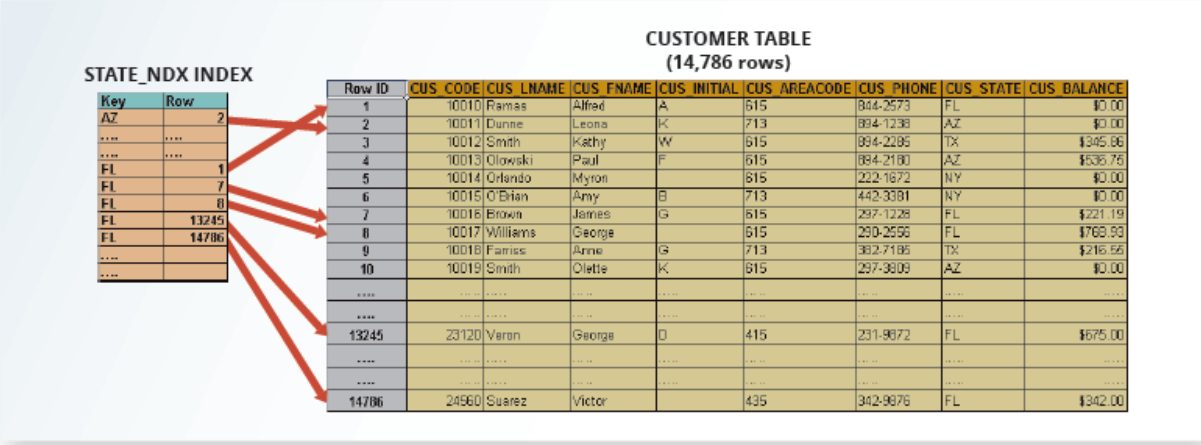


Figure 4-3: Index Representation for the Customer Table (Coronel C., Morris S. 2022)

Suppose you submit the following query:

```
SELECT CUS_NAME, CUS_STATE
FROM CUSTOMER
WHERE CUS_STATE = 'FL';
```

If there is no index, the DBMS will perform a full-table scan and read all 14,786 customer rows. Assuming that the index STATE_NDX is created (and analyzed), the DBMS will automatically use the index to locate the first customer with a state equal to 'FL' and then proceed to read all subsequent CUSTOMER rows, using the row IDs in the index as a guide. Assuming that only five rows meet the condition CUS_STATE 5 'FL' there are five accesses to the index and five accesses to the data, for a total of 10 I/O accesses. The DBMS would be saved from reading approximately 14,776 I/O requests for customer rows that do not meet the criteria. That is a lot of CPU cycles!

If indexes are so important, why not index every column in every table? The simple answer is that it is not practical to do so. Indexing every column in every table overtaxes the DBMS in terms of index-maintenance processing, especially if the table has many attributes and rows, or requires many inserts, updates, and deletes. One measure that determines the need for an index is the data *sparsity* of the column you want to index. **Data sparsity** refers to the number of different values a column could have. For example, a STU_SEX column in a STUDENT table can have only two possible values, M or F; therefore, that column is said to have low sparsity. In contrast, the STU_DOB column that stores the student date of birth can have many different date values; therefore, that column is said to have high sparsity. Knowing the sparsity helps you decide

whether the use of an index is appropriate. For example, when you perform a search in a column with low sparsity, you are likely to read a high percentage of the table rows anyway; therefore, index processing might be unnecessary work. In Section 11-5, you learn how to determine when an index is recommended.

Most DBMSs implement indexes using one of the following data structures:

- **Hash index**. A hash index is based on an ordered list of hash values. A hash algorithm is used to create a hash value from a key column. This value points to an entry in a hash table, which in turn points to the actual location of the data row. This type of index is good for simple and fast lookup operations based on equality conditions—for example, LNAME="Scott" and FNAME="Shannon".
- **B-tree index**. The B-tree index is an ordered data structure organized as an upside-down tree. (See Figure 11.4.) The index tree is stored separately from the data. The lower-level leaves of the B-tree index contain the pointers to the actual data rows. B-tree indexes are "self-balanced," which means that it takes approximately the same amount of time to access any given row in the index. This is the default and most common type of index used in databases. The B-tree index is used mainly in tables in which column values repeat a relatively small number of times.
- **Bitmap index**. A bitmap index uses a bit array (0s and 1s) to represent the existence of a value or condition. These indexes are used mostly in data warehouse applications in tables with a large number of rows in which a small number of column values repeat many times. (See Figure 4.4.) Bitmap indexes tend to use less space than B-tree indexes because they use bits instead of bytes to store their data.

Using the preceding index characteristics, a database designer can determine the best type of index to use. For example, assume that a CUSTOMER table has several thousand rows. The CUSTOMER table has two columns that are used extensively for query purposes: CUS_LNAME, which represents a customer's last name, and REGION_CODE, which can have one of four values (NE, NW, SW, and SE). Based on this information, you could conclude that:

- Because the CUS_LNAME column contains many different values that repeat a relatively small number of times compared to the total number of rows in the table, a B-tree index will be used.
- Because the REGION_CODE column contains only a few different values that repeat a relatively large number of times compared to the total number of rows in the table, a bitmap index will be used. Figure 4.4 shows the B-tree and bitmap representations for the CUSTOMER table used in the previous discussion.

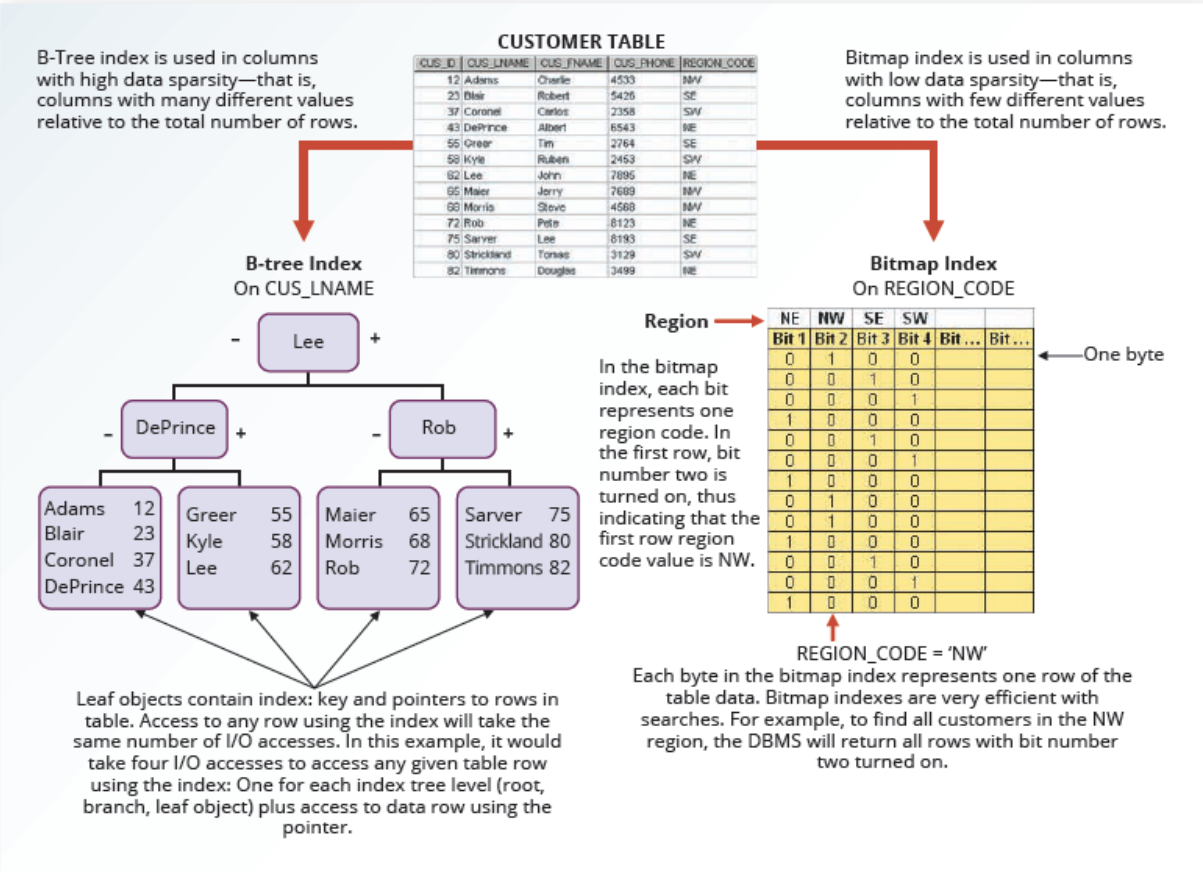


Figure 4-4: B-Tree and Bitmap Index Representation (Coronel C., Morris S. 2022)

Current-generation DBMSs are intelligent enough to determine the best type of index to use under certain circumstances, provided that the DBMS has updated database statistics. Regardless of which index is chosen, the DBMS determines the best plan to execute a given query. Each DBMS has its own algorithms for determining the most efficient way to access the data. The query optimizer can operate in one of two modes:

- A **rule-based optimizer** uses preset rules and points to determine the best approach to execute a query. The rules assign a "fixed cost" to each SQL operation; the costs are then added to yield the cost of the execution plan. For example, a full table scan has a set cost of 10, while a table access by row ID has a set cost of 3.
- A **cost-based optimizer** uses sophisticated algorithms based on statistics about the objects being accessed to determine the best approach to execute a query. In this case, the optimizer process adds up the processing cost, the I/O costs, and the resource costs (RAM and temporary space) to determine the total cost of a given execution plan.

SQL Performance Tuning

SQL performance tuning is evaluated from the client perspective. Therefore, the goal is to illustrate some common practices used to write efficient SQL code. A few words of caution are appropriate:

- Most current-generation relational DBMSs perform automatic query optimization at the server end.

- Most SQL performance optimization techniques are DBMS-specific and, therefore, are rarely portable, even across different versions of the same DBMS. Part of the reason for this behavior is the constant advancement in database technologies.

Does this mean that you should not worry about how a SQL query is written because the DBMS will always optimize it? No, because there is considerable room for improvement. (The DBMS uses *general* optimization techniques rather than focusing on specific techniques dictated by the special circumstances of the query execution.) A poorly written SQL query can, *and usually will*, bring the database system to its knees from a performance point of view. The majority of current database performance problems are related to poorly written SQL code. Therefore, although a DBMS provides general optimizing services, a carefully written query almost always outperforms a poorly written one. Although SQL data manipulation statements include many different commands such as INSERT, UPDATE, DELETE, and SELECT, most recommendations in this section are related to the use of the SELECT statement, and in particular, the use of indexes and how to write conditional expressions.

Index Selectivity

Indexes are the most important technique used in SQL performance optimization. The key is to know when an index is used. As a general rule, indexes are likely to be used:

- When an indexed column appears by itself in the search criteria of a WHERE or HAVING clause
- When an indexed column appears by itself in a GROUP BY or ORDER BY clause
- When a MAX or MIN function is applied to an indexed column
- When the data sparsity on the indexed column is high

Indexes are very useful when you want to select a small subset of rows from a large table based on a given condition. If an index exists for the column *used in the selection*, the DBMS may choose to use it. The objective is to create indexes with high selectivity. **Index selectivity** is a measure of the likelihood that an index will be used in query processing. Here are some general guidelines for creating and using indexes:

- *Create indexes for each single attribute used in a WHERE, HAVING, ORDER BY, or GROUP BY clause*. If you create indexes in all single attributes *used in search conditions*, the DBMS will access the table using an index scan instead of a full table scan. For example, if you have an index for P_PRICE, the condition P_PRICE > 10.00 can be solved by accessing the index instead of sequentially scanning all table rows and evaluating P_PRICE for each row. Indexes are also used in join expressions, such as in CUSTOMER.CUS_CODE = INVOICE.CUS_CODE.
- *Do not use indexes in small tables or tables with low sparsity*. Remember, small tables and low-sparsity tables are not the same thing. A search condition in a table with low sparsity may return a high percentage of table rows anyway, making the index operation too costly and making the full table scan a viable option. Using the same logic, do not create indexes for tables with few rows and few attributes—*unless you must ensure the existence of unique values in a column*.
- *Declare primary and foreign keys so the optimizer can use the indexes in join operations*. All natural joins and old-style joins will benefit if you declare primary keys and foreign keys because the optimizer will use the available indexes at join time. (The declaration of a PK or FK, primary key or foreign key, will automatically create an index for the declared column.) Also, for the same reason, it is better to write joins using the SQL JOIN syntax.

- *Declare indexes in join columns other than PK or FK*. If you perform join operations on columns other than the primary and foreign keys, you might be better off declaring indexes in those columns.

You cannot always use an index to improve performance. For example, using the data shown in Table 4.4 in the next section, the creation of an index for P_MIN will not help the search condition P_QOH > P_MIN * 1.10. The reason is that in some DBMSs, *indexes are ignored when you use functions in the table attributes*. However, major databases such as Oracle, SQL Server, and DB2 now support function-based indexes. A **function-based index** is an index based on a specific SQL function or expression. For example, you could create an index on YEAR(INV_DATE). Function-based indexes are especially useful when dealing with derived attributes. For example, you could create an index on EMP_SALARY > EMP_COMMISSION.
How many indexes should you create? It bears repeating that you should not create an index for every column in a table. Too many indexes will slow down INSERT, UPDATE, and DELETE operations, especially if the table contains many thousands of rows. Furthermore, some query optimizers will choose only one index to be the driving index for a query, even if your query uses conditions in many different indexed columns. Which index does the optimizer use? If you use the cost-based optimizer, the answer will change with time as new rows are added to or deleted from the tables. In any case, you should create indexes in all search columns and then let the optimizer choose. It is important to constantly evaluate the index usage—monitor, test, evaluate, and improve it if performance is not adequate.

Conditional Expressions

A conditional expression is normally placed within the WHERE or HAVING clauses of a SQL statement. Also known as conditional criteria, a conditional expression restricts the output of a query to only the rows that match the conditional criteria. Generally, the conditional criteria have the form shown in Table 4.4.

Table 4-4: Conditional criteria

OPERAND1	CONDITIONAL OPERATOR	OPERAND2
P_PRICE	>	10.00
V_STATE	=	FL
V_CONTACT	LIKE	Smith%
P_QOH	>	P_MIN * 1.10

In Table 4.4, note that an operand can be:
• A simple column name such as P_PRICE or V_STATE
• A literal or a constant such as the value 10.00 or the text 'FL'
• An expression such as P_MIN * 1.10

Most of the query optimization techniques mentioned below are designed to make the optimizer's work easier. The following common practices are used to write efficient conditional expressions in SQL code.

- Use simple columns or literals as operands in a conditional expression—avoid the use of conditional expressions with functions whenever possible. Comparing the contents of a

single column to a literal is faster than comparing to expressions. For example, P_PRICE > 10.00 is faster than P_QOH > P_MIN * 1.10 because the DBMS must evaluate the P_MIN * 1.10 expression first. The use of functions in expressions also adds to the total query execution time. For example, if your condition is UPPER (V_NAME) = 'JIM', try to use V_NAME = 'Jim' if all names in the V_NAME column are stored with proper capitalization.

- *Numeric field comparisons are faster than character, date, and NULL comparisons*. In search conditions, comparing a numeric attribute to a numeric literal is faster than comparing a character attribute to a character literal. In general, the CPU handles numeric comparisons (integer and decimal) faster than character and date comparisons. Because indexes do not store references to null values, NULL conditions involve additional processing, and therefore tend to be the slowest of all conditional operands.

- *Equality comparisons are generally faster than inequality comparisons*. For example, P_PRICE = 10.00 is processed faster because the DBMS can do a direct search using the index in the column. If there are no exact matches, the condition is evaluated as false. However, if you use an inequality symbol (>, >=, <, <=), the DBMS must perform additional processing to complete the request, because there will almost always be more "greater than" or "less than" values in the index than "equal" values. With the exception of NULL, the slowest of all comparison operators is LIKE with wildcard symbols, as in V_CONTACT LIKE "%glo%". Also, using the "not equal" symbol (< >) yields slower searches, especially when the sparsity of the data is high—that is, when there are many more different values than there are equal values.

- *Whenever possible, transform conditional expressions to use literals*. For example, if your condition is P_PRICE – 10 = 7, change it to read P_PRICE = 17. Also, if you have a composite condition such as:

  P_QOH < P_MIN AND P_MIN = P_REORDER AND P_QOH = 10

  change it to read:

  P_QOH = 10 AND P_MIN = P_REORDER AND P_MIN > 10

- *When using multiple conditional expressions, write the equality conditions first*. Note that this was done in the previous example. Remember, equality conditions are faster to process than inequality conditions. Although most RDBMSs will automatically do this for you, paying attention to this detail lightens the load for the query optimizer. The optimizer will not have to do what you have already done.

- *If you use multiple AND conditions, write the condition most likely to be false first*. If you use this technique, the DBMS will stop evaluating the rest of the conditions as soon as it finds a conditional expression that is evaluated as false. Remember, for multiple AND conditions to be found true, all conditions must be evaluated as true. If one of the conditions evaluates to false, the whole set of conditions will be evaluated as false. If you use this technique, the DBMS will not waste time unnecessarily evaluating additional conditions. Naturally, the use of this technique implies knowledge of the sparsity of the data set. For example, look at the following condition list:

  P_PRICE > 10 AND V_STATE = 'FL'

If you know that only a few vendors are located in Florida, you could rewrite this condition as:

V_STATE = 'FL' AND P_PRICE > 10

- *When using multiple OR conditions, put the condition most likely to be true first*. By doing this, the DBMS will stop evaluating the remaining conditions as soon as it finds a conditional expression that is evaluated as true. Remember, for multiple OR conditions to evaluate to true, only one of the conditions must be evaluated as true.

- *Whenever possible, try to avoid the use of the NOT logical operator*. It is best to transform a SQL expression that contains a NOT logical operator into an equivalent expression. For example:

  NOT (P_PRICE > 10.00) can be written as P_PRICE <= 10.00.
  Also, NOT (EMP_SEX = 'M') can be written as EMP_SEX = 'F'.

Query Formulation

Queries are usually written to answer questions. For example, if an end user gives you a sample output and tells you to match that output format, you must write the corresponding SQL. To get the job done, you must carefully evaluate what columns, tables, and computations are required to generate the desired output. To do that, you must have a good understanding of the database environment and the database that will be the focus of your SQL code. This section focuses on SELECT queries because they are the queries you will find in most applications. To formulate a query, you would normally follow these steps:

1. *Identify what columns and computations are required*. The first step is needed to clearly determine what data values you want to return. Do you want to return just the names and addresses, or do you also want to include some computations? Remember that all columns in the SELECT statement should return single values.
   a. Do you need simple expressions? For example, do you need to multiply the price by the quantity on hand to generate the total inventory cost? You might need some single attribute functions such as DATE(), SYSDATE(), or ROUND().
   b. Do you need aggregate functions? If you need to compute the total sales by product, you should use a GROUP BY clause. In some cases, you might need to use a subquery.
   c. Determine the granularity of the raw data required for your output. Sometimes, you might need to summarize data that is not readily available in any table. In such cases, you might consider breaking the query into multiple subqueries and storing those subqueries as views. Then you could create a top-level query that joins those views and generates the final output.
2. *Identify the source tables*. Once you know what columns are required, you can determine the source tables used in the query. Some attributes appear in more than one table. In those cases, try to use the least number of tables in your query to minimize the number of join operations.
3. *Determine how to join the tables*. Once you know what tables you need in your query statement, you must properly identify how to join the tables. In most cases, you will use some type of inner join, but in some instances, you might need to use an outer join.

4. *Determine what selection criteria are needed*. Most queries involve some type of selection criteria. In this case, you must determine what operands and operators are needed in your criteria. Ensure that the data type and granularity of the data in the comparison criteria are correct.
    a. *Simple comparison*. In most cases, you will be comparing single values—for example, P_PRICE > 10.
    b. *Single value to multiple values*. If you are comparing a single value to multiple values, you might need to use an IN comparison operator—for example, V_STATE IN ('FL', 'TN', 'GA').
    c. *Nested comparisons*. In other cases, you might need to have some nested selection criteria involving subqueries—for example, P_PRICE >5 (SELECT AVG (P_PRICE) FROM PRODUCT).
    d. *Grouped data selection*. On other occasions, the selection criteria might apply not to the raw data but to the aggregate data. In those cases, you need to use the HAVING clause.
5. *Determine the order in which to display the output*. Finally, the required output might be ordered by one or more columns. In those cases, you need to use the ORDER BY clause. Remember that the ORDER BY clause is one of the most resource-intensive operations for the DBMS.

DBMS Performance Tuning

DBMS performance tuning includes global tasks such as managing the DBMS processes in primary memory (allocating memory for caching purposes) and managing the structures in physical storage (allocating space for the data files). Fine-tuning the performance of the DBMS also includes applying several practices examined in the previous section. For example, the DBA must work with developers to ensure that the queries perform as expected—creating the indexes to speed up query response time and generating the database statistics required by cost-based optimizers. DBMS performance tuning at the server end focuses on setting the parameters used for:

- *Data cache*. The data cache size must be set large enough to permit as many data requests as possible to be serviced from the cache. Each DBMS has settings that control the size of the data cache; some DBMSs might require a restart. This cache is shared among all database users. The majority of primary memory resources will be allocated to the data cache.
- *SQL cache*. The SQL cache stores the most recently executed SQL statements (after the SQL statements have been parsed by the optimizer). Generally, if you have an application with multiple users accessing a database, the *same* query will likely be submitted by many different users. In those cases, the DBMS will parse the query only once and execute it many times, using the same access plan. In that way, the second and subsequent SQL requests for the same query are served from the SQL cache, skipping the parsing phase.
- *Sort cache*. The sort cache is used as a temporary storage area for ORDER BY or GROUP BY operations, as well as for index-creation functions.
- *Optimizer mode*. Most DBMSs operate in one of two optimization modes: cost-based or rule-based. Others automatically determine the optimization mode based on whether database statistics are available. For example, the DBA is responsible for generating the

database statistics that are used by the cost-based optimizer. If the statistics are not available, the DBMS uses a rule-based optimizer.

From the performance point of view, it would be optimal to have the entire database stored in primary memory to minimize costly disk access. This is why several database vendors offer in-memory database options for their main products. **In-memory database** systems are optimized to store large portions (if not all) of the database in primary (RAM) storage rather than secondary (disk) storage. These systems are becoming popular because increasing performance demands of modern database applications (such as Business Analytics and Big Data), diminishing costs, and technology advances of components (such as flash-memory and solid state drives.) Even though these type of databases "eliminate" disk access bottlenecks, they are still subject to query optimization and performance tuning rules, especially when faced with poorly designed databases or poorly written SQL statements. Although in-memory databases are carving a niche in selected markets, most database implementations still rely on data stored on disk drives. That is why managing the physical storage details of the data files plays an important role in DBMS performance tuning. Note the following general recommendations for physical storage of databases:

- Use **I/O accelerators**. This type of device uses flash solid state drives (SSDs) to store the database. An SSD does not have any moving parts and, therefore performs I/O operations at a higher speed than traditional rotating disk drives. I/O accelerators deliver high transaction performance rates and reduce contention caused by typical storage drives.
- Use **RAID** (Redundant Array of Independent Disks) to provide both performance improvement and fault tolerance, and a balance between them. Fault tolerance means that in case of failure, data can be reconstructed and retrieved. RAID systems use multiple disks to create virtual disks (storage volumes) formed by several individual disks. Table 4.5 describes the most common RAID configurations.

Table 4-5: Common RAID Levels

RAID Level	Description
0	The data blocks are spread over separate drives. Also known as *striped array*. Provides increased performance but no fault tolerance. Requires a minimum of two drives.
1	The same data blocks are written (duplicated) to separate drives. Also referred to as *mirroring* or *duplexing*. Provides increased read performance and fault tolerance via data redundancy. Requires a minimum of two drives.
3	The data is striped across separate drives, and parity data is computed and stored in a dedicated drive. (Parity data is specially generated data that permits the reconstruction of corrupted or missing data.) Provides good read performance and fault tolerance via parity data. Requires a minimum of three drives.
5	The data and the parity data are striped across separate drives. Provides good read performance and fault tolerance via parity data. Requires a minimum of three drives.
1+0	The data blocks are spread over separate drives and mirrored (duplicated). This arrangement provides both speed and fault tolerance. This is the recommended RAID configuration for most database installations (if cost is not an issue).

- Minimize disk contention. Use multiple, independent storage volumes with independent spindles (rotating disks) to minimize hard disk cycles. Remember, a database is composed of many table spaces, each with a particular function. In turn, each table space is composed of several data files in which the data is actually stored. A database should have at least the following table spaces:
    - *System table space*. This is used to store the data dictionary tables. It is the most frequently accessed table space and should be stored in its own volume.
    - *User data table space*. This is used to store end-user data. You should create as many user data table spaces and data files as are required to balance performance and usability. For example, you can create and assign a different

user data table space for each application and each distinct group of users, but this is not necessary for each user.
    - *Index table space*. This is used to store indexes. You can create and assign a different index table space for each application and each group of users. The index table space data files should be stored on a storage volume that is separate from user data files or system data files.
    - *Temporary table space*. This is used as a temporary storage area for merge, sort, or set aggregate operations. You can create and assign a different temporary table space for each application and each group of users.
    - *Rollback segment table space*. This is used for transaction-recovery purposes.

- Put high-usage tables in their own table spaces so the database minimizes conflict with other tables.
- Assign separate data files in separate storage volumes for the indexes, system, and high-usage tables. This ensures that index operations will not conflict with end-user data or data dictionary table access operations. Another advantage of this approach is that you can use different disk block sizes in different volumes. For example, the data volume can use a 16 K block size, while the index volume can use an 8 K block size. Remember that the index record size is generally smaller, and by changing the block size you will reduce contention and minimize I/O operations. This is very important; many database administrators overlook indexes as a source of contention. By using separate storage volumes and different block sizes, the I/O operations on data and indexes will happen asynchronously (at different times); more importantly, the likelihood of write operations blocking read operations is reduced, as page locks tend to lock fewer records.
- Take advantage of the various table storage organizations available in the database. For example, in Oracle consider the use of index-organized tables (IOT); in SQL Server, consider clustered index tables. An **index-organized table** (or **clustered index table**) is a table that stores the end-user data and the index data in consecutive locations on permanent storage. This type of storage organization provides a performance advantage to tables that are commonly accessed through a given index order, because the index contains the index key as well as the data rows. Therefore, the DBMS tends to perform fewer I/O operations.
- Partition tables based on usage. Some RDBMSs support the horizontal partitioning of tables based on attributes. By doing so, a single SQL request can be processed by multiple data processors. Put the table partitions closest to where they are used the most.
- Use denormalized tables where appropriate. In other words, you might be able to improve performance by taking a table from a higher normal form to a lower normal form—typically, from third to second normal form. This technique adds data duplication, but it minimizes join operations.
- Store computed and aggregate attributes in tables. In short, use derived attributes in your tables. For example, you might add the invoice subtotal, the amount of tax, and the total in the INVOICE table. Using derived attributes minimizes computations in queries and join operations, especially during the execution of aggregate queries.

Summary

- Database performance tuning refers to a set of activities and procedures designed to ensure that an end-user query is processed by the DBMS in the least amount of time. SQL performance tuning refers to activities on the client side that are designed to generate SQL code that returns the correct answer in the least amount of time, using the minimum amount of resources at the server end. DBMS performance tuning refers to activities on the server side that are oriented so the DBMS is properly configured to respond to clients' requests in the fastest way possible while making optimum use of existing resources.
- Database statistics refer to a number of measurements gathered by the DBMS that describe a snapshot of the database objects' characteristics. The DBMS gathers statistics about objects such as tables, indexes, and available resources, which include the number of processors used, processor speed, and temporary space available. The DBMS uses the statistics to make critical decisions about improving query processing efficiency.
- DBMSs process queries in three phases. In the parsing phase, the DBMS parses the SQL query and chooses the most efficient access/execution plan. In the execution phase, the DBMS executes the SQL query using the chosen execution plan. In the fetching phase, the DBMS fetches the data and sends the result set back to the client.
- Indexes are crucial in the process that speeds up data access. Indexes facilitate searching, sorting, and using aggregate functions and join operations. The improvement in data access speed occurs because an index is an ordered set of values that contains the index key and pointers. Data sparsity refers to the number of different values a column could have. Indexes are recommended in high-sparsity columns used in search conditions.
- During query optimization, the DBMS must choose what indexes to use, how to perform join operations, which table to use first, and so on. Each DBMS has its own algorithms for determining the most efficient way to access the data. The two most common approaches are rule-based and cost-based optimization.
- A rule-based optimizer uses preset rules and points to determine the best approach to execute a query. A cost-based optimizer uses sophisticated algorithms based on statistics about the objects being accessed to determine the best approach to execute a query. In this case, the optimizer process adds up the processing cost, the I/O costs, and the resource costs (RAM and temporary space) to determine the total cost of a given execution plan.
- SQL performance tuning deals with writing queries that make good use of the statistics. In particular, queries should make good use of indexes. Indexes are very useful when you want to select a small subset of rows from a large table based on a condition.
- Query formulation deals with how to translate business questions into specific SQL code to generate the required results. To do this, you must carefully evaluate which columns, tables, and computations are required to generate the desired output.
- DBMS performance tuning includes tasks such as managing the DBMS processes in primary memory (allocating memory for caching purposes) and managing the structures in physical storage (allocating space for the data files).

Review Questions

1. What is SQL performance tuning?
2. What is database performance tuning?
3. What is the focus of most performance-tuning activities, and why does that focus exist?
4. What are database statistics, and why are they important?
5. How are database statistics obtained?
6. What database statistics measurements are typical of tables, indexes, and resources?
7. How is the processing of SQL DDL statements (such as CREATE TABLE) different from the processing required by DML statements?
8. In simple terms, the DBMS processes a query in three phases. What are the phases, and what is accomplished in each phase?

Chapter 5: Business Intelligence and Data Warehouse

Learning Outcomes

After this chapter you will be able to:

1. Describe the role of business intelligence in providing comprehensive business decision support
2. Describe the architecture, reporting styles, evolution, and benefits of business intelligence
3. Differentiate between operational data and decision support data
4. Identify the purpose, characteristics, and components of a data warehouse
5. Develop star and snowflake schemas for decision-making purposes
6. Describe the characteristics and capabilities of online analytical processing (OLAP)
7. Describe the role and functions of data analytics and data mining
8. Define a data lake

The Need for Data Analysis

Organizations tend to grow and prosper as they gain a better understanding of their environment. Most managers need to track daily transactions to evaluate how the business is performing. By tapping into the operational database, management can develop an understanding of how the company is performing and evaluate whether the current strategies meet organizational goals. In addition, analyzing the company data can provide insightful information about short-term tactical evaluations and strategic questions, such as: Are our sales promotions working? What market percentage are we controlling? Are we attracting new customers? Tactical and strategic decisions are also shaped by constant pressure from external and internal forces, including globalization, the cultural and legal environment, and technology. Organizations are always looking for a competitive advantage through product development, market positioning, sales promotions, and customer service.

Thanks to the Internet, customers are more informed than ever about the products they want and the prices they are willing to pay. Technology advances allow customers to place orders using their smart phones while they commute to work in the morning. Decision makers can no longer wait a couple of days for a report to be generated; they are compelled to make quick decisions if they want to remain competitive. How can companies survive on lower margins and still make a profit? The key is in having the right data at the right time to support the decision-making process. This process takes place at all levels of an organization. For example, transaction-processing systems, based on operational databases, are tailored to serve the information needs of people who deal with short-term inventory, accounts payable, and purchasing. Middle-level managers, general managers, vice presidents, and presidents focus on strategic and tactical decision making. Those managers require summarized information designed to help them make decisions in a complex business environment.

Companies and software vendors addressed these multilevel decision support needs by creating autonomous applications for particular groups of users, such as those in finance, customer management, human resources, and product support. Applications were also tailored to different industries such as education, retail, health care, and finance. This approach worked well for some time, but changes in the business world, such as globalization, expanding markets, mergers and acquisitions, increased regulation, and new technologies, called for new ways of integrating and managing decision support across levels, sectors, and geographic locations. This more comprehensive and integrated decision support framework within organizations became known as business intelligence.

Business Intelligence

Business intelligence (BI) is a term that describes a comprehensive, cohesive, and integrated set of tools and processes used to capture, collect, integrate, store, and analyze data with the purpose of generating and presenting information to support business decision making. This intelligence is based on learning and understanding the facts about the business environment. BI is a framework that allows a business to transform data into information, information into knowledge, and knowledge into wisdom. BI has the potential to positively affect a company's culture by creating continuous business performance improvement through active decision support at all levels in an organization. This business insight empowers users to make sound decisions based on the accumulated knowledge of the business. BI's initial adopters were high-volume industries such as financial services, insurance, and healthcare companies. As BI technology evolved, its usage spread to other industries such as telecommunications, retail/merchandising, manufacturing, media, government, and even education. Table 5.1 lists

some companies that have implemented BI tools and shows how the tools benefited the companies. You will learn about these tools later in the chapter.

Table 5-1: Solving Business Problems and Adding Value with BI Tools

Company	Problem	Benefit
Alliant Energy Wisconsin-based utility company that serves more than 965,000 electric and 415,000 gas customers. Source: ibm.com/products/cognos-analytics	• Needed to meet increasing demand for electric and gas usage • Wanted to expand clean and renewable energy options • Needed to modernize the power grid and upgrade the gas distribution system	• Developed an analytics workflow that evaluates and ranks customer requests • Reduced the company's carbon footprint • Provided access to data that drives decisions about assets and operations
NASDAQ Largest U.S. electronic stock market trading organization Source: Oracle Corp. www.oracle.com	• Inability to provide real-time, ad hoc query and standard reporting for executives, business analysts, and other users • Excessive storage costs for many terabytes of data	• Reduced storage costs by moving to a multitier storage solution • Implemented new data warehouse center with support for ad hoc query and reporting, and near real-time data access for end users
Pfizer Global pharmaceutical company Source: Oracle Corp. www.oracle.com	• Needed a way to control costs and adjust to tougher market conditions, international competition, and increasing government regulations • Needed better analytical capabilities and flexible decision-making framework	• Ability to get and integrate financial data from multiple sources in a reliable way • Streamlined, standards-based financial analysis to improve forecasting process • Faster and smarter decision making for business strategy formulation
Swisscom Switzerland's leading telecommunications provider Source: Microsoft Corp. www.microsoft.com	• Needed a tool to help employees monitor service-level compliance • Had a time-consuming process to generate performance reports • Needed a way to integrate data from 200 different systems	• Ability to monitor performance using dashboard technology • Quick and easy access to real-time performance data • Managers have closer and better control over costs

Implementing BI in an organization involves capturing not only internal and external business data, but also the metadata, or knowledge about the data. In practice, BI is a complex proposition that requires a deep understanding and alignment of the business processes, business data, and information needs of users at all levels in an organization. BI is not a product by itself, but a framework of concepts, practices, tools, and technologies that help a business better understand its core capabilities, provide snapshots of the company situation, and identify key opportunities to create competitive advantage. In general, BI provides a framework for:

- Collecting and storing operational data
- Aggregating the operational data into decision support data
- Analyzing decision support data to generate information
- Presenting such information to the end user to support business decisions
- Making business decisions, which in turn generate more data that is collected, stored, and so on (restarting the process)
- Monitoring results to evaluate outcomes of the business decisions, which again provides more data to be collected, stored, and so on
- Predicting future behaviors and outcomes with a high degree of accuracy

The preceding points represent a system-wide view of the flow of data, processes, and outcomes within the BI framework. In practice, the first point, collecting and storing operational data, does not fall into the realm of a BI system per se; rather, it is the function of an operational system. However, the BI system will use the operational data as input material from which information will be derived. The rest of the processes and outcomes explained in the preceding points are oriented toward generating knowledge, and they are the focus of the BI system. In the following section, you will learn about the basic BI architecture.

Business Intelligence Architecture

BI covers a range of technologies and applications to manage the entire data life cycle from acquisition to storage, transformation, integration, presentation, analysis, monitoring, and archiving. BI functionality ranges from simple data gathering and transformation to very complex data analysis and presentation. BI architecture ranges from highly integrated, single-vendor systems to loosely integrated, multivendor environments. However, some common functions are expected in most BI implementations. Like any critical business IT infrastructure, the BI architecture is composed of many interconnected parts: people, processes, data, and technology working together to facilitate and enhance a business's management and governance. Figure 5.1 depicts how all these components fit together within the BI framework.

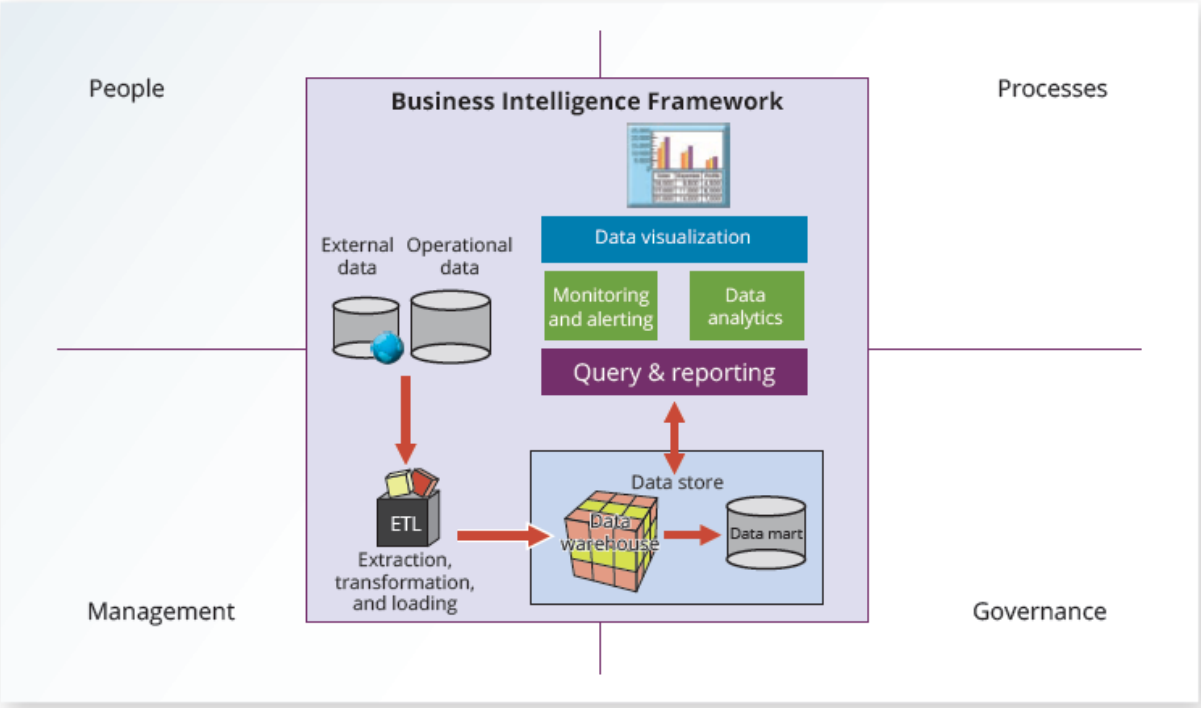


Figure 5-1: BI Framework(Coronel C., Morris S. 2022)

The general BI framework depicted in Figure 5.1 has six basic components that encompass the functionality required on most current-generation BI systems. The components are briefly described in Table 5.2.

Table 5-2: Basic BI Architectural Components

Component	Description
ETL tools	Data **extraction, transformation, and loading (ETL)** tools collect, filter, integrate, and aggregate internal and external data to be saved into a data store optimized for decision support.
Data store	The data store is optimized for decision support and is generally represented by a *data warehouse* or a *data mart*. The data is stored in structures that are optimized for data analysis and query speed.
Query and reporting	This component performs data selection and retrieval, and it is used by the data analyst to create queries that access the database and create the required reports.
Data visualization	This component presents data to the end user in a variety of meaningful and innovative ways. This tool helps the end user select the most appropriate presentation format, such as summary reports, maps, pie or bar graphs, mixed graphs, and static or interactive dashboards.
Data monitoring and alerting	This component allows real-time monitoring of business activities. The BI system presents concise information in a single integrated view with specific metrics about the system performance or activities, such as number of orders placed in the last four hours, number of customer complaints by product by month, and total revenue by region. Alerts can be placed on a given metric; once the value of a metric goes below or above a certain baseline, the system will perform a given action, such as emailing shop floor managers, presenting visual alerts, or starting an application.
Data analytics	This component performs data analysis and data-mining tasks using the data in the data store. This tool advises the user as to which data analysis tool to select and how to build a reliable business data model. Business models are generated by special algorithms that identify and enhance the understanding of business situations and problems.

Each BI component shown in Table 5.2 has generated a fast-growing market for specialized tools. Thanks to technological advancements, the components can interact with other components to form a truly open architecture. As a matter of fact, you can integrate multiple tools from different vendors into a single BI framework. Table 5.3 shows a sample of BI tools and vendors.

Table 5-3: Sample BI Tools

Tool	Description	Sample Vendors
Dashboards and business activity monitoring	**Dashboards** use web-based technologies to present key business performance indicators or information in a single integrated view, generally using graphics that are clear, concise, and easy to understand.	Salesforce IBM/Cognos BusinessObjects Information Builders iDashboards Tableau
Portals	**Portals** provide a unified, single point of entry for information distribution. Portals are a web-based technology that use a web browser to integrate data from multiple sources into a single webpage. Many different types of BI functionality can be accessed through a portal.	Oracle Portal Microsoft SAP
Data analysis and reporting tools	These advanced tools are used to query multiple and diverse data sources to create integrated reports.	Microsoft Reporting Services MicroStrategy SAS WebReportStudio
Data-mining tools	These tools provide advanced statistical analysis to uncover problems and opportunities hidden within business data.	SAP Teradata MS Analytics Services Rapid Miner
Data warehouses (DW)	The data warehouse is the foundation of a BI infrastructure. Data is captured from the production system and placed in the DW on a near real-time basis. BI provides company-wide integration of data and the capability to respond to business issues in a timely manner.	Microsoft Oracle IBM/Cognos Teradata Snowflake
OLAP tools	Online analytical processing provides multidimensional data analysis.	IBM/Cognos BusinessObjects Oracle Microsoft
Data visualization	These tools provide advanced visual analysis and techniques to enhance understanding and create additional insight of business data and its true meaning.	Tableau Dundas QlikView Microsoft PowerBI

The main BI architectural components were illustrated in Figure 5.1 and further explained in Table 5.2 and 5.3. However, the heart of the BI system is its advanced information generation and decision support capabilities. A BI system's advanced decision support functions come to life via its intuitive and informational user interface, and particularly its reporting capabilities. A modern BI system provides three distinctive reporting styles:

• *Advanced reporting*. A BI system presents insightful information about the organization in a variety of presentation formats. Furthermore, the reports provide interactive features that allow the end user to study the data from multiple points of view—from highly summarized to very detailed data. The reports present key actionable information used to support decision making.

- *Monitoring and alerting*. After a decision has been made, the BI system offers ways to monitor the decision's outcome. The BI system provides the end user with ways to define metrics and other key performance indicators to evaluate different aspects of an organization. In addition, exceptions and alerts can be set to warn managers promptly about deviations or problem areas.
- *Advanced data analytics*. A BI system provides tools to help the end user discover relationships, patterns, and trends hidden within the organization's data. These tools are

used to create two types of data analysis: explanatory and predictive. Explanatory analysis provides ways to discover relationships, trends, and patterns among data, while predictive analysis provides the end user with ways to create models that predict future outcomes.

Understanding the architectural components of a BI framework is the first step in properly implementing BI in an organization. A good BI infrastructure promises many benefits to an organization, as outlined in the next section.

Business Intelligence Benefits

A properly implemented BI architecture could provide a framework for continuous performance improvements and business decision making. Improved decision making is the main goal of BI, but BI provides other benefits:

- *Integrating architecture*. Like any other IT project, BI has the potential of becoming the integrating umbrella for a disparate mix of IT systems within an organization. This architecture could support all types of company-generated data from operational to executive, as well as diverse hardware for data collection, processing, and analysis.
- *Common user interface for data reporting and analysis*. BI front ends can provide up-to-the- minute consolidated information with advanced interfaces for all company users. IT departments no longer have to provide multiple training options for diverse interfaces. End users benefit from similar or common interfaces in different devices that use multiple clever and insightful presentation formats.
- *Common data repository fosters single version of company data*. In the past, multiple IT systems supported different aspects of an organization's operations. Such systems collected and stored data in separate data stores. Keeping the data synchronized and up to date has always been difficult. BI provides a framework to integrate such data under a common environment and present a single version of the data.
- *Improved organizational performance*. BI can provide competitive advantages in many different areas, from customer support to manufacturing processes. Such advantages can be reflected in added efficiency, reduced waste, increased sales, reduced employee and customer turnover, and most importantly, an increased bottom line for the business.

Achieving all these benefits takes a lot of human, financial, and technological resources, not to mention time. BI benefits are not achieved overnight, but are the result of a focused company wide effort that could take a long time. As a matter of fact, as you will learn in the next section, the BI field has evolved over a long period of time itself.

Business Intelligence Evolution

Providing useful information to end users has been a priority of IT systems since computing became an integral part of corporations. Business decision support has evolved over many decades. Following computer technology advances, business intelligence started with centralized reporting systems and evolved into today's highly integrated BI environments. Table 5.4 summarizes the evolution of BI systems. Using Table 5.4 as a guide, you can trace business intelligence from the mainframe environment to the desktop and then to the more current, cloud-based, mobile BI environments. The precursor of the modern BI environment was the first-generation decision support system. A **decision support system (DSS)** is an arrangement of computerized tools used to assist managerial decision making. A DSS typically has a much narrower focus and reach than a BI solution. At first, decision support systems were the realm of a few selected managers in an organization. Over time, decision support systems migrated to

agile platforms, such as high-end servers, appliances, and cloud-based offerings. This evolution effectively changed the reach of decision support systems; BI is no longer limited to a small group of top-level managers with training in statistical modeling. Instead, BI is now available to all users in an organization, from line managers to the shop floor to mobile agents in the field.

Table 5-4: BI Evolution

System Type	Data Source	Data Extraction/ Integration Process	Data Store	End-User Query Tool	End User Presentation Tool
Traditional mainframe-based online transaction processing (OLTP)	Operational data	None Reports read and summarized data directly from operational data	None Temporary files used for reporting purposes	Very basic Predefined reporting formats Basic sorting, totaling, and averaging	Very basic Menu-driven, predefined reports, text and numbers only
Managerial information system (MIS)	Operational data	Basic extraction and aggregation Read, filtered, and summarized operational data into intermediate data store	Lightly aggregated data in RDBMS	Same as above, in addition to some ad hoc reporting using SQL	Same as above, in addition to some ad hoc columnar report definitions
First-generation departmental decision support system (DSS)	Operational data External data	Data extraction and integration process populates DSS data store Run periodically	First DSS database generation Usually RDBMS	Query tool with some analytical capabilities and predefined reports	Spreadsheet style Advanced presentation tools with plotting and graphics capabilities
First-generation BI	Operational data External data	Advanced data extraction and integration Access diverse data sources, filters, aggregations, classifications, scheduling, and conflict resolution	Data warehouse RDBMS technology Optimized for query purposes Star schema model	Same as above	Same as above, in addition to multidimensional presentation tools with drill-down capabilities
Second-generation BI Online analytical processing (OLAP)	Same as above	Same as above	Data warehouse stores data in MDBMS Cubes with multiple dimensions	Adds support for end-user-based data analytics	Same as above, but uses cubes and multidimensional matrixes; limited by cube size Dashboards Scorecards Portals
Third-generation Mobile, cloud-based, and Big Data	Same as above Includes social media and machine-generated data	Same as above Cloud-based	Same as above Cloud-based Hadoop and NoSQL databases	Advanced analytics Flexible ad hoc interactions via data visualization	Mobile devices: smartphones and tablets

- Starting in the late 1970s, the need for information distribution was filled by centralized reports running on mainframes, minicomputers, or central server environments. Such reports were predefined and took considerable time to process.
- With the introduction of desktop computers in the 1980s, a new style of information distribution, the spreadsheet, emerged as the dominant format for decision support systems. In this environment, managers downloaded information from centralized data stores and manipulated the data in desktop spreadsheets.
- As the use of spreadsheets multiplied, IT departments tried to manage the flow of data in a more formal way using enterprise reporting systems. These systems were developed in the early 1990s and basically integrated all data into an IT umbrella that started with the first-generation DSS. The systems still used spreadsheet-like features with which end users were familiar.
- Once DSSs were established, the evolution of business intelligence flourished with the introduction of the data warehouse and online analytical processing systems (OLAPs) in the mid-1990s.
- Rapid changes in information technology and the Internet revolution led to the introduction of advanced BI systems such as web-based dashboards in the early and

mid-2000s and mobile BI later in the decade. With mobile BI, end users access BI reports via native applications that run on a mobile device, such as a smartphone or tablet.

- More recently, the social media revolution has generated large amounts of data. At the same time sensor-generated data is being collected and stored. Companies are using Big Data analytics and data visualization to leverage such data and obtain critical information otherwise unavailable to them.

Figure 5.2 depicts the evolution of BI information dissemination.

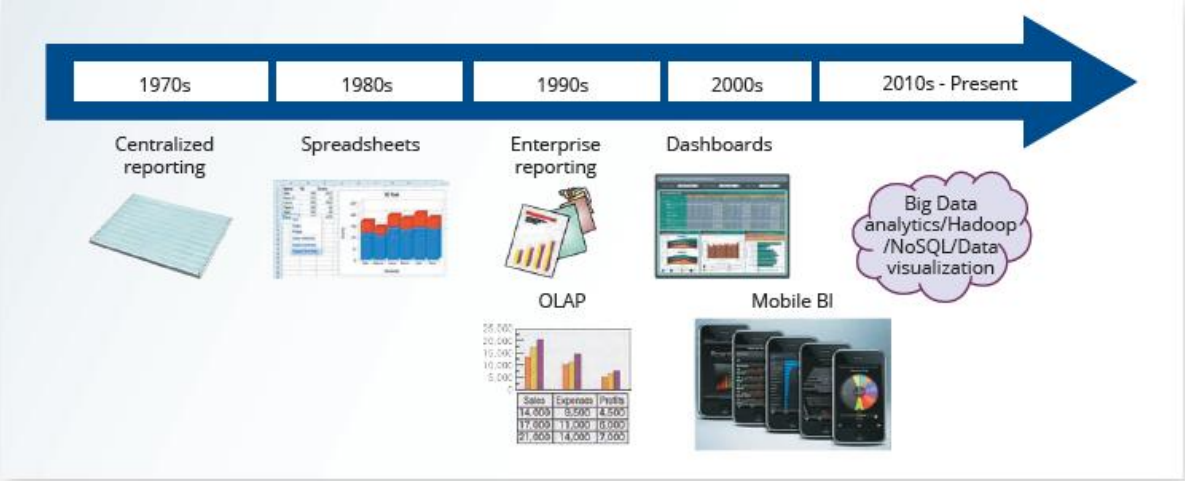


Figure 5-2: Evolution of BI Information Dissemination Formats(AntunHirsman/Shutterstock.com)

Mobile BI technology has a significant impact on the way BI information is disseminated and processed. If the number of students using smartphones to communicate with friends, update their Facebook status, and send tweets on Twitter is any indicator, you can expect the next generation of consumers and workers to be highly mobile. Leading corporations are therefore starting to push decision making to agents in the field to facilitate customer relationships, sales and ordering, and product support. Such mobile technologies are so portable and interactive that some users call them "disruptive" technologies. BI information technology has evolved from centralized reporting styles to the current, mobile BI and Big Data analytics style in the span of just a few years. The rate of technological change is not slowing down; to the contrary, technology advancements are accelerating the adoption of BI to new levels.

Business Intelligence Technology Trends

The need to analyze ever-larger amounts of varied data and technological advances are driving the growth of business intelligence technologies. These advances create new generations of more affordable products and services that are faster and easier to use. In turn, such products and services open new markets and work as driving forces in the increasing adoption of business intelligence technologies within organizations. Some of the more remarkable technological trends are:

- *Data storage improvements*. Newer data storage technologies, such as solid-state drives (SSD), offer increased performance and larger capacity that make data storage faster and more affordable. Currently, you can buy single SSD drives with a capacity ranging from 10 to 100 terabytes.
- *Business intelligence appliances*. Vendors now offer plug-and-play appliances optimized for data warehouse and BI applications. These new appliances offer improved price-performance ratios, simplified administration, rapid installation, scalability, and fast

integration. Some of these vendors include IBM, Netezza, EMC Greenplum, and Teradata Aster.

- *Business intelligence as a service*. Vendors now offer data warehouses and BI as a service. These cloud-based services allow any corporation to rapidly develop a data warehouse store without the need for hardware, software, or extra personnel. These prepackaged services offer "pay-as-you-go" models for specific industries and capacities, and they provide an opportunity for organizations to pilot-test a BI project without incurring large time or cost commitments. For example, such services are offered by IBM, Oracle, Microsoft, Teradata, MicroStrategy, and SAP.
- *Big Data analytics*. The Big Data phenomenon is creating a new market for data analytics. Organizations are turning to social media as the new source for information and knowledge to gain competitive advantages.
- *Personal analytics*. OLAP brought data analytics to the desktop of every end user in an organization. Mobile BI is extending business decision making outside the walls of the organization. BI can now be deployed to mobile users who are closer to customers. The main requirement is for the BI end user to have a key understanding of the business. Some personal analytics vendors include MicroStrategy, QlikView, and Tableau. There is a growing trend toward self-service, personalized data analytics. It is not so far-fetched to imagine that in a few years, end users will have smart data analytics agents on their smartphones tailored to their personal interests. Such personal agents will provide users with up-to-the-minute "intelligent knowledge" about their personal interests.

One constant in this relentless technological evolution is the need for better decision support data and the importance of understanding the difference between decision support data and operational data.

Decision Support Data

Although BI is used at the strategic and tactical managerial levels within organizations, *its effectiveness depends on the quality of data gathered at the operational level*. Yet, operational data is seldom well suited to decision support tasks.

Operational Data versus Decision Support Data

Operational data and decision support data serve different purposes. Therefore, it is not surprising to learn that their formats and structures differ. Most operational data is stored in a relational database in which the structures (tables) tend to be highly normalized. Operational data storage is optimized to support transactions that represent daily operations. For example, each time an item is sold, it must be accounted for. Customer data, inventory data, and other similar data need frequent updating. To provide effective update performance, operational systems store data in many tables, each with a minimum number of fields. Thus, a simple sales transaction might be represented by five or more different tables, such as INVOICE, INVOICE LINE, DISCOUNT, STORE, and DEPARTMENT. Although such an arrangement is excellent in an operational database, it is not efficient for query processing. For example, to extract a simple invoice, you would have to join several tables. Whereas operational data is useful for capturing daily business transactions, decision support data gives tactical and strategic business meaning to the operational data. From the data analyst's point of view, decision support data differs from operational data in three main areas: time span, granularity, and dimensionality.

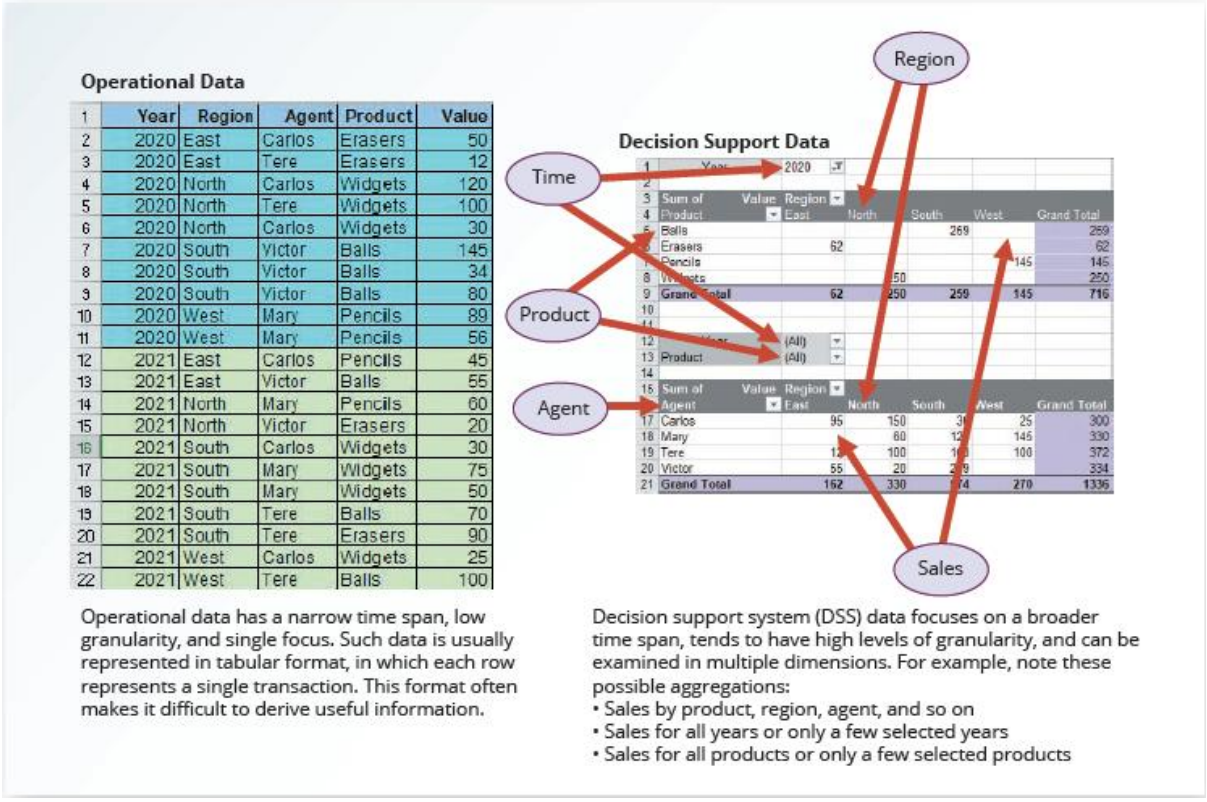
- *Time span*. Operational data covers a short time frame. In contrast, decision support data tends to cover a longer time frame. Managers are seldom interested in a specific sales

invoice to Customer X; rather, they tend to focus on sales generated during the last month, the last year, or the last five years.

- *Granularity (level of aggregation)*. Decision support data must be presented at different levels of aggregation, from highly summarized to nearly atomic. For example, if managers analyze regional sales, they must be able to access data showing the sales by region, by city within the region, by store within the city within the region, and so on. In that case, summarized data to compare the regions is required, along with data in a structure that enables a manager to **drill down**, or decompose, the data into more atomic components—that is, finer-grained data at lower levels of aggregation. In contrast, when you **roll up** the data, you are aggregating the data to a higher level.
- *Dimensionality*. Operational data focuses on representing individual transactions rather than the effects of the transactions over time. In contrast, data analysts tend to include many data dimensions and are interested in how the data relates over those dimensions. For example, an analyst might want to know how Product X fared relative to Product Z during the past six months by region, state, city, store, and customer. In that case, both place and time are part of the picture.

Figure 5.3 shows how decision support data can be examined from multiple dimensions such as product, region, and year, using a variety of filters to produce each dimension. The ability to analyze, extract, and present information in meaningful ways is one of the differences between decision support data and transaction-at-a-time operational data.

Figure 13.3 Transforming Operational Data Into Decision Support Data



	Year	Region	Agent	Product	Value
2	2020	East	Carlos	Erasers	50
3	2020	East	Tere	Erasers	12
4	2020	North	Carlos	Widgets	120
5	2020	North	Tere	Widgets	100
6	2020	North	Carlos	Widgets	30
7	2020	South	Victor	Balls	145
8	2020	South	Victor	Balls	34
9	2020	South	Victor	Balls	80
10	2020	West	Mary	Pencils	89
11	2020	West	Mary	Pencils	56
12	2021	East	Carlos	Pencils	45
13	2021	East	Victor	Balls	55
14	2021	North	Mary	Pencils	60
15	2021	North	Victor	Erasers	20
16	2021	South	Carlos	Widgets	30
17	2021	South	Mary	Widgets	75
18	2021	South	Mary	Widgets	50
19	2021	South	Tere	Balls	70
20	2021	South	Tere	Erasers	90
21	2021	West	Carlos	Widgets	25
22	2021	West	Tere	Balls	100

Figure 5-3: Transforming Operational Data into Decision Support Data (Coronel C., Morris S. 2022)

From the designer's point of view, the differences between operational and decision support data are as follows:

- Operational data represents transactions as they happen in real time. Decision support data is a snapshot of the operational data at a given point in time. Therefore, decision support data is historic, representing a time slice of the operational data.
- Operational and decision support data are different in terms of transaction *type* and transaction *volume*. Whereas operational data is characterized by update transactions, decision support data is mainly characterized by read-only transactions. Decision support data also requires *periodic* updates to load new data that is summarized from the operational data. Finally, the concurrent transaction volume in operational data tends to be very high compared with the low to medium levels in decision support data.
- Operational data is commonly stored in many tables, and the stored data represents information about a given transaction only. Decision support data is generally stored in a few tables derived from the operational data. The decision support data does not include the details of each operational transaction. Instead, decision support data represents transaction *summaries*; therefore, the decision support database stores data that is integrated, aggregated, and summarized for decision support purposes.
- The degree to which decision support data is summarized is very high when contrasted with operational data. Therefore, you will see a great deal of derived data in decision support databases. For example, rather than storing all 10,000 sales transactions for a given store on a given day, the decision support database might simply store the total number of units sold and the total sales dollars generated during that day. Decision support data might be collected to monitor such aggregates as total sales for each store or for each product. The purpose of the summaries is simple: they are used to establish and evaluate sales trends and product sales comparisons and to provide other data that serves decision needs. (How well are items selling? Should this product be discontinued? Has the advertising been effective as measured by increased sales?)
- The data models that govern operational data and decision support data are different. The operational database's frequent and rapid data updates make data duplication a potentially devastating problem. Therefore, the data in a relational transaction (operational) system generally requires normalized structures that yield many tables, each of which contains the minimum number of attributes. In contrast, the decision support database is not subject to such transaction updates, and the focus is on querying capability. Therefore, decision support databases tend to be non-normalized and include fewer tables than their operational counterparts, with each table containing many attributes. However, since decision support databases typically include data integrated from multiple operational databases, the collective effect is a large decision support database with numerous tables containing many attributes.
- The frequency and complexity of query activity in the operational database tends to be low to allow additional processing cycles for the more crucial update transactions. Therefore, queries against operational data typically are narrow in scope and low in complexity, and high speed is critical. In contrast, decision support data exists for the sole purpose of serving query requirements. Queries against decision support data typically are broad in scope and high in complexity.
- Finally, decision support data is characterized by very large amounts of data. The large data volume is the result of two factors. First, data is stored in non-normalized structures that are likely to display many data redundancies and duplications. Second, the same data can be categorized in many different ways to represent different snapshots. For example, sales data might be stored in relation to product, store, customer, region, and manager.

Table 5.5 summarizes the differences between operational and decision support data from the database designer's point of view.

Table 5-5: Operational Data v/s Decision Support Data Characteristics

Characteristic	Operational Data	Decision Support Data
Data currency	Current operations Real-time data	Historic data Snapshot of company data Time component (week/month/year)
Granularity	Atomic-detailed data	Summarized data
Summarization level	Low; some aggregate yields	High; many aggregation levels
Data model	Highly normalized Mostly relational DBMSs	Non-normalized Complex structures Some relational, but mostly multidimensional DBMSs
Transaction type	Mostly updates	Mostly query
Transaction volumes	High-update volumes	Periodic loads and summary calculations
Transaction speed	Updates are critical	Retrievals are critical
Query activity	Low to medium	High
Query scope	Narrow range	Broad range
Query complexity	Simple to medium	Very complex
Data volumes	Hundreds of gigabytes	Terabytes to petabytes

Decision Support Database Requirements

A decision support database is a specialized DBMS tailored to provide fast answers to complex queries. There are three main requirements for a decision support database: the database schema, data extraction and filtering, and database size.

Database Schema

The decision support database schema must support complex (non-normalized) data representations. As noted earlier, the decision support database must contain data that is aggregated and summarized. In addition to meeting those requirements, the queries must be able to extract multidimensional time slices. If you are using an RDBMS, the conditions suggest using non-normalized and even duplicated data. To see why this must be true, take a look at the 10-year sales history for a single store containing a single department. At this point, the data is fully normalized within the single table, as shown in Table 5.6. This structure works well when you have only one store with only one department. However, it is very unlikely that such a simple environment has much need for a decision support database. A decision support database becomes a factor when you are dealing with more than one store, each of which has more than one department. To support all of the decision support requirements, the database must contain data for all of the stores and all of their departments— and the database must be able to support multidimensional queries that track sales by stores, by departments, and over time. For simplicity, suppose that there are only two stores (A and B) and two departments (1 and 2) within each store. Also, change the time dimension to include yearly data. Table 5.7 shows the sales figures under the specified conditions. Only 2012, 2018, and 2021 are shown; ellipses (...) are used to indicate that data values were omitted. You can see in Table 13.7 that the number of rows and attributes already multiplies quickly.

Table 5-6: Ten-Year Sales History (for a Single department – millions of Dollars)

Table 13.6 Ten-Year Sales History for a Single Department, in Millions of Dollars

Year	Sales
2012	8,227
2013	9,109
2014	10,104
2015	11,553
2016	10,018
2017	11,875
2018	12,699
2019	14,875
2020	16,301
2021	19,986

Table 5-7: Yearly Sales Summaries (Two stores and two departments per store – in millions of Dollars)

	Store	Department	Sales
2012	A	1	1,985
2012	A	2	2,401
2012	B	1	1,879
2012	B	2	1,962
…	…	…	…
2018	A	1	3,912
2018	A	2	4,158
2018	B	1	3,426
2018	B	2	1,203
…	…	…	…
2021	A	1	7,683
2021	A	2	6,912
2021	B	1	3,768
2021	B	2	1,623

Now suppose that the company has 10 departments per store and 20 stores nationwide, and suppose that you want to access yearly sales summaries. Now you are dealing with 200 rows and 12 monthly sales attributes per row. (Actually, there are 13 attributes per row if you add each store's sales total for each year.) The decision support database schema must also be optimized for query (read-only) retrievals. To optimize query speed, the DBMS must support features such

as bitmap indexes and data partitioning. In addition, the DBMS query optimizer must be enhanced to support the non-normalized and complex structures in decision support databases.

Data Extraction and Filtering

The decision support database is created largely by extracting data from the operational database and by importing additional data from external sources. Thus, the DBMS must support advanced data extraction and data-filtering tools. To minimize the impact on the operational database, the data extraction capabilities should allow batch and scheduled data extraction, and should support different data sources: flat files and hierarchical, network, and relational databases, as well as multiple vendors. Data-filtering capabilities must include the ability to check for inconsistent data or data validation rules. Finally, to filter and integrate the operational data into the decision support database, the DBMS must support advanced data integration, aggregation, and classification. Using data from multiple external sources also usually means having to solve data-formatting conflicts. For example, data such as Social Security numbers and dates can occur in different formats; measurements can be based on different scales, and the same data elements can have different names. In short, data must be filtered and purified to ensure that only the pertinent decision support data is stored in the database and that it is stored in a standard format.

Database Size

Decision support databases tend to be very large; terabyte and petabyte ranges are not unusual. For example, Walmart has more than 4 petabytes of data in its data warehouses. Therefore, the DBMS must be capable of supporting **very large databases (VLDBs)**. To support a VLDB adequately, the DBMS might be required to support advanced storage technologies, and even more importantly, to support multiple-processor technologies, such as a symmetric multiprocessor (SMP) or a massively parallel processor (MPP). The complex information requirements and the ever-growing demand for sophisticated data analysis sparked the creation of a new type of data repository. This repository, called a data warehouse, contains data in formats that facilitate data extraction, data analysis, and decision making. It has become the foundation for a new generation of decision support systems.

The Data Warehouse

A **data warehouse** is an integrated, subject-oriented, time-variant, non-volatile collection of data that provides support for decision making. Here's a breakdown of the components:

- *Integrated*. The data warehouse is a centralized, consolidated database that integrates data derived from the entire organization and from multiple sources with diverse formats. Data integration implies that all business entities, data elements, data characteristics, and business metrics are *described in the same way throughout the enterprise*. Although this requirement sounds logical, you would be amazed to discover how many different measurements for "sales performance" can exist within an organization; the same scenario can be true for any other business element. For instance, the status of an order might be indicated with text labels such as "open," "received," "canceled," and "closed" in one department and as "1," "2," "3," and "4" in another department. A student's status might be defined as "freshman," "sophomore," "junior," or "senior" in the accounting department and as "FR," "SO," "JR," or "SR" in the computer information systems department. To avoid the potential format tangle, the data in the data warehouse must conform to a common format that is acceptable throughout the organization. This integration can be time-consuming, but once accomplished, it enhances decision

making and helps managers better understand the company's operations. This understanding can be translated into recognition of strategic business opportunities.

- *Subject-oriented*. Data warehouse data is arranged and optimized to provide answers to questions from diverse functional areas within a company. Data warehouse data is organized and summarized by topic, such as sales, marketing, finance, distribution, and transportation. For each topic, the data warehouse contains specific subjects of interest—products, customers, departments, regions, promotions, and so on. This form of data organization is quite different from the more functional or process-oriented organization of typical transaction systems. For example, an invoicing system designer concentrates on designing normalized data structures to support the business process by storing invoice components in two tables: INVOICE and INVLINE. In contrast, the data warehouse has a *subject* orientation. Data warehouse designers focus specifically on the data rather than on the processes that modify the data. (After all, data warehouse data is not subject to numerous real-time data updates!) Therefore, instead of storing an invoice, the data warehouse stores its "sales by product" and "sales by customer" components because decision support activities require the retrieval of sales summaries by product or customer.
- *Time-variant*. In contrast to operational data, which focuses on current transactions, warehouse data represents the flow of data through time. The data warehouse can even contain projected data generated through statistical and other models. It is also time-variant in the sense that when data is periodically uploaded to the data warehouse, all time-dependent aggregations are recomputed. For example, when data for previous weekly sales is uploaded to the data warehouse, the weekly, monthly, yearly, and other time-dependent aggregates for products, customers, stores, and other variables are also updated. Because data in a data warehouse constitutes a snapshot of the company history as measured by its variables, the time component is crucial. The data warehouse contains a time ID that is used to generate summaries and aggregations by week, month, quarter, year, and so on. Once the data enters the data warehouse, the time ID assigned to the data cannot be changed.
- *Non-volatile*. Once data enters the data warehouse, it is never removed. Because the data in the warehouse represents the company's history, the operational data, which represents the near-term history, is always added to it. Because data is never deleted and new data is continually added, the data warehouse is always growing. Therefore, the DBMS must be able to support multi-terabyte or greater databases operating on multiprocessor hardware.

In summary, the data warehouse is a read-only database optimized for data analysis and query processing. Typically, data is extracted from various sources and are then transformed and integrated—in other words, passed through a data filter—before being loaded into the data warehouse. As mentioned, this process is known as ETL. Figure 5.4 illustrates the ETL process to create a data warehouse from operational data. Although the centralized and integrated data warehouse can be an attractive proposition that yields many benefits, managers may be reluctant to embrace this strategy. Creating a data warehouse requires time, money, and considerable managerial effort. Therefore, it is not surprising that many companies begin their foray into data warehousing by focusing on more manageable data sets that are targeted to meet the special needs of small groups within the organization. These smaller data stores are called data marts.

Figure 13.4 The ETL Process

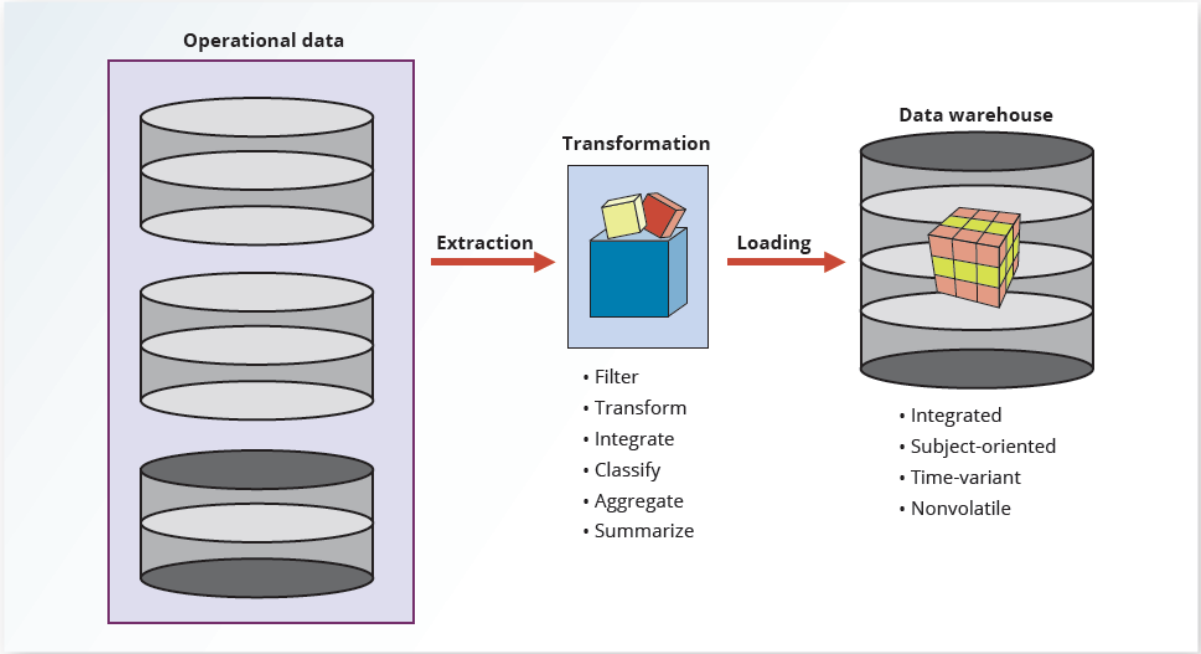


Figure 5-4: The ETL Process (Coronel C., Morris S. 2022)

Data Marts

A **data mart** is a small, single-subject data warehouse subset that provides decision support to a small group of people. In addition, a data mart could be created from data extracted from a larger data warehouse for the specific purpose of supporting faster data access to a target group or function. That is, data marts and data warehouses can coexist within a business intelligence environment. Some organizations choose to implement data marts not only because of the lower cost and shorter implementation time but because of the technological advances and inevitable "people issues" that make data marts attractive. Powerful computers can provide a customized decision support system to small groups in ways that might not be possible with a centralized system. Also, a company's culture may predispose its employees to resist major changes, but they might quickly embrace relatively minor changes that lead to demonstrably improved decision support.

In addition, people at different organizational levels are likely to require data with different summarization, aggregation, and presentation formats. Data marts can serve as a test vehicle for companies exploring the potential benefits of data warehouses. By gradually migrating from data marts to data warehouses, a specific department's decision support needs can be addressed within six months to one year, as opposed to the one- to three-year time frame usually required to implement a data warehouse. Information technology (IT) departments also benefit from this approach because their personnel can learn the issues and develop the skills required to create a data warehouse. The only difference between a data mart and a data warehouse is the size and scope of the problem being solved. The problem definitions and data requirements are essentially the same for both. To be useful, the data warehouse must conform to uniform structures and formats to avoid data conflicts and support decision making.

Twelve Rules That Define a Data Warehouse

In 1994, Bill Inmon and Chuck Kelley created a set of rules to define a data warehouse. These rules summarize many of the points made in this chapter about data warehouses.3 The 12 rules for a data warehouse are shown in Table 5.8. Note how the 12 rules capture the complete data warehouse life cycle—from its introduction as an entity separate from the operational data store to its components, functionality, and management processes. Most data warehouse implementations are based on the relational database model, and their market share suggests that their popularity will not fade anytime soon. Relational data warehouses use the star schema design technique to handle multidimensional data.

Table 5-8: Twelve Rules for a Data Warehouse

Rule No.	Description
1	The data warehouse and operational environments are separated.
2	The data warehouse data is integrated.
3	The data warehouse contains historical data over a long time.
4	The data warehouse data is snapshot data captured at a given point in time.
5	The data warehouse data is subject oriented.
6	The data warehouse data is mainly read-only with periodic batch updates from operational data. No online updates are allowed.
7	The data warehouse development life cycle differs from classical systems development. Data warehouse development is data-driven; the classical approach is process-driven.
8	The data warehouse contains data with several levels of detail: current detail data, old detail data, lightly summarized data, and highly summarized data.
9	The data warehouse environment is characterized by read-only transactions to very large data sets. The operational environment is characterized by numerous update transactions to a few data entities at a time.
10	The data warehouse environment has a system that traces data sources, transformations, and storage.
11	The data warehouse's metadata is a critical component of this environment. The metadata identifies and defines all data elements. The metadata provides the source, transformation, integration, storage, usage, relationships, and history of each data element.
12	The data warehouse contains a chargeback mechanism for resource usage that enforces optimal use of the data by end users.

Star Schemas

The **star schema** is a data-modelling technique used to map multidimensional decision support data into a relational database. In effect, the star schema creates the near equivalent of a multidimensional database schema from the existing relational database. Star schemas yield an easily implemented model for multidimensional data analysis while preserving the relational structures on which the operational database is built. The basic star schema has four components:
facts, dimensions, attributes, and attribute hierarchies.

Facts

Facts are numeric measurements (values) that represent a specific business aspect or activity. For example, sales figures are numeric measurements that represent product and service sales. Facts commonly used in business data analysis are units, costs, prices, and revenues. Facts are normally stored in a fact table that is the center of the star schema. The **fact table** contains facts that are linked through their dimensions, which are explained in the next section. Facts can also be computed or derived at run time. Such computed or derived facts are sometimes called **metrics** to differentiate them from stored facts. The fact table is updated periodically with data from operational databases.

Dimensions

Dimensions are qualifying characteristics that provide additional perspectives to a given fact. Recall that dimensions are of interest because *decision support data is almost always viewed in relation to other data*. For instance, sales might be compared by product from region to region and from one time period to the next. The kind of problem typically addressed by a BI system might be to compare the sales of unit X by region for the first quarters of 2010 through 2020. In that example, sales have product, location, and time dimensions. In effect, dimensions are the magnifying glass through which you study the facts. Such dimensions are normally stored in **dimension tables**. Figure 5.5 depicts a star schema for sales with product, location, and time dimensions.

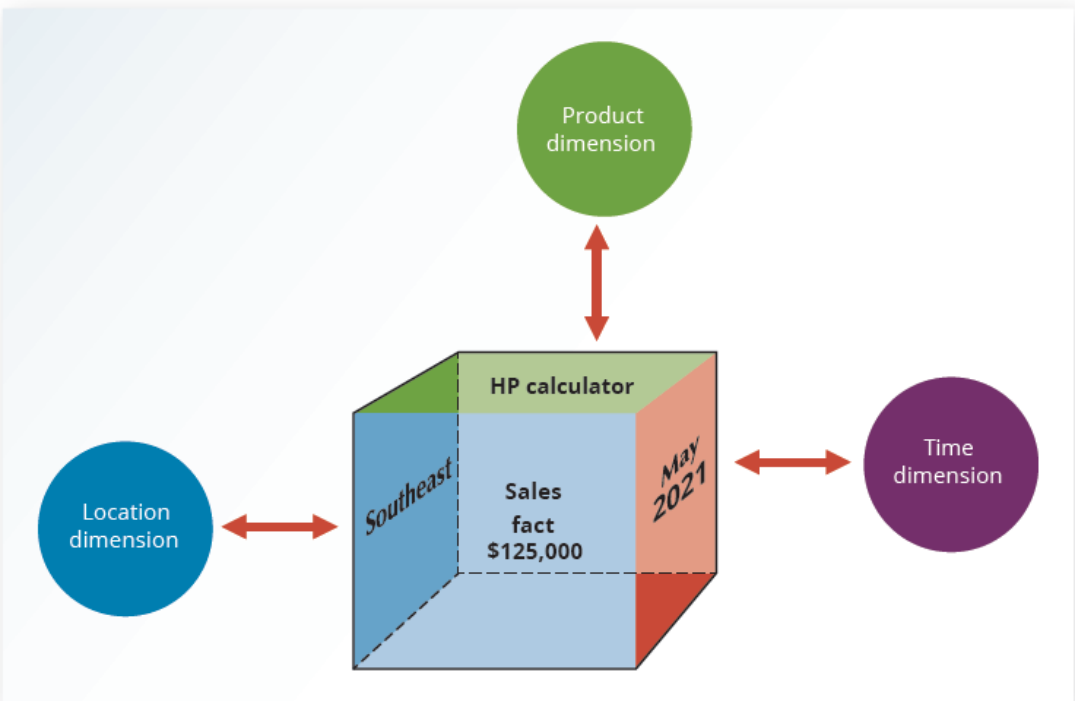


Figure 5-5: Simple Star Schema (Coronel C., Morris S. 2022)

Attributes

Each dimension table contains attributes. Attributes are often used to search, filter, or classify facts. *Dimensions provide descriptive characteristics about the facts through their attributes*. Therefore, the data warehouse designer must define common business attributes that will be used by the data analyst to narrow a search, group information, or describe dimensions. Using a sales example, some possible attributes for each dimension are illustrated in Table 5.9.

Table 5-9: Possible Attributes for Sales Dimensions

Dimension Name	Description	Possible Attributes
Location	Anything that provides a description of the location—for example, Nashville, Store 101, South Region, and TN	Region, state, city, store, and so on
Product	Anything that provides a description of the product sold—for example, hair care product, shampoo, Natural Essence brand, 5.5-oz. bottle, and blue liquid	Product type, product ID, brand, package, presentation, color, size, and so on
Time	Anything that provides a time frame for the sales fact—for example, the year 2021, the month of July, the date 07/29/2021, and the time 4:46 p.m.	Year, quarter, month, week, day, time of day, and so on

These product, location, and time dimensions add a business perspective to the sales facts. The data analyst can now group the sales figures for a given product, in a given region, and at a given time. The star schema, through its facts and dimensions, can provide the data in a format suited for data analysis. Also, it can do so without imposing the burden of additional and unnecessary data, such as order number, purchase order number, and status that commonly exists in operational databases. Conceptually, the sales example's multidimensional data model is best represented by a three-dimensional cube. Of course, this does not imply that there is a limit on the number of dimensions you can associate to a fact table. There is no mathematical limit to the number of dimensions used. However, using a three-dimensional model makes it easy to visualize the problem. The three-dimensional cube illustrated in Figure 5.6 represents a view of sales with product, location, and time dimensions.

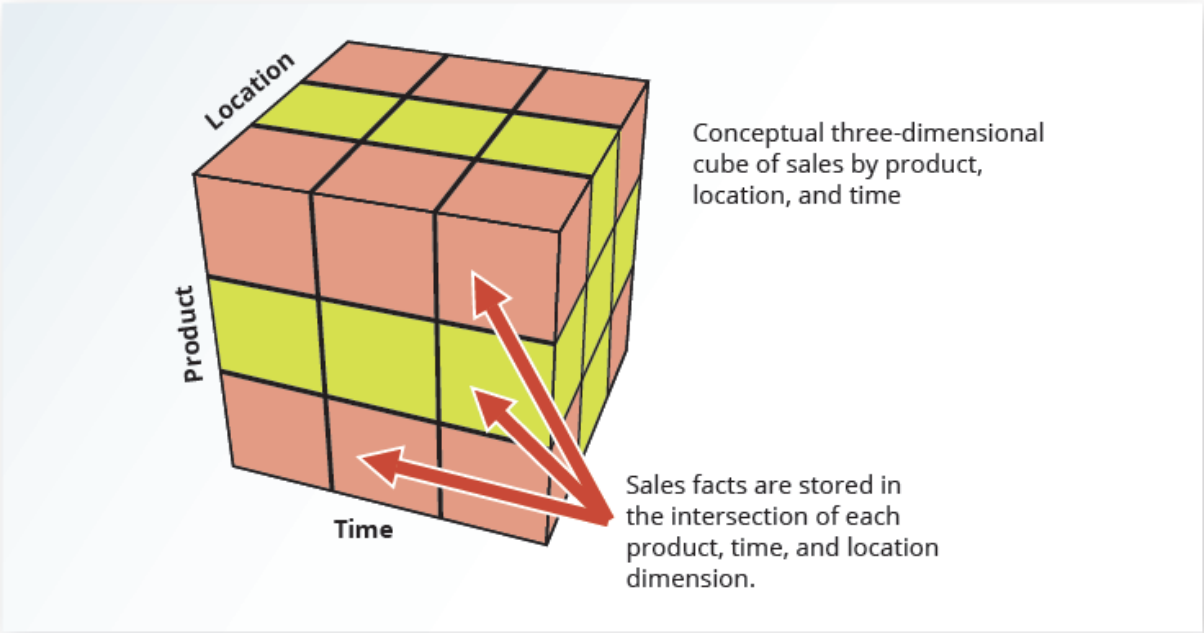


Figure 5-6: Three-Dimensional View of Sales (Coronel C., Morris S. 2022)

Keep in mind that this cube is only a *conceptual* representation of multidimensional data; it does not show how the data is physically stored in a data warehouse. Whatever the underlying database technology, one of the main features of multidimensional analysis is its ability to focus on specific "slices" of the cube. For example, the product manager may be interested in examining the sales of a product while the store manager is interested in examining the sales made by a particular store. In multidimensional terms, the ability to focus on slices of the cube to perform a more detailed analysis is known as **slice and dice**. Figure 5.7 illustrates the slice-and-dice concept; note that each cut across the cube yields a slice. Intersecting slices produce small cubes that constitute the "dice" part of the slice-and-dice operation.

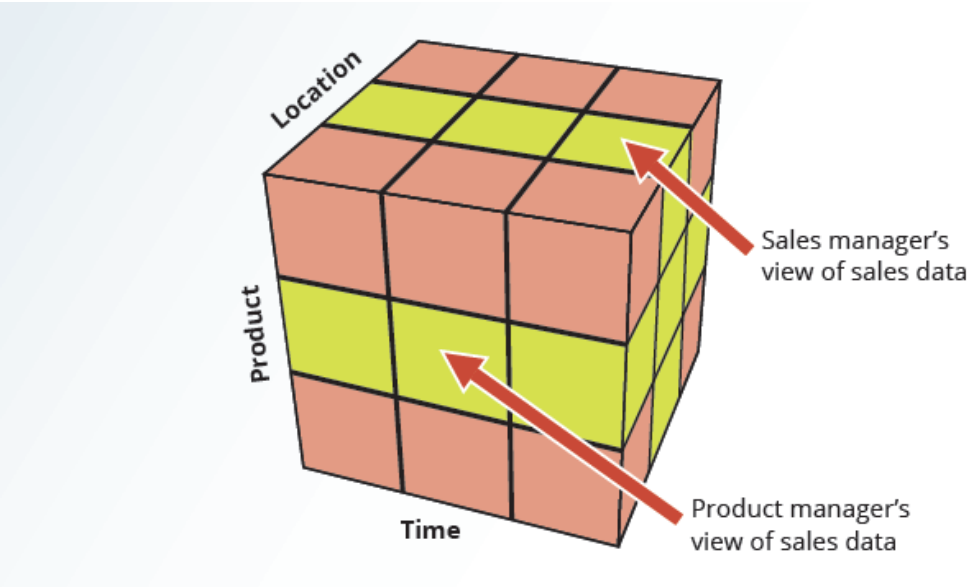


Figure 5-7: Slice-and-Dice View of Sales (Coronel C., Morris S. 2022)

To slice and dice, it must be possible to identify each slice of the cube. To do so, you use the values of each attribute in a given dimension. For example, to use the location dimension, you might need to define a STORE_ID attribute to focus on a particular store. Given the requirement

for attribute values in a slice-and-dice environment, re-examine Table 5.9. Note that each attribute adds perspective to the sales facts, thus setting the stage for finding new ways to search, classify, and possibly aggregate information. For example, the location dimension adds a geographic perspective of where the sales took place: in which region, state, city, store, and so on. All of the attributes are selected with the objective of providing decision support data to end users so they can study sales by each of the dimension's attributes.
Time is an especially important dimension; it provides a framework from which sales patterns can be analyzed and possibly predicted. Also, the time dimension plays an important role when the data analyst is interested in studying sales aggregates by quarter, month, week, and so on. Given the importance and universality of the time dimension from a data analysis perspective, many vendors have added automatic time dimension management features to their data-warehousing products.

Attribute Hierarchies

Attributes within dimensions can be ordered in a well-defined attribute hierarchy. The **Attribute hierarchy** provides a top-down data organization that is used for two main purposes: aggregation and drill-down/roll-up data analysis. For example, Figure 5.8 shows how the location dimension attributes can be organized in a hierarchy by region, state, city, and store.

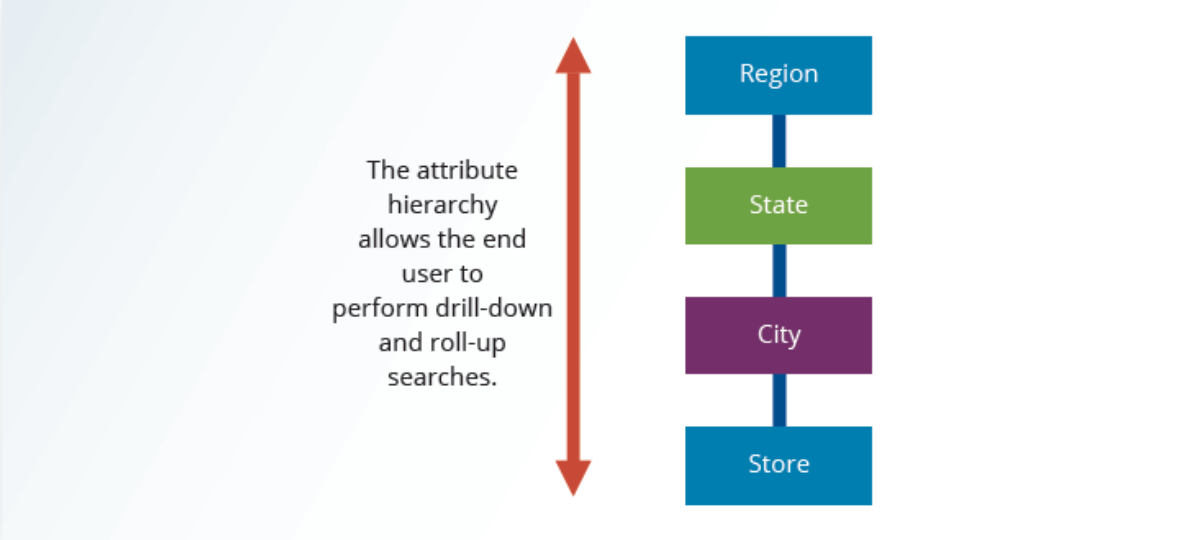


Figure 5-8: Location Attribute Hierarchy (Coronel C., Morris S. 2022)

The attribute hierarchy provides the capability to perform drill-down and roll-up searches in a data warehouse. For example, suppose a data analyst looks at the answers to the following query: How does the 2020 month-to-date sales performance compare to the 2021 month-to-date sales performance? The data analyst spots a sharp sales decline for March 2020, and thus might decide to drill down inside the month of March to see how sales by regions compared to the previous year. By doing that, the analyst can determine whether the low March sales were reflected in all regions or in only a particular region. This type of drill-down operation can even be extended until the data analyst identifies the store that is performing below the norm. It is not necessary for all attributes to be part of an attribute hierarchy; some attributes exist merely to provide narrative descriptions of the dimensions. However, keep in mind that the attributes from different dimensions can be grouped to form a hierarchy. For example, after you drill down from

city to store, you might want to drill down using the product dimension so the manager can identify slow-selling products in the store. The product dimension can be based on the product group (dairy, meat, and so on) or the product brand (Brand A, Brand B, and so on). Figure 5.9 illustrates a scenario in which the data analyst studies sales facts using the product, time, and location dimensions. In this example, the product dimension is set to "All products," meaning that the data analyst will see all products on the *y*-axis. The time dimension (*x*-axis) is set to "Quarter," meaning that the data is aggregated by quarters—for example, total sales of products A, B, and C in Q1, Q2, Q3, and Q4. Finally, the location dimension is initially set to "Region," thus ensuring that each cell contains the total regional sales for a given product in a given quarter.

Figure 13.9 Attribute Hierarchies in Multidimensional Analysis

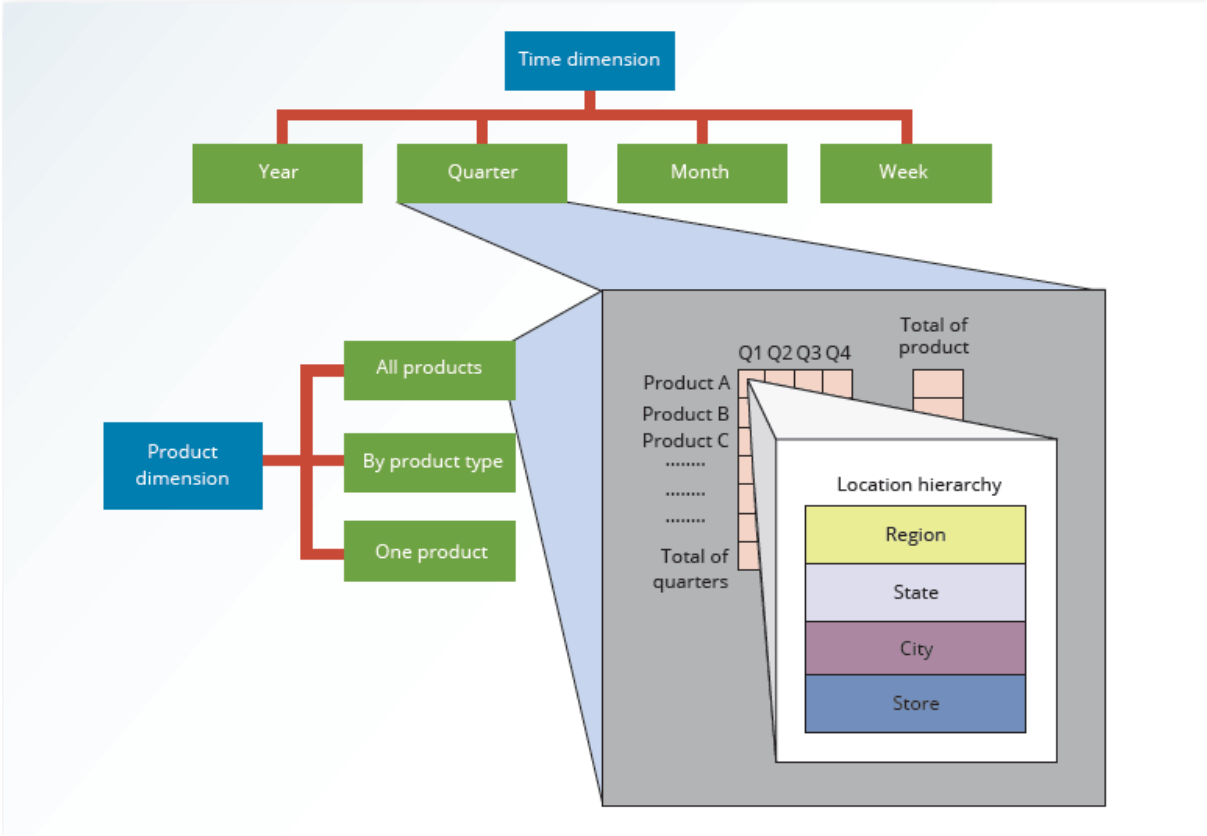


Figure 5-9: Attribute Hierarchies in Multidimensional Analysis (Coronel C., Morris S. 2022)

The simple scenario illustrated in Figure 13.9 provides the data analyst with three different information paths. On the product dimension (the *y*-axis), the data analyst can request to see all products, products grouped by type, or just one product. On the time dimension (the *x*-axis), the data analyst can request time–variant data at different levels of aggregation: year, quarter, month, or week. Each sales value initially shows the total sales, by region, of each product. When a GUI is used, clicking the region cell enables the data analyst to drill down to see sales by states within the region. Clicking one of the state values again yields the sales for each city in the state, and so forth. As the preceding examples illustrate, attribute hierarchies determine how the data in the data warehouse is extracted and presented. The attribute hierarchy information is stored in the DBMS's data dictionary and is used by the BI tool to access the data warehouse properly. Once such access is ensured, query tools must be closely integrated with the data warehouse's metadata, and they must support powerful analytical capabilities.

Star Schema Representation

Facts and dimensions are normally represented by physical tables in the data warehouse database. The fact table is related to each dimension table in a many-to-one (M:1) relationship. In other words, many fact rows are related to each dimension row. Using the sales example, you can conclude that each product appears many times in the SALES fact table. Fact and dimension tables are related by foreign keys and are subject to the familiar primary key and foreign key constraints. The primary key on the "1" side, the dimension table, is stored as part of the primary key on the "many" side, the fact table. *Because the fact table is related to many dimension tables, the primary key of the fact table is a composite primary key*. Figure 5.10 illustrates the relationships among the sales fact table and the product, location, and time dimension tables. To show you how easily the star schema can be expanded, a customer dimension has been added to the mix. Adding the customer dimension merely required including the CUST_ID in the SALES fact table and adding the CUSTOMER table to the database.

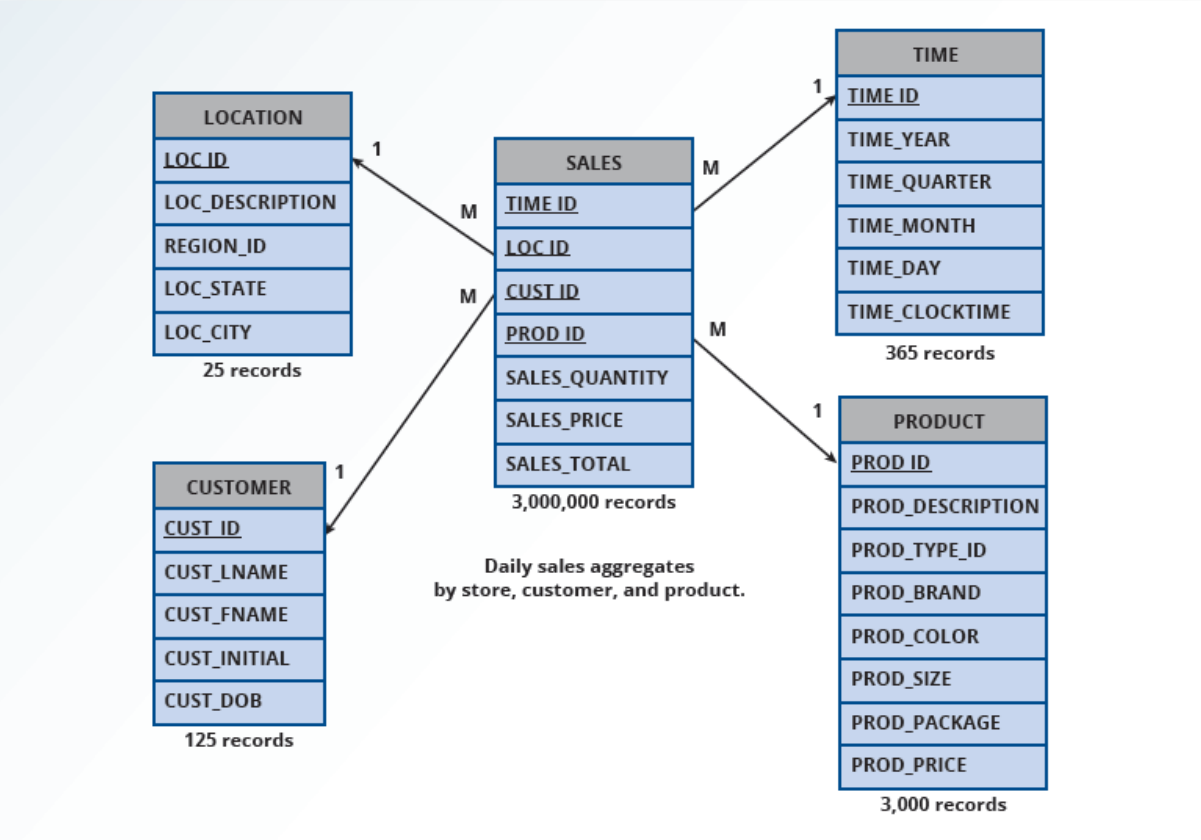


Figure 5-10: Star Schema for Sales (Coronel C., Morris S. 2022)

The composite primary key for the SALES fact table is composed of TIME_ID, LOC_ID, CUST_ID, and PROD_ID. Each record in the SALES fact table is uniquely identified by the combination of values for each of the fact table's foreign keys. *By default, the fact table's primary key is always formed by combining the foreign keys pointing to the dimension tables to which they are related*. In this case, each sales record represents each product sold to a specific customer, at a specific time, and in a specific location. In this schema, the TIME dimension table represents daily periods, so the SALES fact table represents daily sales aggregates by product and by customer. Because fact tables contain the actual values used in the decision support process, those values are repeated many times in the fact tables. Therefore, the fact tables are always the largest tables

in the star schema. Because the dimension tables contain only nonrepetitive information, such as all unique salespersons and all unique products, the dimension tables are always smaller than the fact tables. In a typical star schema, each dimension record is related to thousands of fact records. For example, “widget” appears only once in the product dimension, but it has thousands of corresponding records in the SALES fact table. This characteristic of the star schema facilitates data retrieval because the data analyst usually looks at the facts through the dimension’s attributes. Therefore, a data warehouse DBMS that is optimized for decision support first searches the smaller dimension tables before accessing the larger fact tables.

Although this configuration within the star schema would yield desirable results, there are always performance-enhancing techniques that can be implemented so that it performs even better. Once such technique is called the snowflake schema and it is achieved by normalizing dimensional tables to achieve semantic simplicity and facilitate end-user navigation through the dimensions. The star schema shown in Figure 5.11 is known as a **snowflake schema**, which is a type of star schema in which the dimension tables can have their own dimension tables. The snowflake schema is usually the result of normalizing dimension tables. By normalizing the dimension tables, you simplify the data-filtering operations related to the dimensions. In this example, the region, state, city, and location contain very few records compared to the SALES fact table. Only the location table is directly related to the SALES fact table.

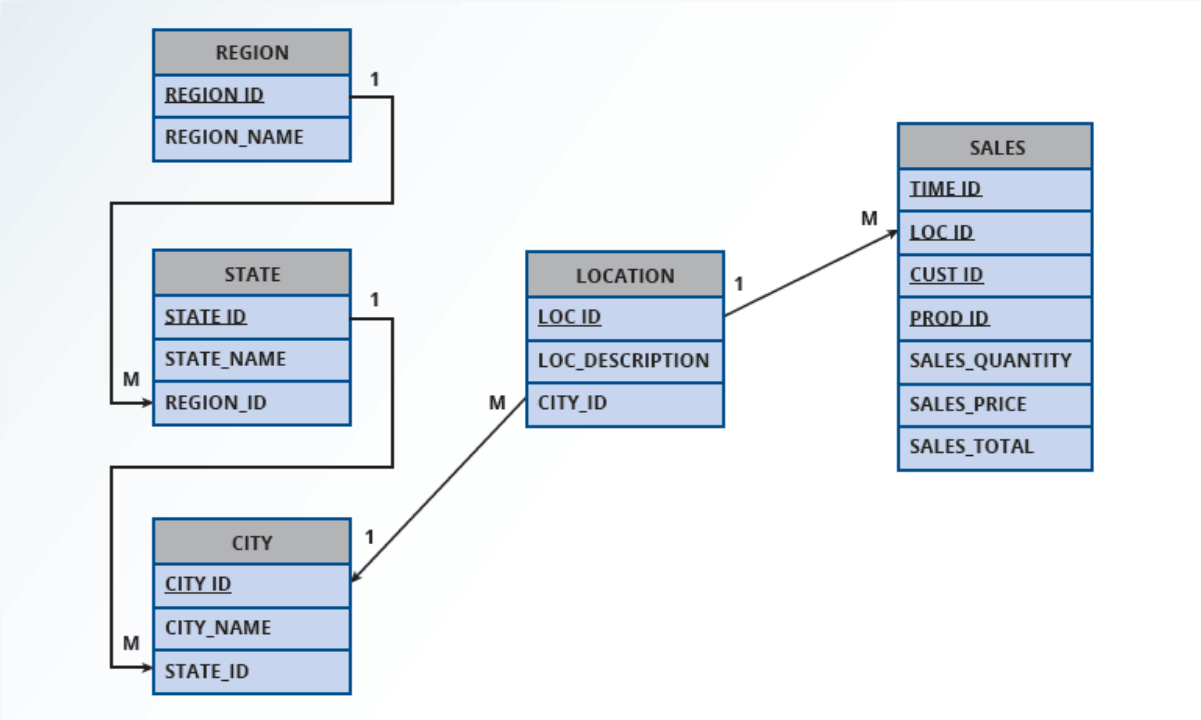


Figure 5-11: Normalized Dimension Tables (Snowflake Schema) (Coronel C., Morris S. 2022)

Online Analytical Processing

Online analytical processing (OLAP) is a BI style whose systems share three main characteristics:

- Multidimensional data analysis techniques
- Advanced database support
- Easy-to-use end-user interfaces

Multidimensional Data Analysis Techniques

The most distinctive characteristic of modern OLAP tools is their capacity for multidimensional analysis, in which data is processed and viewed as part of a multidimensional structure. This type of data analysis is particularly attractive to business decision makers because they tend to view business data as being related to other business data. To better understand this view, you can examine how a business data analyst might investigate sales figures. In this case, the analyst is probably interested in the sales figures as they relate to other business variables such as customers and time. In other words, customers and time are viewed as different dimensions of sales. Figure 5.12 illustrates how the operational (one-dimensional) view differs from the multidimensional view of sales. Note in Figure 5.12 that the operational (tabular) view of sales data is not well suited to decision support because the relationship between INVOICE and LINE does not provide a business perspective of the sales data. On the other hand, the end user's view of sales data *from a business perspective* is more closely represented by the multidimensional view of sales than by the tabular view of separate tables. Note also that the multidimensional view allows end users to consolidate or aggregate data at different levels: total sales figures by customers and by date. Finally, the multidimensional view of data allows a business data analyst to easily switch business perspectives (dimensions) from sales by customer to sales by division, by region, and so on. Multidimensional data analysis techniques are augmented by the following functions:

- *Advanced data presentation functions*. These functions include 3D graphics, pivot tables, crosstabs, data rotation, and three-dimensional cubes. Such tools are compatible with desktop spreadsheets, statistical packages, and query and report packages.
- *Advanced data aggregation, consolidation, and classification functions*. These allow the data analyst to create multiple data aggregation levels, slice and dice, and drill down and roll up data across different dimensions and aggregation levels. For example, aggregating data by week, month, quarter, and year allows the data analyst to drill down and roll up across time dimensions.
- *Advanced computational functions*. These include business-oriented variables such as market share, period comparisons, sales margins, product margins, and percentage changes; financial and accounting ratios, including profitability, overhead, cost allocations, and returns; and statistical and forecasting functions. These functions are provided automatically, so the end user does not need to redefine the components each time they are accessed.
- *Advanced data-modeling functions*. These provide support for what-if scenarios, variable assessment, contributions to outcome, linear programming, and predictive modeling tools. Predictive modeling allows the system to build advanced statistical models to predict future values (business outcomes) with a high percentage of accuracy.

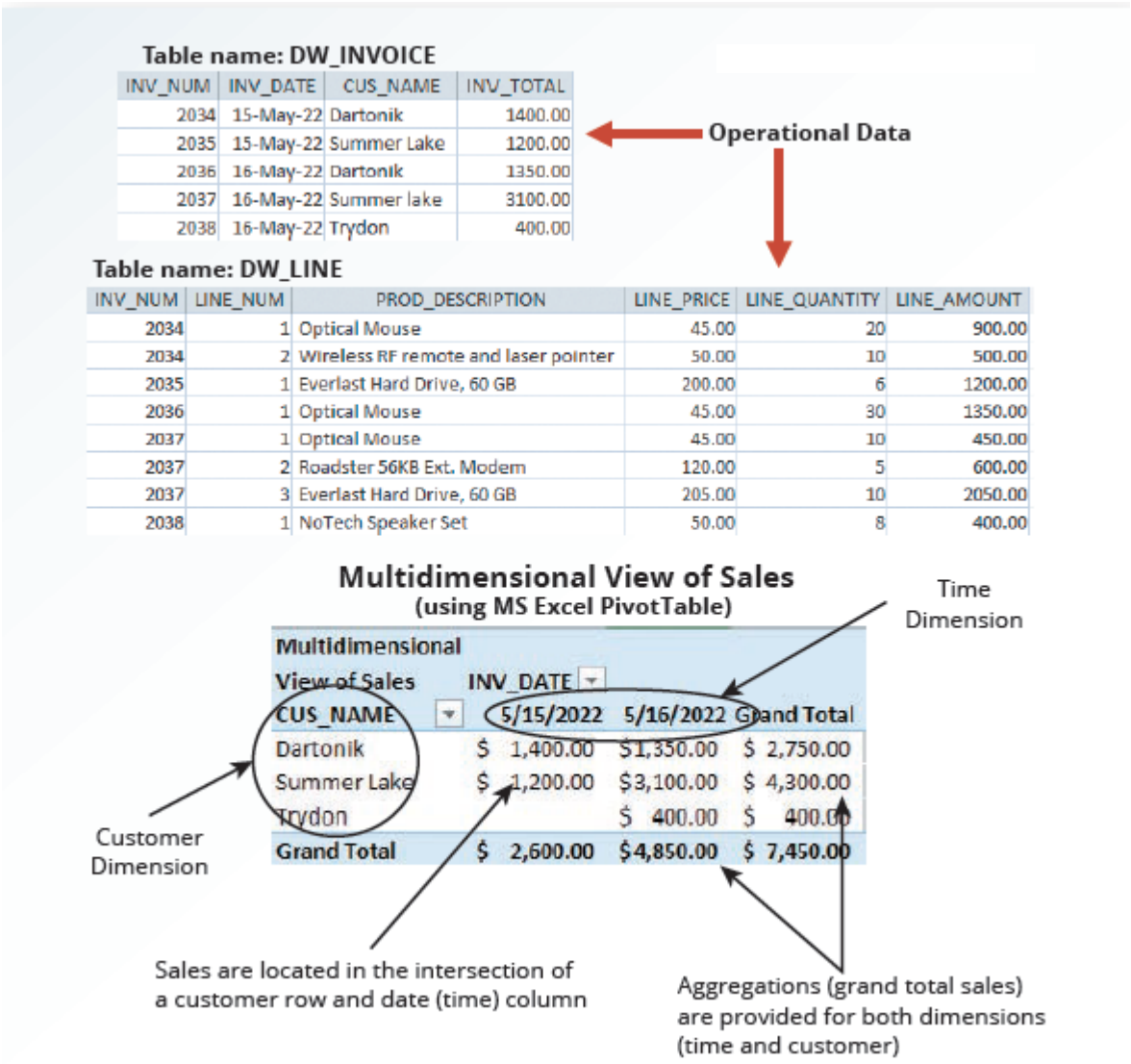


Figure 5-12: Operation vs Multidimensional View of Sales (Coronel C., Morris S. 2022)

Advanced Database Support

To deliver efficient decision support, OLAP tools must have the following advanced data access features:

- Access to many different kinds of DBMSs, flat files, and internal and external data sources
- Access to aggregated data warehouse data as well as to the detail data found in operational databases
- Advanced data navigation features such as drill-down and roll-up
- Rapid and consistent query response times
- The ability to map end-user requests, expressed in either business or model terms, to the appropriate data source and then to the proper data access language (usually SQL). The query code must be optimized to match the data source, regardless of whether the source is operational or data warehouse data.
- Support for very large databases. As explained earlier, the data warehouse could easily and quickly grow to multiple terabytes in size.

To provide a seamless interface, OLAP tools map the data elements from the data warehouse and the operational database to their own data dictionaries. This metadata is used to translate end-user data analysis requests into the proper (optimized) query codes, which are then directed to the appropriate data sources.

Easy-to-Use End-User Interfaces

The end-user analytical interface is one of the most critical OLAP components. When properly implemented, an analytical interface permits the user to navigate the data in a way that simplifies and accelerates decision making or data analysis. Advanced OLAP features become more useful when access to them is kept simple. OLAP tool vendors learned this lesson early and have equipped their sophisticated data extraction and analysis tools with easy-to-use graphical interfaces. Many of the interface features are “borrowed” from previous generations of data analysis tools that are already familiar to end users.

Because many analysis and presentation functions are common to desktop spreadsheet packages, most OLAP vendors have closely integrated their systems with spreadsheets such as Microsoft Excel. Using the features available in graphical end-user interfaces, OLAP simply becomes another option within the spreadsheet menu bar, as shown in Figure 5.13. This seamless integration is an advantage for OLAP systems and spreadsheet vendors because end users gain access to advanced data analysis features by using familiar programs and interfaces. Therefore, additional training and development costs are minimized.

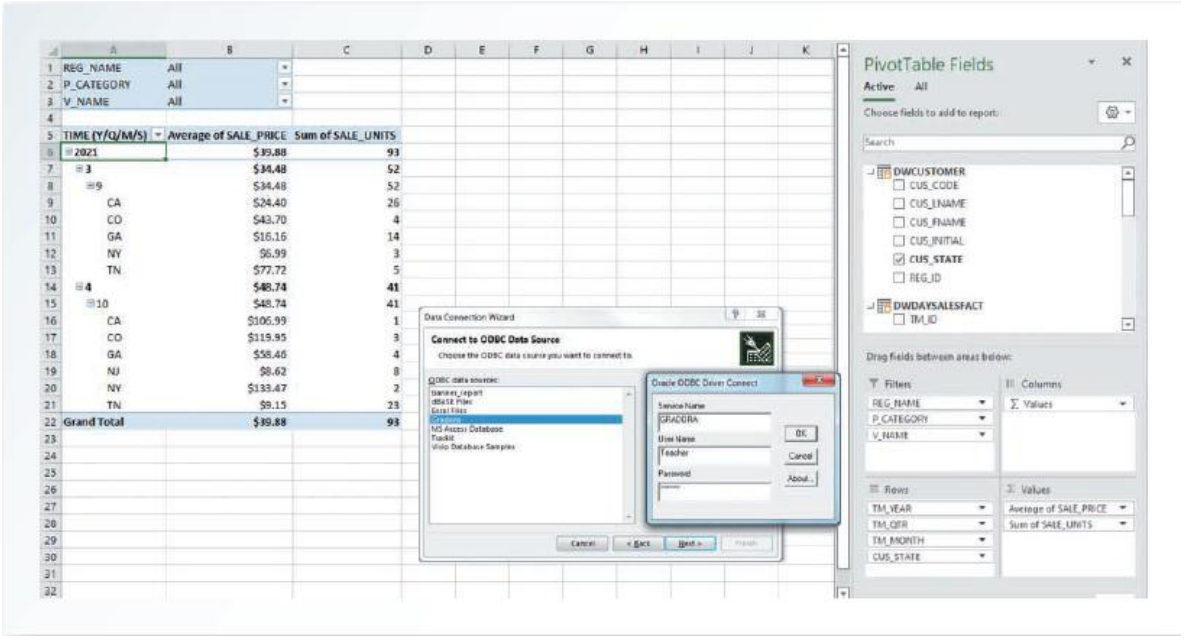


Figure 5-13: Integration of OLAP with a Spreadsheet Program (Coronel C., Morris S. 2022)

OLAP Architecture

The OLAP architecture is designed to meet ease-of-use requirements while keeping the system flexible. An OLAP system has three main architectural components:

- Graphical user interface (GUI)
- Analytical processing logic
- Data-processing logic

These three components can exist on the same computer or be distributed among several computers. Figure 5.14 illustrates OLAP's architectural components.

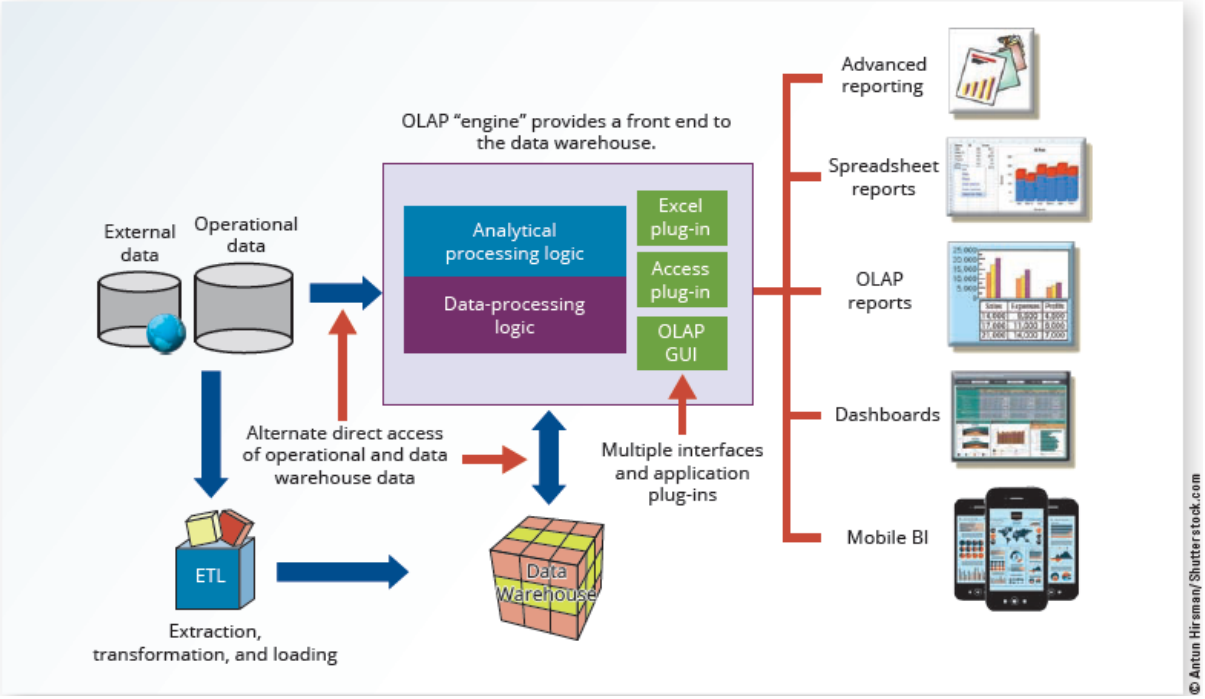


Figure 5-14: OLAP Architecture (Antun Hirsman/Shutterstock.com via Coronel C., Morris S. 2022)

As Figure 5.14 illustrates, OLAP systems are designed to use both operational and data warehouse data. The figure shows the OLAP system components on a single computer, but this single-user scenario is only one of many. In fact, one problem with the installation shown here is that each data analyst must have a powerful computer to store the OLAP system and perform all data processing locally.

A more common and practical architecture is one in which the OLAP GUI runs on client workstations while the OLAP data-processing logic (or OLAP "server") runs on a shared server computer. The OLAP analytical processing logic could be located on a client workstation, the OLAP server, or be split between the two sides. In any case, the OLAP server component acts as an intermediary between the OLAP GUI and the data warehouse. This middle layer accepts and handles the data-processing requests generated by the many end-user OLAP workstations. This flexible architecture allows for many different OLAP configurations. Figure 5.15 illustrates an OLAP server with local miniature data marts.

Figure 13.17 OLAP Server with Local Mini Data Marts

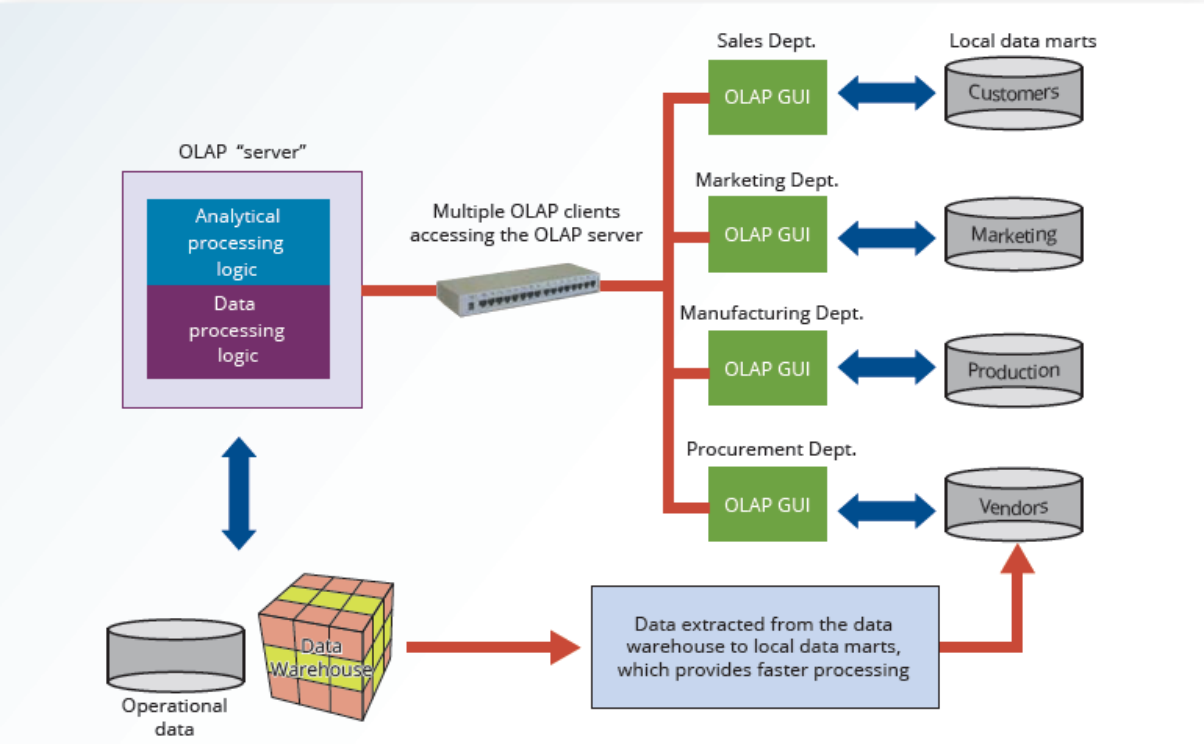


Figure 5-15: OLAP Server with Local Mini Data Marts (Coronel C., Morris S. 2022)

Relational OLAP

Relational online analytical processing (ROLAP) provides OLAP functionality by using relational databases and familiar relational query tools to store and analyze multidimensional data. This approach builds on existing relational technologies and represents a natural extension to companies that already use relational database management systems within their organizations. ROLAP adds the following extensions to traditional RDBMS technology:

- Multidimensional data schema support within the RDBMS
- Data access language and query performance optimized for multidimensional data
- Support for very large databases (VLDBs)

Multidimensional Data Schema Support within the RDBMS

Relational databases use normalized tables to reduce redundancies, thereby eliminating data anomalies, and to facilitate data updates. Unfortunately, for decision support purposes, it is easier to understand data when it is seen with respect to other data. (See the example in Figure 5.12) Given that view of the data environment, this text has emphasized that decision support data tends to be non-normalized, duplicated, and pre-aggregated. Those characteristics seem to preclude the use of standard relational design techniques and RDBMSs as the foundation for multidimensional data.

Fortunately for companies heavily invested in relational technology, ROLAP uses a special design technique that enables RDBMS technology to support multidimensional data representations. The star schema is designed to optimize data query operations rather than data update operations. Naturally, changing the data design foundation means that the tools used to access such data will have to change. End users who are familiar with traditional relational query tools will discover that those tools do not work efficiently with the star schema. However, ROLAP saves the day by adding support for the star schema when familiar query tools are used. ROLAP

provides advanced data analysis functions and improves query optimization and data visualization methods.

Data Access Language Optimized for Multidimensional Data

Another criticism of relational databases is that SQL is not suited for performing advanced data analysis. Most decision support data requests require the use of multiple-pass SQL queries or multiple nested SQL statements. To answer this criticism, ROLAP extends SQL so that it can differentiate between access requirements for data warehouse data (based on the star schema) and operational data (normalized tables). A ROLAP system, therefore, can generate the SQL code required to access the star schema data. Query performance is also improved because the query optimizer is modified to identify the SQL code's intended query targets. For example, if the query target is the data warehouse, the optimizer passes the requests to the data warehouse. However, if the end user performs drill-down queries against operational data, the query optimizer identifies that operation and properly optimizes the SQL requests before passing them to the operational DBMS. Another source of improved query performance is the use of advanced indexing techniques such as bitmapped indexes within relational databases. As the name suggests, a bitmapped index is based on 0 and 1 bits to represent a given condition. For example, if the REGION attribute in Figure 5.3 has only four outcomes—North, South, East, and West—those outcomes may be represented as shown in Table 5.10. Only the first 10 rows from Figure 5.3 are represented in the table. The "1" represents "bit on," and the "0" represents "bit off." For example, to represent a row with a REGION attribute = "East," only the "East" bit would be on. Note that each row must be represented in the index table.

Note that the index in Table 5.10 takes a minimal amount of space. Therefore, bitmapped indexes are more efficient at handling large amounts of data than the indexes typically found in many relational databases. However, keep in mind that bitmapped indexes are primarily used when the number of possible values for an attribute is fairly small. For example, REGION has only four outcomes in this example. Marital status—married, single, widowed, or divorced—would be another good, bitmapped index candidate, as would gender—M or F.

Table 5-10: Bitmap Representation of Region Values

North	South	East	West
0	0	1	0
0	0	1	0
1	0	0	0
1	0	0	0
1	0	0	0
0	1	0	0
0	1	0	0
0	1	0	0
0	0	0	1
0	0	0	1

Support for Very Large Databases

Recall that support for VLDBs is a requirement for decision support databases. Therefore, relational databases must have the proper tools to import, integrate, and populate large data sets into the data warehouse. Decision support data is normally loaded in bulk (batch) mode from the operational data. However, batch operations require that both the source and the destination databases be reserved (locked). The speed of the data-loading operations is important, especially when you realize that most operational systems run 24 hours a day, 7 days a week. Therefore, the window of opportunity for maintenance and batch loading is open only briefly, typically during slack periods. Clearly, ROLAP is a logical choice for companies that already use relational databases for their operational data. Given the size of the relational database market, it is hardly surprising that most current RDBMS vendors have extended their products to support data warehouses and OLAP capabilities.

Multidimensional OLAP

Multidimensional online analytical processing (MOLAP) extends OLAP functionality to **multidimensional database management systems (MDBMSs)**. An MDBMS uses proprietary techniques to store data in matrix-like *n*-dimensional arrays. MOLAP's premise is that multidimensional databases are best suited to manage, store, and analyze multidimensional data. Most of the proprietary techniques used in MDBMSs are derived from engineering fields such as computer-aided design/computer-aided manufacturing (CAD/CAM) and geographic information systems (GIS). MOLAP tools store data using multidimensional arrays, row stores, or column stores. Conceptually, MDBMS end users visualize the stored data as a three dimensional cube known as a **data cube**. The location of each data value in the data cube is a function of the *x*-, *y*-, and *z*-axes in a three-dimensional space. The three axes represent the dimensions of the data value. The data cubes can grow to *n* number of dimensions, thus becoming *hypercubes*. Data cubes are created by extracting data from the operational databases or from the data warehouse. One important characteristic of data cubes is that they are static; that is, they are not subject to change and must be created before they can be used. Data cubes cannot be created by ad hoc queries. Instead, you query precreated cubes with defined axes; for example, a cube for sales will have the product, location, and time dimensions, and you can query only those dimensions. Therefore, the data cube creation process is critical and requires in-depth front-end design work. This design work may be well justified because MOLAP databases are known to be much faster than their ROLAP counterparts, especially when dealing with large data sets. To speed data access, data cubes are normally held in memory in the **cube cache**. (A data cube is only a window to a predefined subset of data in the database. A data cube and a database are not the same thing.) Because MOLAP also benefits from a client/server infrastructure, the cube cache can be located at the MOLAP server, the MOLAP client, or both. Because the data cube is predefined with a set number of dimensions, the addition of a new dimension requires that the entire data cube be re-created, which is time-consuming. Therefore, when data cubes are created too often, the MDBMS loses some of its speed advantage over the relational database. In addition, the MDBMS uses proprietary data storage techniques that in turn require proprietary data access methods using a multidimensional query language.

Multidimensional data analysis is also affected by how the database system handles sparsity. **Sparsity** measures the density of the data held in the data cube; it is computed by dividing the total number of actual values in the cube by its total number of cells. Because the data cube's dimensions are predefined, not all cells are populated. In other words, some cells are empty. Returning to the sales example, many products might not be sold during a given time period in a given location. In fact, you will often find that a significant percentage of the data cube's cells are

not populated. In any case, multidimensional databases must handle sparsity effectively to reduce processing overhead and resource requirements.

Relational versus Multidimensional OLAP

Table 5.11 summarizes some pros and cons of ROLAP and MOLAP. Keep in mind that the selection of one or the other often depends on the evaluator's vantage point. For example, a proper evaluation of OLAP must include price, supported hardware platforms, compatibility with the existing DBMS, programming requirements, performance, and availability of administrative tools. The summary in Table 5.11 provides a useful starting point for comparison. ROLAP and MOLAP vendors are working to integrate their respective solutions within a unified decision support framework. Many OLAP products can handle tabular and multidimensional data with the same ease. The advent of OLAP techniques has also fueled the growth of other data analysis techniques that operate under the umbrella term of data analytics.

Table 5-11: ROLAP vs MOLAP

Characteristic	Rolap	Molap
Schema	Uses star schema Additional dimensions can be added dynamically	Uses data cubes Multidimensional arrays, row stores, column stores Additional dimensions require re-creation of the data cube
Database size	Medium to large	Large
Architecture	Client/server Standards-based	Client/server Open or proprietary, depending on vendor
Access	Supports ad hoc requests Unlimited dimensions	Limited to predefined dimensions Proprietary access languages
Speed	Good with small data sets; average for medium-sized to large data sets	Faster for large data sets with predefined dimensions

Data Analytics

Data analytics is a subset of business intelligence (BI) functionality that encompasses a wide range of mathematical, statistical, and modeling techniques with the purpose of extracting knowledge from data. Data analytics is used at all levels within the BI framework, including queries and reporting, monitoring and alerting, and data visualization. Hence, data analytics is a "shared" service that is crucial to what BI adds to an organization. Data analytics represents what business managers really want from BI: the ability to extract actionable business insight from current events and foresee future problems or opportunities. Data analytics discovers characteristics, relationships, dependencies, or trends in the organization's data, and then explains the discoveries and predicts future events based on the discoveries. In practice, data analytics is better understood as a continuous spectrum of knowledge acquisition that goes from *discovery* to *explanation* to *prediction*. The outcomes of data analytics then become part of the information framework on which decisions are built. Data analytics tools can be grouped into two separate (but closely related and often overlapping) areas:

- **Explanatory analytics** focuses on discovering and explaining data characteristics and relationships based on existing data. Explanatory analytics uses statistical tools to formulate hypotheses, test them, and answer the *how* and *why* of such relationships—for example, how do past sales relate to previous customer promotions?

- **Predictive analytics** focuses on *predicting future data outcomes* with a high degree of accuracy. Predictive analytics uses sophisticated statistical tools to help the end user create advanced models that answer questions about future data occurrences—for example, what would next month's sales be based on a given customer promotion?

You can think of explanatory analytics as explaining the past and present, while predictive analytics forecasts the future. However, you need to understand that both sciences work together; predictive analytics uses explanatory analytics as a stepping stone to create predictive models. Data analytics has evolved over the years from simple statistical analysis of business data to dimensional analysis with OLAP tools, and then from data mining that discovers data patterns, relationships, and trends to its current status of predictive analytics. The next sections illustrate the basic characteristics of data mining and predictive analytics.

Data Mining

Data mining refers to analyzing massive amounts of data to uncover hidden trends, patterns, and relationships; to form computer models to simulate and explain the findings; and then to use such models to support business decision making. In other words, data mining focuses on the discovery and explanation stages of knowledge acquisition. To put data mining in perspective, look at the pyramid in Figure 5.16, which represents how knowledge is extracted from data. *Data* forms the pyramid base and represents what most organizations collect in their operational databases. The second level contains *information* that represents the purified and processed data. Information forms the basis for decision making and business understanding. *Knowledge* is found at the pyramid's apex and represents highly distilled information that provides concise, actionable business insight.

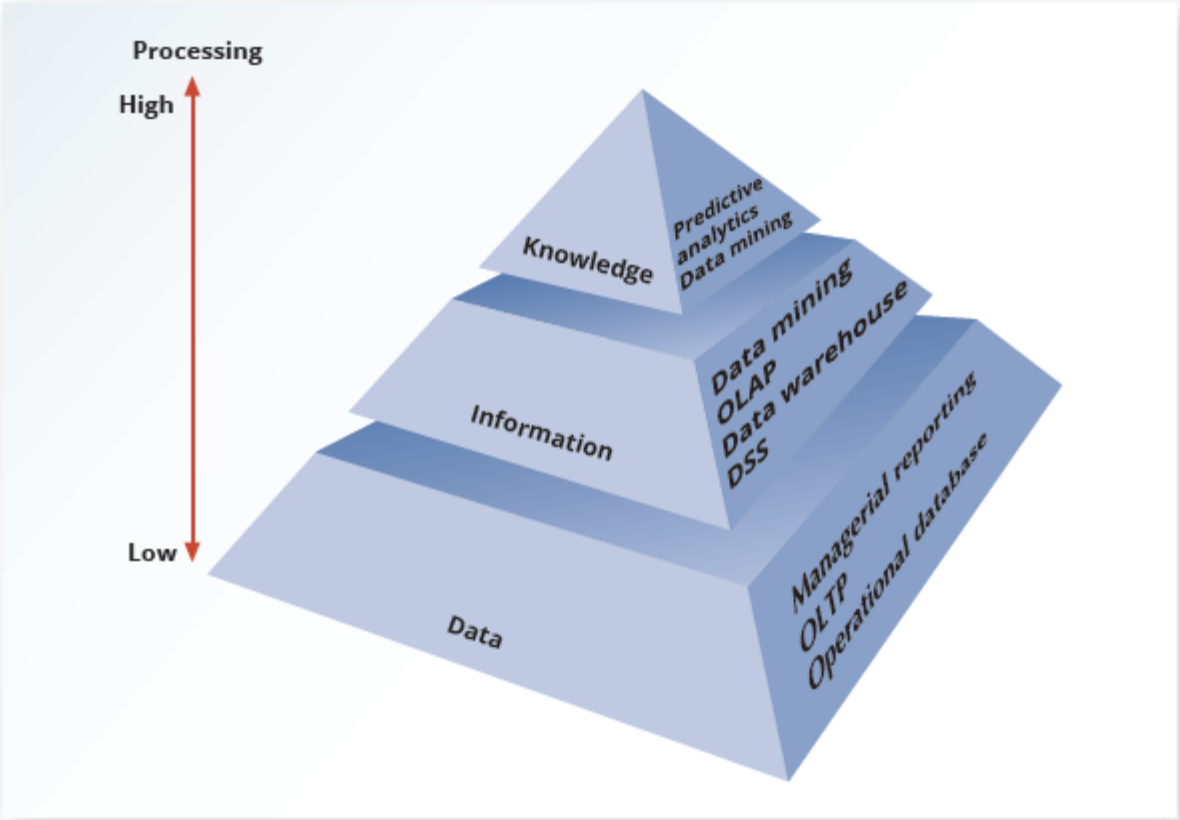


Figure 5-16: Knowledge extraction from Data (Coronel C., Morris S. 2022)

Current-generation data-mining tools contain many design and application variations to fit specific business requirements. Depending on the problem domain, data-mining tools focus on market niches such as banking, insurance, marketing, retailing, finance, and health care. Within a given niche, data-mining tools can use certain algorithms that are implemented in different ways and applied over different data. Despite the lack of precise standards, data mining consists of four general phases:

- Data preparation
- Data analysis and classification
- Knowledge acquisition
- Prognosis

In the *data preparation phase*, the main data sets to be used by the data-mining operation are identified and cleansed of any data impurities. Depending on the problem domain, the data sets source could be operational data, data warehouse data, or even raw data, such as text files, emails, social media comments, etc. Because the data in the data warehouse is already integrated and filtered, the data warehouse usually is the target set for data-mining operations. The *data analysis and classification phase* studies the data to identify common data characteristics or patterns. During this phase, the data-mining tool applies specific algorithms to find:

- Data groupings, classifications, clusters, or sequences
- Data dependencies, links, or relationships
- Data patterns, trends, and deviations

The *knowledge acquisition phase* uses the results of the data analysis and classification phase. During the knowledge acquisition phase, the data-mining tool (with possible intervention by the end user) selects the appropriate modeling or knowledge acquisition algorithms. The most common algorithms used in data mining are based on neural networks, decision trees, rules induction, genetic algorithms, classification and regression trees, memory-based reasoning,

and nearest neighbor. A data-mining tool may use many of these algorithms in any combination to generate a computer model that reflects the behavior of the target data set.

Although many data-mining tools focus on the knowledge–discovery phase, others continue to the *prognosis phase*. In that phase, the data-mining findings are used to predict future behavior and forecast business outcomes. Examples of data-mining findings can be:

- Sixty-five percent of customers who did not use a particular credit card in the last six months are 88 percent likely to cancel that account.
- Eighty-two percent of customers who bought a 42-inch or larger LCD TV are 90 percent likely to buy an entertainment center within the next four weeks.
- If age < 30, income <= 25,000, credit rating < 3, and credit amount > 25,000, then the minimum loan term is 10 years.

The complete set of findings can be represented in a decision tree, a neural network, a forecasting model, or a visual presentation interface that is used to project future events or results. For example, the prognosis phase might project the likely outcome of a new product rollout or a new marketing promotion. Figure 5.17 illustrates the different phases of the data-mining process. Because of the nature of the data-mining process, some findings might fall outside the boundaries of what business managers expect. For example, a data-mining tool might find a close relationship between a customer's favorite brand of soda and the brand of tires on the customer's car. Clearly, that relationship might not be held in high regard among sales managers. (In regression analysis, those relationships are commonly described by the label "idiot correlation.") Fortunately, data mining usually yields more meaningful results. In fact, data mining has proven helpful in finding practical relationships among data that help define customer buying patterns, improve product development and acceptance, reduce healthcare and credit card fraud, analyze stock markets, and so on. Data mining can be run in two modes:

- *Guided*. The end user guides the data-mining tool step by step to explore and explain known patterns or relationships. In this mode, the end user decides what techniques to apply to the data.
- *Automated*. In this mode, the end user sets up the data-mining tool to run automatically and uncover hidden patterns, trends, and relationships. The data-mining tool applies multiple techniques to find significant relationships.

As you learned in this section, data-mining methodologies focus on discovering and extracting information that describes and explains the data. For example, an explanatory model could create a customer profile that describes a given customer group. However, data mining can also be used as the basis to create advanced predictive data models. For example, a predictive model could be used to predict future customer behavior, such as a customer response to a target marketing campaign.

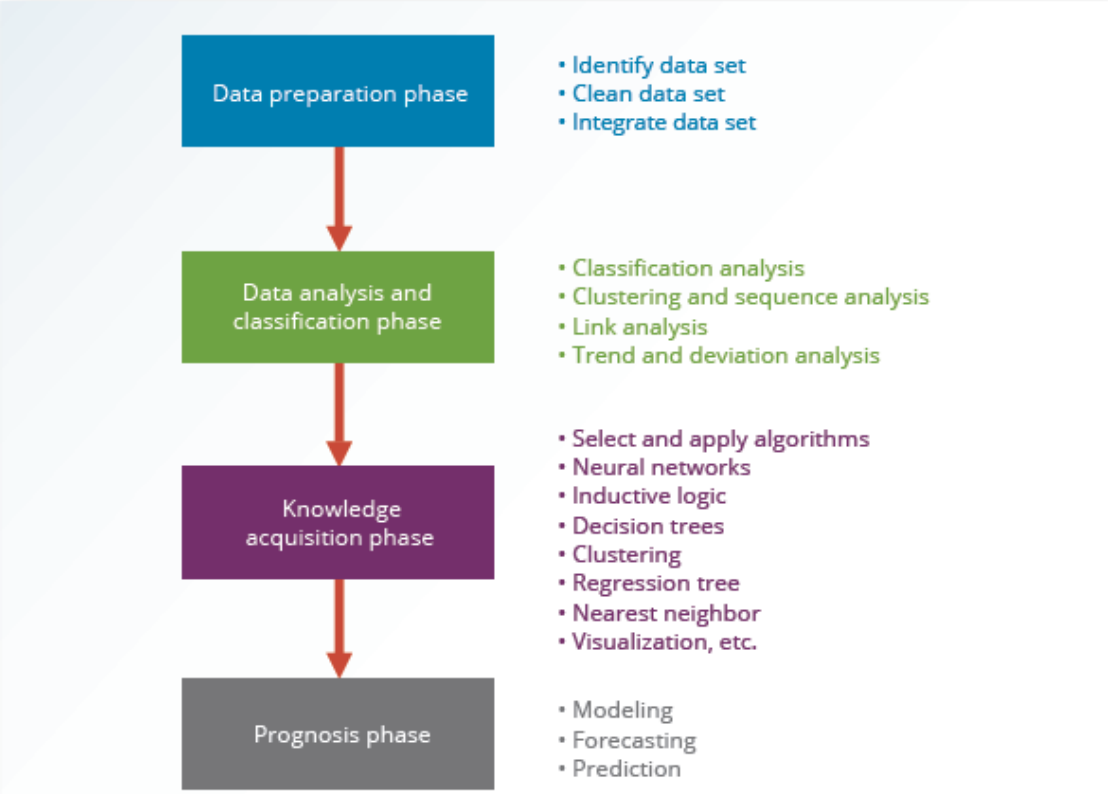


Figure 5-17: Data Mining Phases (Coronel C., Morris S. 2022)

Predictive Analytics

Although the term *predictive analytics* is used by many BI vendors to indicate many different levels of functionality, the promise of predictive analytics is very attractive for businesses looking for ways to improve their bottom line. Therefore, predictive analytics is receiving a lot of marketing buzz; vendors and businesses are dedicating extensive resources to this BI area. Predictive analytics refers to the use of advanced mathematical, statistical, and modeling tools to predict future business outcomes with high degrees of accuracy.

What is the difference between data mining and predictive analytics? As you learned earlier, data mining also has predictive capabilities. In fact, data mining and predictive analytics use similar and overlapping sets of tools, but with a slightly different focus. Data mining focuses on answering the "how" and "what" of *past* data, while predictive analytics focuses on creating actionable models to predict *future* behaviors and events. In some ways, you can think of predictive analytics as the next logical step after data mining; once you understand your data, you can use the data to predict future behaviors. In fact, most BI vendors are dropping the term *data mining* and replacing it with the more alluring term *predictive analytics*.

The origins of predictive analytics can be traced back to the banking and credit card industries. The need to profile customers and predict customer buying patterns in these industries was a critical driving force for the evolution of many modeling methodologies used in BI data analytics today. For example, based on your demographic information and purchasing history, a credit card company can use data-mining models to determine what credit limit to offer, what offers

you are more likely to accept, and when to send those offers. Predictive analytics received a big stimulus with the advent of social media. Companies turned to data mining and predictive analytics to harvest the mountains of data stored on social media sites. Google was one of the first companies that offered targeted ads to increase and personalize search experiences. Similar initiatives were used by all types of organizations to increase customer loyalty and drive-up sales. Note the example of the airline and credit card industries and their frequent flyer and affinity card programs. Today, many organizations use predictive analytics to profile customers to get and keep the right ones, which in turn will increase loyalty and sales.

Predictive analytics employs mathematical and statistical algorithms, neural networks, artificial intelligence, machine learning, and other advanced modeling tools to create actionable predictive models based on available data. The algorithms used to build the predictive model are specific to certain types of problems and work with certain types of data. Therefore, it is important that the end user, who typically is trained in statistics and understands business, applies the proper algorithms to the problem in hand. However, thanks to constant technology advances, modern BI tools automatically apply multiple algorithms to find the optimum model.

Most predictive analytics models are used in areas such as healthcare services, customer relationships, customer service, customer retention, fraud detection, targeted marketing, and optimized pricing. Predictive analytics can add value to an organization in many different ways. For example, it can help optimize existing processes, identify hidden problems, and anticipate future problems or opportunities. However, predictive analytics is not the "secret sauce" to fix all business problems. Managers should carefully monitor and evaluate the value of predictive analytics models to determine their return on investment.

Data mining is not free of challenges. Take the ever-increasing scrutiny of the rights of people to limit the collection of personal data. Federal and local governments are creating legislation that regulates the collection and usage of personal data. Take the example of the General Data Protection Regulation (GDPR), established in 2018, to regulate the collection and use of European citizens' personal data. Companies, public and private, must balance data mining benefits with the user's right to privacy. Because data analytics is so prevalent in today's business environment, SQL has been upgraded to support new business analytics functions.

Data Lake

A **data lake** is a mega data repository that stores all company data (structured, semi-structured, and unstructured) in its raw/natural format. For example, the data lake can contain relational data, semi-structured data from Word, Excel, emails, XML, JSON documents, etc., and raw data such as binary files (images, movies, etc.) The data lake can also store social media and sensor data. The term "data lake" was first introduced by James Dixon, Pentaho, in 2011. In its presentation, Dixon envisioned a large data repository as "a new way to capture, organize and analyze data." Companies like Google, Microsoft, Amazon, Snowflake, and others now offer "data lake" products, but there is no common architecture. Most offerings provide a flexible universal storage component and a way to organize and navigate the lake. The idea of the data lake is to store the data in its raw/natural state before it is processed. Contrast this with the data warehouse approach where the data is "processed" before it is stored. The data lake can also serve as the source data for data scientists to "fish" for data relationships and patterns in the raw data before it is processed.

Summary

- Business intelligence (BI) is a term for a comprehensive, cohesive, and integrated set of applications used to capture, collect, integrate, store, and analyze data with the purpose of generating and presenting information to support business decision making.
- Decision support systems (DSSs) refer to an arrangement of computerized tools used to assist managerial decision making within a business. DSSs were the original precursor of current-generation BI systems.
- Operational data is not well suited for decision support. From the end user's point of view, decision support data differs from operational data in three main areas: time span, granularity, and dimensionality.
- The data warehouse is an integrated, subject-oriented, time-variant, non-volatile collection of data that provides support for decision making. The data warehouse is usually a read-only database optimized for data analysis and query processing. A data mart is a small, single-subject data warehouse subset that provides decision support to a small group of people.
- The star schema is a data-modeling technique used to map multidimensional decision support data into a relational database for advanced data analysis. The basic star schema has four components: facts, dimensions, attributes, and attribute hierarchies. Facts are numeric
- measurements or values that represent a specific business aspect or activity. Dimensions are general qualifying categories that provide additional perspectives to facts. Conceptually, the multidimensional data model is best represented by a three-dimensional cube. Attributes can
- be ordered in well-defined hierarchies, which provide a top-down organization that is used for two main purposes: to permit aggregation and provide drill-down and roll-up data analysis.
- Online analytical processing (OLAP) refers to an advanced data analysis environment that supports decision making, business modeling, and operations research.
- Data analytics is a subset of BI functionality that provides advanced data analysis tools to extract knowledge from business data. Data analytics can be divided into explanatory and predictive analytics. Explanatory analytics focuses on discovering and explaining data characteristics and relationships. Predictive analytics focuses on creating models to predict future outcomes or events based on the existing data.
- Data mining automates the analysis of operational data to find previously unknown data characteristics, relationships, dependencies, and trends. The data-mining process has four phases: data preparation, data analysis and classification, knowledge acquisition, and prognosis.
- A data lake is a large, centralized repository of data to store all company data (structured, semi-structured, and unstructured) in its raw/natural format before it is processed.

Review Questions

1. What is business intelligence? Give some recent examples of BI usage, using the Internet for assistance. What BI benefits have companies found?
2. Describe the BI framework. Illustrate the evolution of BI.
3. What are decision support systems, and what role do they play in the business environment?
4. Explain how the main components of the BI architecture interact to form a system. Describe the evolution of BI information dissemination formats.
5. What are the most relevant differences between operational data and decision support data?
6. What is a data warehouse, and what are its main characteristics? How does it differ from a data mart?
7. Give three examples of likely problems when operational data is integrated into the data warehouse. Use the following scenario to answer Questions 8–13. While working as a database analyst for a national sales organization, you are asked to be part of its data warehouse project team.
8. Prepare a high-level summary of the main requirements for evaluating DBMS products for data warehousing.
9. Your data warehousing project group is debating whether to create a prototype of a data warehouse before its implementation. The project group members are especially concerned about the need to acquire some data warehousing skills before implementing the enterprise-wide data warehouse. What would you recommend? Explain your recommendations.
10. Suppose that you are selling the data warehouse idea to your users. How would you define multidimensional data analysis for them? How would you explain its advantages to them?
11. The data warehousing project group has invited you to provide an OLAP overview. The group's members are particularly concerned about the OLAP client/server architecture requirements and how OLAP will fit the existing environment. Your job is to explain the main OLAP client/server components and architectures.
12. The project group is ready to make a final decision, choosing between ROLAP and MOLAP. What should be the basis for this decision? Why?
13. The data warehouse project is in the design phase. Explain to your fellow designers how you would use a star schema in the design.
14. Briefly discuss the OLAP architectural styles with and without data marts.
15. What is OLAP, and what are its main characteristics?
16. Explain ROLAP and list the reasons you would recommend its use in the relational database environment.
17. Explain the use of facts, dimensions, and attributes in the star schema.
18. Explain multidimensional cubes and describe how the slice-and-dice technique fits into this model.
19. In the star schema context, what are attribute hierarchies and aggregation levels, and what is their purpose?
20. Discuss the most common performance improvement techniques used in star schemas.
21. What is data analytics? Briefly define explanatory and predictive analytics.
22. Describe and contrast the focus of data mining and predictive analytics. Give some examples.

Chapter 6: Big Data and NoSQL

Learning Outcomes

After this chapter you will be able to:

1. Explain the role of Big Data in modern business
2. Describe how the primary characteristics of Big Data go beyond the traditional "3 Vs"
3. Explain how the core components of the Hadoop framework operate
4. Identify the major components of the Hadoop ecosystem
5. Summarize the four major approaches of the NoSQL data model
6. Describe how NoSQL data models differ from the relational model

Big Data

Big Data refers to a set of data that displays the characteristics of volume, velocity, and variety (the 3 Vs) to an extent that makes the data unsuitable for management by a relational database management system. These characteristics can be defined as follows:

- **Volume**—the quantity of data to be stored
- **Velocity**—the speed at which data is entering the system
- **Variety**—the variations in the structure of the data to be stored

Notice the lack of specific values associated with these characteristics. This lack of specificity is what leads to the ambiguity in defining Big Data. What was Big Data five years ago might not be considered Big Data now. Similarly, something considered Big Data now might not be considered Big Data five years from now. The key is that the characteristics are present to an extent that the current relational database technology struggles with managing the data. Further adding to the problem of defining Big Data is that pundits disagree about which of the 3 Vs must be present for a data set to be considered Big Data. Originally, Big Data was conceived as shown in Figure 6.1 as a combination of the 3 Vs. Web data, a combination of text, graphics, video, and audio sources combined into complex structures, created new challenges for data management that involve all three characteristics. After the dot-com bubble burst in the 1990s, many start-up, web-based companies failed, but the companies that survived experienced significant growth as web commerce consolidated into a smaller set of businesses. As a result, companies like Google and Amazon experienced significant growth and were among the first to feel the pressure of managing Big Data. The success of social media giant Facebook quickly followed, and these companies became pioneers in creating technologies to address Big Data problems. Google created the BigTable data store, Amazon created Dynamo, and Facebook created Cassandra, technologies that are discussed later in this chapter, to deal with the growing need to store and manage large sets of data that had the characteristics of the 3 Vs. Although social media and web data have been at the forefront of perceptions of Big Data issues, other organizations have Big Data issues, too.

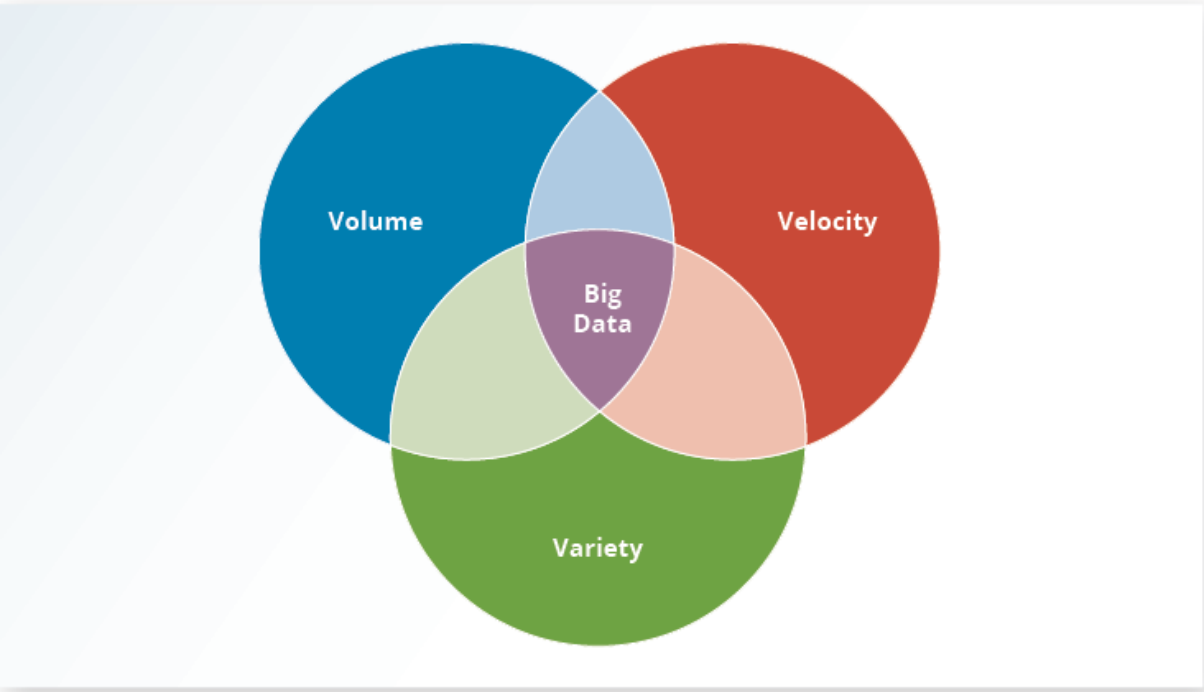


Figure 6-1: Original View of Big Data (Coronel C., Morris S. 2022)

More recently, changes in technology have increased the opportunities for businesses to generate and track data so that Big Data has been redefined as involving any, but not necessarily all, of the 3 Vs, as shown in Figure 6.2. Advances in technology have led to a vast array of user-generated data and machine-generated data that can spur growth in specific areas. For example, Disney World has introduced "Magic Bands" for park visitors to wear on their wrists. Each visitor's Magic Band is connected to much of the data that Disney stores about that person. These bands use radio frequency identification (RFID) and near-field communications (NFC) to act as tickets for rides, hotel room keys, and even credit cards within the park. The bands can be tracked so that Disney systems can track people as they move through the park and record with which Disney characters (who are also tracked) they interact, purchases made, wait time in lines, and more. Visitors can make reservations at restaurants and order meals through a Disney app on their smartphones, and by tracking the Magic Bands, the restaurant staff know when the visitors arrive for their reservation, can track at which table they are seated, and deliver their meals within minutes of the guests being seated. With the many cameras mounted throughout the park, Disney can also capture pictures and short videos of the visitors throughout their stay in the park to produce a personalized movie of their vacation experience, which can then be sold to the visitors as souvenirs. All of this involves the capture of a constant stream of data from each band, processed in real time. Considering the tens of thousands of visitors in Disney World each day, each with a Magic Band, the volume, velocity, and variety of the data are enormous.

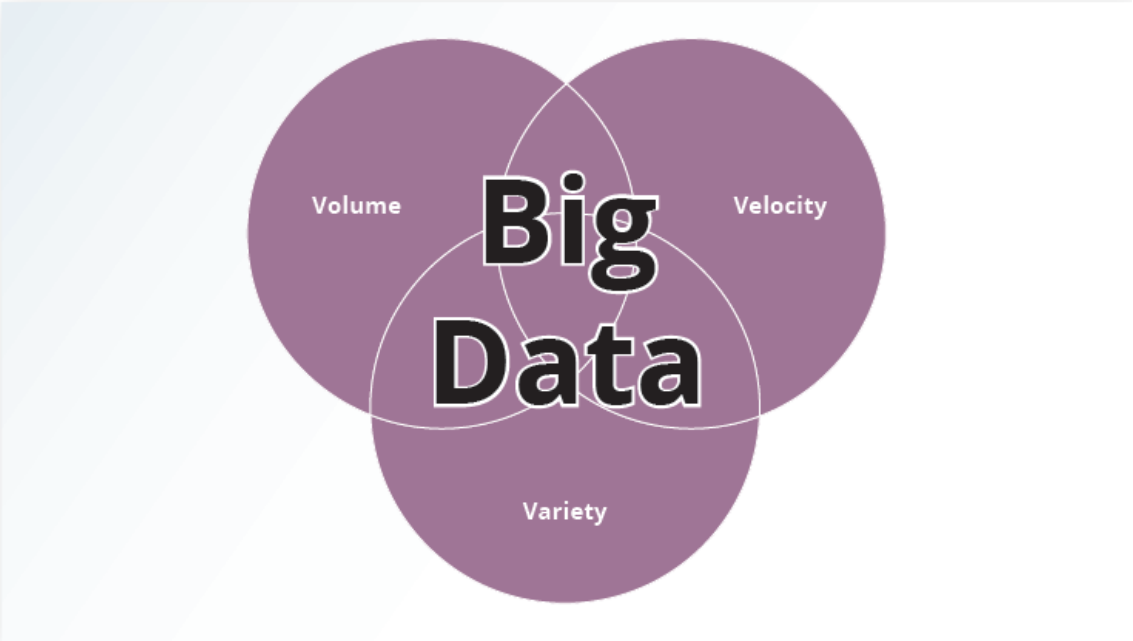


Figure 6-2: Current View of Big Data (Coronel C., Morris S. 2022)

Volume

Volume, the quantity of data to be stored, is a key characteristic of Big Data. The storage capacities associated with Big Data are extremely large. Table 6.1 provides definitions for units of data storage capacity.

Table 6-1: Storage Capacity Units

Term	Capacity	Abbreviation
Bit	0 or 1 value	b
Byte	8 bits	B
Kilobyte	1024* bytes	KB
Megabyte	1024 KB	MB
Gigabyte	1024 MB	GB
Terabyte	1024 GB	TB
Petabyte	1024 TB	PB
Exabyte	1024 PB	EB
Zettabyte	1024 EB	ZB
Yottabyte	1024 ZB	YB

Naturally, as the quantity of data needing to be stored increases, the need for larger storage devices increases as well. When this occurs, systems can either scale up or scale out. **Scaling up** is keeping the same number of systems but migrating each system to a larger system: for example, changing from a server with 16 CPU cores and a 1 TB storage system to a server with 64 CPU cores and a 100 TB storage system. Scaling up involves moving to larger and faster systems. However, there are limits to how large and fast a single system can be. Further, the costs of these high-powered systems increase at a dramatic rate.

On the other hand, **scaling out** means that when the workload exceeds the capacity of a server, the workload is spread out across a number of servers. This is also referred to as *clustering*—creating a cluster of low-cost servers to share a workload. This can help to reduce the overall cost of the computing resources since it is cheaper to buy ten 100 TB storage systems than it is to buy a single 1 PB storage system. Make no mistake, organizations need storage capacities in these extreme sizes. Organizations such as eBay collect clickstream data that easily reaches into the dozens of petabytes. This is in addition to the enterprise data warehouses, which can also be petabytes in size and spread over hundreds of thousands of nodes. To carry out these functions, the DBMS acts as the "brain" of the database system and must maintain control over all of the data within the database. However, because the DBMS must act as a single point of control for all of the data in the database, distributing the database across multiple systems requires a high degree of communication and coordination across the systems. Significant constraints are associated with the ability to distribute the DBMS due to the increased performance costs of communication and coordination as the number of nodes grows. This limits the degree to which a relational database can be scaled out as data volume grows, and it makes RDBMSs ill-suited for clusters.

Velocity

Velocity, another key characteristic of Big Data, refers to the rate at which new data enters the system as well as the rate at which the data must be processed. In many ways, the issues of velocity mirror those of volume. For example, consider a web retailer such as Amazon. In the past, a retail store might capture only the data about the final transaction of a customer making a purchase. Today, a retailer like Amazon captures not only the final transaction but also every

click of the mouse in the searching, browsing, comparing, and purchase process. Instead of capturing one event (the final sale) in a 20-minute shopping experience, it might capture data on 30 events during that 20-minute period—a 30-times increase in the velocity of the data. Other advances in technology, such as RFID, GPS, and NFC, add new layers of data-gathering opportunities that often generate large amounts of data that must be stored in real time. For example, RFID tags can be used to track items for inventory and warehouse management. The tags do not require line-of-sight between the tag and the reader, and the reader can read hundreds of tags simultaneously while the products are still in boxes. This means that instead of a single record for tracking a given quantity of a product being produced, each individual product is tracked, creating an increase of several orders of magnitude in the amount of data being delivered to the system at any one time. In addition to the speed with which data is entering the system, that data must be processed at a very rapid pace for Big Data to be actionable. The velocity of processing can be broken down into two categories.

- Stream processing
- Feedback loop processing

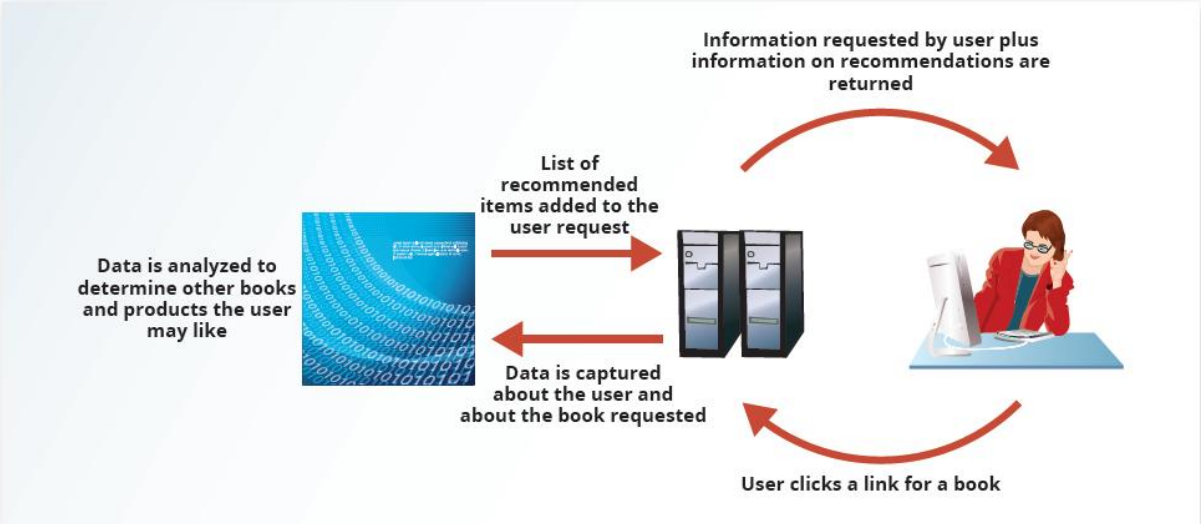


Figure 6-3: Feedback Loop Processing (Coronel C., Morris S. 2022)

Stream processing focuses on input processing, and it requires analysis of the data stream as it enters the system. In some situations, large volumes of data can enter the system at such a rapid pace that it is not feasible to try to store all of the data. The data must be processed and filtered as it enters the system to determine which data to keep and which data to discard. For example, at the CERN Large Hadron Collider, the largest and most powerful particle accelerator in the world, experiments produce about 600 TB per second of raw data. Scientists have created **algorithms** to decide ahead of time which data will be kept. These algorithms are applied in a two-step process to filter the data down to only about 1 GB per second of data that will actually be stored.1

Feedback loop processing refers to the analysis of the data to produce actionable results. While stream processing could be thought of as focused on inputs, feedback loop processing can be thought of as focused on outputs. Capturing the data, processing it into usable information, and then acting on that information is a feedback loop. Figure 6.3 shows a feedback loop for providing recommendations for book purchases. Feedback loop processing to provide immediate results requires analyzing large amounts of data within just a few seconds so that the results of the analysis can become a part of the product delivered to the user in real time. Not all feedback loops are used to include results within immediate data products. Feedback loop processing is

also used to help organizations sift through terabytes and petabytes of data to inform decision makers to help them make faster strategic and tactical decisions, and it is a key component in data analytics.

Variety

In a Big Data context, variety refers to the vast array of formats and structures in which the data may be captured. Data can be considered to be structured, unstructured, or semi-structured. **Structured data** is data that has been organized to fit a predefined data model. **Unstructured data** is data that is not organized to fit into a predefined data model. Semi-structured data combines elements of both—some parts of the data fit a predefined model while other parts do not. Relational databases rely on structured data. A data model is created by the database designer based on the business rules. As data enters the database, the data is decomposed and routed for storage in the corresponding tables and columns as defined in the data model. Although much of the transactional data that organizations use works well in a structured environment, most of the data in the world is semi-structured or unstructured. Unstructured data includes maps, satellite images, emails, texts, tweets, videos, transcripts, and a whole host of other data forms. Over the decades that the relational model has been dominant, relational databases have evolved to address some forms of unstructured data. For example, most large-scale RDBMSs support a binary large object (BLOB) data type that allows the storage of unstructured objects like audio, video, and graphic data as a single, atomic value. One problem with BLOB data is that the semantic value of the data, the meaning that the object conveys, is inaccessible and uninterpretable by data processing. Big Data requires that the data be captured in whatever format it naturally exists, without any attempt to impose a data model or structure to the data. This is one of the key differences between processing data in a relational database and Big Data processing. Relational databases impose a structure on the data when the data is captured and stored. Big Data processing imposes a structure on the data as needed for applications as a part of retrieval and processing. One advantage to providing structure during retrieval and processing is the flexibility of being able to structure the data in different ways for different applications.

Other Characteristics

Characterizing Big Data with the 3 Vs is fairly standard. However, as the industry matures, other characteristics have been proposed as equally important. Keeping with the spirit of the 3 Vs, these characteristics are typically presented as additional Vs, as summarized in Table 6.2. **Variability** refers to the changes in the meaning of the data based on context. While *variety* and *variability* are similar terms, they mean distinctly different things in Big Data. Variety is about differences in structure. Variability is about differences in meaning. Variability is especially relevant in areas such as sentiment analysis that attempt to understand the meanings of words. **Sentiment analysis** is a method of text analysis that attempts to determine if a statement conveys a positive, negative, or neutral attitude about a topic. For example, the statements, “I just bought a new smartphone—I love it!” and “The screen on my new smartphone shattered the first time I dropped it—I love it!” In the first statement the presence of the phrase “I love it” might help an algorithm correctly interpret the statement as expressing a positive attitude. However, the second statement uses sarcasm to express a negative attitude, so the presence of the phrase “I love it” may cause the analysis to interpret the meaning of the phrase incorrectly.

Table 6-2: Additional Vs of Big Data

Characteristic	Description
Variability	Data meaning changes based on context.
Veracity	Data is correct.
Value (Viability)	Data can provide meaningful information.
Visualization	Data can be presented in such a way as to make it understandable.

Veracity refers to the trustworthiness of the data. Can decision makers reasonably rely on the accuracy of the data and the information generated from it? This is especially pertinent given the automation of data capture and some of the analysis. Uncertainty about the data can arise from several causes, such as having to capture only selected portions of the data due to high velocity. Also, in terms of sentiment analysis, customers' opinions and preferences can change over time, so comments at one point in time might not be suitable for action at another point in time.

Increasingly, value is being touted as an important characteristic for Big Data. **Value**, also called *viability*, refers to the degree to which the data can be analyzed to provide meaningful information that can add value to the organization. Just because a set of data *can* be captured does not mean that it *should* be captured. Only data that can form the basis for analysis that has the potential to impact organizational behavior should be included in a company's Big Data efforts. The final characteristic of Big Data is visualization. **Visualization** is the ability to graphically present the data in such a way as to make it understandable. Volumes of data can leave decision makers awash in facts but with little understanding of what the facts mean. Visualization is a way of presenting the facts so that decision makers can comprehend the meaning of the information to gain insights. An argument could be made that these additional Vs are not necessarily characteristics of Big Data; or, perhaps more accurately, they are not characteristics of *only* Big Data. Veracity of data is an issue with even the smallest data store, which is why data management is so important in relational databases. Value of data also applies to traditional, structured data in a relational database. One of the keys to data modeling is that only the data that is of interest to the users should be included in the data model. Data that is not of value should not be recorded in any data store—Big Data or not. The important thing to remember is that these characteristics that play an important part in working with data in the relational model are universal and also apply to Big Data.

Hadoop

Big Data requires a different approach to distributed data storage that is designed for largescale clusters. Although other implementation technologies are possible, Hadoop has become the de facto standard for most Big Data storage and processing. Hadoop is not a database. Hadoop is a Java-based framework for distributing and processing very large data sets across clusters of computers. While the Hadoop framework includes many parts, the two most important components are the Hadoop Distributed File System (HDFS) and MapReduce. HDFS is a low-level distributed file processing system, which means that it can be used directly for data storage. MapReduce is a programming model that supports processing large data sets in a highly parallel, distributed manner. While it is possible to use HDFS and MapReduce separately, the two technologies complement each other so that they work better together as a Hadoop system.

Hadoop was engineered specifically to distribute and process enormous amounts of data across vast clusters of servers.

HDFS

The **Hadoop Distributed File System (HDFS)** approach to distributing data is based on several key assumptions:

• *High volume*. The volume of data in Big Data applications is expected to be in terabytes, petabytes, or larger. Hadoop assumes that files in the HDFS will be extremely large. Data in the HDFS is organized into physical blocks, just as in other types of file storage. For example, on a typical personal computer, file storage is organized into blocks that are often 512 bytes in size, depending on the hardware and operating system involved. Relational databases often aggregate these into database blocks. By default, Oracle organizes data into 8 KB physical blocks. Hadoop, on the other hand, has a default block size of 64 MB (8,000 times the size of an Oracle block!), and it can be configured to even larger values. As a result, the number of blocks per file is greatly reduced, simplifying the metadata overhead of tracking the blocks in each file.

- *Write-once, read-many*. Using a write-once, read-many model simplifies concurrency issues and improves overall data throughput. Using this model, a file is created, written to the file system, and then closed. Once the file is closed, changes cannot be made to its contents. This improves overall system performance and works well for the types of tasks performed by many Big Data applications. Although existing contents of the file cannot be changed, recent advancements in the HDFS allow for files to have new data appended to the end of the file. This is a key advancement for NoSQL databases because it allows for database logs to be updated.

- *Streaming access*. Unlike transaction processing systems where queries often retrieve small pieces of data from several different tables, Big Data applications typically process entire files. Instead of optimizing the file system to randomly access individual data elements, Hadoop is optimized for batch processing of entire files as a continuous stream of data.

- *Fault tolerance*. Hadoop is designed to be distributed across thousands of low-cost, commodity computers. It is assumed that with thousands of such devices, at any point in time, some will experience hardware errors. Therefore, the HDFS is designed to replicate data across many different devices so that when one device fails, the data is still available from another device. By default, Hadoop uses a replication factor of three, meaning that each block of data is stored on three different devices. Different replication factors can be specified for each file, if desired.

Hadoop uses several types of nodes. A *node* is just a computer that performs one or more types of tasks within the system. Within the HDFS, there are three types of nodes: the client node, the name node, and one or more data nodes, as depicted in Figure 6.4. Data nodes store the actual file data within the HDFS. Recall that files in HDFS are broken into blocks and are replicated to ensure fault tolerance. As a result, each block is duplicated on more than one data node. Figure 6.4 shows the default replication factor of three, so each block appears on three data nodes. The name node contains the metadata for the file system. There is typically only one name node within a HDFS cluster. The metadata is designed to be small, simple, and easily recoverable. Keeping the metadata small allows the name node to hold all of the metadata in memory to reduce disk accesses and improve system performance. This is important because there is only

one name node so contention for the name node is minimized. The metadata is composed primarily of the name of each file, the block numbers that comprise each file, and the desired replication factor for each file. The client node makes requests to the file system, either to read files or to write new files, as needed to support the user application.

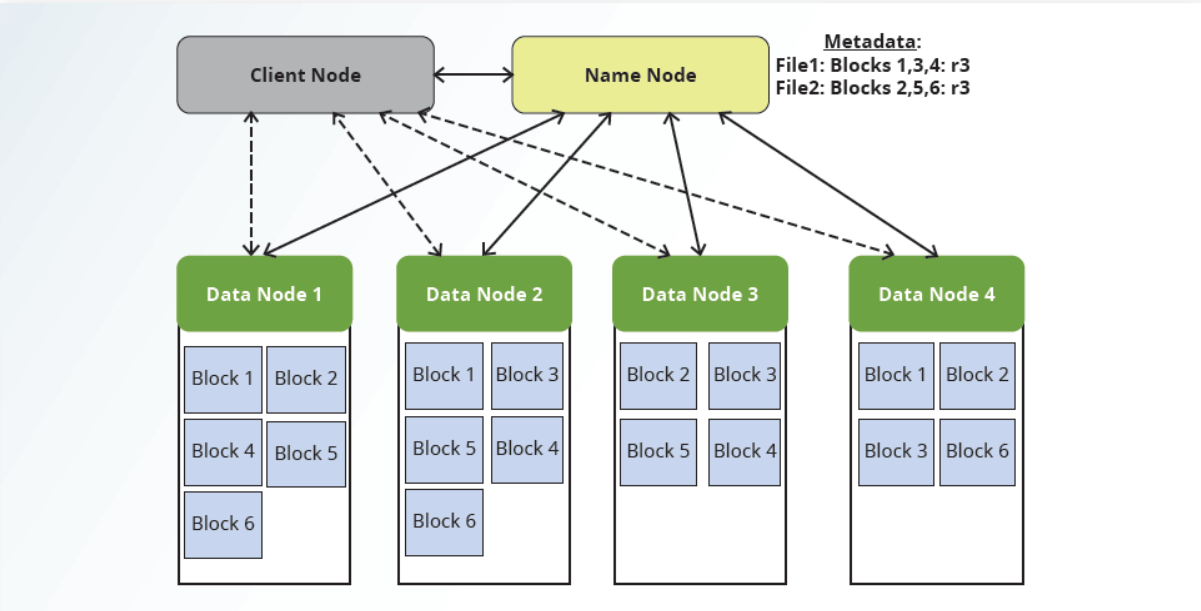


Figure 6-4: Hadoop Distribute File System (HDFS) (Coronel C., Morris S. 2022)

When a client node needs to create a new file, it communicates with the name node. The name node:

- Adds the new file name to the metadata.
- Determines a new block number for the file.
- Determines a list of which data nodes the block will be stored in.
- Passes that information back to the client node.

The client node contacts the first data node specified by the name node and begins writing the file on that data node. At the same time, the client node sends the data node the list of other data nodes that will be replicating the block. As the data is received from the client node, the data node contacts the next data node in the list and begins sending the data to this node for replication. This second data node then contacts the next data node in the list and the process continues with the data being streamed across all of the data nodes that are storing the block. Once the first block is written, the client node can get another block number and list of data nodes from the name node for the next block. When the entire file has been written, the client node informs the name node that the file is closed. It is important to note that at no time was any of the data file actually transmitted to the name node. This helps to reduce the data flow to the name node to avoid congestion that could slow system performance. Similarly, if a client node needs to read a file, it contacts the name node to request the list of blocks associated with that file and the data nodes that hold them. Given that each block may appear in many data nodes, for each block, the client attempts to retrieve the block from the data node that is closest to it on the network. Using this information, the client node reads the data directly from each of those nodes.

Periodically, each data node communicates with the name node. The data nodes send block reports and heartbeats. A **block report** is sent every 6 hours and informs the name node of which blocks are on that data node. Heartbeats are sent every 3 seconds. A **heartbeat** is used to let the name node know that the data node is still available. If a data node experiences a fault, due to

hardware failure, power outage, and so on, then the name node will not receive a heartbeat from that data node. As a result, the name node knows not to include that data node in lists to client nodes for reading or writing files. If the lack of a heartbeat from a data node causes a block to have fewer than the desired number of replicas, the name node can have a "live" data node initiate replicating the block on another data node. Taken together, the components of the HDFS produce a powerful, yet highly specialized distributed file system that works well for the specialized processing requirements of Big Data applications. Next, we will consider how MapReduce provides data processing to complement data storage of HDFS.

The Hadoop system uses batch processing. **Batch processing** is when a program runs from beginning to end, either completing the task or halting with an error, without any interaction with the user. Batch processing is often used when the computing task requires an extended period of time or a large portion of the system's processing capacity. Businesses often use batch processing to run year-end financial reports in the evenings when systems are often idle, and universities might use batch processing for student fee payment processing. Batch processing is not bad, but it has limitations. As a result, a number of complementary programs have been developed to improve the integration of Hadoop within the larger IT infrastructure.

Hadoop Ecosystem

Hadoop is widely used by organizations tapping into the potential of analyzing extremely large data sets. Unfortunately, because Hadoop is a very low-level tool requiring considerable effort to create, manage, and use, it presents quite a few obstacles. As a result, a host of related applications have grown up around Hadoop to attempt to make it easier to use and more accessible to users who are not skilled at complex Java programming. Figure 6.5 shows examples of some of these types of applications. Most organizations that use Hadoop also use a set of other related products that interact and complement each other to produce an entire ecosystem of applications and tools. Like any ecosystem, the interconnected pieces are constantly evolving and their relationships are changing, so it is a rather fluid situation. The following are some of the more popular components in a Hadoop ecosystem and how they relate to each other.

MapReduce Simplification Applications

Creating MapReduce jobs requires significant programming skills. As the mapper and reducer programs become more complex, the skill requirements increase and the time to produce the programs becomes significant. These skills are beyond the capabilities of most data users. In fact, the number of organizations that create significant MapReduce jobs directly in Java has diminished greatly in recent years, with projections for direct, low-level MapReduce programming to continue to decline. Therefore, applications to simplify the process of creating MapReduce jobs have been developed. Two of the most popular are Hive and Pig. *Hive* is a data warehousing system that sits on top of HDFS. It is not a relational database, but it supports its own SQL-like language, called HiveQL, that mimics SQL commands to run ad hoc queries. HiveQL commands are processed by the Hive query engine into sets of MapReduce jobs. As a result, the underlying processing tends to be batch-oriented, producing jobs that are very scalable over extremely large sets of data. However, the batch nature of the jobs makes Hive a poor choice for jobs that only require a small subset of data to be returned very quickly. *Pig* is a tool for compiling a high-level scripting language, named Pig Latin, into MapReduce jobs for executing in Hadoop. In concept it is similar to Hive in that it provides a means of producing MapReduce jobs without the burden of low-level Java programming. The primary difference is that Pig Latin is a scripting language, which means it is procedural, while HiveQL, like SQL, is declarative. Declarative languages allow the user to specify what they want, not how to get it.

This is very useful for query processing. Procedural languages require the user to specify how the data is to be manipulated. This is very useful for performing data transformations. As a result, Pig is often used for producing data pipeline tasks that transform data in a series of steps. This is often seen in ETL (extraction, transformation, and loading) processes.

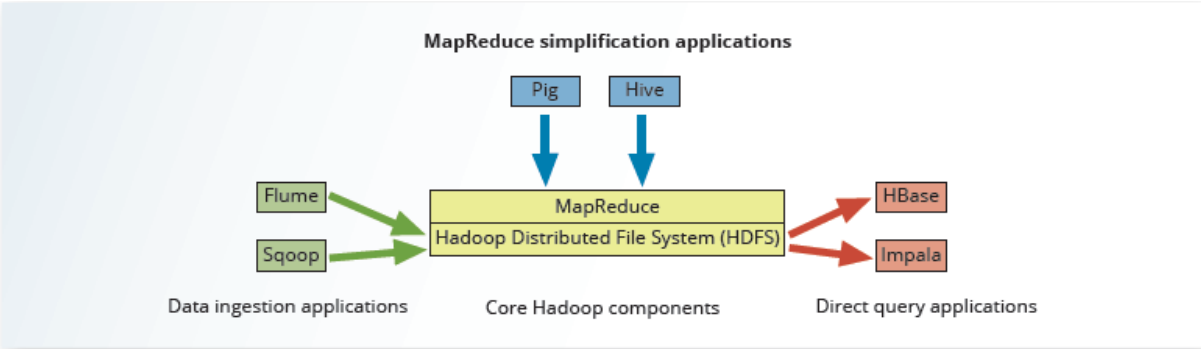


Figure 6-5: Hadoop Ecosystem Sample (Coronel C., Morris S. 2022)

Data Ingestion Applications

One challenge faced by organizations that are taking advantage of Hadoop's massive data storage and data processing capabilities is the issue of actually getting data from their existing systems into the Hadoop cluster. To simplify this task, applications have been developed to "ingest" or gather this data into Hadoop.

Flume is a component for ingesting data into Hadoop. It is designed primarily for harvesting large sets of data from server log files, like clickstream data from web server logs. It can be configured to import the data on a regular schedule or based on specified events. In addition to simply bringing the data into Hadoop, Flume contains a simple query-processing component so the possibility exists of performing some transformations on the data as it is being harvested. Typically, Flume would move the data into the HDFS, but it can also be configured to input the data directly into another component of the Hadoop ecosystem named HBase.

Sqoop is a more recent addition to the Hadoop ecosystem. It is a tool for converting data back and forth between a relational database and the HDFS. The name Sqoop (pronounced, "scoop," as in a scoop of ice cream) is an amalgam of "SQL-to-Hadoop." In concept, Sqoop is similar to Flume in that it provides a way of bringing data into the HDFS. However, while Flume works primarily with log files, Sqoop works with relational databases such as Oracle, MySQL, and SQL Server. Further, while Flume operates in one direction only, Sqoop can transfer data in both directions—into and out of HDFS. When transferring data from a relational database into HDFS, the data is imported one table at a time with the process reading the table row by row. This is done in a highly parallelized manner using MapReduce, so the contents of the table will usually be distributed into several files with the rows stored in a delimited format. Once the data has been imported into HDFS, it can be processed by MapReduce jobs or using Hive. The resulting data can then be exported from HDFS back to the relational database, most often a traditional data warehouse.

Direct Query Applications

Direct query applications attempt to provide faster query access than is possible through MapReduce. These applications interact with HDFS directly, instead of going through the MapReduce processing layer.

HBase is a column-oriented NoSQL database designed to sit on top of the HDFS. One of HBase's primary characteristics is that it is highly distributed and designed to scale out easily. It does not support SQL or SQL-like languages, relying instead on lower-level languages such as Java for interaction. The system does not rely on MapReduce jobs, so it avoids the delays caused by batch processing, making it more suitable for fast processing involving smaller subsets of the data. HBase is very good at quickly processing sparse data sets. HBase is one of the more popular components of the Hadoop ecosystem and is used by Facebook for its messaging system. Column-oriented databases will be discussed in more detail in the next section. *Impala* was the first SQL on Hadoop application. It was produced by Cloudera as a query engine that supports SQL queries that pull data directly from HDFS. Prior to Impala, if an organization needed to make data from Hadoop available to analysts through an SQL interface, data would be extracted from HDFS and imported into a relational database. With Impala, analysts can write SQL queries directly against the data while it is still in HDFS. Impala makes heavy use of in-memory caching on data nodes. It is generally considered an appropriate tool for processing large amounts of data into a relatively small result set.

Hadoop Pushback

The Hadoop Ecosystem can create a powerful, highly customized solution to dealing with Big Data. However, it is not without its critics. The modular approach to the ecosystem creates a significant challenge for implementation. For example, if MS Word were developed using this type of modular approach, you would require one program to create and edit a document, another program to print the document, another program for spell checking, another program for layout, another program for adding graphics or images, and so on. You could customize your word processor to exactly the capabilities that you want, and you can customize exactly how your documents are processed. On the other hand, it would require you to install and successfully integrate all of these programs, each developed independently by different programmers with only a degree of coordination.

To be certain, many organizations benefit from having a customized Hadoop ecosystem that is tailored to their specific needs in a manner that no other solution can duplicate. However, the learning curve can be steep. Companies such as IBM and Cloudera offer out-of-the-box Hadoop ecosystems called data platforms. While these can significantly reduce the effort required to install and integrate a number of ecosystem components, an array of many dozens of configuration settings may still need to be adjusted to tailor the platform to your organization's specific needs, and the entire ecosystem must still be integrated into your organization's workflow. The perceived complications of Hadoop, among other reasons discussed below, have helped to propel interest in alternative solutions, such as NoSQL databases.

NoSQL

NoSQL is the name given to a broad array of nonrelational database technologies that have developed to address the challenges represented by Big Data. The name does not describe what the NoSQL technologies are, but rather what they are not. In fact, the name also does a poor job of explaining what the technologies are not! The term *NoSQL* was never meant to imply that products in this category should never include support for SQL. In fact, many such products support query languages that mimic SQL in important ways. Although no one has yet produced a

NoSQL system that implements standard SQL, given the large base of SQL users, the appeal of creating such a product is obvious. More recently, some industry observers have tried to interject that NoSQL could stand for "Not Only SQL." In fact, if the requirement to be considered a NoSQL product were simply that languages beyond SQL are supported, then all of the traditional RDBMS products such as Oracle, SQL Server, MySQL, and MS Access would qualify. Regardless, you are better off focusing on understanding the array of technologies to which the term refers than worrying about the name itself.

There are literally hundreds of products that can be considered as being under the broadly defined term NoSQL. Most of these fit roughly into one of four categories: key-value data stores, document databases, column-oriented databases, and graph databases. Table 6.3 shows some popular NoSQL databases of each type. Although not all NoSQL databases have been produced as open-source software, most have been. As a result, NoSQL databases are generally perceived as a part of the open-source movement. Accordingly, they also tend to be associated with the Linux operating system. It makes sense from a cost standpoint that, if an organization is going to create a cluster containing tens of thousands of nodes, the organization does not want to purchase licenses for Windows or Mac OS for all of those nodes. The preference is to use a platform, like Linux, that is freely available and highly customizable. Therefore, most of the NoSQL products run only in a Linux or Unix environment. The following sections discuss each of the major NoSQL approaches.

Table 6-3: NoSQL Databases

Table 14.3 NoSQL Databases

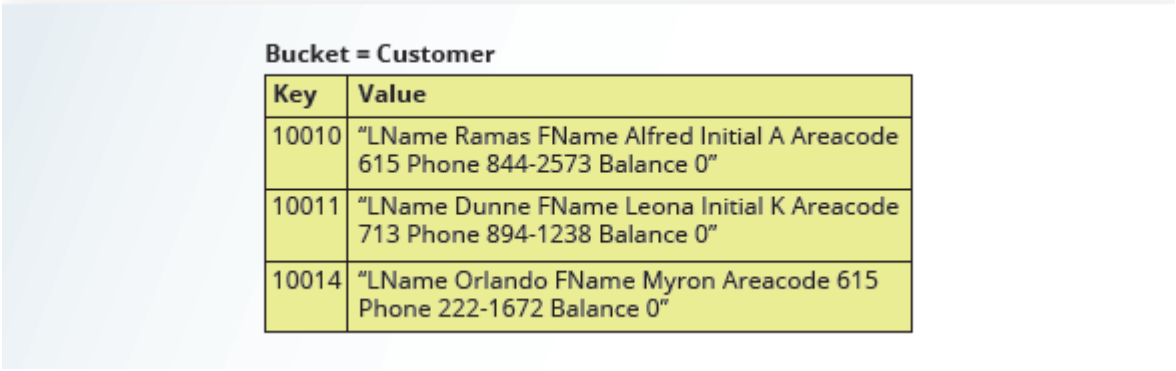
NoSQL Category	Example Databases	Developer
Key-value database	Dynamo Riak Redis Voldemort	Amazon Basho Redis Labs LinkedIn
Document databases	MongoDB CouchDB OrientDB RavenDB	MongoDB, Inc. Apache OrientDB Ltd. Hibernating Rhinos
Column-oriented databases	HBase Cassandra Hypertable	Apache Apache (originally Facebook) Hypertable, Inc.
Graph databases	Neo4j ArangoDB GraphBase	Neo4j ArangoDB, LLC FactNexus

Key-Value Databases

Key-value (KV) databases are conceptually the simplest of the NoSQL data models. A KV database is a NoSQL database that stores data as a collection of key-value pairs. The key acts as an identifier for the value. The value can be anything such as text, an XML document, or an image. The database does not attempt to understand the contents of the value component or its meaning—the database simply stores whatever value is provided for the key. It is the job of the

applications that use the data to understand the meaning of the data in the value component. There are no foreign keys; in fact, relationships cannot be tracked among keys at all. This simplifies the work that the DBMS must perform, making KV databases extremely fast and scalable for basic processing.

Key-value pairs are typically organized into "buckets." A **bucket** can roughly be thought of as the KV database equivalent of a table. A bucket is a logical grouping of keys. Key values must be unique within a bucket, but they can be duplicated across buckets. All data operations are based on the bucket plus the key. In other words, it is not possible to query the data based on anything in the value component of the key-value pair. All queries are performed by specifying the bucket and key. Operations on KV databases are rather simple—only *get*, *store*, and *delete* operations are used. *Get* or *fetch* is used to retrieve the value component of the pair. *Store* is used to place a value in a key. If the bucket + key combination does not exist, then it is added as a new key-value pair. If the bucket + key combination does exist, then the existing value component is replaced with the new value. *Delete* is used to remove a key-value pair. Figure 6.6 shows a customer bucket with three key-value pairs. Since the KV model does not allow queries based on data in the value component, it is not possible to query for a key-value pair based on customer last name, for example. In fact, the KV DBMS does not even know that there is such a thing as a customer last name because it does not understand the content of the value component. An application could issue a *get* command to have the KV DBMS return the key-value pair for bucket customer and key 10011, but it would be up to the application to know how to parse the value component to find the customer's last name, first name, and other characteristics. (One important note about Figure 6.6: Be aware that although key-value pairs appear in tabular form in the figure, the tabular format is just a convenience to help visually distinguish the components. Actual key-value pairs are not stored in a table-like structure.)



Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"

Figure 6-6: Key-Value Database Storage (Coronel C., Morris S. 2022)

Document Databases

Document databases are conceptually similar to key-value databases, and they can almost be considered a subtype of KV databases. A document database is a NoSQL database that stores data in tagged documents in key-value pairs. Unlike a KV database where the value component can contain any type of data, a document database always stores a document in the value component. The document can be in any encoded format, such as XML, **JSON (JavaScript Object Notation)**, or **BSON (Binary JSON)**. Another important difference is that while KV databases do not attempt to understand the content of the value component, document databases do. Tags are named portions of a document. For example, a document may have tags to identify which text in the document represents the title, author, and body of the document. Within the body of the document, there may be additional tags to indicate chapters and sections.

Despite the use of tags in documents, document databases are considered schema-less, that is, they do not impose a predefined structure on the data that is stored. For a document database, being schema-less means that although all documents have tags, not all documents are required to have the same tags, so each document can have its own structure. The tags in a document database are extremely important because they are the basis for most of the additional capabilities that document databases have over KV databases. Tags inside the document are accessible to the DBMS, which makes sophisticated querying possible.

Just as KV databases group key-value pairs into logical groups called buckets, document databases group documents into logical groups called **collections**. While a document may be retrieved by specifying the collection and key, it is also possible to query based on the contents of tags. For example, Figure 6.7 represents the same data from Figure 6.6, but in a tagged format for a document database. Because the DBMS is aware of the tags within the documents, it is possible to write queries that retrieve all of the documents where the Balance tag has the value 0. Document databases even support some aggregate functions such as summing or averaging balances in queries. You learn some basic operations in the MongoDB document database later in this chapter, and Appendix P, Working with MongoDB, includes a hands-on tutorial. Document databases tend to operate on an implied assumption that a document is relatively self-contained, not a fragment of the data about a given topic. Relational databases decompose complex data in the business environment into a set of related tables. For example, data about orders may be decomposed into customer, invoice, line, and product tables. A document database would expect all of the data related to an order to be in a single order document. Therefore, each order document in an Orders collection would contain data on the customer, the order itself, and the products purchased in that order all as a single self-contained document. Document databases do not store relationships as perceived in the relational model and generally have no support for join operations.

Collection = Customer

Key	Document
10010	{LName: "Ramas", FName: "Alfred", Initial: "A", Areacode: "615", Phone: "844-2573", Balance: "0"}
10011	{LName: "Dunne", FName: "Leona", Initial: "K", Areacode: "713", Phone: "894-1238", Balance: "0"}
10014	{LName: "Orlando", FName: "Myron", Areacode: "615", Phone: "222-1672", Balance: "0"}

Figure 6-7: Document Database Tagged Format (Coronel C., Morris S. 2022)

Column-Oriented Databases

The term "*column-oriented database*" can refer to two different sets of technologies that are often confused with each other. In one sense, column-oriented database or columnar database can refer to traditional, relational database technologies that use **column-centric storage** instead of **row-centric storage**. Relational databases present data in logical tables; however, the data is actually stored in data blocks containing rows of data. All of the data for a given row is stored together in sequence with many rows in the same data block. If a table has many rows of data, the rows will be spread across many data blocks. Figure 6.8 illustrates a relational table with 10 rows of data that is physically stored across five data blocks. Row-centric storage

minimizes the number of disk reads necessary to retrieve a row of data. Retrieving one row of data requires accessing just one data block, as shown in Figure 6.8 Remember, in transactional systems, normalization is used to decompose complex data into related tables to reduce redundancy and to improve the speed of rapid manipulation of small sets of data. These manipulations tend to be row-oriented, so row-oriented storage works very well. However, in queries that retrieve a small set of columns across a large set of rows, a large number of disk accesses are required. For example, a query that wants to retrieve only the city and state of every customer will have to access every data block that contains a customer row to retrieve that data. In Figure 6.8, that would mean accessing five data blocks to get the city and state of every customer.

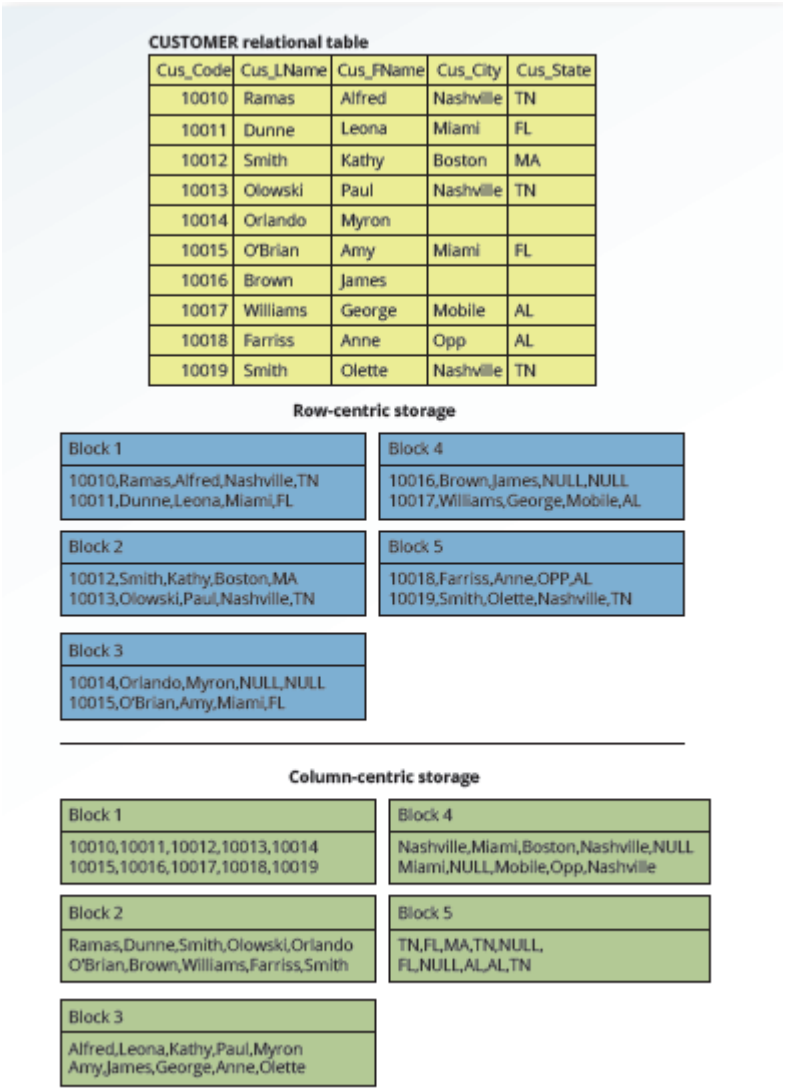


Figure 6-8: Row-Centric vs Column-Centric Storage (Coronel C., Morris S. 2022)

A column-oriented or columnar database stores the data in blocks by column instead of by row. A single customer's data will be spread across several blocks, but all of the data from a single column will be in just a few blocks. In Figure 6.8, all of the city data for customers will be stored together, just as all of the state data will be stored together. In that case, retrieving the city and state for every customer might require accessing only two data blocks. This type of column-centric storage works very well for databases that are primarily used to run queries over few columns but many rows, as is done in many reporting systems and data warehouses.

Though Figure 6.8 shows only a few rows and data blocks, it is easy to imagine that the gains would be significant if the table size grew to millions or billions of rows across hundreds of thousands of data blocks. At the same time, column-centric storage would be very inefficient for processing transactions since insert, update, and delete activities would be very disk intensive. It is worth noting that column-centric storage can be achieved within relational database technology, meaning that it still requires structured data and has the advantage of supporting SQL for queries. The other use of the term *column-oriented database*, also called *column family database*, is to describe a type of NoSQL database that takes the concept of column-centric storage beyond the confines of the relational model. As NoSQL databases, these products do not require the data to conform to predefined structures nor do they support SQL for queries. This database model originated with Google's BigTable product. Other column-oriented database products include Hbase, described earlier, Hypertable, and Cassandra. Cassandra began as a project at Facebook, but Facebook released it to the open-source community, which has continued to develop Cassandra into one of the most popular column-oriented databases.

A **column family database** is a NoSQL database that organizes data in key-value pairs with keys mapped to a set of columns in the value component. While column family databases use many of the same terms as relational databases, the terms don't mean quite the same things. Fortunately, the column family databases are conceptually simple and are conceptually close enough to the relational model that your understanding of the relational model can help you understand the column family model. A column is a key-value pair that is similar to a cell of data in a relational database. The key is the name of the column, and the value component is the data that is stored in that column. Therefore, "cus_lname: Ramas" is a column; *cus_lname* is the name of the column, and *Ramas* is the data value in the column. Similarly, "cus_city: Nashville" is another column, with *cus_city* as the column name and *Nashville* as the data value.

As more columns are added, it becomes clear that some columns form natural groups, such as cus_fname, cus_lname, and cus_initial, which would logically group together to form a customer's name. Similarly, cus_street, cus_city, cus_state, and cus_zip would logically group together to form a customer's address. These groupings are used to create super columns. A **super column** is a group of columns that are logically related. In many cases, super columns can be thought of as the composite attribute and the columns that compose the super column as the simple attributes. Just as all simple attributes do not have to belong to a composite attribute, not all columns have to belong to a super column. Although this analogy is helpful in many contexts, it is not perfect. It is possible to group columns into a super column that logically belongs together for application processing reasons but does not conform to the relational idea of a composite attribute. Row keys are created to identify objects in the environment. All of the columns or super columns that describe these objects are grouped together to create a **column family**; therefore, a column family is conceptually similar to a table in the relational model. Although a column family is similar in concept to a relational table, Figure 6.9 shows that it is structurally very different. Notice in Figure 6.9 that each row key in the column family can have different columns.

Column Family Name	CUSTOMERS
Key	Rowkey 1
Columns	City: Nashville Fname: Alfred Lname: Ramas State: TN
Key	Rowkey 2
Columns	Balance: 345.86 Fname: Kathy Lname: Smith
Key	Rowkey 3
Columns	Company: Local Markets, Inc. Lname: Dunne

Figure 6-9: Column Family Database (Coronel C., Morris S. 2022)

Graph Databases

A **graph database** is a NoSQL database based on graph theory to store data about relationship-rich environments. Graph theory is a mathematical and computer science field that models relationships, or *edges*, between objects called nodes. Modeling and storing data about relationships is the focus of graph databases. Graph theory is a well-established field of study going back hundreds of years. As a result, creating a database model based on graph theory immediately provides a rich source for algorithms and applications that have helped graph databases quickly gain sophistication. Because much of the data explosion over the last decade has involved data that is relationship-rich, graph databases have been poised to experience significant interest in the business environment. Interest in graph databases originated in the area of social networks. Social networks include a wide range of applications beyond the typical Facebook, Twitter, and Instagram that immediately come to mind. Dating websites, knowledge management, logistics and routing, master data management, and identity and access management are all areas that rely heavily on tracking complex relationships among objects. Of course, relational databases support relationships too. One of the great advances of the relational model was that relationships are easy to maintain. A relationship between a customer and an agent is as easy to implement in the relational model as adding a foreign key to create a common attribute, and the customer and agent rows are related by having the same value in the common attributes. If the customer changes to a different agent, then simply changing the value in the foreign key will change the relationship between the rows to maintain the integrity of the data. The relational model does all of these things very well. However, what if we want a “like” option so customers can “like” agents on our website? This would require a structural change to

the database to add a new foreign key to support this second relationship. Next, what if the company wants to allow customers on its website to "friend" each other so a customer can see which agents their friends like, or the friends of their friends? In social networking data, there can be dozens of different relationships among individuals that need to be tracked, and often the relationships are tracked many layers deep (e.g., friends, friends of friends, and friends of friends of friends). This results in a situation where the relationships become just as important as the data itself. This is the area where graph databases shine.

The primary components of graph databases are nodes, edges, and properties, as shown in Figure 6.10. A node corresponds to the idea of a relational entity instance. The **node** is a specific instance of something we want to keep data about. Each node (circle) in Figure 6.10 represents a single agent. Properties are like attributes; they are the data that we need to store about the node. All agent nodes might have properties like first name and last name, but all nodes are not required to have the same properties. An **edge** is a relationship between nodes. Edges (shown as arrows in Figure 6.10) can be in one direction, or they can be bidirectional. For example, in Figure 6.10, the *friends* relationships are bidirectional, but the *likes* relationships are not. Note that edges can also have **properties**. In Figure 6.10, the date on which customer Alfred Ramas *liked* agent Alex Alby is recorded in the graph database. A query in a graph database is called a **traversal**. Instead of *querying the database*, the correct terminology would be *traversing the graph*. Graph databases excel at traversals that focus on relationships between nodes, such as shortest path and degree of connectedness.

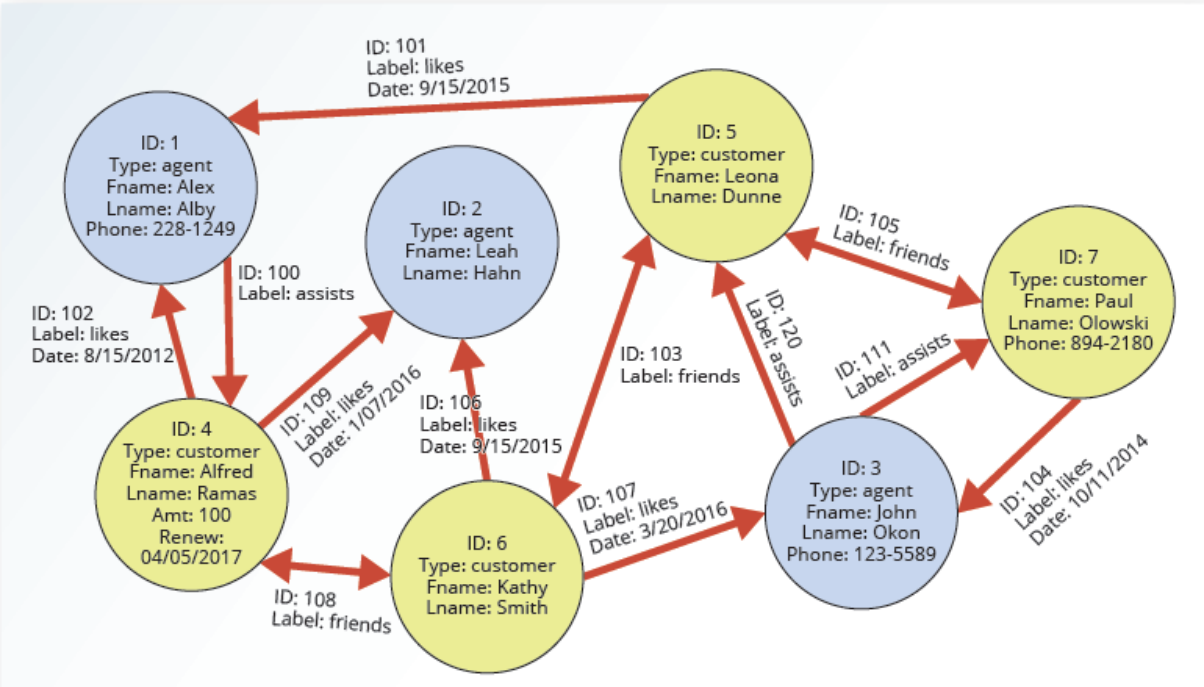


Figure 6-10: Graph Database Visual Representation (Coronel C., Morris S. 2022)

Graph databases share some characteristics with other NoSQL databases in that graph databases do not force data to fit predefined structures, do not support SQL, and are optimized to provide velocity of processing, at least for relationship-intensive data. However, other key characteristics do not apply to graph databases. Graph databases do not scale out very well to clusters due to differences in aggregate awareness.

Aggregate Awareness

Key-value, document, and column family databases are **aggregate aware**. Aggregate aware means that the data is collected or aggregated around a central topic or entity. For example, a blog website might organize data around individual blog posts. All data related to each blog post is aggregated into a single denormalized collection that might include data about the blog post (title, content, and date posted), the poster (user name and screen name), and all comments made on the post (comment content and commenter's user name and screen name). In a normalized, relational database, this same data might call for USER, BLOGPOST, and COMMENT tables. Determining the best central entity for forming aggregates is one of the most important tasks in designing most NoSQL databases and is determined by how the application will use the data. The aggregate aware database models achieve clustering efficiency by making each piece of data relatively independent. That allows a key-value pair to be stored on one node in the cluster without the DBMS needing to associate it with another key-value pair that may be on a different node on the cluster. The greater the number of nodes involved in a data operation, the greater the need for coordination and centralized control of resources. Separating independent pieces of data, often called *shards*, across nodes in the cluster is what allows NoSQL databases to scale out so effectively. Graph databases, like relational databases, are **aggregate ignorant**. Aggregate ignorant models do not organize the data into collections based on a central entity. Data about each topic is stored separately and joins are used to aggregate individual pieces of data as needed. Aggregate ignorant databases, therefore, tend to be more flexible at allowing applications to combine data elements in a greater variety of ways. Graph databases specialize in highly related data, not independent pieces of data. As a result, graph databases tend to perform best in centralized or lightly clustered environments, similar to relational databases.

Summary

- Big Data is characterized by data of such volume, velocity, and/or variety that the relational model struggles to adapt to it. Volume refers to the quantity of data that must be stored. Velocity refers to both the speed at which data is entering storage as well as the speed with which it must be processed. Variety refers to the lack of uniformity in the structure of the data being stored. As a result of Big Data, organizations are having to employ a variety of data storage solutions that include technologies in addition to relational databases, a situation referred to as polyglot persistence.
- Volume, velocity, and variety are collectively referred to as the 3 Vs of Big Data. However, these are not the only characteristics of Big Data to which data administrators must be sensitive. Additional Vs that have been suggested by the data management industry include variability, veracity, value, and visualization. Variability is the variation in the meaning of data that can occur over time. Veracity is the trustworthiness of the data. Value is concerned with whether or not the data is useful. Finally, visualization is the requirement that the data must be able to be presented in a manner that makes it comprehendible to decision makers. Most of these additional Vs are not unique to Big Data. There are also concerns for data in relational databases as well.
- The Hadoop framework has quickly emerged as a standard for the physical storage of Big Data. The primary components of the framework include the Hadoop Distributed File System (HDFS) and MapReduce. HDFS is a coordinated technology for reliably distributing data over a very large cluster of commodity servers. MapReduce is a complementary process for distributing data processing across distributed data. One of the key concepts for MapReduce is to move the computations to the data instead of moving the data to the computations. MapReduce works by combining the functions of *map*, which distributes subtasks to the cluster servers that hold data to be processed, and *reduce*, which combines the map results into a single result set. The Hadoop framework also supports an entire ecosystem of additional tools and technologies, such as Hive, Pig, and Flume, that work together to produce a complex system of Big Data processing.
- NoSQL is a broad term to refer to any of several nonrelational database approaches to data management. Most NoSQL databases fall into one of four categories: key-value databases, document databases, column-oriented databases, or graph databases. Due to the wide variability of products under the NoSQL umbrella, these categories are not necessarily all-encompassing, and many products can fit into multiple categories.
- Key-value databases store data in key-value pairs. In a key-value pair, the value of the key must be known to the DBMS, but the data in the value component can be of any type, and the DBMS makes no attempt to understand the meaning of the data in it. These types of databases are very fast when the data is completely independent, and the application programs can be relied on to understand the meaning of the data.
- Document databases also store data in key-value pairs, but the data in the value component is an encoded document. The document must be encoded using tags, such as in XML or JSON. The DBMS is aware of the tags in the documents, which makes querying on tags possible. Document databases expect documents to be self-contained and relatively independent of each other.
- Column-oriented databases, also called column family databases, organize data into key-value pairs in which the value component is composed of a series of columns, which are themselves key-value pairs. Columns can be grouped into super columns, similar to a composite attribute in the relational model being composed of simple attributes. All objects of a similar type are identified as rows, given a row key, and placed within a

column family. Rows within a column family are not required to have the same structure, that is, they are not required to have the same columns.

- Graph databases are based on graph theory and represent data through nodes, edges, and properties. A node is similar to an instance of an entity in the relational model. Edges are the relationships between nodes. Both nodes and edges can have properties, which are attributes that describe the corresponding node or edge. Graph databases excel at tracking data that is highly interrelated, such as social media data. Due to the many relationships among the nodes, it is difficult to distribute a graph database across a cluster in a highly distributed manner.

Review Questions

1. What is Big Data? Give a brief definition.
2. What are the traditional 3 Vs of Big Data? Briefly define each.
3. Explain why companies like Google and Amazon were among the first to address the Big Data problem.
4. Explain the difference between *scaling up* and *scaling out*.
5. What is stream processing, and why is it sometimes necessary?
6. How is stream processing different from feedback loop processing?
7. Explain why veracity, value, and visualization can also be said to apply to relational databases as well as Big Data.
8. What is polyglot persistence, and why is it considered a new approach?
9. What are the key assumptions made by the Hadoop Distributed File System approach?
10. What is the difference between a name node and a data node in HDFS?
11. Explain the basic steps in MapReduce processing.
12. Briefly explain how HDFS and MapReduce are complementary to each other.
13. What are the four basic categories of NoSQL databases?
14. How are the value components of a key-value database and a document database different?
15. Briefly explain the difference between row-centric and column-centric data storage.
16. What is the difference between a column and a super column in a column family database?
17. Explain why graph databases tend to struggle with scaling out.
18. Explain what it means for a database to be aggregate aware.

Chapter 7: Database Connectivity and Web Technologies

Learning Outcomes

After this chapter you will be able to:

1. Explain the purpose of standard database connectivity interfaces
2. Describe the functionality and features of various database connectivity technologies: ODBC, OLE, ADO.NET, and JDBC
3. Describe how web-to-database middleware is used to integrate databases with the Internet
4. Identify the services provided by web application servers
5. Describe the advantages and disadvantages of using cloud computing for the database-as-a-service model

Database Connectivity

Database connectivity refers to the mechanisms through which application programs connect and communicate with data repositories. Databases store data in persistent storage structures so it can be retrieved later for processing. As you already learned, the database management system (DBMS) functions as an intermediary between the data (stored in the database) and the end-user's applications. Thus far, we know the following database fundamentals:

- DBMSs provide means to interact with the data in their databases. This could be in the form of administrative tools and data manipulation tools. DBMSs also provide a proprietary way for external application programs to connect to the database by the means of an application programming interface.
- Modern DBMSs have the option to store data locally or distributed in multiple locations. Locally stored data resides in the same processing host as the DBMS. A distributed database stores data in multiple geographically distributed nodes with data management capability.
- The database connectivity software we discuss in this chapter supports Structured Query Language (SQL) as the standard data manipulation language. However, depending on the type of database model, some database connectivity interfaces may support other proprietary data manipulation languages.
- Database connectivity software works in a client/server architecture, by which processing tasks are split among multiple software layers. In this model, the multiple layers exchange control messages and data. To better understand database connectivity software, we use client/server concepts in which an application is broken down into interconnected functional layers. In the case of database connectivity software, you could break down its basic functionality into three broad layers:

1. A data layer where the data resides. You could think of this layer as the actual data repository interface. This layer resides closest to the database itself and normally is provided by the DBMS vendor.
2. A middle layer that manages multiple connectivity and data transformation issues. This layer is in charge of dealing with data logic issues, data transformations, ways to "talk" to the database below it, and so on. This would also include translating multiple data manipulation languages to the native language supported by the specific data repository.
3. A top layer that interfaces with the actual external application. This mostly comes in the form of an application programming interface that publishes specific protocols for the external programs to interact with the data.

From the previous discussion, you can understand why the database connectivity software is also known as **database middleware**—because it provides an interface between the application program and the database or data repository. The data repository, also known as the *data source*, represents the data management application, such as Oracle, SQL Server, MySQL, IBM DB2, or NoSQL that will be used to store the data generated by the application program. **Database middleware** is database connectivity software through which application programs connect and communicate with data repositories.

Ideally, a data source or data repository could be located anywhere and hold any type of data. Furthermore, the same database connectivity middleware could support multiple data sources at the same time. For example, the data source could be a relational database, a NoSQL database, a spreadsheet, a MS Access database, or a text data file. This multi-data-source type capability is based on the support of well-established data access standards. The need for standard database connectivity interfaces cannot be overstated. Just as SQL has become the de facto data manipulation language, a standard database connectivity interface is necessary for

enabling applications to connect to data repositories. Although there are many ways to achieve database connectivity, this section covers only the following interfaces:

- Native SQL connectivity (vendor provided)
- Microsoft's Open Database Connectivity (ODBC), Data Access Objects (DAO), and Remote Data Objects (RDO)
- Microsoft's Object Linking and Embedding for Database (OLE-DB)
- Microsoft's ActiveX Data Objects (ADO.NET)
- Oracle's Java Database Connectivity (JDBC)

The data connectivity interfaces illustrated here are dominant players in the market and, more importantly, they enjoy the support of most database vendors. In fact, ODBC, OLE-DB, and ADO.NET form the backbone of Microsoft's **Universal Data Access (UDA)** architecture, a collection of technologies used to access any type of data source and manage the data through a common interface. As you will see, Microsoft's database connectivity interfaces have evolved over time: each interface builds on top of the other, thus providing enhanced functionality, features, flexibility, and support.

Native SQL Connectivity

Most DBMS vendors provide their own methods for connecting to their databases. Native SQL connectivity refers to the connection interface that is provided by the database vendor and is unique to that vendor. The best example of this type of native interface is the Oracle RDBMS. To connect a client application to an Oracle database, you must install and configure Oracle's SQL*Net interface on the client computer. Figure 7.1 shows the configuration of the Oracle SQL*Net interface on the client computer.

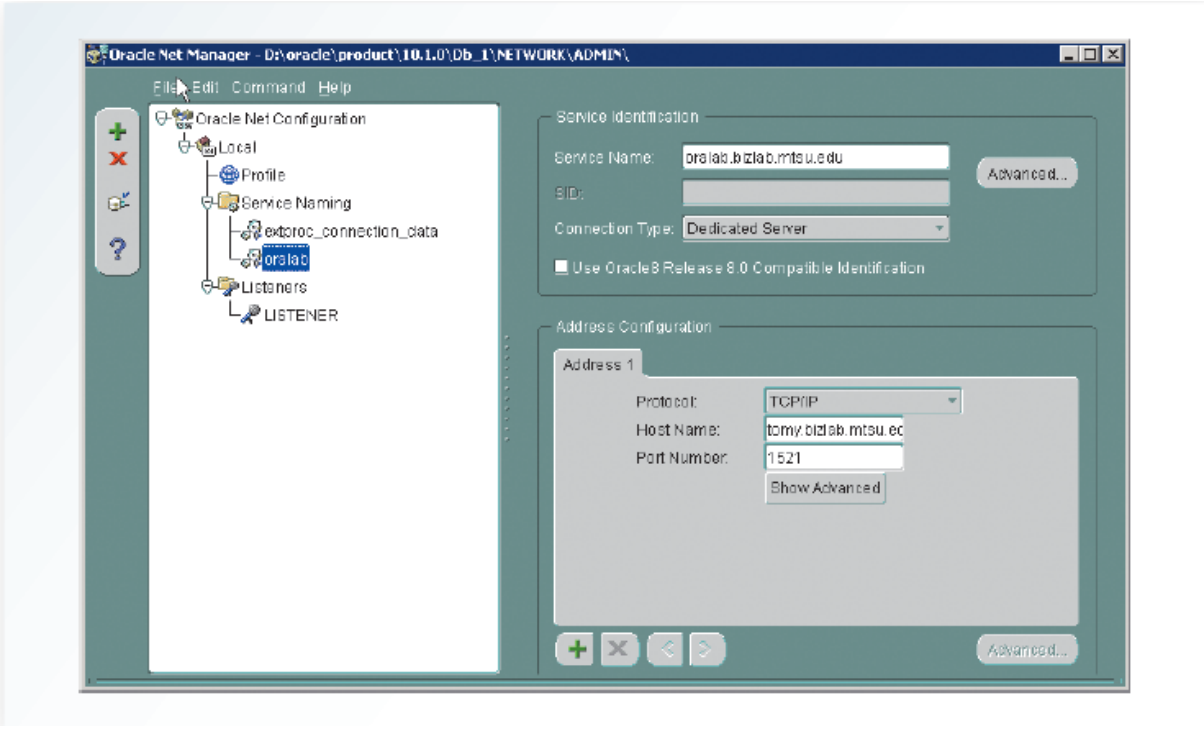


Figure 7-1: Oracle Native Connectivity (Coronel C., Morris S. 2022)

Native database connectivity interfaces are optimized for "their" DBMS, and those interfaces support access to most or all of the database features. However, maintaining multiple native interfaces for different databases can become a burden for the programmer. Therefore, the need for universal database connectivity arises. Usually, the native database connectivity interface provided by the vendor is not the only way to connect to a database; most current DBMS products support other database connectivity standards, the most common being ODBC.

ODBC, DAO, and RDO

Developed in the early 1990s, **Open Database Connectivity (ODBC)** is Microsoft's implementation of a superset of the SQL Access Group **Call Level Interface (CLI)** standard for database access. ODBC is a widely supported database connectivity interface that allows any Windows application to access relational data sources using SQL via a standard **application programming interface (API)**. The Webopedia online dictionary (*www.webopedia.com*) defines an API as "a set of routines, protocols, and tools for building software applications." A good API makes it easy to develop a program by providing all of the building blocks; the programmer puts the blocks together. Most operating environments, such as Windows, provide an API so that programmers can write applications consistent with the operating environment. Although APIs are designed for programmers, they are ultimately good for users because they guarantee that all programs using a common API will have similar interfaces. That makes it easy for users to learn new programs. ODBC was the first widely adopted database middleware standard, and it enjoyed rapid adoption in Windows applications. As programming languages evolved, ODBC did not provide significant functionality beyond the ability to execute SQL to manipulate relational-style data. Therefore, programmers needed a better way to access data. To answer that need, Microsoft developed two other data access interfaces:

- **Data Access Objects (DAO)** is an object-oriented API used to access desktop databases, such as MS Access and FileMaker Pro. DAO provides an optimized interface that exposes programmers to the functionality of the Jet data engine, on which MS Access is based. The DAO interface can also be used to access other relational-style data sources.
- **Remote Data Objects (RDO)** is a higher-level, object-oriented application interface used to access remote database servers. RDO uses the lower-level DAO and ODBC for direct access to databases. RDO is optimized to deal with server-based databases such as MS SQL Server, Oracle, and DB2.

Figure 7.2 illustrates how Windows applications can use ODBC, DAO, and RDO to access local and remote relational data sources.

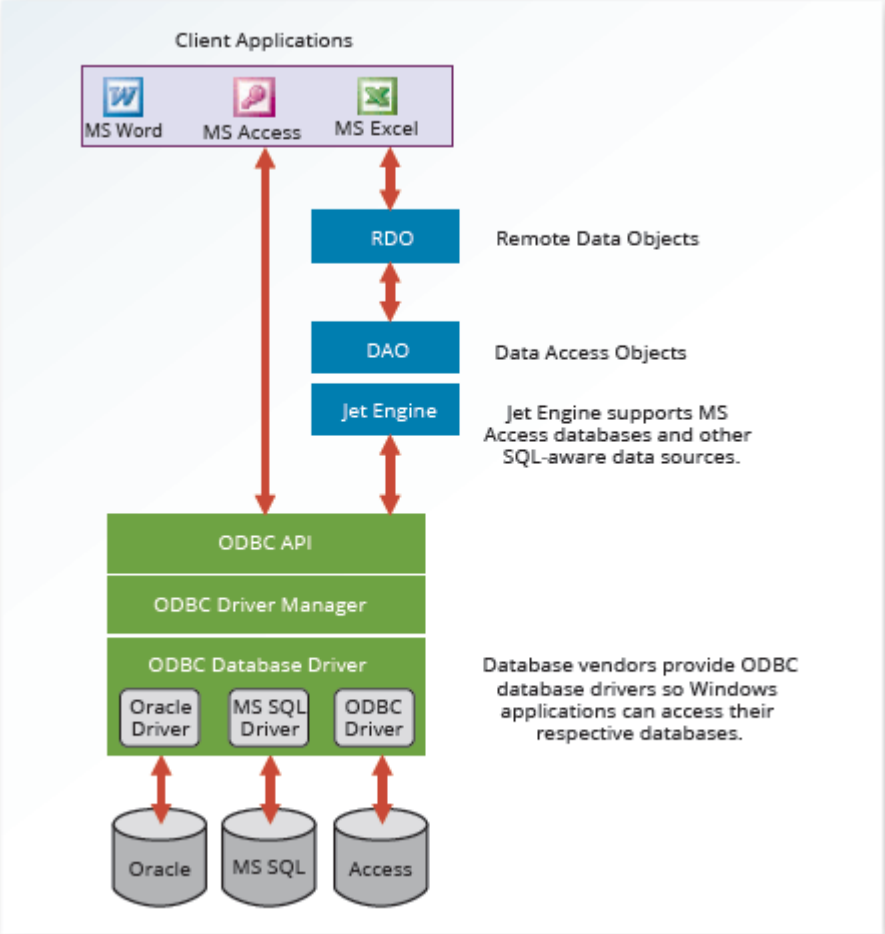


Figure 7-2: Using ODBC, DAO and RDO to Access Databases (Coronel C., Morris S. 2022)

The DAO and RDO interfaces provide more functionality than ODBC. DAO and RDO make use of the underlying ODBC data services. ODBC, DAO, and RDO are implemented as shared code that is dynamically linked to the Windows operating environment through **dynamic-link libraries (DLLs)**, which are stored as files with a .dll extension. Running as a DLL, the code speeds up load and run times. The basic ODBC architecture has three main components:

- A high-level *ODBC API* through which application programs access ODBC functionality
- A *driver manager* that is in charge of managing all database connections
- An *ODBC driver* that communicates directly to the DBMS

Defining a data source is the first step in using ODBC. To define a data source, you must create a **data source name (DSN)** for it. To create a DSN, you need to provide the following:

- *An ODBC driver*. You must identify the driver to use to connect to the data source. The ODBC driver is normally provided by the database vendor, although Microsoft provides several drivers that connect to most common databases. For example, if you are using an Oracle DBMS, you would select the Oracle ODBC driver provided by Oracle. Or, you could instead select the Microsoft-provided ODBC driver for Oracle.
- *A name*. This is a unique name by which the data source will be known to ODBC, and, therefore, to applications. ODBC offers two types of data sources: user and system. *User data sources* are available only to the user. *System data sources* are available to all users, including operating system services.

- *ODBC driver parameters*. Most ODBC drivers require specific parameters to establish a connection to the database. For example, if you are using an MS Access database, you must point to the location of the MS Access file and then provide a username and password if necessary. If you are using a DBMS server, you must provide the server name, the database name, the username, and the password needed to connect to the database. Figure 7.3 shows the ODBC screens required to create a system ODBC data source for an Oracle DBMS. Note that some ODBC drivers use the native driver provided by the DBMS vendor.

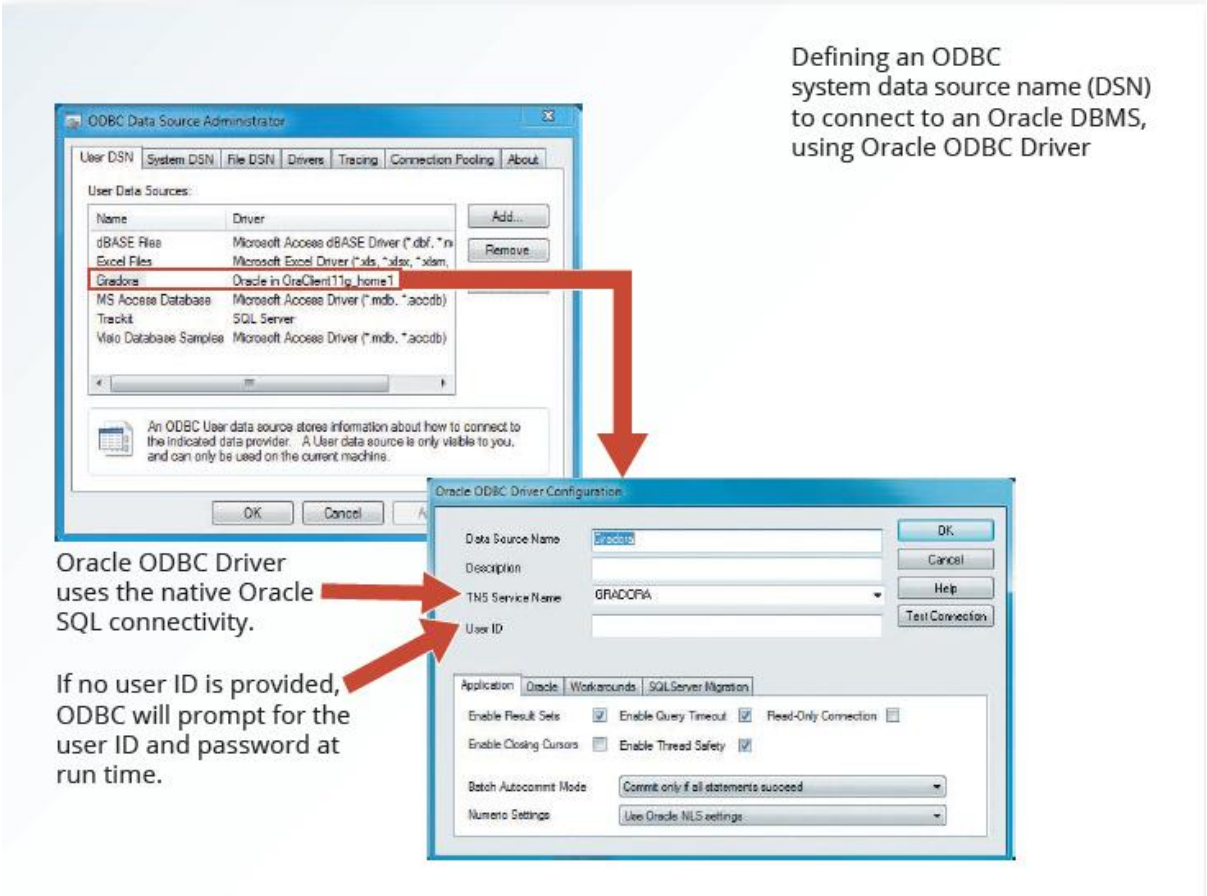


Figure 7-3: Configuring an Oracle ODBC Data Source (Coronel C., Morris S. 2022)

Once the ODBC data source is defined, application programmers can write to the ODBC API by issuing specific commands and providing the required parameters. The ODBC Driver Manager will properly route the calls to the appropriate data source. The ODBC API standard defines three levels of compliance: Core, Level-1, and Level-2, which provide increasing levels of functionality. For example, Level-1 might provide support for most SQL DDL and DML statements, including subqueries and aggregate functions, but not for procedural SQL or cursors. The database vendors can choose which level to support. However, to interact with ODBC, the database vendor must implement all of the features in the specified ODBC API support level.
Figure 7.4 shows how you could use MS Excel to retrieve data from an Oracle RDBMS using ODBC. Because much of the functionality provided by these interfaces is oriented toward accessing relational data sources, the use of the interfaces was limited with other data source types. With the advent of object-oriented programming languages, it has become more important to provide access to other nonrelational data sources.

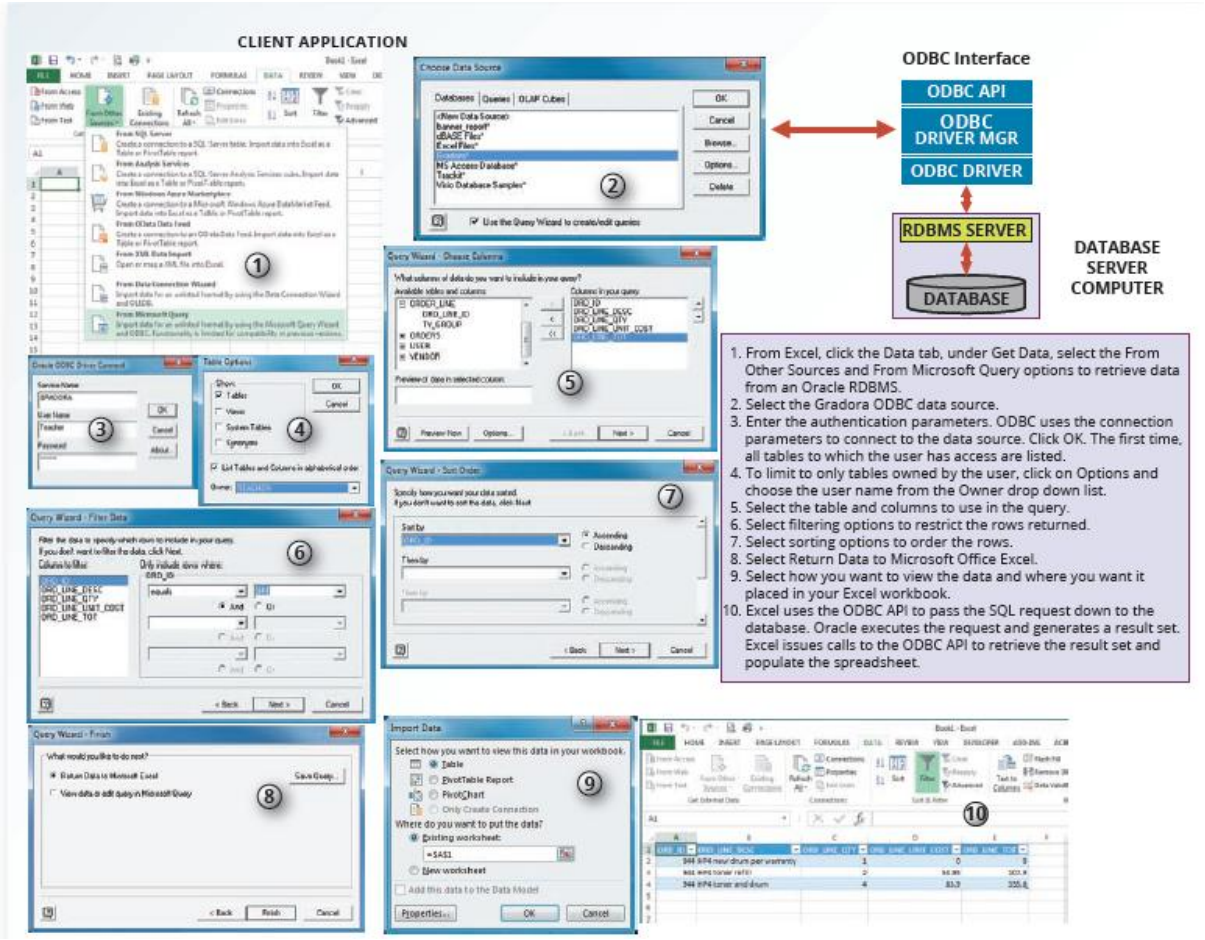


Figure 7-4: Using ODBC in MS Excel to Connect to an Oracle Database (Coronel C., Morris S. 2022)

OLE-DB

Although ODBC, DAO, and RDO are used, they do not provide support for nonrelational data. To answer that need and to simplify data connectivity, Microsoft developed **Object Linking and Embedding for Database (OLE-DB)**. Based on Microsoft's Component Object Model (COM), OLE-DB is database middleware that adds object-oriented functionality for access to relational and nonrelational data. OLE-DB was the first part of Microsoft's strategy to provide a unified object-oriented framework for the development of next-generation applications. OLE-DB is composed of a series of COM objects that provide low-level database connectivity for applications. Because OLE-DB is based on COM, the objects contain data and methods, also known as the interface. The OLE-DB model is better understood when you divide its functionality into two types of objects:

- *Consumers* are objects (applications or processes) that request and use data. Consumers request data by invoking the methods exposed by the data provider objects (public interface) and passing the required parameters.
- *Providers* are objects that manage the connection with a data source and provide data to the consumers. Providers are divided into two categories: data providers and service providers.
    - *Data providers* provide data to other processes. Database vendors create data provider objects that expose the functionality of the underlying data source (relational, object-oriented, text, and so on).

- *Service providers* provide additional functionality to consumers. The service provider is located between the data provider and the consumer. The service provider requests data from the data provider, transforms the data, and then provides the transformed data to the data consumer. In other words, the service provider acts like a data consumer of the data provider and as a data provider for the data consumer (end-user application).

For example, a service provider could offer cursor management services, transaction management services, query processing services, and indexing services.

As a common practice, many vendors provide OLE-DB objects to augment their ODBC support, effectively creating a shared object layer on top of their existing database connectivity (ODBC or native) through which applications can interact. The OLE-DB objects expose functionality about the database; for example, there are objects that deal with relational data, hierarchical data, and flat-file text data. Additionally, the objects implement specific tasks, such as establishing a connection, executing a query, invoking a stored procedure, defining a transaction, or invoking an OLAP function. By using OLE-DB objects, the database vendor can choose what functionality to implement in a modular way, instead of being forced to include all of the functionality all of the time. Table 7.1 shows a sample of the object-oriented classes used by OLE-DB and some of the methods (interfaces) exposed by the objects.

Table 7-1: Sample OLE-DB Classes and Interfaces

Object Class	Usage	Sample Interfaces
Session	Used to create an OLE-DB session between a data consumer application and a data provider	IGetDataSource
Command	Used to process commands to manipulate a data provider's data; generally, the command object will create RowSet objects to hold the data returned by a data provider	ICommandPrepare
RowSet	Used to hold the result set returned by a relational-style database or a database that supports SQL; represents a collection of rows in a tabular format	IRowsetInfo IRowsetFind IRowsetScroll

OLE-DB provides additional capabilities for the applications accessing the data. However, it does not provide support for scripting languages, especially the ones used for web development, such as Active Server Pages (ASP) and ActiveX. (A **script** is written in a programming language that is not compiled but is interpreted and executed at run time.) To provide that support, Microsoft developed a new object framework called **ActiveX Data Objects (ADO)**, which provides a high-level, application-oriented interface to interact with OLE-DB, DAO, and RDO. ADO provides a unified interface to access data from any programming language that uses the underlying OLE-DB objects. Figure 7.5 illustrates the ADO/OLE-DB architecture and how it interacts with ODBC and native connectivity options. ADO introduced a simpler object model that was composed of only a few interacting objects to provide the data manipulation services required by the applications. Sample objects in ADO are shown in Table 7.2.

Table 7-2: Sample ADO Objects

Object Class	Usage
Connection	Used to set up and establish a connection with a data source. ADO will connect to any OLE-DB data source. The data source can be of any type.
Command	Used to execute commands against a specific connection (data source)
Recordset	Contains the data generated by the execution of a command. It will also contain any new data to be written to the data source. The Recordset can be disconnected from the data source.
Fields	Contains a collection of field descriptions for each column in the Recordset

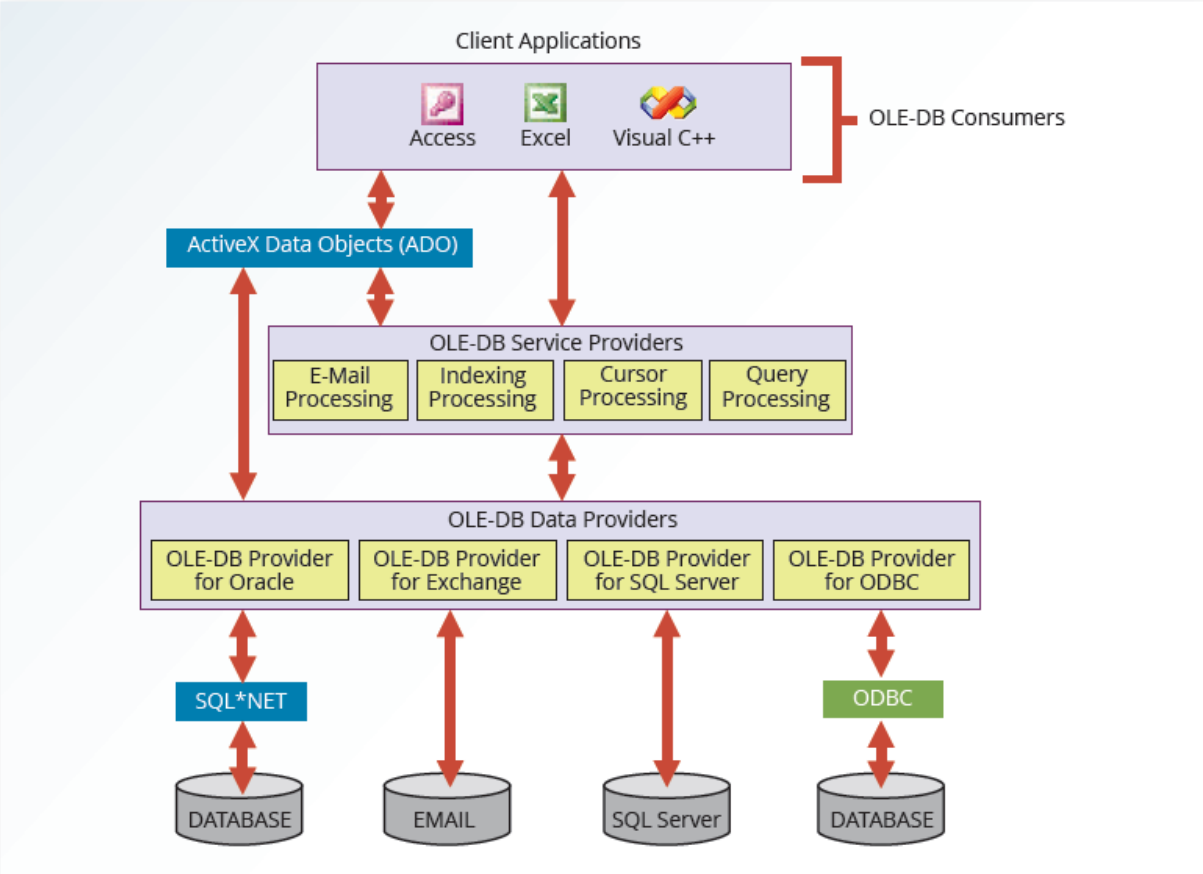


Figure 7-5: OLE-DB Architecture (Coronel C., Morris S. 2022)

ADO.NET

Based on ADO, **ADO.NET** is the data access component of Microsoft's .NET application development framework. The **Microsoft .NET framework** is a component-based platform for developing distributed, heterogeneous, interoperable applications aimed at manipulating any type of data using any combination of network, operating system, and programming language. Comprehensive coverage of the .NET framework is beyond the scope of this book. Therefore, this section only introduces the basic data access component of the .NET architecture, ADO.NET. It is important to understand that the .NET framework extends and enhances the functionality provided by the ADO/OLE-DB duo. ADO.NET introduced two new features that are critical for the development of distributed applications: DataSets and XML support.

To understand the importance of this new model, you should know that a **DataSet** is a disconnected, memory-resident representation of the database. That is, the DataSet contains tables, columns, rows, relationships, and constraints. Once the data is read from a data provider, it is placed in a memory-resident DataSet, which is then disconnected from the data provider. The data consumer application interacts with the data in the DataSet object to make inserts, updates, and deletes in the DataSet. Once the processing is done, the DataSet data is synchronized with the data source and the changes are made permanent.

The DataSet is internally stored in XML format, and the data in the DataSet can be made persistent as XML documents. This is critical in today's distributed environments. You can think of the DataSet as an XML-based, in-memory database that represents the persistent data stored in the data source. (You will learn about XML later in this chapter.) Figure 15.6 illustrates the main

components of the ADO.NET object model. The ADO.NET framework consolidates all data access functionality under one integrated object model. In this object model, several objects interact with one another to perform specific data manipulations. These objects can be grouped as data providers and consumers. Database vendors provide data provider objects. However, ADO.NET comes with two standard data providers: one for OLE-DB data sources and one for SQL Server. That way, ADO.NET can work with any previously supported database, including an ODBC database with an OLE-DB data provider. At the same time, ADO.NET includes a highly optimized data provider for SQL Server. Whatever the data provider is, it must support a set of specific objects to manipulate the data in the data source. Some of those objects are shown in Figure 7.6. A brief description of the objects follows.

- *Connection*. The Connection object defines the data source used, the name of the server, the database, and so on. This object enables the client application to open and close a connection to a database.
- *Command*. The Command object represents a database command to be executed within a specified database connection. This object contains the actual SQL code or a stored procedure call to be run by the database. When a SELECT statement is executed, the Command object returns a set of rows and columns.
- *DataReader*. The DataReader object is a specialized object that creates a read-only session with the database to retrieve data sequentially (forward only) and very quickly.
- *DataAdapter*. The DataAdapter object is in charge of managing a DataSet object, and it is the most specialized object in the ADO.NET framework. The DataAdapter object contains the following objects that aid in managing the data in the DataSet: SelectCommand, InsertCommand, UpdateCommand, and DeleteCommand. The DataAdapter object uses these objects to populate and synchronize the data in the DataSet with the permanent data source data.
- *DataSet*. The DataSet object is the in-memory representation of the data in the database. This object contains two main objects. The DataTableCollection object contains a collection of DataTable objects that make up the "in-memory" database, and the DataRelation-Collection object contains a collection of objects that describe the data relationships and ways to associate one row in a table to the related row in another table.
- *DataTable*. The DataTable object represents the data in tabular format. This object has one important property: PrimaryKey, which allows the enforcement of entity integrity. In turn, the DataTable object is composed of three main objects:
    - *DataColumnCollection* contains one or more column descriptions. Each column description has properties such as column name, data type, nulls allowed, maximum value, and minimum value.
    - *DataRowCollection* contains zero rows, one row, or more than one row with data as described in the DataColumnCollection. *ConstraintCollection* contains the definition of the constraints for the table. Two types of constraints are supported: ForeignKeyConstraint and UniqueConstraint.

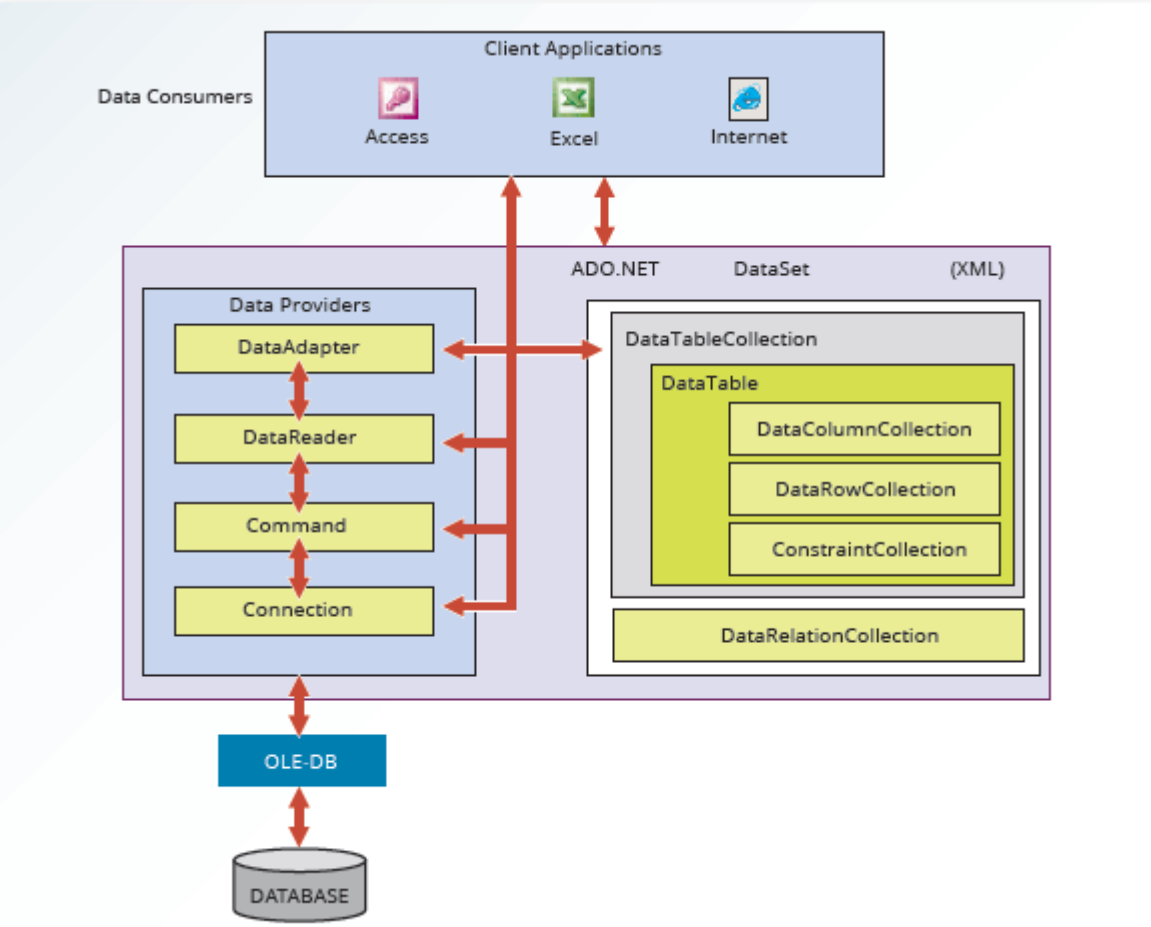


Figure 7-6: ADO.NET Framework (Coronel C., Morris S. 2022)

As you can see, a DataSet is a simple database with tables, rows, and constraints. Even more importantly, the DataSet does not require a permanent connection to the data source. The DataAdapter uses the SelectCommand object to populate the DataSet from a data source. However, once the DataSet is populated, it is completely independent of the data source, which is why it is called disconnected. Additionally, DataTable objects in a DataSet can come from different data sources. This means that you could have an EMPLOYEE table in an Oracle database and a SALES table in a SQL Server database. You could then create a DataSet that relates both tables as though they were in the same database. In short, the DataSet object paves the way for truly heterogeneous, distributed database support within applications. The ADO.NET framework is optimized to work in disconnected environments. In a disconnected environment, applications exchange messages in request/reply format. The most common example of a disconnected system is the Internet. Modern applications rely on the Internet as the network platform and on the web browser as the graphical user interface.

Java Database Connectivity (JDBC)

Java is an object-oriented programming language developed by Sun Microsystems (acquired by Oracle in 2010) that runs on top of web browser software. Java is one of the most common programming languages for web development. Sun Microsystems created Java as a “write once,

run anywhere" environment, which means that a programmer can write a Java application once and then run it in multiple environments without any modification. The cross-platform capabilities of Java are based on its portable architecture. Java code is normally stored in pre-processed "chunks" known as applets that run in a virtual machine environment in the host operating system. This environment has well-defined boundaries, and all interactivity with the host operating system is closely monitored. Java run-time environments are available for most operating systems, from computers to handheld mobile devices to TV set-top boxes. Another advantage of using Java is its "on-demand" architecture. When a Java application loads, it can dynamically download all its modules or required components via the Internet. When Java applications need to access data outside the Java runtime environment, they use predefined application programming interfaces. **Java Database Connectivity (JDBC)** is an application programming interface that allows a Java program to interact with a wide range of data sources, including relational databases, tabular data sources, spreadsheets, and text files. JDBC allows a Java program to establish a connection with a data source, prepare and send the SQL code to the database server, and process the result set. One main advantage of JDBC is that it allows a company to leverage its existing investment in technology and personnel training. JDBC allows programmers to use their SQL skills to manipulate the data in the company's databases. As a matter of fact, JDBC allows direct access to a database server or access via database middleware. Furthermore, JDBC provides a way to connect to databases through an ODBC driver. Figure 7.7 illustrates the basic JDBC architecture and the various database access styles.

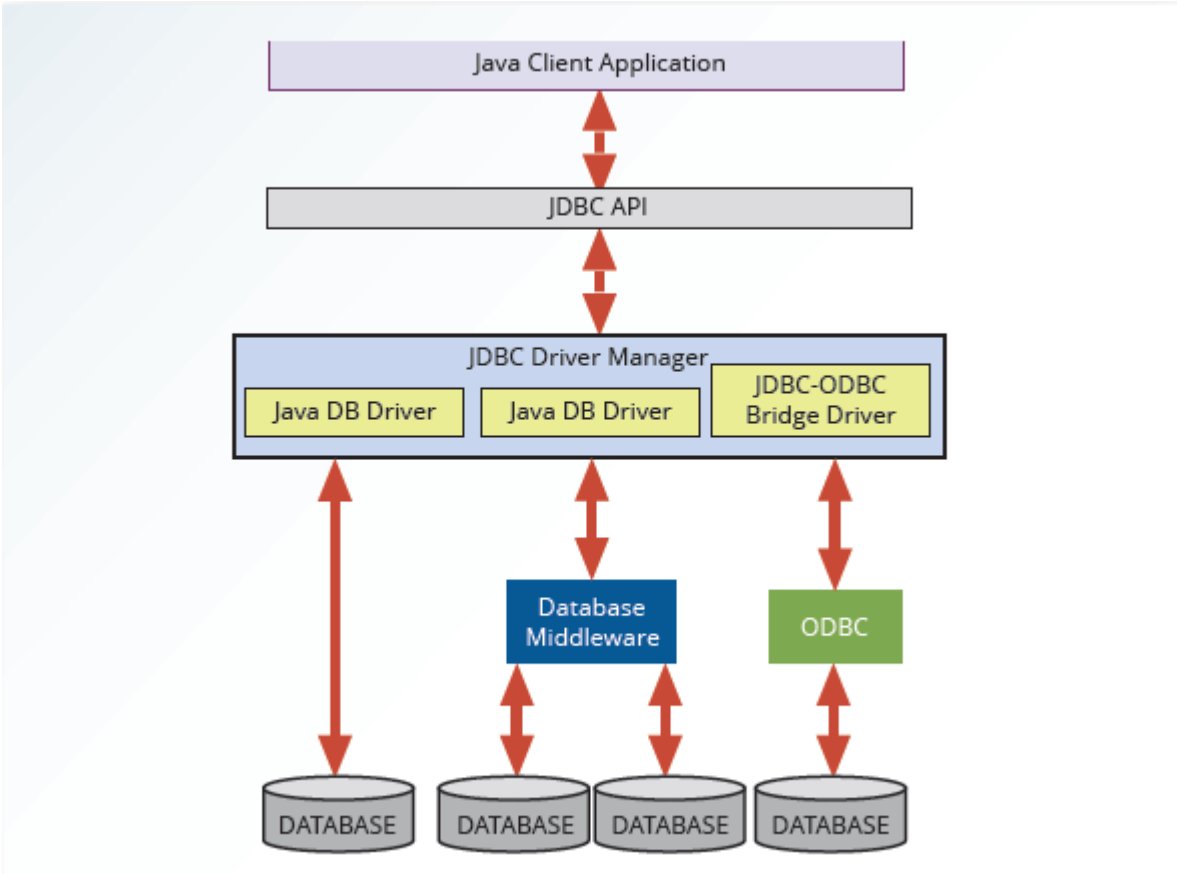


Figure 7-7: JDBC Architecture (Coronel C., Morris S. 2022)

As you see in Figure 7.7, the database access architecture in JDBC is similar to the ODBC/OLE/ADO.NET architecture. All database access middleware shares similar components and functionality. One advantage of JDBC over other middleware is that it requires no configuration on the client side. The JDBC driver is automatically downloaded and installed as part of the Java applet download. Because Java is a web-based technology, applications can connect to a database directly using a simple URL. Once the URL is invoked, the Java architecture comes into play, the necessary applets are downloaded to the client (including the JDBC database driver and all configuration information), and then the applets are executed securely in the client's runtime environment. This framework is used successfully in many vertical database markets, in particular in the rapidly growing data analytics market, where open source players like Hadoop and MapReduce provide end-users with advanced application programming interfaces to high-performance data analytics functions using large-scale clusters of interconnected data stores. Every day, more and more companies are investing resources to develop and expand their web presence and are finding ways to do more business on the Internet. Such business generates increasing amounts of data to be stored in databases. Java and the .NET framework are part of the trend toward increasing reliance on the Internet as a critical business resource. In fact, the Internet has become a major development platform for most businesses. In the next section, you will learn more about Internet databases and how they are used.

Database Internet Connectivity

Millions of people all over the world access the Internet and connect to databases via web browsers or data services. For example, they can use a smartphone app to get weather forecasts, stock prices, driving directions, concert tickets, or music downloads. Internet database connectivity opens the door for companies to bring new and innovative services to market quickly, increase customer satisfaction by providing "location aware" services that combine mapping and GPS (Global Positioning System) data, and allow anywhere, anytime access to data and services. Given these advantages, many organizations rely on their IT departments to create universal data access architectures based on Internet standards. Table 7.3 shows a sample of Internet technology characteristics and the benefits they provide. As you will learn in the following sections, database application development— particularly the creation and management of user interfaces and database connectivity—is profoundly affected by the web. However, having a web-based database interface does not negate the design and implementation issues that were addressed in the previous chapters. In the final analysis, whether you make a purchase by going online or by standing in line, the system-level transaction details are essentially the same, and they require the same basic database structures and relationships.

Table 7-3: Characteristics and Benefits of Internet Technologies

Internet Characteristic	Benefit
Hardware and software independence	Savings in equipment and software acquisition Ability to run on most existing equipment Platform independence and portability No need for multiple platform development
Common and simple user interface	Reduced training time and cost Reduced end-user support cost No need for multiple platform development
Location independence	Global access through Internet infrastructure and mobile smart devices Creation of new location-aware services Reduced requirements (and costs!) for dedicated connections
Rapid development at manageable costs	Availability of multiple development tools Plug-and-play development tools (open standards) More interactive development Reduced development times Relatively inexpensive tools Free client access tools (web browsers) Low entry costs; frequent availability of free web servers Reduced costs of maintaining private networks Distributed processing and scalability using multiple servers

Web-to-Database Middleware: Server-Side Extensions

In general, the web server is the main hub through which all Internet services are accessed. For example, when an end user uses a web browser to dynamically query a database, the client browser requests a webpage from the web server. The web server receives the page request, looks for the page on the hard disk, finds it, executes the query to retrieve the data, and sends it back to the client. Dynamic webpages are at the heart of current websites. In this database query scenario, the web server generates the webpage contents before it sends the page to the client web browser. The only problem with the preceding query scenario is that the web server must include the database query result on the page *before* it sends that page back to the client. Unfortunately, neither the web browser nor the web server knows how to connect to, and read data from, the database. Therefore, to support this type of request, the web server's capability must be extended so it can understand and process database requests. This job is known as a server-side extension.

A **server-side extension** is a program that interacts directly with the web server to handle specific types of requests. In the preceding database query example, the server-side extension program retrieves the data from databases and passes the retrieved data to the web server, which in turn sends the data to the client's browser for display. The server-side extension makes it possible to retrieve and present the query results, but more importantly, *it provides its services to the web server in a way that is transparent to the client browser*. In short, the server-side extension adds significant functionality to the web server, and therefore to the Internet.

A database server-side extension program is also known as **web-to-database middleware**. Figure 7.8 shows the interaction between the browser, the web server, and the web-to-database middleware.

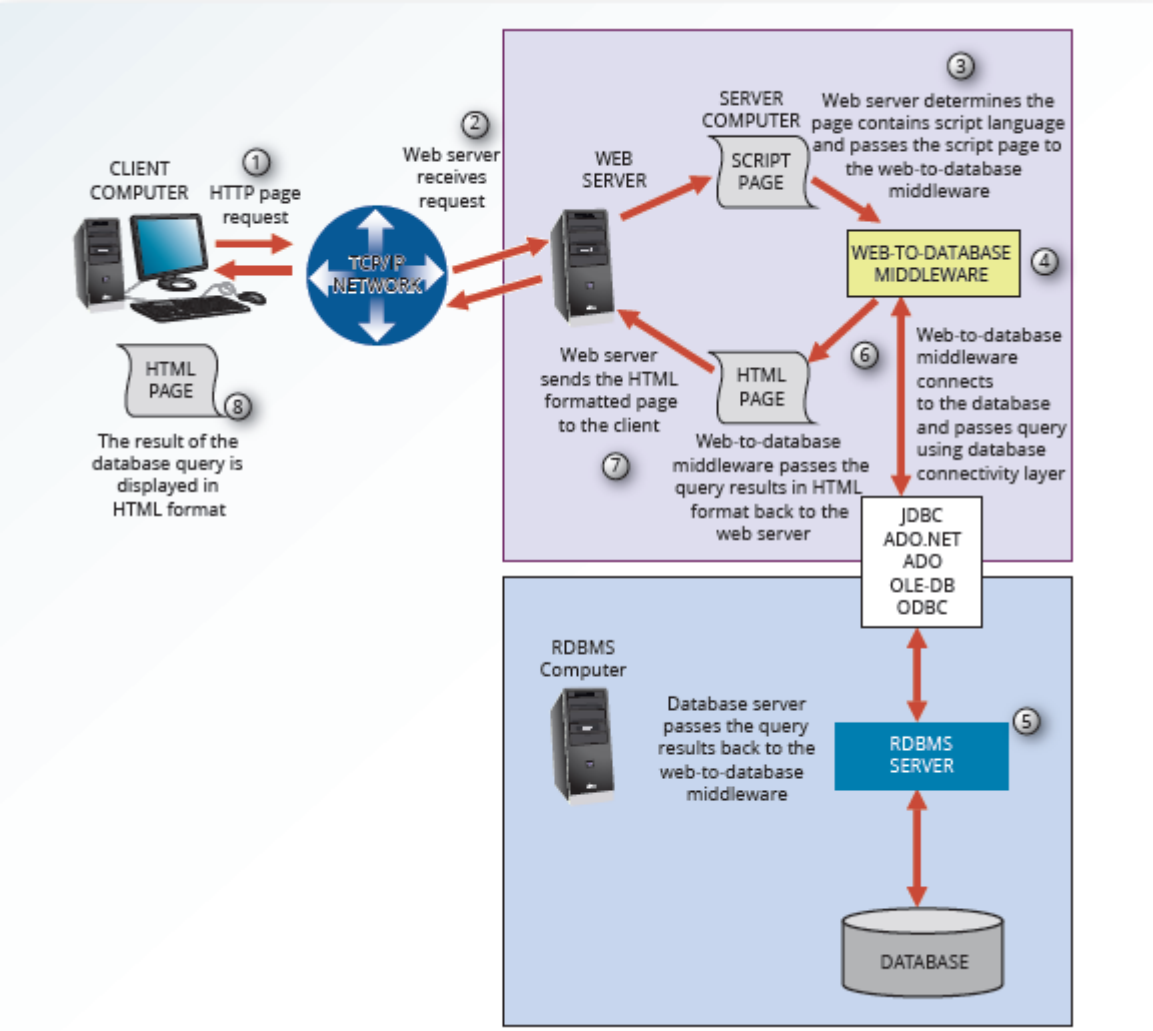


Figure 7-8: Web-to-Database Middleware (Coronel C., Morris S. 2022)

Trace the web-to-database middleware actions in Figure 7.8:

1. The client browser sends a page request to the web server.
2. The web server receives and passes the request to the web-to-database middleware for processing.
3. Generally, the requested page contains some type of scripting language to enable the database interaction. The web server passes the script to the web-to-database middleware.
4. The web-to-database middleware reads, validates, and executes the script. In this case, it connects to the database and passes the query using the database connectivity layer.
5. The database server executes the query and passes the result back to the web-to-database middleware.
6. The web-to-database middleware compiles the result set, dynamically generates an HTML-formatted page that includes the data retrieved from the database, and sends it to the web server.
7. The web server returns the just-created HTML page, which now includes the query result, to the client browser.
8. The client browser displays the page on the client's computer.

The interaction between the web server and the web-to-database middleware is crucial to the development of a successful Internet database implementation. Therefore, the middleware must integrate closely via a well-defined web server interface.

Web Server Interfaces

Extending web server functionality implies that the web server and the web-to-database middleware will properly communicate with each other. (Database professionals often use the word *interoperate* to indicate that each party can respond to the communications of the other.) A web server interface defines a standard way to exchange messages with external programs. Currently, there are two well-defined web server interfaces:

- Common Gateway Interface (CGI)
- Application programming interface (API)

The **Common Gateway Interface (CGI)** uses script files that perform specific functions based on the client's parameters that are passed to the web server. The script file is a small program containing commands written in a programming language—usually Perl, C#, or Visual Basic. The script file's contents can be used to connect to the database and to retrieve data from it, using the parameters passed by the web server. Next, the script converts the retrieved data to HTML format and passes the data to the web server, which sends the HTML-formatted page to the client.

The main disadvantage of using CGI scripts is that the script file is an external program that executes separately for each user request and therefore causes a resource bottleneck. Performance also could be degraded by using an interpreted language or by writing the script inefficiently. An application programming interface (API) is a newer web server interface standard that is more efficient and faster than a CGI script. APIs are more efficient because they are implemented as shared code or as dynamic-link libraries (DLLs). That means the API is treated as part of the web server program that is dynamically invoked when needed. APIs are faster than CGI scripts because the code resides in memory, so there is no need to run an external program for each request. Instead, the same API serves all requests. Another advantage is that an API can use a shared connection to the database instead of creating a new one every time, as is the case with CGI scripts.

Although APIs are more efficient in handling requests, they have some disadvantages. Because the APIs share the same memory space as the web server, an API error can bring down the web server. Another disadvantage is that APIs are specific to the web server and to the operating system.

The web interface architecture is illustrated in Figure 7.9.

Regardless of the type of web server interface used, the web-to-database middleware program must be able to connect with the database. That connection can be accomplished in one of two ways:

- Use the native SQL access middleware provided by the vendor. For example, you can use SQL*Net if you are using Oracle.
- Use the services of general database connectivity standards such as ODBC, OLE-DB, ADO, ADO.NET, or JDBC.

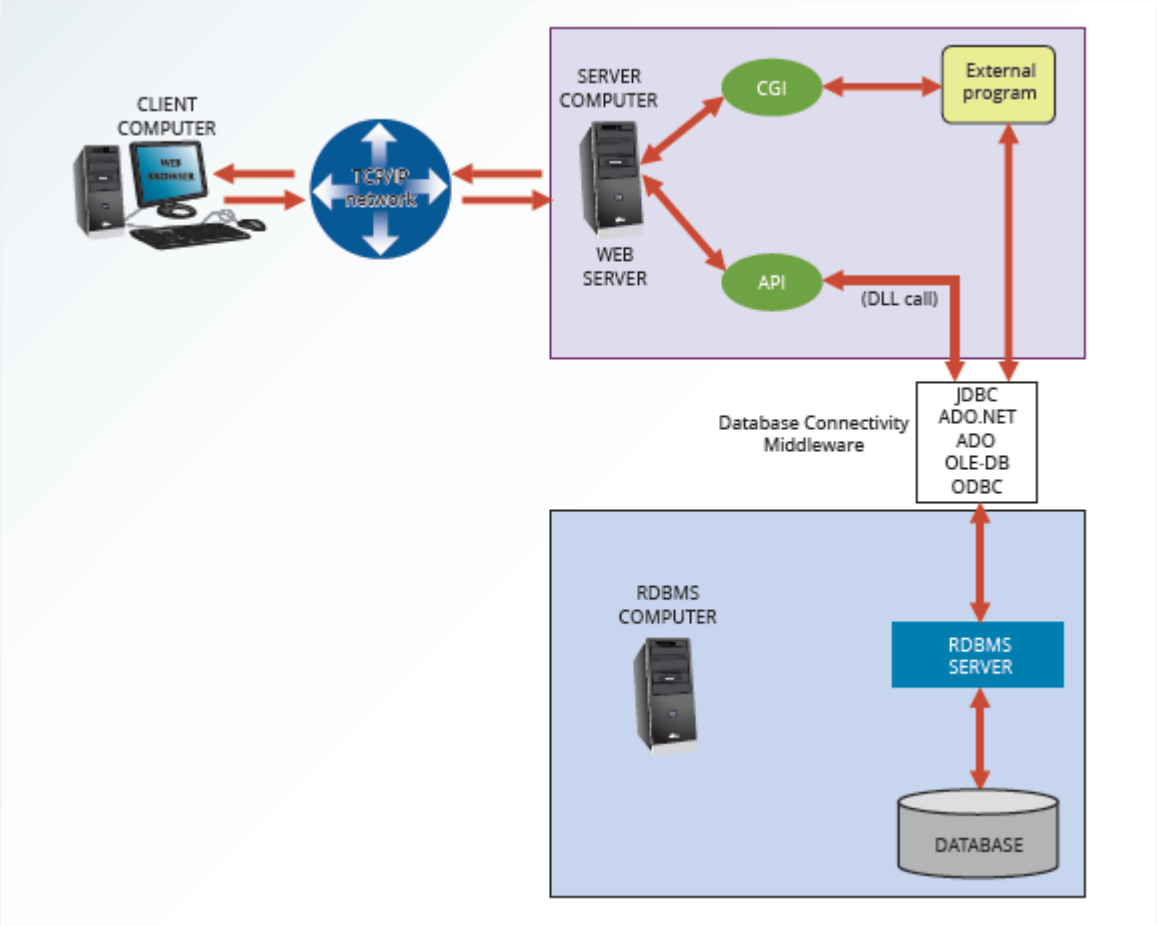


Figure 7-9: Web Server CGI and API Interfaces (Coronel C., Morris S. 2022)

The Web Browser

The web browser is software such as Microsoft Internet Explorer, Microsoft Edge, Google Chrome, Apple Safari, or Mozilla Firefox that lets end users navigate the web from their client computer. Each time the end user clicks a hyperlink, the browser generates an HTTP GET page request that is sent to the designated web server using the TCP/IP Internet protocol. The web browser's job is to *interpret* the HTML code that it receives from the web server and to present the various page components in a standard formatted way. Unfortunately, the browser's interpretation and presentation capabilities are not sufficient to develop web-based applications. The web is a **stateless system**—at any given time, a web server does not know the status of any of the clients communicating with it. That is, there is no open communication line between the server and each client accessing it, which of course is impractical in a *worldwide* web! Instead, client and server computers interact in short "conversations" that follow the request-reply model.

For example, the browser is concerned only with the *current* page, so there is no way for the second page to know what was done in the first page. The only time the client and server computers communicate is when the client requests a page—when the user clicks a link—and

the server sends the requested page to the client. Once the client receives the page and its components, the client/server communication is ended. Therefore, although you may be browsing a page and *think* that the communication is open, you are actually just browsing the HTML document stored in the local cache (temporary directory) of your browser. The server does not have any idea what the end user is doing with the document, what data is entered in a form, what option is selected, and so on. On the web, if you want to act on a client's selection, you need to jump to a new page (go back to the web server), thus losing track of what was done before.

The web browser, through its use of HTML, does not have computational abilities beyond formatting output text and accepting form field inputs. Even when the browser accepts form field data, there is no way to perform immediate data entry validation. Therefore, to perform such crucial processing in the client, the web defers to other web programming languages such as Java, JavaScript, and VBScript. The browser resembles a dumb terminal that displays only data and can perform only rudimentary processing such as accepting form data inputs. To improve the capabilities of the web browser, you must use plug-ins and other client-side extensions. On the server side, web application servers provide the necessary processing power.

Client-Side Extensions

Client-side extensions add functionality to the web browser. Although client-side extensions are available in various forms, the most common are:

- Plug-ins
- Java and JavaScript
- ActiveX and VBScript

A **plug-in** is an external application that is automatically invoked by the browser when needed. The plug-in is associated with a data object—generally using the file extension—to allow the web server to properly handle data that is not originally supported. For example, if one of the page components is a PDF document, the web server will receive the data, recognize it as a Portable Document Format object, and launch Adobe Reader to present the document on the client computer.

JavaScript is a scripting language (one that enables the execution of a series of commands or macros) that allows web authors to design interactive sites. JavaScript code is embedded in the webpage and executed after a specific event, such as a mouse click on an object or a page being loaded from the server into memory.

ActiveX is Microsoft's alternative to Java. ActiveX is a specification for writing programs that run inside the Microsoft client browser, Internet Explorer. Because ActiveX is oriented toward Windows applications, it has low portability. ActiveX extends the web browser by adding controls to webpages, including drop-down lists, a slider, a calendar, and a calculator. Those controls are downloaded from the web server when needed so you can manipulate data inside the browser. ActiveX controls can be created in several programming languages, including C++. Microsoft's .NET framework allows for wider interoperability of ActiveX-based applications (such as ADO.NET) across multiple operating environments.

VBScript is another Microsoft product that is used to extend browser functionality. VBScript is derived from Microsoft Visual Basic. Like JavaScript, VBScript code is embedded inside an HTML page and is activated by triggering events such as clicking a link.

From the developer's point of view, using routines that permit data validation on the client side is an absolute necessity. For example, when data is entered in a web form and no data validation

is done on the client side, the entire data set must be sent to the web server. That scenario requires the server to perform all data validation, thus wasting valuable CPU processing cycles. Therefore, client-side data input validation is one of the most basic requirements for web applications. Most of the data validation routines are done in Java, JavaScript, ActiveX, or VBScript.

Web Application Servers

A **web application server** is a middleware application that expands the functionality of web servers by linking them to a wide range of services, such as databases, directory systems, and search engines. The web application server also provides a consistent runtime environment for web applications. Web application servers can be used to perform the following:

- Connect to and query a database from a webpage
- Present database data in a webpage using various formats
- Create dynamic web search pages
- Create webpages to insert, update, and delete database data
- Enforce referential integrity in the application program logic
- Use simple and nested queries and programming logic to represent business rules

Web application servers provide features such as:

- An integrated development environment with session management and support for persistent application variables
- Security and authentication of users through user IDs and passwords
- Computational languages to represent and store business logic in the application server
- Automatic generation of HTML pages integrated with Java, JavaScript, VBScript, ASP, and so on
- Performance and fault-tolerant features
- Database access with transaction management capabilities
- Access to multiple services, such as file transfers (FTP), database connectivity, email, and directory services

Examples of web application servers include ColdFusion by Adobe, WebSphere Application Server by IBM, and WebLogic Server by Oracle. All web application servers offer the ability to connect web servers to multiple data sources and other services. They vary in their range of available features, robustness, scalability, compatibility with other web and database tools, and extent of the development environment.

Cloud Computing Services

You have almost certainly heard about the “cloud” from the thousands of publications and TV ads that have used the term over the years, although it has represented different concepts. In the late 1980s, the term *cloud* was used by telecommunication companies to describe their data networks. In the late 1990s, during the peak of Internet growth, the term depicted the Internet itself. Then, in 2006, Google and Amazon began using the term *cloud computing* to describe a new set of innovative web-based services. Google, Yahoo, eBay, and Amazon were early adopters of this new computing paradigm. But what exactly is cloud computing? According to the National Institute of Standards and Technology (NIST), **cloud computing** is “a computing model for enabling ubiquitous, convenient, on-demand network access to a shared pool of

configurable computer resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction." The term **cloud services** is used in this book to refer to the services provided by cloud computing. Cloud services allow any organization to quickly and economically add information technology services such as applications, storage, servers, processing power, databases, and infrastructure to its IT portfolio. Figure 7.10 shows a representation of cloud computing services on the Internet.

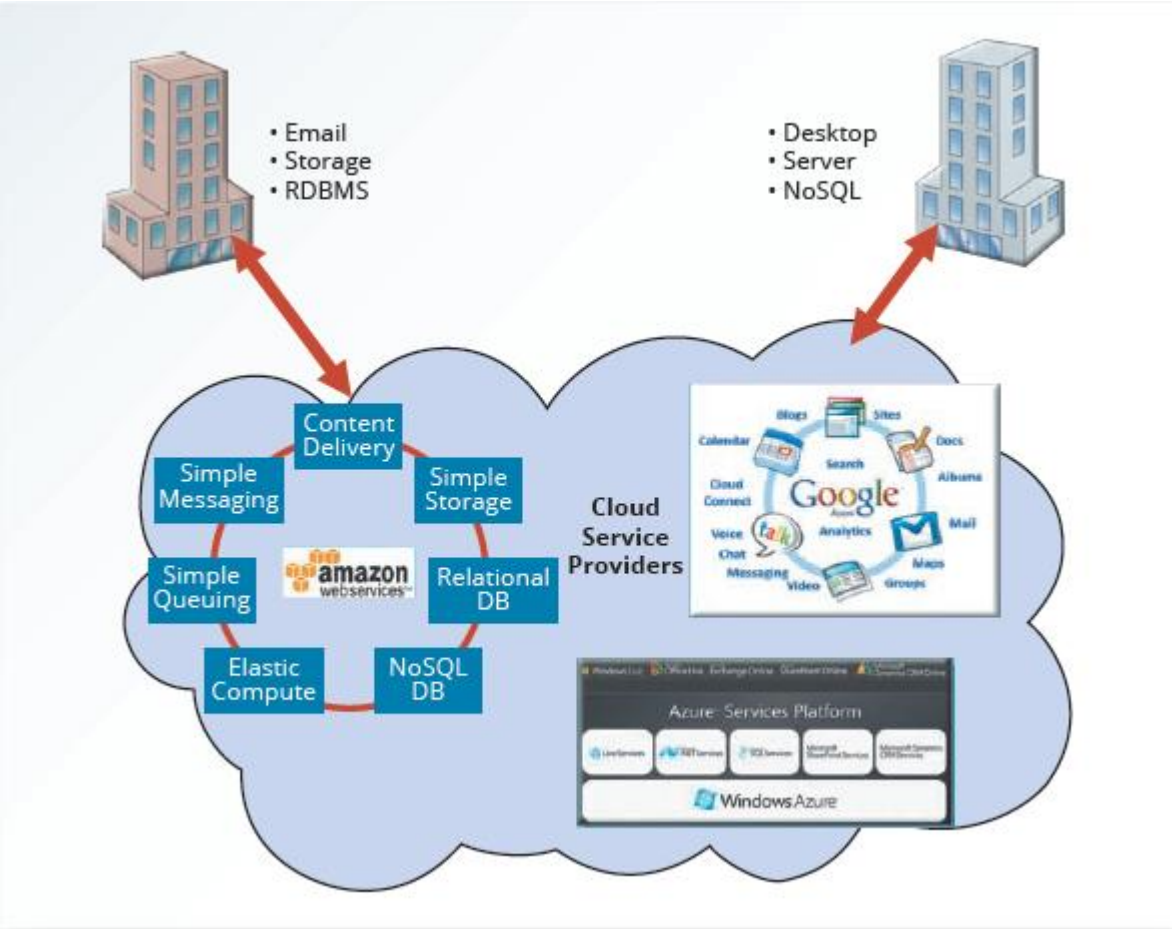


Figure 7-10: Cloud Services (Coronel C., Morris S. 2022)

Cloud computing allows IT-savvy organizations such as Amazon, Google, and Microsoft to build high-performance, fault-tolerant, flexible, and scalable IT services. These services include applications, storage, servers, processing power, databases, and email, which are delivered via the Internet to individuals and organizations using a pay-as-you-go price model. For example, imagine that the chief technology officer of a nonprofit organization wants to add email services to the IT portfolio. A few years ago, this proposition would have implied building the email system's infrastructure from the ground up, including hardware, software, setup, configuration, operation, and maintenance. However, in today's cloud computing era, you can use Google Apps for Business or Microsoft 365 Online and get a scalable, flexible, and more reliable email solution for a fraction of the cost. The best part is that you do not have to worry about the daily chores of managing and maintaining the IT infrastructure, such as OS updates, patches, security, fault tolerance, and recovery. What used to take months or years to implement can now be done in a matter of minutes. If you need more space, you just add another storage unit to your storage cloud. If you need more processing power to handle last-minute orders during the busy holiday season, you simply add more processing units to your cloud servers. Even more importantly, you

can scale down as easily as you scaled up. Once your need for additional processing or storage subsides, you can go back to your previous levels of usage and pay only for what you use. The beauty of cloud services is that you can scale down automatically, without an administrator's intervention. Cloud computing has become a "game changer" for database technologies because it eliminates financial and technological barriers so organizations can leverage database technologies in their business processes with minimal effort and cost. In fact, the cloud has turned basic IT services into "commodity" services such as electricity, gas, and water, and enabled a revolution that changed not only the way that companies do business, but the IT business itself. The technologies that make cloud computing work have been around for a few years now; these technologies include the web, messaging, virtualization, remote desktop protocols, VPN, and XML. Cloud computing has reached the maturity level that allows companies to widely adopt the technology to secure advanced database and analytical services for their organizations. Currently, you can log in to Amazon Web Services (AWS) or Microsoft Azure and have a relational database ready for use in a matter of minutes. Instead of spending large amounts of cash buying hardware and software, organizations can employ a pay-per-use model for their IT services. Figure 7.11 depicts the cost of provisioning a relational database instance in Microsoft Azure and Amazon RDS services, respectively. Although Figure 7.11 shows a cloud that requires some degree of customization on the customer's part, other cloud computing services are more transparent to the user and require less customization. For example, Dropbox and Microsoft OneDrive are simple cloud services that let you synchronize your documents, photos, music, and other files transparently over the Internet across many devices. Apple provides a similar service, known as iCloud, to seamlessly exchange data among all its mobile and nonmobile devices. Both services work transparently behind the scenes with minimal end-user intervention. As you can see, cloud computing implementations vary; the next section explains the basic types.

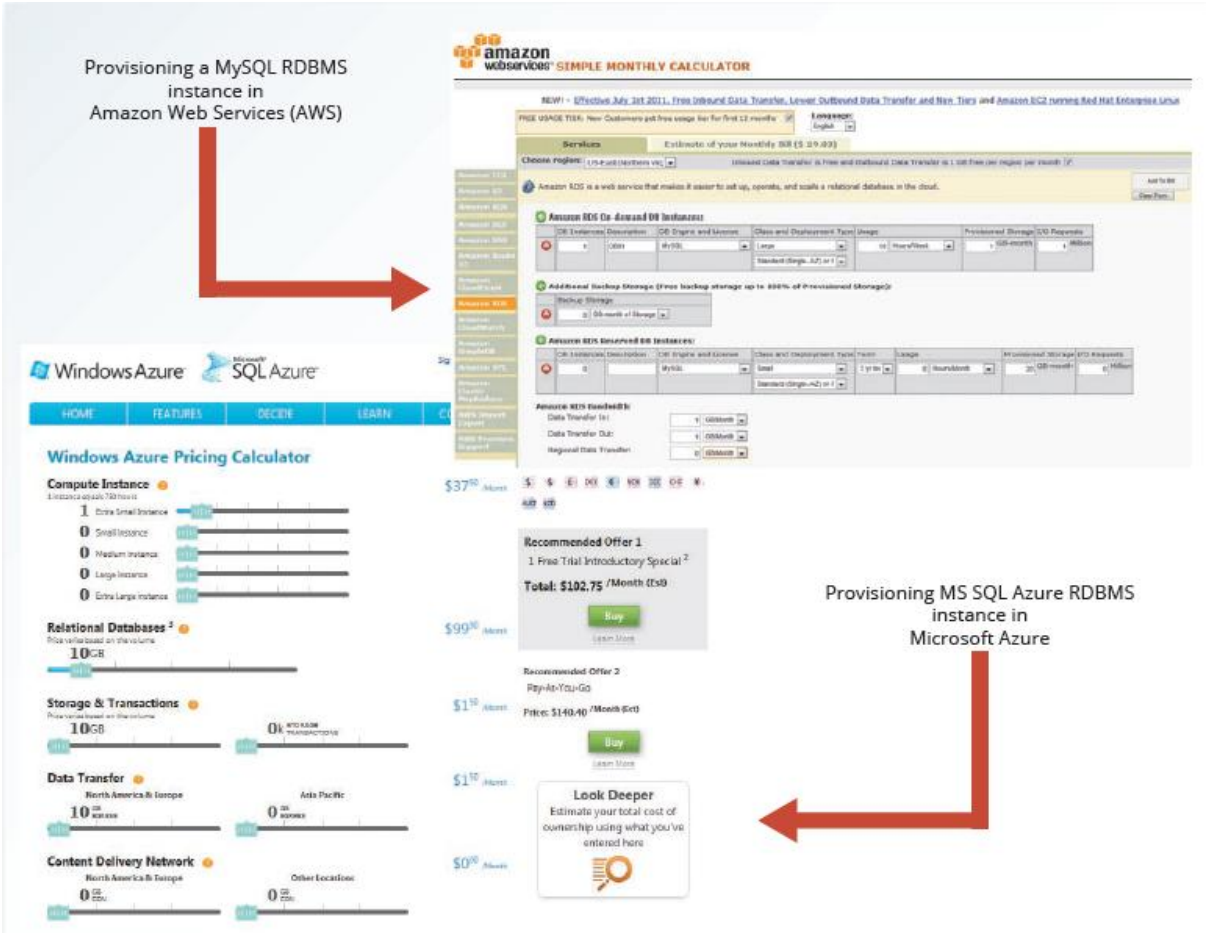


Figure 7-11: Provisioning a RDBMS in the Cloud (Coronel C., Morris S. 2022)

Cloud Implementation Types

Cloud computing has different types of implementations based on who the target customers are:

- **Public cloud**. This type of cloud infrastructure is built by a third-party organization to sell cloud services to the general public. The public cloud is the most common type of cloud implementation; examples include Amazon Web Services (AWS), Google App Engine, and Microsoft Azure. In this model, cloud consumers share resources with other consumers transparently. The public cloud infrastructure is managed exclusively by the third-party provider.
- **Private cloud**. This type of internal cloud is built by an organization for the sole purpose of servicing its own needs. Private clouds are often used by large, geographically dispersed organizations to add agility and flexibility to internal IT services. The cloud infrastructure could be managed by internal IT staff or an external third party.
- **Community cloud**. This type of cloud is built by and for a specific group of organizations that share a common trade, such as agencies of the federal government, the military, or higher education. The cloud infrastructure could be managed by internal IT staff or an external third party.

Regardless of the implementation an organization uses, most cloud services share a common set of core characteristics.

Characteristics of Cloud Services

Cloud computing services share a set of guiding principles. The characteristics listed in this section are shared by prominent public cloud providers such as Amazon, Google, Salesforce, SAP, and Microsoft. The prevalent characteristics are:

- *Ubiquitous access via Internet technologies*. All cloud services use Internet and web technologies to provision, deliver, and manage the services they provide. The basic requirement is that the device has access to the Internet.
- *Shared infrastructure*. The cloud service infrastructure is shared by multiple users. Sharing is made possible by web and virtualization technologies. Cloud services effectively provide an organization with a virtual IT infrastructure, which is locally managed by the consumer's organization as if it were the only user of the infrastructure.
- *Lower startup costs and variable pricing*. The initial costs of using cloud services tend to be significantly lower than building on-premise IT infrastructures. According to some studies, the savings could range from 35 percent to 55 percent depending on company size, although more research is needed in this area. Because the web service's usage is metered per volume and time utilization, consumers benefit from lower and flexible pricing options. These options range from pay-as-you-go to fixed pricing based on minimum levels of service. There is a break-even point where enterprises are better off using in-house cloud infrastructure, so your organization should analyze the costs closely for their particular needs.
- *Flexible and scalable services*. The cloud services are built on an infrastructure that is highly scalable, fault tolerant, and very reliable. The services can scale up and down depending on resource demands.
- *Dynamic provisioning*. The consumer can quickly provision any needed resources, including servers, processing power, storage, and email, by accessing the web management dashboard and then adding and removing services on demand. This process also could be automated via other services.
- *Service orientation*. Cloud computing focuses on providing consumers with specific, well-defined services that use well-known interfaces. These interfaces hide the complexity from the end user and can be delivered anytime and anywhere.
- *Managed operations*. Cloud computing minimizes the need for extensive and expensive in-house IT staff. The system infrastructure is managed by the cloud provider. The consumer organization's IT staff is free from routine management and maintenance tasks so they can focus on other tasks within the organization. Managed operations apply to organizations that use public clouds and that outsource cloud management to an external third party.

The preceding list is not exhaustive, but it is a starting point to understand most cloud computing offerings. Although most companies move to cloud services because of cost savings, some companies move to them because they are the best way to gain access to specific IT resources that would otherwise be unavailable.

Types of Cloud Services

Cloud services come in different shapes and forms; no single type of service works for all consumers. In fact, cloud services often follow an a la carte model; consumers can choose multiple service options according to their individual needs. These services can build on top of

each other to provide sophisticated solutions. Based on the types of services provided, cloud services can be classified by the following categories:

- **Software as a Service (SaaS)**. The cloud service provider offers turnkey applications that run in the cloud. Consumers can run the provider's applications internally in their organizations via the web or any mobile device. The consumer can customize certain aspects of the application but cannot make changes to the application itself. The application is actually shared among users from multiple organizations. Examples of SaaS include Microsoft 365, Google Docs, Intuit's TurboTax Online, and SCALA digital signage.
- **Platform as a Service (PaaS)**. The cloud service provider offers the capability to build and deploy consumer-created applications using the provider's cloud infrastructure. In this scenario, the consumer can build, deploy, and manage applications using the provider's cloud tools, languages, and interfaces. However, the consumer does not manage the underlying cloud infrastructure. Examples of PaaS include the Microsoft Azure platform with .NET and the Java development environment, and Google Application Engine with Python or Java.
- **Infrastructure as a Service (IaaS)**. In this case, the cloud service provider offers consumers the ability to provision their own resources on demand; these resources include storage, servers, databases, processing units, and even a complete virtualized desktop. The consumer then can add or remove the resources as needed. For example, a consumer can use Amazon Web Services (AWS) and provision a server computer that runs Linux and Apache Web server using 16 GB of RAM and 256 GB of storage.

Figure 7.12 illustrates a sample of the different types of cloud services; these services can be accessed from any computing device. Cloud computing services have evolved in their sophistication and flexibility. The merging of new technologies has enabled the creation of new options such as "desktop as a service," which effectively creates a virtual computer on the cloud that can be accessed from any device over the Internet



Figure 7-12: Types of Cloud Services (Coronel C., Morris S. 2022)

Cloud Services: Advantages and Disadvantages

Cloud computing has grown remarkably in the past few years. Companies of all sizes are enjoying the advantages of cloud computing, but its widespread adoption is still limited by several factors. Table 7.4 summarizes the main advantages and disadvantages of cloud computing. As the table shows, the top-perceived benefit of cloud computing is the lower cost of entry. At the same time, the chief concern of cloud computing is data security and privacy, particularly in companies that deal with sensitive data and are subject to high levels of regulation and compliance. This concern leads to the perception that cloud services are mainly implemented in small to medium-sized companies where the risk of service loss is minimal. In fact, some companies that are subject to strict data security regulations tend to favor private clouds rather than public ones. One of the biggest growth segments in cloud services is mobile computing. For example, Netflix, the video-on-demand trailblazer, moved significant parts of its IT infrastructure to AWS. Netflix decided to move to the cloud because of the challenges of building IT infrastructure fast enough to keep up with its relentless growth.

Table 7-4: Advantages and Disadvantages of Cloud Computing

Advantage	Disadvantage
Low initial cost of entry. Cloud computing has lower costs of entry when compared with the alternative of building in house.	*Issues of security, privacy, and compliance*. Trusting sensitive company data to external entities is difficult for most data-cautious organizations.
Scalability/elasticity. It is easy to add and remove resources on demand.	*Hidden costs of implementation and operation*. It is hard to estimate bandwidth and data migration costs.
Support for mobile computing. Cloud computing providers support multiple types of mobile computing devices.	*Data migration is a difficult and lengthy process*. Migrating large amounts of data to and from the cloud infrastructure can be difficult and time-consuming.
Ubiquitous access. Consumers can access the cloud resources from anywhere at any time, as long as they have Internet access.	*Complex licensing schemes*. Organizations that implement cloud services are faced with complex licensing schemes and complicated service-level agreements.
High reliability and performance. Cloud providers build solid infrastructures that otherwise are difficult for the average organization to leverage.	*Loss of ownership and control*. Companies that use cloud services are no longer in complete control of their data. What is the responsibility of the cloud provider if data are breached? Can the vendor use your data without your consent?
Fast provisioning. Resources can be provisioned on demand in a matter of minutes with minimal effort.	*Organization culture*. End users tend to be resistant to change. Do the savings justify being dependent on a single provider? Will the cloud provider be around in 10 years?
Managed infrastructure. Most cloud implementations are managed by dedicated internal or external staff. This allows the organization's IT staff to focus on other areas.	*Difficult integration with internal IT system*. Configuring the cloud services to integrate transparently with internal authentication and other internal services could be a daunting task.

Summary

- Database connectivity refers to the mechanisms through which application programs connect and communicate with data repositories. Database connectivity software is also known as *database middleware*.
- Microsoft database connectivity interfaces are dominant players in the market and enjoy the support of most database vendors. ODBC, OLE-DB, and ADO.NET form the backbone of Microsoft's Universal Data Access (UDA) architecture.
- Native database connectivity refers to the connection interface that is provided by the database vendor and is unique to that vendor. ODBC is probably the most widely supported database connectivity interface. ODBC allows any Windows application to access relational data sources using standard SQL. Data Access Objects (DAO) is an older, object-oriented application interface. Remote Data Objects (RDO) is a higher-level, object-oriented application interface used to access remote database servers. RDO was optimized to deal with server-based databases such as MS SQL Server and Oracle.
- Object Linking and Embedding for Database (OLE-DB) is database middleware developed with the goal of adding object-oriented functionality for access to relational and nonrelational data. ActiveX Data Objects (ADO) provides a high-level, application-oriented interface to interact with OLE-DB, DAO, and RDO. Based on ADO, ADO.NET is the data access component of Microsoft's .NET application development framework. Java Database Connectivity (JDBC) is the standard way to interface Java applications with data sources.
- Database access through the web is achieved through middleware. To improve the capabilities on the client side of the web browser, you must use plug-ins and other client-side extensions such as Java and JavaScript, or ActiveX and VBScript. On the server side, web application servers are middleware that expand the functionality of web servers by linking them to a wide range of services, such as databases, directory systems, and search engines.
- Cloud computing is a computing model that provides ubiquitous, on-demand access to a shared pool of configurable resources that can be rapidly provisioned.

Review Questions

1. Give some examples of database connectivity options and what they are used for.
2. What are ODBC, DAO, and RDO? How are they related?
3. What is the difference between DAO and RDO?
4. What are the three basic components of the ODBC architecture?
5. What steps are required to create an ODBC data source name?
6. What is OLE-DB used for, and how does it differ from ODBC?
7. Explain the OLE-DB model based on its two types of objects.
8. How does ADO complement OLE-DB?
9. What is ADO.NET, and what two new features make it important for application development?
10. What are web server interfaces used for? Give some examples.
11. Search the Internet for web application servers. Choose one and prepare a short presentation for your class.
12. What does this statement mean: "The web is a stateless system." What implications does a stateless system have for database application developers?
13. What is a web application server, and how does it work from a database perspective?
14. What are scripts, and what is their function? (Think in terms of database application development.)
15. What is JDBC, and what is it used for?
16. What is cloud computing, and why is it a "game changer"?
17. Name and contrast the types of cloud computing implementation.
18. Name and describe the most prevalent characteristics of cloud computing services.
19. Using the Internet, search for providers of cloud services. Then, classify the types of services they provide (SaaS, PaaS, and IaaS).
20. Summarize the main advantages and disadvantages of cloud computing services.

Chapter 8: Database Administration

Learning Outcomes

After this chapter you will be able to:

1. Describe the impact of data quality on a company's assets and competitive position
2. Describe the role of the database in supporting operational, tactical, and strategic decision making
3. Describe the impact that the introduction of a DBMS has on technological, managerial, and cultural aspects of an organization
4. Describe the managerial and technical roles of the database administrator
5. Describe the processes and systems in the information security framework that support the three database security goals
6. Identify the standards, strategies, and tools used in database administration
7. Describe the impact that cloud-based data services have on the role of the DBA

Data as a Corporate Asset

Management of data is complex because it is so widely dispersed throughout the organization. While you have learned a lot about the importance of reducing redundancy, particularly within a database, that can be quite difficult when looking at the number of databases across an organization. A large organization can have a dozen or more major databases and thousands of smaller databases scattered throughout the organization. For example, as a part of data management efforts, one organization with which the authors are familiar did a thorough scan of all computers connected to their network to identify all the databases in use. The organization found more than 20,000 MS Access databases within their network. Certainly, not all of these were active, but this example illustrates the potentially massive number of databases that can exist, which can lead to enormous amounts of redundancy across databases.

To assess data's monetary value, consider what is stored in a company database: data about customers, suppliers, inventory, operations, and so on. How many opportunities are lost if the data is lost? What is the actual cost of data loss? For example, an accounting firm that lost its entire database would incur significant direct and indirect costs. The firm's problems would be magnified if the data loss occurred during tax season. Data loss puts any company in a difficult position. The company might be unable to handle daily operations effectively, it might lose customers who require quick and efficient service, and it might lose the opportunity to gain new customers.

Data is a valuable *resource* that can translate into *information*. If the information is accurate and timely, it can enhance the company's competitive position and generate wealth. In effect, an organization is subject to a *data-information-decision cycle*; that is, the data *user* applies intelligence to data to produce information that is the basis of knowledge used in *decision making*. This cycle is illustrated in Figure 8.1 Note in Figure 8.1 that decisions made by high-level managers trigger actions within the organization's lower levels. Such actions produce additional data to be used for monitoring company performance. In turn, the additional data must be recycled within the data-information-decision framework. Thus, data forms the basis for decision making, strategic planning, control, and operations monitoring.

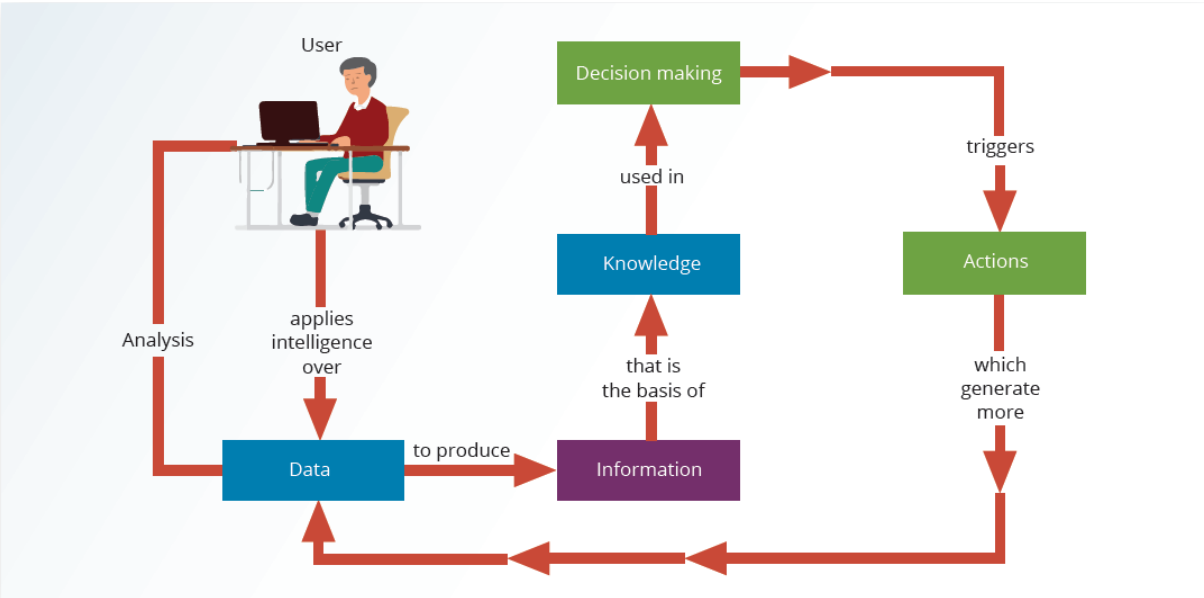


Figure 8-1: The Data-Information-Decision-Making Cycle (Coronel C., Morris S. 2022)

Efficient asset management is critical to the success of an organization. To manage data as a corporate asset, managers must understand the value of information. For some companies, such as credit reporting agencies, their only product is information, and their success is solely a function of information management.

Most organizations continually seek new ways to leverage their data resources to get greater returns. This leverage can take many forms, from data warehouses that support improved customer relationships to tighter integration with customers and suppliers in support of the electronic supply chain. As organizations become more dependent on information, that information's accuracy becomes more critical. **Dirty data**, or data that suffers from inaccuracies and inconsistencies, becomes an even greater threat. Data can become dirty for many reasons:

- Lack of enforcement of integrity constraints, such as not null, uniqueness, and referential integrity
- Data-entry errors and typographical errors
- Use of synonyms and homonyms across systems
- Nonstandard use of abbreviations in character data
- Different decompositions of composite attributes into simple attributes across systems

Some causes of dirty data, such as improper implementation of constraints, can be addressed within an individual database. However, addressing other causes is more complicated. Some dirty data comes from the movement of data across systems, as in the creation of a data warehouse. Efforts to control dirty data are referred to as data quality initiatives. **Data quality** is a comprehensive approach to ensuring the accuracy, validity, and timeliness of data. This comprehensive approach is important because data quality involves more than just cleaning dirty data; it also focuses on preventing future inaccuracies and building user confidence in the data. Large-scale data quality initiatives tend to be complex and expensive projects, so the alignment of these initiatives with business goals is a must, as is buy-in from top management. While data quality efforts vary from one organization to another, most involve the following:

- A data governance structure that is responsible for data quality
- Measurements of current data quality
- Definition of data quality standards in alignment with business goals
- Implementation of tools and processes to ensure future data quality

A number of tools can assist in data quality initiatives. In particular, data-profiling and master data management software are available from many vendors. **Data-profiling software** gathers statistics, analyzes existing data sources and metadata to determine data patterns, and compares the patterns against standards that the organization has defined. This analysis can help to assess the quality of existing data and identify sources of dirty data. **Master data management (MDM)** helps to prevent dirty data by coordinating common data across multiple systems. MDM software provides a "master" copy of entities, such as customers, that appear in numerous systems throughout the organization. MDM systems can provide a single, common definition of important entities so that every database that requires those entities can pull that data from a single source, which will make that data be consistent with every other database in the organization. This can help to reduce redundancy across databases and help ensure data consistency and improve data quality. While these technological approaches provide an important part of data quality, the overall solution to high-quality data within an organization still relies heavily on data administration and management.

The Need for a Database and Its Role in an Organization

Data is used by different people in different departments for various reasons. Therefore, data management must address the concept of shared data. Used properly, the DBMS facilitates:

- *Interpretation* and *presentation* of data in useful formats by transforming raw data into information
- *Distribution* of data and information to the right people at the right time
- Data *preservation* and *monitoring* data usage for adequate periods of time
- *Control* over data duplication and use, both internally and externally

Regardless of the organization, the database's predominant role is *to support managerial decision making at all levels in the organization while preserving data privacy and security*. An organization's managerial structure might be divided into three levels: top-level management makes strategic decisions, middle management makes tactical decisions, and operational management makes daily working decisions. *Operational* decisions are short term; for example, a manager might change the price of a product to clear it from inventory. *Tactical* decisions involve a longer time frame and affect larger-scale operations—for example, changing the price of a product in response to competitive pressures. *Strategic* decisions affect the long-term well-being of the company or even its survival—for example, changing the pricing strategy across product lines to capture market share. The DBMS must give each level of management a useful view of the data and support the required level of decision making. The following activities are typical of each management level. At the *top management* level, the database must be able to:

- Provide the information necessary for strategic decision making, strategic planning, policy formulation, and goals definition.
- Provide access to external and internal data to identify growth opportunities and to chart the direction of such growth. (Direction refers to the nature of the operations: will a company become a service organization, a manufacturing organization, or some combination of the two?)
- Provide a framework for defining and enforcing organizational policies that are translated into business rules at lower levels in the organization.
- Improve the likelihood of a positive return on investment by searching for new ways to reduce costs and boost productivity in the company.
- Provide feedback to monitor whether the company is achieving its goals.

At the *middle management* level, the database must be able to:

• Deliver the data necessary for tactical decisions and planning.
• Monitor and control the allocation and use of company resources and evaluate the performance of various departments.
• Provide a framework for enforcing and ensuring the security and privacy of the data in the database. **Security** means protecting the data against accidental or intentional use by unauthorized users. In the context of database administration, **privacy** is the extent to which individuals and organizations have the right to determine the details of data usage (who, what, when, where, and how). At the *operational management* level, the database must be able to:

- Represent and support company operations as closely as possible. The data model must be flexible enough to incorporate all current and future data.
- Produce query results within specified performance levels. Keep in mind that the performance requirements increase for lower levels of management and

operations. Thus, the database must support fast responses to a greater number of transactions at the operational management level.
- Enhance the company's short-term operations by providing timely information for customer support and for application development and computer operations.

A general objective for any database is to provide a seamless flow of information throughout the company. The company's database is also known as the corporate or enterprise database. The **Enterprise database** might be defined as the company's data representation that provides support for all present and expected future operations. Most of today's successful organizations depend on the enterprise database to provide support for all of their operations—from design to implementation, from sales to services, and from daily decision making to strategic planning.

Introduction of a Database: Special Considerations

Having a computerized database management system does not guarantee that the data will be properly used to provide the best solutions required by managers. A DBMS is a tool for managing data; like any tool, it must be used effectively to produce the desired results. In the hands of a carpenter, a hammer can help produce furniture, but in the hands of a child, it might do damage. The solution to company problems is not the mere existence of a computer system or its database, but its effective management and use. The introduction of a DBMS represents a substantial change and challenge. Throughout the organization, the DBMS is likely to have a profound impact, which might be positive or negative depending on how it is administered. For example, one key consideration is to adapt the DBMS to the organization rather than forcing the organization to adapt to the DBMS. The main issue should be the organization's needs rather than the DBMS's technical capabilities. However, the introduction of a DBMS (internally hosted or outsourced to a cloud service) cannot be accomplished without affecting the organization. The flood of new information has a profound effect on the way the organization functions and, therefore, on its corporate culture. The introduction of a DBMS has been described as a process that includes three important aspects:

- *Technological*—DBMS software and hardware
- *Managerial*—Administrative functions
- *Cultural*—Corporate resistance to change

The *technological* aspect includes selecting, installing, configuring, and monitoring the DBMS to make sure that it efficiently handles data storage, access, and security. The personnel in charge of installing the DBMS must have the technical skills to provide or secure adequate support for various users of the system: programmers, managers, and end users. Therefore, database administration staffing is a key technological consideration. The selected personnel must have the right mix of technical and managerial skills to provide a smooth transition to the new shared-data environment. In today's IT world, the technological aspects would apply to both internally hosted DBMS as well as cloud-based data environments. The *managerial* aspect of the DBMS introduction should not be taken lightly. A high-quality DBMS does not guarantee a high-quality information system, just as having the best race car does not guarantee winning a race. Such managerial aspects would also include the management of the services and the relationship with the cloud-based data services provider.

The introduction of a DBMS requires careful planning to create an appropriate organizational structure and accommodate the personnel responsible for administering the system. This structure must also be subject to well-developed monitoring and controls. The administrative personnel must have excellent interpersonal and communications skills combined with broad organizational and business understanding. Top management must be committed to the new

system and must define and support data administration functions, goals, and roles within the organization. The *cultural* impact of the new database system must be assessed carefully. The DBMS is likely to have an effect on people, functions, and interactions. For example, additional personnel might be hired, new roles might be allocated to existing personnel, and employee performance might be evaluated using new standards. A cultural impact is likely because the database approach creates a more controlled and structured information flow. Department managers who are accustomed to handling their own data must surrender ownership and share their data with the rest of the company. Application programmers must learn and follow new design and development standards. Managers might perceive an information overload and require time to adjust to the new environment. When the new database comes online, people might be reluctant to use its information and might question its value or accuracy. Many might be disappointed that the information does not fit their preconceived notions and strongly held beliefs. Database administrators must be prepared to open their doors to end users, listen to their concerns, act on those concerns when possible, and explain the system's uses and benefits.

The Evolution of Database Administration

Data administration has its roots in the old, decentralized world of the file system. The cost of data and managerial duplication in these systems gave rise to centralized data administration known as the electronic data processing (EDP) or data processing (DP) department. The DP department's task was to pool all computer resources to support all departments *at the operational level*. DP administrators were given the authority to manage all company file systems as well as resolve data and managerial conflicts created by the duplication and misuse of data. The advent of the DBMS and its shared view of data produced a new level of data management sophistication and led the DP department to evolve into an **information systems (IS) department** (A department responsible for all information technology services and production functions in an organization). The responsibilities of the IS department were broadened to include the following:

- A *service* function to provide end users with data management support
- A *production* function to provide end users with solutions for their information needs through integrated application or management information systems

The function of the IS department was reflected in its internal organizational structure; a modern organizational structure of an IS department (in a medium- to large-size company) is shown in Figure 8.2. As demand grew, the IS application development segment was subdivided by the type of system it supported: accounting, inventory, marketing, data warehousing, business intelligence, and so on. However, this development meant that database administration responsibilities were divided. The application development segment was in charge of gathering database requirements and logical database design, whereas the database operations segment took charge of implementing, monitoring, and controlling DBMS operations.

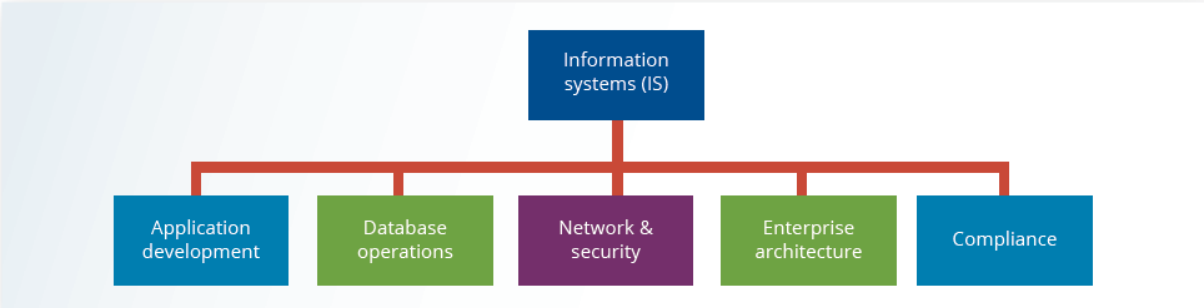


Figure 8-2: The Information Systems Department's Internal Organization (Coronel C., Morris S. 2022)

As the number of database applications grew, data management became increasingly complex, thus leading to the development of database administration. The person responsible for control of the centralized and shared database became known as the **database administrator (DBA)**. The size and role of the DBA function varies from company to company, as does its placement within the organizational structure. On the organizational chart, the DBA function might be defined as either a staff or line position. In a staff position, the DBA often takes on a consulting role; the DBA can devise the data administration strategy but does not have the authority to enforce it or resolve conflicts. In a line position, the DBA has both the responsibility and authority to plan, define, implement, and enforce the policies, standards, and procedures used in data administration. The two possible DBA positions are illustrated in Figure 8.3.

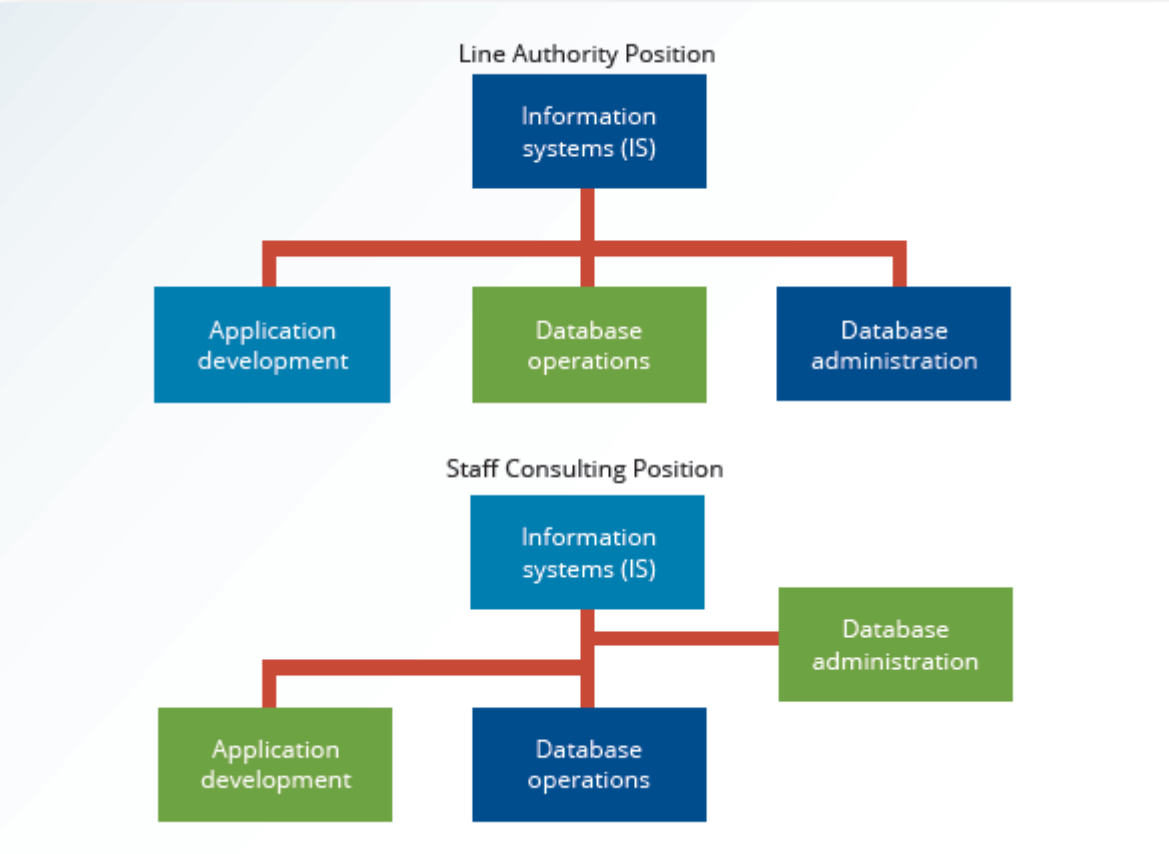


Figure 8-3: Placement of the DBA Function (Coronel C., Morris S. 2022)

There is no standard for how the DBA function fits in an organization's structure, partly because the function itself is probably the most dynamic of any in an organization. In fact, the fast-paced changes in DBMS technology dictate changing organizational styles. For example:

- The development of distributed databases can force an organization to decentralize data administration further. The distributed database requires the system DBA to define and delegate the responsibilities of each local DBA, thus imposing new and more complex *coordinating* activities on the system DBA.
- The growing use of Internet-accessible data and the growing number of data warehousing applications are likely to expand the DBA's data-modeling and design activities.
- The increasing sophistication and power of personal-computer-based DBMS packages provide an easy platform for developing user-friendly, cost-effective, and efficient solutions. However, such an environment also invites data duplication, not to mention

the problems created by people who lack the technical qualifications to produce good database designs. In short, the new computing environment requires the DBA to develop a new set of technical and managerial skills.

- The increasing use of cloud data services is pushing many database platforms and infrastructures into the cloud. This can free DBAs from many lower-level technology-oriented tasks, allowing them to focus on higher-value strategic issues. In such environments, the DBA becomes a data use service provider and advisor for the organization.
- Conversely, the growing use of Big Data in organizations can force the DBA to become more technology-oriented. Ongoing efforts to integrate Hadoop storage systems with both NoSQL and relational databases require DBAs to be familiar with the lower-level storage and access issues that are still dominant in those emerging disciplines.

DBA operations are commonly defined and divided according to the phases of the Database Life Cycle (DBLC). If that approach is used, the DBA function requires personnel to cover the following activities:

- Database planning, including the definition of standards, procedures, and enforcement
- Database requirements gathering and conceptual design
- Database logical and transaction design
- Database physical design and implementation
- Database testing and debugging
- Database operations and maintenance, including installation, conversion, and migration
- Database training and support
- Data quality monitoring and management

Figure 8.4 represents a DBA functional organization according to the preceding model.

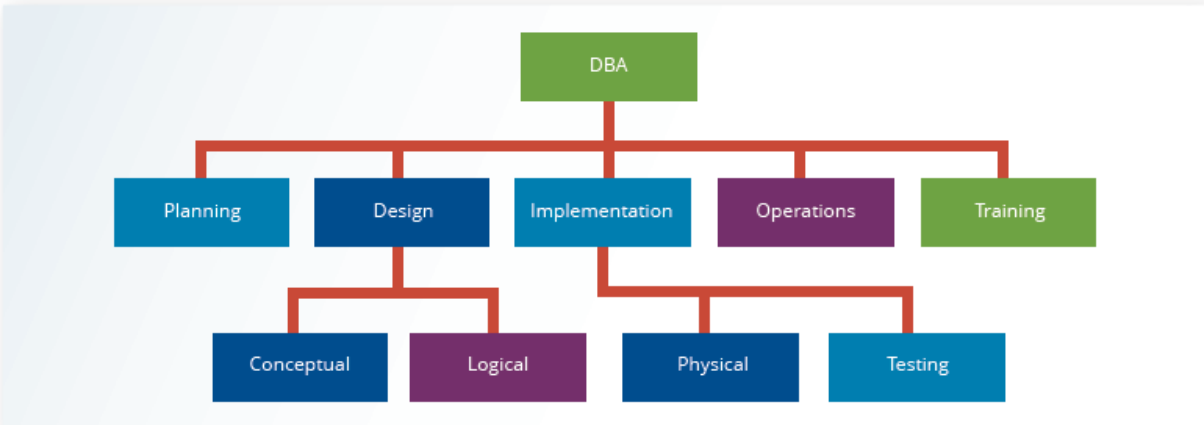


Figure 8-4: A DBA Functional Organization (Coronel C., Morris S. 2022)

Keep in mind that a company might have several incompatible DBMSs installed to support different operations. For example, some corporations have a hierarchical DBMS to support daily transactions at the operational level and a relational database to support middle and top management's ad hoc information needs. A variety of personal computer DBMSs might be installed in different departments. In such an environment, the company might have one DBA assigned for each DBMS. The general coordinator of all DBAs is sometimes known as the **systems administrator**; that position is illustrated in Figure 8.5.

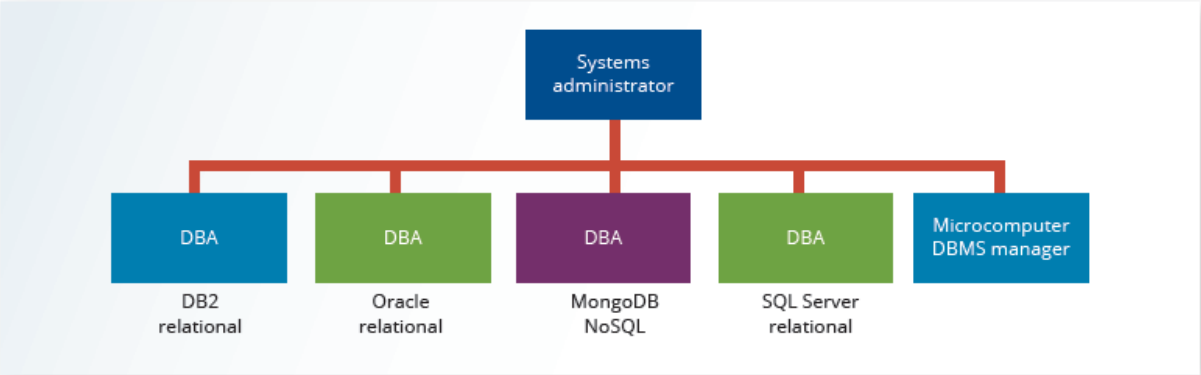


Figure 8-5: Multiple Database Admins in an Organization (Coronel C., Morris S. 2022)

There is a growing trend toward specialization in data management. For example, the organizational charts used by some larger corporations make a distinction between a DBA and the **data administrator (DA)**. The DA, also known as the **information resource manager (IRM)**, usually reports directly to top management and is given a higher degree of responsibility and authority than the DBA, although the two roles can overlap. The DA is responsible for controlling the overall corporate data resources, both computerized and manual. Thus, the DA's job covers more operations than the DBA's because the DA controls data outside the scope of the DBMS in addition to computerized data. Depending on an organization's structure, the DBA might report to the DA, the IRM, the IS manager, or directly to the company's CEO.

The Database Environment's Human Component

A substantial portion of this book is devoted to relational database design and implementation, and to DBMS features and characteristics. Thus far, the book has focused on important technical aspects of the database. However, even the most carefully crafted database system cannot operate without human assistance. In this section, you will explore how people perform the data administration activities that make a good database design useful. Effective data administration requires both technical and managerial skills. For example, the DA's job typically has a strong managerial orientation with company-wide scope, along with a technical orientation that has a narrower, DBMS-specific scope. However, the DBA also must have considerable people skills. For example, both the DA and DBA direct and control personnel staffing and training within their respective departments. Table 8.1 contrasts the characteristics of both positions by summarizing typical DA and DBA activities. All of these activities are assigned to the DBA if the organization does not employ both a DA and a DBA.

Table 8-1: Data Admin vs Database Admin Activities and Characteristics

Data Administrator (DA)	Database Administrator (DBA)
Performs strategic planning	Controls and supervises
Sets long-term goals	Executes plans to reach goals
Sets policies and standards	Enforces policies and procedures Enforces programming standards
Job is broad in scope	Job is narrow in scope
Focuses on the long term	Focuses on the short term (daily operations)
Has a managerial orientation	Has a technical orientation
Is DBMS-independent	Is DBMS-specific

Note that the DA provides a global and comprehensive administrative strategy for the organization's data. In other words, the DA's plans must consider the entire data spectrum. Thus, the DA is responsible for the consolidation and consistency of both manual and computerized data. The DA must also set data administration goals. Those goals are defined by issues such as:

- Data "shareability" and time availability
- Data consistency and integrity
- Data security and privacy
- Data quality standards
- Extent and type of data use

Naturally, the list can be expanded to fit an organization's specific data needs. Regardless of how data management is conducted—and despite the fact that great authority is invested in the DA or DBA to define and control the way company data is used—the DA and DBA do not own the data. Instead, their functions are defined to emphasize that data is a shared company asset. The preceding discussion should not lead you to believe that there are universally accepted DA and DBA administrative standards. The style, duties, organizational placement, and internal structure of both functions vary from company to company. For example, many companies distribute DA duties between the DBA and the manager of information systems. For simplicity and to avoid confusion, the label *DBA* is used here as a general title that encompasses all appropriate data administration. The arbitration of interactions between the two most important assets of any organization, people and data, places the DBA in the dynamic environment portrayed in Figure 8.6.

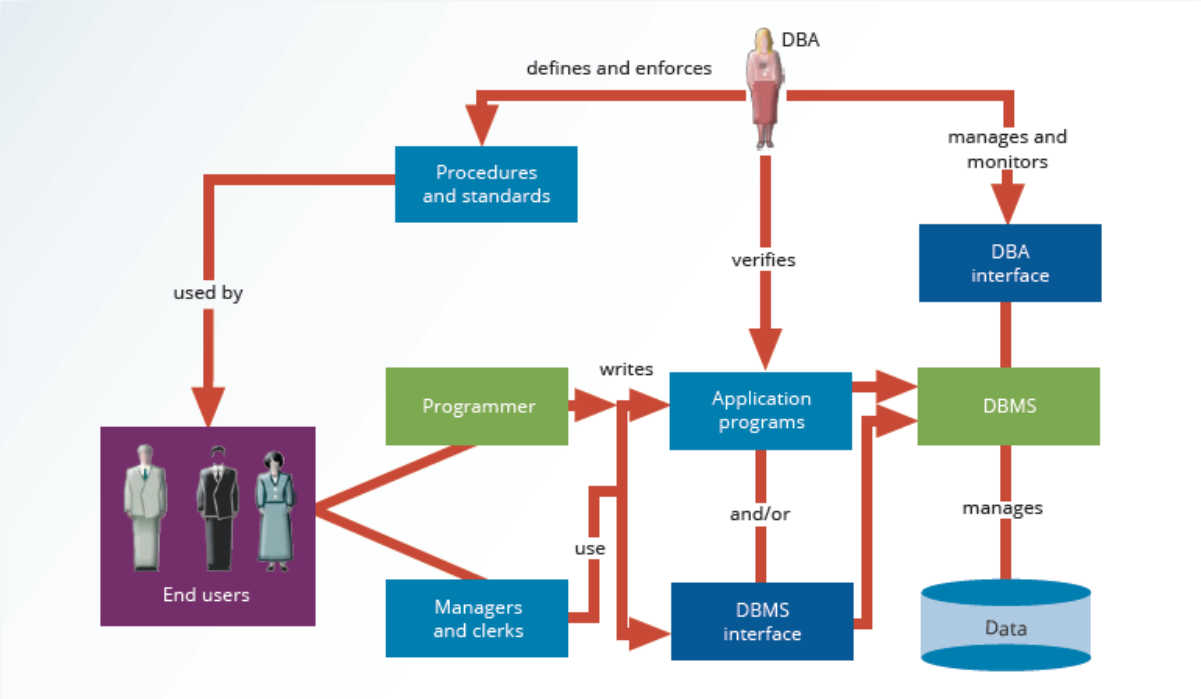


Figure 8-6: A Summary of DBA Activities (Coronel C., Morris S. 2022)

As you examine Figure 8.6, note that the DBA is the focal point for data and user interaction. The DBA defines and enforces the procedures and standards to be used by programmers and end users during their work with the DBMS. The DBA also verifies that programmer and end-user access meet the required quality and security standards. Database users might be classified by the following criteria:

- Type of decision-making support required (operational, tactical, or strategic)
- Degree of computer knowledge (novice, proficient, or expert)
- Frequency of access (casual, periodic, or frequent)

These classifications are not exclusive and usually overlap. For example, an operational user can be an expert with casual database access, or a top-level manager might be a strategic novice user with periodic database access. On the other hand, a database application programmer is an operational expert and frequent database user. Thus, each organization employs people whose levels of database expertise span an entire spectrum. The DBA must be able to interact with all of them, understand their needs, answer questions at all levels of expertise, and communicate effectively. The DBA activities portrayed in Figure 8.6 suggest the need for a diverse mix of skills. In large companies, such skills are likely to be distributed among several DBAs. In small companies, the skills might be the domain of just one DBA. The skills can be divided into two categories—managerial and technical—as summarized in Table 8.2.

Table 8-2: Desired DBA Skills

Managerial	Technical
Broad business understanding	Broad data-processing background and up-to-date knowledge of database technologies
Coordination skills	Understanding of Systems Development Life Cycle
Analytical skills	Structured methodologies Data flow diagrams Structure charts Programming languages
Conflict resolution skills	Knowledge of Database Life Cycle
Communication skills (oral and written)	Database modeling and design skills • Conceptual • Logical • Physical
Negotiation skills	Operational skills: Database implementation, data dictionary management, security, and so on

The DBA's Managerial Role

As a manager, the DBA must concentrate on the control and planning of database administration. Therefore, the DBA is responsible for the following:

- Coordinating, monitoring, and allocating database administration resources: people and data
- Defining goals and formulating strategic plans for database administration

More specifically, the DBA's responsibilities are shown in Table 8.3.

Table 8-3: DBA Activities and Services

DBA Activity		DBA Service
Planning		End-user support
Organizing		Policies, procedures, and standards
Testing	of	Data security, privacy, and integrity
Monitoring		Data backup and recovery
Delivering		Data distribution and use

Table 8.3 illustrates that the DBA is responsible for planning, organizing, testing, monitoring, and delivering quite a few services. Those services might be performed by the DBA, although they are more likely to be performed by the DBA's personnel.

End-User Support

The DBA interacts with end users by providing data and information support to their departments. Because end users usually have dissimilar computer backgrounds, support services include the following:

- *Gathering user requirements*. The DBA must work with end users to help gather the data required to identify and describe their present and future information needs. The DBA's communication skills are important in working closely with people who have varying computer backgrounds and communication styles.

- *Building end-user confidence*. Finding adequate solutions to end users' problems increases their trust and confidence in the DBA. The DBA also should educate end users about the services provided and how they enhance data stewardship and data security.
- *Resolving conflicts and problems*. Finding solutions to end users' problems in one department might trigger conflicts with other departments. End users are typically concerned with their own data needs rather than those of others, and they might not consider how their data might affect other departments within the organization. When conflicts arise, the DBA must have the authority and responsibility to resolve them.
- *Finding solutions to information needs*. The ability and authority to resolve data conflicts enables the DBA to develop solutions that will properly fit within the data management framework and address end users' information needs. Given the growing importance of the Internet, those solutions are likely to require the development and management of web servers to interface with the databases. In fact, the explosive growth of e-commerce requires the use of *dynamic* interfaces to facilitate interactive product queries and product sales.
- *Ensuring quality and integrity of data and applications*. Once the right solution has been found, it must be properly implemented and used. The DBA must work with application programmers and end users to teach them the database standards and procedures required for data quality, access, and manipulation. The DBA must also make sure that the database transactions do not adversely affect data quality. Likewise, certifying the quality of application programs that access the database is a crucial DBA function. Special attention must be given to DBMS Internet interfaces because they are prone to security issues, particularly when using cloud data services.
- *Managing the training and support of DBMS users*. One of the most time-consuming DBA activities is teaching end users how to use the database. The DBA must ensure that all users understand the basic functions of the DBMS software. The DBA coordinates and monitors all DBMS training activities.

Policies, Procedures, and Standards

A successful data administration strategy requires the continuous enforcement of policies, procedures, and standards for correct data creation, usage, and distribution within the database. The DBA must define, document, and communicate the following before they can be enforced:

- **Policies** are general statements of direction or action that communicate and support DBA goals.
- **Standards** describe the minimum requirements of a given DBA activity; they are more detailed and specific than policies. In effect, standards are rules that evaluate the quality of the activity. For example, standards define the structure of application programs and the naming conventions programmers must use.
- **Procedures** are written instructions that describe a series of steps to be followed during the performance of a given activity. Procedures must be developed within existing working conditions, and they must support and enhance the work environment. To illustrate the distinctions among policies, standards, and procedures, look at the following examples:

  Policies
  - All users must have passwords.
  - Passwords must be changed every six months.

  Standards
  - A password must have a minimum of five characters.

    - A password must have a maximum of 12 characters.
    - Social Security numbers, names, and birth dates cannot be used as passwords.

Standards and procedures defined by the DBA apply to all end users who want to benefit from the database. Standards and procedures must complement each other and must constitute an extension of data administration policies. Procedures must facilitate the work of end users and the DBA. The DBA must define, communicate, and enforce procedures that cover areas such as:

- *End-user database requirements gathering*. What documentation is required? What forms must be used?
- *Database design and modeling*. What database design methodology will be used (normalization or object-oriented)? What tools will be used (CASE tools, data dictionaries, UML or ER diagrams)?
- *Documentation and naming conventions*. What documentation must be used in the definition of all data elements, sets, and programs that access the database?
- *Design, coding, and testing of database application programs*. The DBA must define the standards for application program coding, documentation, and testing. The DBA standards and procedures are given to the application programmers, and the DBA must enforce those standards.
- *Database software selection*. The selected DBMS must properly interface with existing software, have the features needed by the organization, and provide a positive return on investment. In today's Internet environment, the DBA must also work with web and network administrators to implement efficient and secure web and cloud database connectivity.
- *Database security and integrity*. The DBA must define policies that govern security and integrity. Database security is especially crucial. Security standards must be clearly defined and strictly enforced. Security procedures must handle a multitude of scenarios to ensure that problems are minimized. Although no system can ever be completely secure, procedures must meet critical standards. The growing use of Internet interfaces to databases opens the door to new security threats that are far more complex and difficult to manage than those in traditional interfaces—this is particularly important when working with cloud data services. Therefore, the DBA must work closely with Internet security specialists to ensure that the databases are properly protected from attacks.
- *Database backup and recovery*. Database backup and recovery procedures must include information that guarantees proper execution and management of the backups. The DBA must work closely with any cloud-based data services provider to ensure the proper procedures are in place to manage data backups and restores and to ensure ownership and security of the data.
- *Database maintenance and operation*. The DBMS's daily operations must be clearly documented. Operators must keep job logs and must write operator instructions and notes. Such notes help pinpoint the causes of, and solutions to, problems. Operational procedures must also include precise instructions for backup and recovery procedures.
- *End-user training*. A full-featured training program must be established within the organization, and training procedures must be clearly specified. Each end user must be aware of available training.

Procedures and standards must be revised at least annually to keep them up to date and to ensure that the organization can adapt quickly to changes in the work environment. Naturally, the introduction of new DBMS software, the discovery of security or integrity violations, company reorganizations, and similar changes require revision of procedures and standards.

Data Security, Privacy, and Integrity

Data security, privacy, and integrity are of great concern to DBAs who manage DBMS installations. Technology has pointed the way to greater productivity through information management, and it has enabled the distribution of data across multiple sites, making it more difficult to maintain data control, security, and integrity. Thus, the DBA must use the security and integrity mechanisms provided by the DBMS to enforce the database administration policies defined in the previous section. In addition, DBAs must team up with Internet security experts to build security mechanisms that safeguard data from possible attacks or unauthorized access.

Data Backup and Recovery

When data is not readily available, companies face potentially ruinous losses. Therefore, data backup and recovery procedures are critical in all database installations. The DBA must also ensure that data can be fully recovered in case of data loss or loss of database integrity. These losses can be partial or total; therefore, backup and recovery procedures are the cheapest database insurance you can buy. The management of database security, integrity, backup, and recovery is so critical that many DBA departments have created a position called the **database security officer (DSO)**. The DSO's sole job is to ensure database security and integrity. In large organizations, the DSO's activities are often classified as *disaster management*.

Disaster management includes all of the DBA activities designed to secure data availability following a physical disaster or a database integrity failure. Disaster management includes all planning, organizing, and testing of database contingency plans and recovery procedures. The backup and recovery measures must include at least the following:

- *Periodic data and application backups*. Some DBMSs include tools to ensure automatic backup and recovery of the database. Products such as IBM's DB2 allow different types of backups: full, incremental, and concurrent. A **full backup**, also known as a **database dump**, produces a complete copy of the entire database. An **incremental backup** produces a backup of all data since the last backup date. A **concurrent backup** takes place while the user is working on the database.
- *Proper backup identification*. Backups must be clearly identified through detailed descriptions and date information, thus enabling the DBA to ensure that the correct backups are used to recover the database. The most common backup medium has traditionally been tape; computer operators must diligently store and label the tapes, and the DBA must keep track of the current tape's location. However, organizations that are large enough to hire a DBA do not typically use tapes for enterprise backup. Other solutions include optical and disk-based backup devices. Such backup solutions include online storage based on network-attached storage (NAS), storage area networks (SAN), and cloud-based data storage. Enterprise backup solutions use a layered approach in which the data is first backed up to fast disk media for intermediate storage and fast restoration. Later, the data is transferred to tape for archival storage.
- *Convenient and safe backup storage*. Multiple backups of the same data are required, and each backup copy must be stored in a different location. The storage locations must include sites inside and outside the organization. (Keeping different backups in the same place defeats the purpose of having multiple backups.) The storage locations must be properly prepared, and they may include fire-safe and quakeproof vaults as well as humidity and temperature controls. The DBA must establish a policy to respond to two questions: (1) Where are the backups to be stored? (2) How long are backups to be stored?
- *Physical protection of both hardware and software*. Protection might include the use of closed installations with restricted access, as well as preparation of the computer sites

to provide air conditioning, backup power, and fire protection. Physical protection also includes a backup computer and DBMS to be used in case of emergency. For example, when Hurricane Sandy hit the east coast of North America in 2012, the U.S. northeast suffered widespread destruction of its communications infrastructure. The storm served as a wake-up call for many organizations and educational institutions that did not have adequate disaster recovery plans for such an extreme level of service interruption.

- *Personal access control to the software of a database installation*. Multilevel passwords and privileges as well as hardware and software challenge/response tokens can be used to identify authorized users of resources.
- *Insurance coverage for the data in the database*. The DBA or security officer must buy an insurance policy to provide financial protection in the event of a database failure. The insurance might be expensive, but it is less expensive than the disaster created by massive data loss.

Two additional points worth making are:

- Data recovery and contingency plans must be thoroughly tested and evaluated, and they must be practiced frequently. So-called fire drills should not be disparaged, and they require top-level management's support and enforcement.
- A backup and recovery program is not likely to cover all components of an information system. Therefore, it is appropriate to establish priorities for the nature and extent of data recovery.

Data Distribution and Use

Data is useful only when it reaches the right users in a timely fashion. The DBA is responsible for ensuring that data is distributed to the right people, at the right time, and in the right format. These tasks can become very time-consuming, especially when data delivery capacity is based on a typical applications programming environment, where users depend on programmers to deliver the programs that access the database. Although the Internet and its intranet and extranet extensions have opened databases to corporate users, they have also created a new set of challenges for the DBA. Current data distribution philosophy makes it easy for *authorized* end users to access the database. One way to accomplish this task is to facilitate the use of new, more sophisticated query tools and new web front ends. They enable the DBA to educate end users to produce required information without being dependent on applications programmers. Naturally, the DBA must ensure that users adhere to appropriate standards and procedures. This data-sharing philosophy is common today, and it probably will become more common as database technology marches on. Such an environment is more flexible for end users; by becoming more self-sufficient in the acquisition and use of data, they can make better decisions. Yet, this "data democracy" can also produce some troublesome side effects. Letting end users micromanage their data subsets could inadvertently sever the connection between those users and data administrators. The DBA's job could become more complicated, and the efficiency of data administration could be compromised. Data duplication might flourish again without checks at the organizational level to ensure the uniqueness of data elements. Thus, end users who do not completely understand the nature and sources of data might use the data elements improperly.

The DBA's Technical Role

The DBA's technical role requires a broad understanding of DBMS functions, configuration, programming languages, and data-modeling and design methodologies. For example, the DBA's technical activities include the selection, installation, operation, maintenance, and upgrading of the DBMS and utility software, as well as the design, development, implementation, and maintenance of application programs that interact with the database. Many of the DBA's technical activities are a logical extension of the DBA's managerial activities. For example, the DBA deals with database security and integrity, backup and recovery, and training and support. The technical aspects of the DBA's job are rooted in the following areas of operation:

- Evaluating, selecting, and installing the DBMS and related utilities
- Designing and implementing databases and applications
- Testing and evaluating databases and applications
- Operating the DBMS, utilities, and applications
- Training and supporting users
- Maintaining the DBMS, utilities, and applications

Security

Information system security refers to activities and measures that ensure the confidentiality, integrity, and availability of an information system and its main asset, data. Securing data requires a comprehensive, company-wide approach. That is, you cannot secure data if you do not secure all the processes and systems around it, including hardware systems, software applications, the network and its devices, internal and external users, procedures, and the data itself. To understand the scope of data security, consider each of the three security goals in more detail:

- **Confidentiality** deals with ensuring that data is protected against unauthorized access, and if an authorized user accesses the data, that it is used only for an authorized purpose. In other words, confidentiality entails safeguarding data against disclosure of any information that would violate the privacy rights of a person or organization. Data must be evaluated and classified according to the level of confidentiality: highly restricted (very few people have access), confidential (only certain groups have access), and unrestricted (can be accessed by all users). The data security officer spends a great amount of time ensuring that the organization is in compliance with desired levels of confidentiality.
- **Compliance** refers to activities that meet data privacy and security reporting guidelines. These guidelines are either part of internal procedures or are imposed by external regulatory agencies such as the federal government. Examples of U.S. legislation enacted to ensure data privacy and confidentiality include the Health Insurance Portability and Accountability Act (HIPAA), the Gramm-Leach-Bliley Act (GLBA), and the Sarbanes-Oxley Act (SOX).
- **Integrity**, within the data security framework, is concerned with keeping data consistent and free of errors or anomalies. (See Chapter 1 to review the concepts of data inconsistencies and data anomalies.) The DBMS plays a pivotal role in ensuring the integrity of the data in the database. However, from the security point of view, the organizational processes, users, and usage patterns also must maintain integrity. For example, a work-at-home employee using the Internet to access product costing could be considered an acceptable use; however, security standards might require the employee to use a secure connection and follow strict procedures to manage the data at home, such as shredding printed reports and using encryption to copy data to the local

hard drive. Maintaining data integrity is a process that starts with data collection and continues with data storage, processing, usage, and archiving. The rationale behind integrity is to treat data as the most valuable asset in the organization and to ensure that rigorous data validation is carried out at all levels within the organization.

- **Availability** refers to the accessibility of data whenever required by authorized users and for authorized purposes. To ensure data availability, the entire system must be protected from service degradation or interruption caused by any internal or external source. Service interruptions could be very costly for companies and users alike. System availability is an important goal of security.

Security Policies

Normally, the tasks of securing the system and its main asset, the data, are performed by the database security officer and the database administrator(s), who work together to establish a cohesive data security strategy. Such a strategy begins with defining a sound and comprehensive security policy. A **security policy** is a collection of standards, policies, and procedures created to guarantee the security of a system and ensure auditing and compliance. The security audit process starts by identifying security vulnerabilities in the organization's information system infrastructure and identifying measures to protect the system and data against those vulnerabilities.

Security Vulnerabilities

A **security vulnerability** is a weakness in a system component that could be exploited to allow unauthorized access or cause service disruptions. Such vulnerabilities could fall under one of the following categories:

- *Technical*. An example would be a flaw in the operating system or web browser.
- *Managerial*. For example, an organization might not educate users about critical security issues.
- *Cultural*. Users might hide passwords under their keyboards or forget to shred confidential reports.
- *Procedural*. Company procedures might not require complex passwords or the checking of user IDs.

When a security vulnerability is left unchecked, it could become a security threat. A **security threat** is an imminent security violation.

A **security breach** occurs when a security threat is exploited to endanger the integrity, confidentiality, or availability of the system. Security breaches can lead to a database whose integrity is either preserved or corrupted.

- *Preserved*. In these cases, action is required to avoid the recurrence of similar security problems, but data recovery may not be necessary. As a matter of fact, most security violations are produced by unauthorized and unnoticed access for information purposes, but such snooping does not disrupt the database.
- *Corrupted*. Action is required to avoid the recurrence of similar security problems, and the database must be recovered to a consistent state. Corrupting security breaches include database access by computer viruses and by hackers who intend to destroy or alter data.

Table 8.4 illustrates some security vulnerabilities of system components and typical protective measures against them.

Table 8-4: A Sample of Security Vulnerabilities and Related Protective Measures

System Component	Security Vulnerability	Security Measures
People	• The user sets a blank password. • The password is short or includes a birth date. • The user leaves the office door open all the time. • The user leaves payroll information on the screen for long periods of time.	• Enforce complex password policies. • Use multilevel authentication. • Use security screens and screen savers. • Educate users about sensitive data. • Install security cameras. • Use automatic door locks.
Workstation and servers	• The user copies data to a flash drive. • Many people use the workstation. • A power failure crashes the computer. • Unauthorized personnel can use the computer. • Sensitive data is stored on a laptop computer. • Data is lost due to a stolen hard disk or laptop. • A natural disaster occurs.	• Use group policies to restrict the use of flash drives. • Assign user access rights to workstations. • Install uninterrupted power supplies (UPSs). • Add security locks to computers. • Implement a kill switch for stolen laptops. • Create and test data backup and recovery plans. • Protect the system against natural disasters—use co-location strategies.
Operating system	• Buffer overflow attacks • Virus attacks • Root kits and worm attacks • Denial-of-service attacks • Trojan horses • Spyware applications • Password crackers	• Apply OS security patches and updates. • Apply application server patches. • Install antivirus and antispyware software. • Enforce audit trails on the computers. • Perform periodic system backups. • Install only authorized applications. • Use group policies to prevent unauthorized installations.
Applications	• Application bugs—buffer overflow • SQL injection, session hijacking, etc. • Application vulnerabilities—cross-site scripting, nonvalidated inputs • Email attacks—spamming, phishing, etc. • Social engineering emails	• Test application programs extensively. • Build safeguards into code. • Do extensive vulnerability testing in applications. • Install spam filters and antivirus software for email systems. • Use secure coding techniques (see www.owasp.org). • Educate users about social engineering attacks.
Network	• IP spoofing • Packet sniffers • Hacker attacks • Clear passwords on network	• Install firewalls. • Use virtual private networks (VPNs). • Use intrusion detection systems (IDSs). • Use network access control (NAC). • Use network activity monitoring.
Data	• Data shares are open to all users. • Data can be accessed remotely. • Data can be deleted from a shared resource.	• Implement file system security. • Implement share access security. • Use access permission. • Encrypt data at the file system or database level.

Database Security

Database security refers to DBMS features and other related measures that comply with the organization's security requirements. From the DBA's point of view, security measures should be implemented to protect the DBMS against service degradation and to protect the database against loss, corruption, or mishandling. In short, the DBA should secure the DBMS from the point of installation through operation and maintenance.

To protect the DBMS against service degradation, some security safeguards are recommended. For example:

- Change default system passwords

- Change default installation paths
- Apply the latest patches
- Secure installation folders with proper access rights
- Make sure that only required services are running
- Set up auditing logs
- Set up session logging
- Require session encryption

Furthermore, the DBA should work closely with the network administrator to implement network security that protects the DBMS and all services running on the network. In modern organizations, one of the most critical components in the information architecture is the network.

Protecting the data in the database is a function of authorization management. **Authorization management** defines procedures to protect and guarantee database security and integrity. Those procedures include the following:

- *User access management*. This function is designed to limit access to the database; it includes at least the following procedures:
    - *Define each user to the database*. The DBA performs this function at the operating system level and the DBMS level. At the operating system level, the DBA can request the creation of a unique user ID for each end user who logs on to the computer system. At the DBMS level, the DBA can either create a different user ID or employ the same one to authorize the end user to access the DBMS.
    - *Assign passwords to each user*. The DBA also performs this function at both the operating system and DBMS levels. The database passwords can be assigned with predetermined expiration dates, which enable the DBA to screen end users periodically and remind them to change their passwords, thus making unauthorized access less likely.
    - *Define user groups*. Classifying users into groups according to common access needs can help the DBA control and manage the access privileges of individual users. Also, the DBA can use database roles and resource limits to minimize the impact of rogue users in the system.
    - *Assign access privileges*. The DBA assigns access privileges to specific users to access certain databases. Access rights may be limited to read-only, or the authorized access might include read, write, and delete privileges. Access privileges in relational databases are assigned through SQL GRANT and REVOKE commands.
    - *Control physical access*. Physical security can prevent unauthorized users from directly accessing the DBMS installation and facilities. Common physical security for large database installations includes secured entrances, password-protected workstations, electronic personnel badges, closed-circuit video, voice recognition, and biometric technology.

- *View definition*. The DBA must define data views to protect and control the scope of the data that are accessible to an authorized user. The DBMS must provide tools that allow the definition of views composed of one or more tables and must assign access rights to users. The SQL CREATE VIEW command is used in relational databases to define views. Oracle DBMS offers Virtual Private Database (VPD), which allows the DBA to create customized views of the data for different users. With this feature, the DBA could restrict regular users who query a payroll database to see only the necessary rows and columns,

while department managers would see only the rows and columns pertinent to their departments.

- *DBMS access control*. Database access can be controlled by placing limits on the use of DBMS query and reporting tools. The DBA must make sure the tools are used properly and only by authorized personnel.
- *DBMS usage monitoring*. The DBA must also audit the use of data in the database. Several DBMS packages contain features that allow the creation of an **audit log**, which automatically records a brief description of database operations performed by all users. Such audit trails enable the DBA to pinpoint access violations. The audit trails can be tailored to record all database accesses or just failed ones. The integrity of a database could be lost because of external factors beyond the DBA's control. For example, the database might be damaged or destroyed by an explosion, a fire, or an earthquake. Whatever the reason, the specter of database corruption or destruction makes backup and recovery procedures crucial to any DBA.

Database Administration Tools

The extraordinary growth of data management activities within organizations created the need for better management standards, processes, and tools. Over the years, a new industry arose dedicated exclusively to data administration tools. These tools cover the entire spectrum of data administration tasks, from selection to inception, deployment, migration, and day-to-day operations. For example, you can find sophisticated data administration tools for:

- Database monitoring
- Database load testing
- Database performance tuning
- SQL code optimization
- Database bottleneck identification and remediation
- Database modeling and design
- Database data extraction, transformation, and loading

All these data administration tools have something in common. They all expand the database's metadata or data dictionary. The importance of the data dictionary as a DBA tool cannot be overstated. This section examines the data dictionary as a data administration tool, as well as the DBA's use of computer-aided systems engineering (CASE) tools to support database analysis and design.

The Data Dictionary

The DBMS data dictionary provides the DBMS with its self-describing characteristic. In effect, the data dictionary resembles an x-ray of the company's entire data set, and it is a crucial element in data administration. Two main types of data dictionaries exist: *integrated* and *standalone*. An integrated data dictionary is included with the DBMS. For example, all relational DBMSs include a built-in data dictionary or system catalog that is frequently accessed and updated by the RDBMS. Other DBMSs, especially older types, do not have a built-in data dictionary; instead, the DBA may use third-party *standalone* systems. Data dictionaries can also be classified as *active* or *passive*. An **active data dictionary** is automatically updated by the DBMS with every database access to keep its access information up to date. A **passive data dictionary** is not updated

automatically and usually requires running a batch process. Data dictionary access information is normally used by the DBMS for query optimization.

The data dictionary's main function is to store the description of all objects that interact with the database. Integrated data dictionaries tend to limit their metadata to the data managed by the DBMS. Standalone data dictionary systems are usually more flexible and allow the DBA to describe and manage all of the organization's data, whether they are computerized or not. Whatever the data dictionary's format, it provides database designers and end users with a much-improved ability to communicate. In addition, the data dictionary is the tool that helps the DBA resolve data conflicts. Although there is no standard format for the information stored in the data dictionary, several features are common. For example, the data dictionary typically stores descriptions of the following:

- *Data elements that are defined in all tables of all databases*. Specifically, the data dictionary stores element names, data types, display format, internal storage format, and validation rules. The data dictionary explains where an element is used, who used it, and so on.
- *Tables defined in all databases*. For example, the data dictionary is likely to store the name of the table creator, the date of creation, access authorizations, and the number of columns.
- *Indexes defined for each database table*. For each index, the DBMS stores at least the index name, the attributes used, the location, specific index characteristics, and the creation date.
- *Defined databases*. This information includes who created each database, when the database was created, where the database is located, and the DBA's name.
- *End users and administrators of the database*. This information defines the users of the database.
- *Programs that access the database*. This information includes screen formats, report formats, application programs, and SQL queries.
- *Access authorizations for all users of all databases*. This information defines who can manipulate which objects and what types of operations can be performed.
- *Relationships among data elements*. This information includes which elements are involved, whether the relationships are mandatory or optional, and connectivity and cardinality requirements.

If the data dictionary can be organized to include data external to the DBMS itself, it becomes an especially flexible tool for more general corporate resource management. Such an extensive data dictionary thus makes it possible to manage the use and allocation of all of the organization's information, regardless of whether it has its roots in the database data. For this reason, some managers consider the data dictionary to be a key element of information resource management, which is why the data dictionary can be described as the **information resource dictionary**. The metadata stored in the data dictionary is often the basis for monitoring database use and for assigning access rights to database users. The information stored in the data dictionary is usually based on a relational table format, thus enabling the DBA to query the database with SQL commands. For example, SQL commands can be used to extract information about the users of a specific table or the access rights of a particular user. In the following section, the IBM DB2 system catalog tables are the basis for several examples of how a data dictionary is used to derive information:

- SYSTABLES stores one row for each table or view.
- SYSCOLUMNS stores one row for each column of each table or view.
- SYSTABAUTH stores one row for each authorization given to a user for a table or view in a database.

Examples of Data Dictionary Usage

Example 1

List the names and creation dates of all tables created by the user JONESVI in the current database.

```
SELECT NAME, CTIME
FROM SYSTABLES
WHERE CREATOR = 'JONESVI';
```

Example 2

List the names of the columns for all tables created by JONESVI in the current database.

```
SELECT NAME
FROM SYSCOLUMNS
WHERE TBCREATOR = 'JONESVI';
```

Example 3

List the names of all tables for which the user JONESVI has DELETE authorization.

```
SELECT TTNAME
FROM SYSTABAUTH
WHERE GRANTEE = 'JONESVI' AND DELETEAUTH = 'Y';
```

Example 4

List the names of all users who have some type of authority over the INVENTORY table.

```
SELECT DISTINCT GRANTEE
FROM SYSTABAUTH
WHERE TTNAME = 'INVENTORY';
```

Example 5

List the user and table names for all users who can alter the database structure for any table in the database.

```
SELECT GRANTEE, TTNAME
FROM SYSTABAUTH
WHERE ALTERAUTH = 'Y'
ORDER BY GRANTEE, TTNAME;
```

As you can see in the preceding examples, the data dictionary can be a tool for monitoring database security by checking the assignment of data access privileges. Although the preceding examples targeted database tables and users, information about the application programs that access the database can also be drawn from the data dictionary. The DBA can use the data dictionary to support data analysis and design. For example, the DBA can create a report that lists all data elements to be used in a particular application; a list of all users who access a particular program; a report that checks for data redundancies, duplications, and the use of homonyms and synonyms; and a number of other reports that describe data users, data access, and data structure. The data dictionary can also be used to ensure that application programmers have met the naming standards for data elements in the database, and that the data validation rules are correct. Thus, the data dictionary can be used to support a wide range of data administration activities and facilitate the design and implementation of information systems. Integrated data dictionaries are also essential to the use of computer-aided systems engineering tools.

Case Tools

CASE is the acronym for **computer-aided systems engineering**. A CASE tool provides an automated framework for the Systems Development Life Cycle (SDLC). CASE uses structured methodologies and powerful graphical interfaces. Because they automate many tedious system design and implementation activities, CASE tools play an increasingly important role in information systems development. CASE tools are usually classified according to the extent of support they provide for the SDLC. For example, **front-end CASE tools** provide support for the planning, analysis, and design phases; **back-end CASE tools** provide support for the coding and implementation phases. The benefits associated with CASE tools include:

- A reduction in development time and costs
- Automation of the SDLC
- Standardization of systems development methodologies
- Easier maintenance of application systems developed with CASE tools

One of the CASE tools' most important components is an extensive data dictionary, which keeps track of all objects created by the systems designer. For example, the CASE data dictionary stores data flow diagrams, structure charts, descriptions of all external and internal entities, data stores, data items, report formats, and screen formats. A CASE data dictionary also describes the relationships among system components.

Several CASE tools provide interfaces that work with the DBMS and allow the CASE tool to store its data dictionary information using the DBMS. Such interaction demonstrates the interdependence between systems development and database development, and it helps create a fully integrated development environment. In a CASE development environment, database and application designers use the CASE tool to store the description of the database schema, data elements, application processes, screens, reports, and other data relevant to development. The CASE tool integrates all systems development information in a common repository, which the DBA can check for consistency and accuracy.

As an additional benefit, a CASE environment tends to improve the extent and quality of communication among the DBA, application designers, and end users. The DBA can use the CASE tool to check the definition of the application's data schema, the observance of naming conventions, the duplication of data elements, validation rules for the data elements, and a host of other developmental and managerial variables. When the CASE tool finds conflicts, rules violations, and inconsistencies, it facilitates making corrections. Better yet, the CASE tool can make a correction and then cascade its effects throughout the applications environment, which simplifies the job of the DBA and the application designer.

A typical CASE tool provides five components:

- Graphics designed to produce structured diagrams such as data flow diagrams, ER diagrams, class diagrams, and object diagrams
- Screen painters and report generators to produce the information system's input and output formats (for example, the end-user interface)
- An integrated repository for storing and cross-referencing the system design data; this repository includes a comprehensive data dictionary
- An analysis segment to provide a fully automated check on system consistency, syntax, and completeness
- A program documentation generator

Figure 8.7 illustrates how Microsoft Visio Professional can be used to produce an ER diagram. Most CASE tools produce fully documented ER diagrams that can be displayed at different abstraction levels. For example, erwin Data Modeler by Computer Associates can produce

detailed relational designs. The user specifies the attributes and primary keys for each entity and describes the relations. Current generation data modeling tools assign foreign keys based on the specified relationships among the entities. Changes in primary keys are always updated automatically throughout the system. Table 8.5 lists some of the many available CASE Data Modeling tool vendors.

Major relational DBMS vendors, such as Oracle, now provide fully integrated CASE tools for their own DBMS software as well as for RDBMSs supplied by other vendors. For example, Oracle's CASE tools can be used with IBM's DB2, and Microsoft's SQL Server to produce fully documented database designs. Some vendors even take nonrelational DBMSs, develop their schemas, and produce the equivalent relational designs automatically. There is no doubt that CASE tools have enhanced the efficiency of database designers and application programmers. However, no matter how sophisticated the CASE tool, its users must be well versed in conceptual design. In the hands of database novices, CASE tools produce impressive-looking but bad designs.

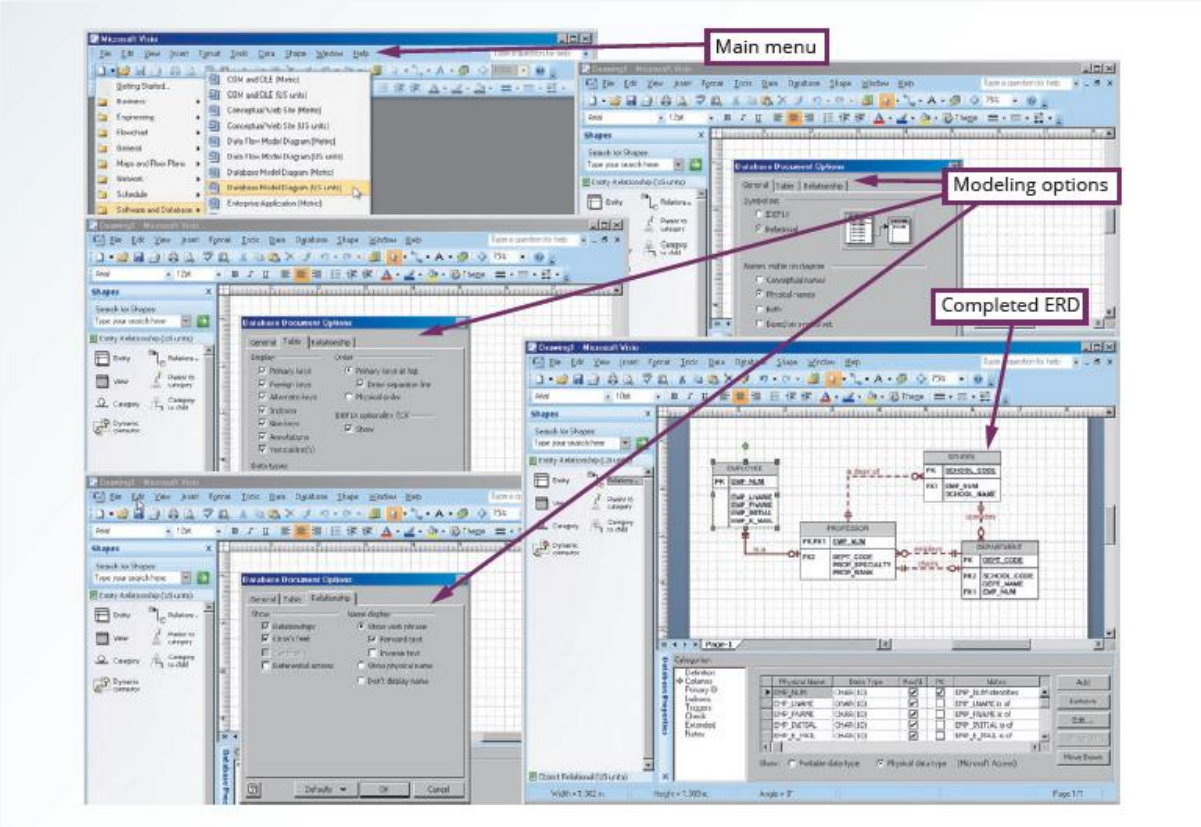


Figure 8-7: Case Tool Example (MS Visio Professional) (Coronel C., Morris S. 2022)

Table 8-5: Case Data Modeling Tools

Company	Product	Website
Erwin Inc.	erwin Data Modeler	www.erwin.com
Idera Inc.	ER/Studio Data Architect	www.idera.com
Microsoft	Visio	office.microsoft.com/en-us/visio
Oracle	SQL Developer Data Modeler	www.oracle.com/tools/downloads/sql-data-modeler-downloads.html
IBM	Rational Software Architect Designer	www.ibm.com/products/rational-software-architect-designer
SAP	Power Designer	www.sap.com/products/powerdesigner-data-modeling-tools.html
Visible Systems	Visible Analyst	www.visiblesystemscorp.com

The DBA's Role in the Cloud

The use of cloud-based data services does not signal the end of DBAs, but it does have a significant impact on their role. As discussed in previous chapters, services such as Microsoft Azure and Amazon Web Services (AWS) allow outsourcing database technology as a highly scalable, capability-on-demand service. In this new world, some of the tasks that once resided in a single "in-house" DBA function are now split between the internal DBA and the cloud service provider. As a result, the use of cloud-based data services alters and expands the typical DBA's role in both technical and managerial dimensions. In general, the cloud services partner company provides:

- *DBMS installation and updates*. The DBMS is installed on a virtual server by the service provider. As the DBMS vendor releases required updates and security fixes to the DBMS software, the service provider manages the application of the updates within a specified maintenance window. The DBA's role now has to carefully coordinate such updates with the external cloud-based data service provider.
- *Server/network management*. The service provider configures and manages the server where the DBMS resides, including scaling the database across multiple servers as needed. If the database is distributed across multiple servers, the service provider can supply load balancing to ensure a high level of performance. However, DBAs must work with their company's network departments to ensure that the network is properly configured for security, performance, availability, and management.
- *Backup and recovery operations*. The service provider performs regular backups and stores backups in secure facilities. The DBA must ensure that internal data privacy and retention policies are enforced and maintained.

Although these services are valuable and free the DBA from these tasks, the primary benefit of cloud-based data services is their ability to provide and manage computing hardware and software configuration at a low cost. The preceding tasks are only a small part of the DBA's responsibilities; the DBA's managerial role is largely unchanged and sometimes is even augmented with the new cloud data services dimension. User requirements must still be gathered; data solutions must still be designed; end users need training; and policies, standards, and procedures must be developed and enforced. Even the technical role of the DBA still exists with the use of cloud data services. There are many cloud data service providers, and some offer a variety of DBMS products, including proprietary systems. Only some versions of these DBMSs are available, including multiple versions of the same DBMS. For example, a given service provider may support both MySQL 5.7 and MySQL 8.0. In this environment, the DBA evaluates different DBMSs to determine which software product to use and from which provider to purchase the DBMS. In addition, the DBA must work with the cloud data services provider to reconcile the required database technical features with the ones supported by the cloud data service provider and ensure data availability, security, and integrity within the expanded boundaries of the company network.

Cloud data service providers offer a variety of pricing schemes. Pricing is typically based on factors such as storage space, computing resources (CPU cycles and memory), and data transfer sizes. Service users are billed monthly for the amount of resources used. Service providers have a vested interest in their clients' databases being as large as possible; it is also in their interest for database designs to be inefficient in processing queries because clients will have to buy more memory and CPU capacity. Service providers benefit if your database is filled with poorly designed tables that contain lots of unnecessarily redundant data, with every attribute in every table indexed, and queries that take a long time to run or return thousands of rows of data that must be transferred to a front-end application for additional processing.

Therefore, the DBA can save the organization time and money by ensuring that databases are properly designed with minimal redundancy and that database coding is efficient. Clearly, the DBA's technical role is still critical to organizations that use cloud-based data services. The DBA's efforts in efficient and effective database design, coding, monitoring database performance, and database tuning still affect the organization's ability to use data and information as a resource, and they have an immediate visible impact on the monthly data service bill. Regardless of whether the database is stored in the enterprise's server or in the cloud, the DBA must ensure the data's availability, security, and integrity.

Summary

- Data management is a critical activity for any organization, so data must be treated as a corporate asset. The value of a data set is measured by the utility of the information derived from it. Good data management is likely to produce good information, which is the basis for better decision making.
- Data quality is a comprehensive approach to ensure the accuracy, validity, and timeliness of data. Data quality focuses on correcting dirty data, preventing future inaccuracies in the data, and building user confidence in the data.
- The DBMS is the most commonly used tool for corporate data management. The DBMS supports strategic, tactical, and operational decision making at all levels of the organization. The introduction of a DBMS into an organization is a delicate job; the impact of the DBMS on the organization's managerial and cultural framework must be carefully examined.
- The database administrator (DBA) is responsible for managing the corporate database. The internal organization of database administration varies from company to company. Although no standard exists, it is common practice to divide DBA operations according to phases of the Database Life Cycle. Some companies have created a position with a broader mandate to manage computerized data and other data; the data administrator (DA) handles this activity.
- The DA and DBA functions tend to overlap. Generally speaking, the DA has more managerial tasks than the more technically oriented DBA. Compared to the DBA function, the DA function is DBMS-independent, with a broader and longer-term focus. However, when the organization does not include a DA position, the DBA executes all of the DA's functions. In this combined role, the DBA must have a diverse mix of technical and managerial skills.
- A DBA's managerial services include supporting end users; defining and enforcing policies, procedures, and standards for the database; ensuring data security, privacy, and integrity; providing data backup and recovery services ;and monitoring distribution and use of the data in the database.
- The DBA's technical role requires involvement in at least the following activities: evaluating, selecting, and installing the DBMS; designing and implementing databases and applications; testing and evaluating databases and applications; operating and maintaining the DBMS, utilities, and applications; and training and supporting users.
- Security refers to activities and measures that ensure the confidentiality, integrity, and availability of an information system and its main asset, data. A security policy is a collection of standards, policies, and practices that guarantee the security of a system and ensure auditing and compliance.
- A security vulnerability is a weakness in a system component that could be exploited to allow unauthorized access or service disruption. A security threat is an imminent security violation caused by an unchecked vulnerability. Security vulnerabilities exist in all components of an information system: people, hardware, software, network, procedures, and data. Therefore, it is critical to have robust database security. Database security refers to DBMS features and related measures that comply with the organization's security requirements.
- The development of a data administration strategy is closely related to the company's mission and objectives. Therefore, the strategic plan requires a detailed analysis of company goals, its situation, and its business needs. To guide the development of this data administration plan, an integrating methodology is required. The most commonly used integrating methodology is known as information engineering (IE).

- To help translate strategic plans into operational plans, the DBA has access to an arsenal of database administration tools, including a data dictionary and computer- aided systems engineering (CASE) tools.
- With the introduction of reliable cloud-based data services, the role of the DBA has expanded beyond corporate walls.

Review Questions

1. Explain the difference between data and information. Give some examples of raw data and information.
2. Define dirty data and identify some of its sources.
3. What is data quality, and why is it important?
4. Explain the interactions among end users, data, information, and decision making. Draw a diagram and explain the interactions.
5. Suppose that you are a DBA. What data dimensions would you describe to top-level managers to obtain their support for data administration?
6. How and why did database management systems become the data management standard in organizations? Discuss some advantages of the database approach over the file-system approach.
7. Using a single sentence, explain the role of databases in organizations. Then explain your answer in more detail.
8. Define *security* and *privacy*. How are the two concepts related?
9. Describe and contrast information needs at the strategic, tactical, and operational levels of an organization. Use examples to explain your answer.
10. What special considerations must you take into account when introducing a DBMS into an organization?
11. Describe the DBA's responsibilities.
12. How can the DBA function be placed within the organization chart? What effects will that placement have on the DBA function?
13. Why and how are new technological advances in computers and databases changing the DBA's role?
14. Explain the DBA department's internal organization based on the DBLC approach.
15. Explain and contrast differences and similarities between the DBA and DA.
16. Explain how the DBA plays an arbitration role between an organization's two main assets. Draw a diagram to illustrate your explanation.
17. Describe and characterize the skills desired for a DBA.
18. What are the DBA's managerial roles? Describe the managerial activities and services provided by the DBA.
19. What DBA activities support end users?
20. Explain the DBA's managerial role in the definition and enforcement of policies, procedures, and standards.
21. Protecting data security, privacy, and integrity are important database functions. What activities are required in the DBA's managerial role of enforcing those functions?
22. Discuss the importance and characteristics of database backup and recovery procedures. Then describe the actions that must be detailed in backup and recovery plans.
23. Assume that your company has assigned you the responsibility of selecting the corporate DBMS. Develop a checklist of the technical issues and other aspects involved in the selection process.
24. Describe the activities that are typically associated with the design and implementation services of the DBA's technical function. What technical skills are desirable in a DBA?
25. Why are testing and evaluation of the database and applications not done by the same people who are responsible for design and implementation? What minimum standards must be met during testing and evaluation?
26. Identify some bottlenecks in DBMS performance, and then propose some solutions used in DBMS performance tuning.

27. What are typical activities in the maintenance of the DBMS and its utilities and applications? Would you consider application performance tuning to be part of the maintenance activities? Explain your answer.
28. How do you normally define security? How is your definition similar to or different from the definition of database security in this chapter?
29. What are the levels of data confidentiality?
30. What are security vulnerabilities? What is a security threat? Give some examples of security vulnerabilities in different IS components.
31. Define the concept of a data dictionary and discuss the different types of data dictionaries. If you managed an organization's entire data set, what characteristics would you want for the data dictionary?
32. Using SQL statements, give some examples of how you would use the data dictionary to monitor database security.
33. What characteristics do a CASE tool and a DBMS have in common? How can those characteristics be used to enhance data administration?
34. Briefly explain the concepts of information engineering (IE) and information systems architecture (ISA). How do those concepts affect the data administration strategy?
35. Identify and explain some critical success factors in the development and implementation of a good data administration strategy.
36. How have cloud-based data services affected the DBA's role?

References

Coronel, C., & Morris, S. (2022). *Database Systems: Design, Implementation & Management* (14th ed.). Boston: Cengage.

Webopedia. (2025, July 27). *Definitions*. Retrieved from Webopedia: https://www.webopedia.com/definitions/

Blair, Ann, et al., editors. Information: A Historical Companion. Princeton University Press, 2021. JSTOR, https://doi.org/10.2307/j.ctv1pdrrbs. Accessed 27 Jan. 2025.

Hancock, Joelie. “Learning with Databases.” Journal of Reading, vol. 32, no. 7, 1989, pp. 582–89. JSTOR, http://www.jstor.org/stable/40029998. Accessed 27 Jan. 2025.

Silberschatz, Abraham, et al. “Database Systems.” Encyclopedia of Information Systems, 2003, pp. 411–422, https://doi.org/10.1016/b0-12-227240-4/00028-9. Accessed 15 Nov. 2021.

K. Munir. (2015). Security model for cloud database as a service (DBaaS). International

Conference on Cloud Technologies and Applications (CloudTech), Marrakech, 2015, pp. 1-

5.doi: 10.1109/CloudTech.2015.7336974